

RockFace Suite

User guide

From the thin section to the database.

Petro, Atlas, Explorer, Lab and MC, with RockFace AI and Mio. Every example uses ILLUSTRATIVE data; every picture comes from the real program.

Version 0.29.0 • 2026-11-13

This guide is also on the web, where it is updated with each version:

https://laribeirojr.github.io/rockface/guide/

Contents

Getting started 5

Getting started 6

Projects and descriptions 10

Python scripts 13

Working together on a server 16

Help, support, plug-ins and updates 21

Sending your opinion 23

RockFace Petro 25

Identification of a sample 26

Macroscopic and microscopic description 29

Pore system 32

Composition 35

Colour of the selected lines (composition) 39

Point counting 41

Totals 45

Classifications and ternary diagrams 48

McBride, Bramkamp & Powers and the packing classes 52

Diagenesis: environments, paragenetic sequence and knowledge graph 55

Analyses and analysis requests 58

Reports 61

Exporting and importing descriptions 64

Exporting wells to WITSML and samples to OSDU 67

Sample catalogue, Touchstone-ready table and XML by mapping 69

Well trajectory 72

Nomenclature, scales and references (the knowledge base) **74**

RockFace Atlas **77**

RockFace Atlas: whole slides, detail photos and the image library **78**

Atlas annotations and the search by concept **82**

Mineral maps: registration, overlay, composite and quantification **85**

Spot analyses: geochemistry, dating and fluid inclusions **88**

Files of the reference programs **91**

From the 2D composite to the 3D composite, and publishing **94**

RockFace Explorer **96**

RockFace Explorer: queries, result tables and exports **97**

RockFace Explorer: the dashboard of a collection **101**

Groups, figures and presentations **104**

Wells to LAS **107**

U-Pb dating on the Tera-Wasserburg diagram **109**

Paragenetic evolution and the stratigraphic column **112**

RockFace Lab **115**

RockFace Lab: crop, filters and the sample mask **116**

RockFace Lab: registration and the image calculator **119**

RockFace Lab: segmentation and porosity **122**

Segmenting by painted examples (random forest) **125**

RockFace Lab: editing maps and objects **128**

RockFace Lab: measurements and size distributions **131**

Transitions between phases **134**

RockFace Lab: flows, the whole section and NetCDF **136**

A mineral map aligned without landmarks; effects on big volumes **139**

Suggest description: from a segmentation map to the description **141**

RockFace MC **144**

RockFace MC: micro-CT volumes — import, views and processing **145**

RockFace MC: segmentation, porosity and registration of volumes **148**

Volume tools: filters, shading, segment editor effects, objects, registration, analyses **151**

Pore network: extraction, permeability, mercury intrusion, relative permeability, production **155**

Open rock data: public micro-CT images **159**

Microporosity: porosity maps, Darcy permeability and the Microtom simulations **161**

Multiscale: image generation (MPS), comparison of the realizations and the two-scale pore network **165**

3D surfaces, saving into an existing NetCDF file and the Geolog package **168**

Jobs: batch and remote computing **171**

Permeability model of image logs **174**

RockFace MC: whole cores and core-box photographs **177**

RockFace MC: image logs **180**

Editing a log segmentation and its objects **183**

MC → More: cups, distance map, folder watch, reconstruction, upscaling and two-phase flow **185**

RockFace AI and Mio **188**

RockFace AI: installation, device and the model hub **189**

RockFace AI: segmenting with models, training, fine-tuning and validating **193**

RockFace AI: generative model of a volume (SinGAN) and the spot cycle **197**

Mio: questions in plain language and the local language model **200**

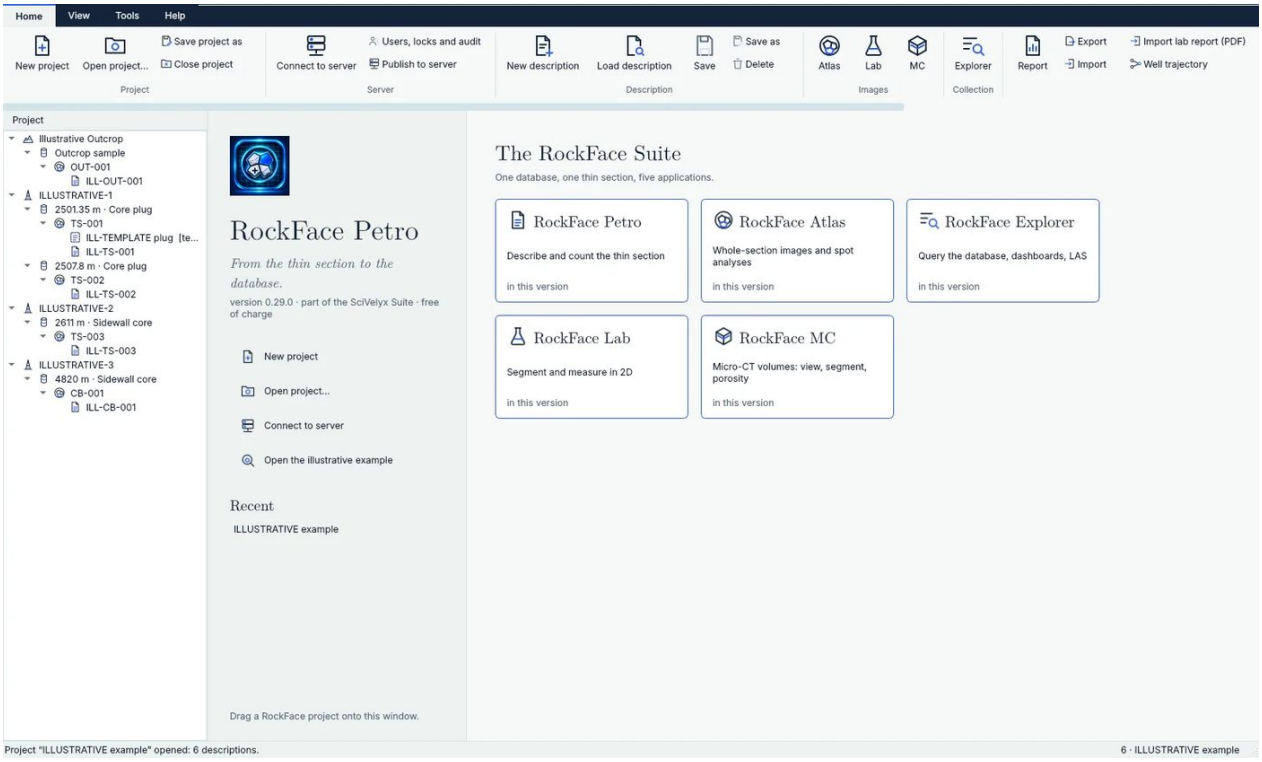
Mio’s skills **204**

Getting started

Getting started

What it is for

RockFace Suite describes, images, queries and measures sedimentary rocks, from the thin section to the database. It is five programs that open the same project and the same thin section, so what one writes the others read at once:



In the program: Getting started

Program	For	Opens with
RockFace Petro	the description of a thin section: identification, composition, point count, classifications, diagenesis, reports	`rockface-petro`
RockFace Atlas	whole-slide images and detail photos at their place on the section, annotations, mineral maps, spot analyses	Home → **Atlas** (Ctrl+Shift+A) or `rockface-atlas`
RockFace Explorer	queries over a collection, the dashboard, groups, figures, presentations, LAS, dating, paragenetic evolution	Home → **Explorer** (Ctrl+Shift+E) or `rockface-explorer`

Program	For	Opens with
RockFace Lab	image analysis of a section: crop, filters, registration, segmentation, objects, measurements	Home → **Lab** (Ctrl+Shift+L) or `rockface-lab`
RockFace MC	micro-CT volumes, whole cores and image logs of a well	Home → **MC** (Ctrl+Shift+M) or `rockface-mc`

Before you start

- RockFace installed on your computer with the installer the RockFace team gave you (public downloads start with version 1.0). RockFace AI and the local language model are installed apart (pages RockFace AI: installation, device and the model hub and Mio: questions in plain language and the local language model).
- Nothing else: the first project can be the ILLUSTRATIVE example.

Step by step

The start page

1. Open **RockFace Petro**. The start page shows **New project**, **Open project...**, **Connect to server** and **Open the illustrative example**, the **Recent** projects, and the five programs of **The RockFace Suite**.
2. Choose **Open the illustrative example** to look around first. It writes a small project with invented samples (every name is marked ILLUSTRATIVE) and opens it.
3. Or drag a `.rockface` file onto the window to open it.

The window

- The **ribbon** at the top has four tabs:
  - **Home** — the groups **Project**, **Server**, **Description**, **Images** (Atlas, Lab, MC), **Collection** (Explorer), **Reports and exchange** and **Form**;
  - **View** — **Dark theme** and the **Language**;
  - **Tools** — the **Knowledge base** (Nomenclature, Scales, References, Knowledge graph), **Python** (Scripts) and **Mio**;
  - **Help** — **Help contents** (F1), **About RockFace**, **Licence**, **Support...**, **Plug-ins and updates...**, the knowledge graph and the two ILLUSTRATIVE projects.
- The **Project** panel on the left lists the wells, their thin sections and the descriptions of each section.
- The **description tabs** in the middle: Identification, Macroscopic, Microscopic, Composition, Point count, Pore system, Classification, Diagenesis, Analyses and Requests.
- The **status bar** at the bottom gives the result of every action; on the right, the number of descriptions and the project's name.

Language and theme

1. **View → Language**: English, Español or Português. Every window changes at once, and the choice is kept for the next start.
2. **View → Dark theme** switches between the light and the dark theme.

The project does not depend on the language: a project written in Portuguese opens, and reports, in English or Spanish. Only the labels change; the stored values are the same codes.

Help inside the program

- **Help → Help contents** (F1) opens this guide in the interface language, with **Contents**, **Index** and **Search**.
- **Mio**, RockFace's assistant, answers "how do I..." questions with the page of this guide that explains it (page Mio: questions in plain language and the local language model).
- **Help → Support...** gathers the versions of RockFace and its libraries to send with a question (no file names and no project content).

The ILLUSTRATIVE collection

Help → Open the ILLUSTRATIVE collection writes a project of 48 invented descriptions from four wells, with images, analyses, U-Pb spots and a stratigraphic table. It is meant for trying RockFace Explorer.

Options

Where	What it changes
View → Language	The interface language (English, Spanish, Portuguese) of every program.
View → Dark theme	Light or dark windows. Diagrams and figures are always drawn on paper colours.
Help → Plug-ins and updates...	Plug-ins installed and the check for a newer RockFace (page Help, support, plug-ins and updates).

How it works

A project is one `.rockface` file (an SQLite database) or a database on a PostgreSQL server shared by a team (page Working together on a server). Every image, counted point and spot analysis is stored in the coordinate frame of its thin section (millimetres), never only in image pixels, so the five programs agree on where things are. Files written by an older version are upgraded when opened; a file written by a newer version is refused with a message.

From a script

```
import rockface.api as rf
with rf.example("demo.rockface") as prj:          # the ILLUSTRATIVE example project
    print(len(prj), [d.name for d in prj])
```

From a terminal:

```
rockface example demo.rockface        # writes the ILLUSTRATIVE example project
rockface collection coll.rockface     # writes the ILLUSTRATIVE collection (48 descriptions)
rockface petro demo.rockface          # opens it in RockFace Petro (also atlas, explorer, lab, mc)
```

Limits and common problems

- “This project was written by a newer RockFace”: update RockFace to open it.
- “… is not a RockFace project”: the file is not a RockFace database (or is damaged).
- The preferences (language, theme, recent projects) belong to your user on this computer, not to the project.

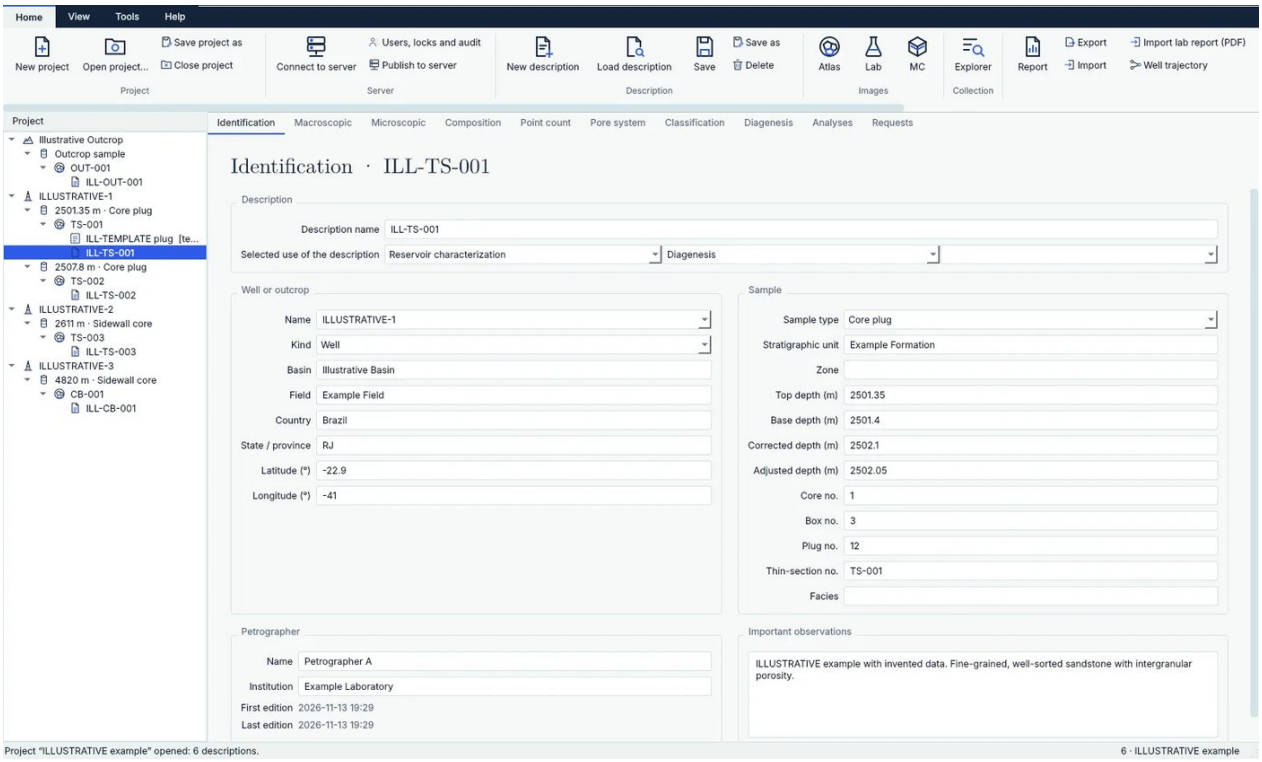
Coverage

P1 (main window), P3 (status bar), P10 (interface in English, Portuguese and Spanish), P40 (help contents), O1 (scripting).

Projects and descriptions

What it is for

A **project** holds a collection: its wells and outcrops, samples, thin sections, descriptions, images, analyses, volumes and logs. A **description** is the petrographer's record of one thin section. This page covers creating and opening projects, and creating, loading, saving, copying, deleting and using descriptions as templates.



In the program: Projects and descriptions

Before you start

RockFace Petro open (page Getting started).

Step by step

A project

1. **Home → Project → New project** (Ctrl+Shift+N): choose a folder and a name. The file ends in `.rockface`.

2. **Open project...** (Ctrl+O) opens an existing one; the **Recent** list of the start page keeps the last ones.
3. **Save project as...** writes a copy of the whole project under a new name and goes on in the copy.
4. **Close project** returns to the start page.

A project on a team server is opened with **Connect to server** instead (page Working together on a server).

A new description

1. **Home → Description → New description** (Ctrl+N).
2. Type the **Name of the new description** and choose its **Thin section**:
   - a new thin section, or
   - an existing one, when several petrographers (or several studies) describe the same section. The identification then starts from what is known of that section: well, depths, sample, numbers.
3. The **Identification** tab opens. Fill it in (page Identification of a sample) and the other tabs as needed.
4. **Save** (Ctrl+S). Every tab is checked together: any error blocks the whole save, and the first tab with an error is shown, with the field marked.

In the **Project** panel, a double click on a thin section also starts a new description on it.

Loading, saving and copying

- **Load description...** (Ctrl+L) lists the descriptions with their well, depth and thin section; type in **Filter...** to narrow the list. A double click in the **Project** panel does the same.
- **Save** (Ctrl+S) writes the description open. If you load another one or close the project with changes not saved, RockFace asks first.
- **Save as...** (Ctrl+Shift+S) writes a copy under a new name (**Name of the copy**). The name of a saved description cannot be changed: use Save as and delete the old one.
- **Delete** removes the description open, with its parts and point count, after a confirmation. It cannot be undone. When it was the last description of a thin section, the section goes too, unless it has Atlas images or spot analyses, or its sample has analyses or requests: those are never thrown away. The well always stays.
- **Home → Form → Clear** empties the tab shown, to start it again.

Templates

A template is a description kept as a starting point: a usual list of constituents, the petrographer and institution, a standard set of uses.

1. Open a description that is a good start, then **Save as...**.
2. Tick **Mark the copy as a template**.
3. Templates appear with the label *template* in the panel and are left out of the count of descriptions and of the queries of Explorer; an export includes them only when asked.
4. To use one: load it, **Save as...** under the new sample's name (without the tick), then change what differs.

Options

Field	What it changes
Thin section (New description)	New section, or a section already in the project: several descriptions of one section share its images, point count frame and analyses.
Mark the copy as a template	The copy is a template: kept apart from the collection.

How it works

The data model follows the rock: basin → field → well (or outcrop) → core → sample → thin section → description, with the images, analyses, volumes and logs attached to the sample or the thin section. Every change is written in the project's audit table (who, when, what). A description is saved as one transaction: identification and all its parts, or nothing.

From a script

```
import rockface.api as rf
with rf.create("study.rockface") as prj:                    # a new, empty project
    d = prj.new("TS-9", well_name="ILLUSTRATIVE W-1", top_depth=2501.3, thin_section_no="9")
    prj.copy("TS-9", "TS-9 template", template=True)     # a template
    print([x.name for x in prj], len(prj.descriptions(templates=True)))
with rf.open("study.rockface") as prj:
    prj.delete("TS-9")
```

Limits and common problems

- "A description named … already exists in this project": names are unique in a project.
- "The name of a saved description cannot be changed; use Save as."
- "Not saved: …" in the status bar names the field to correct; the tab with the error is shown.
- On a server, descriptions written by someone else open read-only for a petrographer (page Working together on a server).

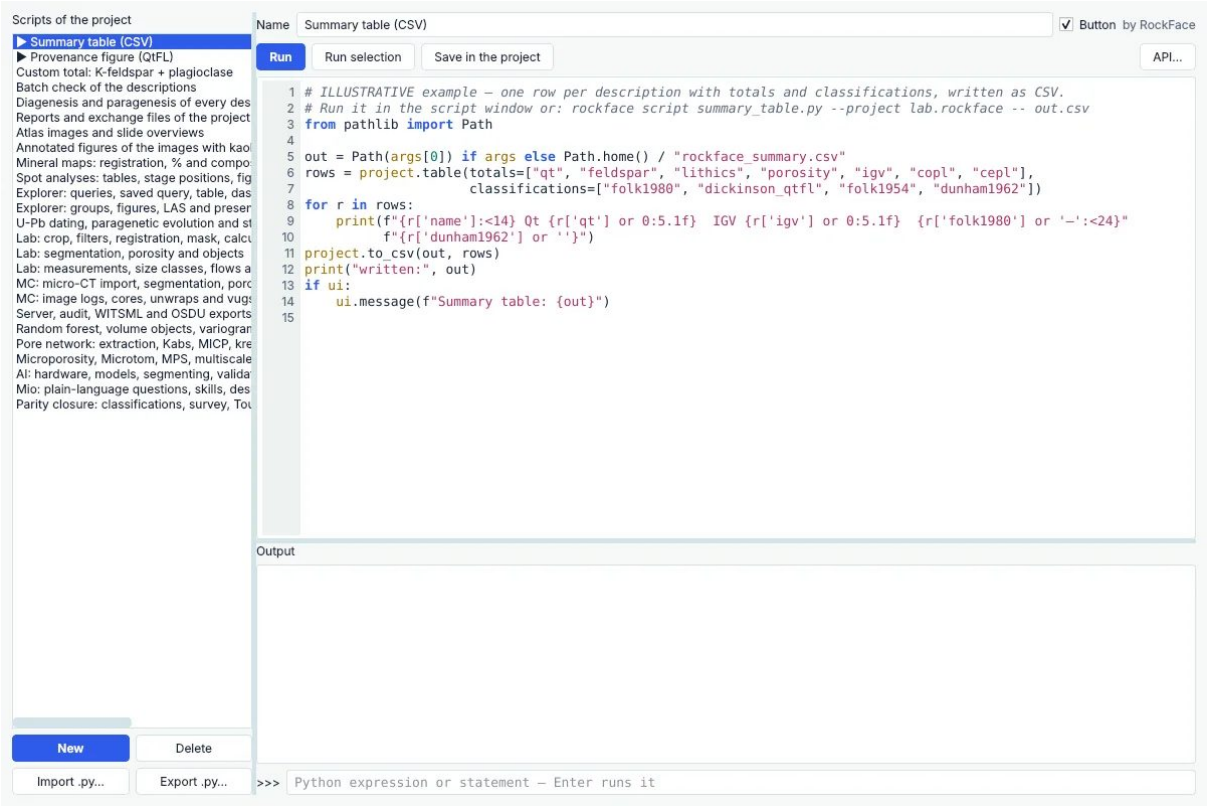
Coverage

P1 (main window), P2 (New, Load, Save, Save as, templates, Delete), P6 (thin section chosen for a description), P9 (Load description).

Python scripts

What it is for

Every RockFace program can be driven by Python scripts through one public API, `rockface.api`: to fill in many descriptions at once, to write the tables and figures of a study, to check a whole collection, or to automate any task of Atlas, Explorer, Lab and MC. Scripts are kept in the project, can be listed as buttons of the ribbon, and also run without the interface, from a terminal or a Jupyter notebook.



In the program: Python scripts

Before you start

- A project open, to keep scripts in it (“Open a project to keep scripts in it.”).
- Basic Python. The **API...** button and `docs/SCRIPTING.md` of the source code list every call.

Step by step

The script window

1. **Tools → Python → Scripts** (Ctrl+Shift+P). The **Python scripts** window opens.
2. **Scripts of the project** on the left: **New**, **Delete**, **Import .py...**, **Export .py...**.
3. Give the script a **Name**. Tick **Button** to list it under the arrow of the **Scripts** button of the ribbon.
4. Write the script in the editor (line numbers, Python highlighting). In a script:
   - `rf` — the API; `project` — the open project; `current` — the description open in the interface;
   - `ui.message(text)` writes in the status bar; `ui.open(name)` opens a description after the script ends.
5. **Run** (F5 or Ctrl+Shift+J) runs the whole script; **Run selection** (Ctrl+J) runs the selected lines or the current line.
6. **Output** shows what the script prints and its errors, the line of an error marked in the editor ("The script stopped with an error at line ...").
7. The **console** under the output evaluates one line in the same namespace as the last run — Enter runs it.
8. **Save in the project** (Ctrl+S).

If the open description has unsaved changes, RockFace asks to save it before the run, and reloads it after.

A first script

```
for d in project:
    t = d.totals()
    print(d.name, round(t["porosity"], 1), d.classify("folk1980", label=True))
rows = project.table(totals=["volume", "qt", "feldspar", "igv"], classifications=["folk1980"])
project.to_csv("summary.csv", rows)
ui.message(f"{len(rows)} descriptions written")
```

Without the interface

```
rockface script my_script.py --project study.rockface -- arg1 arg2
```

In Jupyter or any Python program:

```
import rockface.api as rf
with rf.open("study.rockface") as prj:
    print(len(prj))
```

`rockface notebook study.rockface` writes a notebook that opens the project (page MC → More: cups, distance map, folder watch, reconstruction, upscaling and two-phase flow).

The example scripts

The ILLUSTRATIVE example project holds the example scripts: a summary table, a provenance figure, a custom total, a check of every description, the diagenesis summary, reports in batch, the Atlas overview, annotated figures, mineral maps, spot analyses, Explorer queries and figures, dating and evolution, and the Lab and MC examples.

Options

Option	What it does
Button	Lists the script under the Scripts button of the ribbon (Tools tab).
Run selection	Runs only the selected lines, or the current line.

How it works

- **Same checks.** A script cannot store what the forms would refuse: `d.save()` validates exactly as the forms and raises `rf.ScriptError` with the messages, writing nothing.
- **Audit.** Writes made by a script are recorded under the user's name followed by `[script]`.
- **Scripts written by others.** A script kept in a project and written by someone else runs only after a confirmation ("A script can read and change the project and the files of this computer. Run it?").
- **Stability.** `rf.API_VERSION` follows semantic versioning: within 1.x names are only added.
- **Licence.** Scripts that use the API are the user's own work: writing, running and sharing them is allowed.

From a script

This page is about scripts; the "From a script" section of every other page of this guide shows the calls for that function.

Limits and common problems

- "The script stopped with an error at line ...": the traceback in the output shows only the script's own lines.
- A script runs with your rights on the computer: run only scripts you trust.
- A long script blocks the window while it runs; for long calculations of MC use jobs (page Jobs: batch and remote computing).

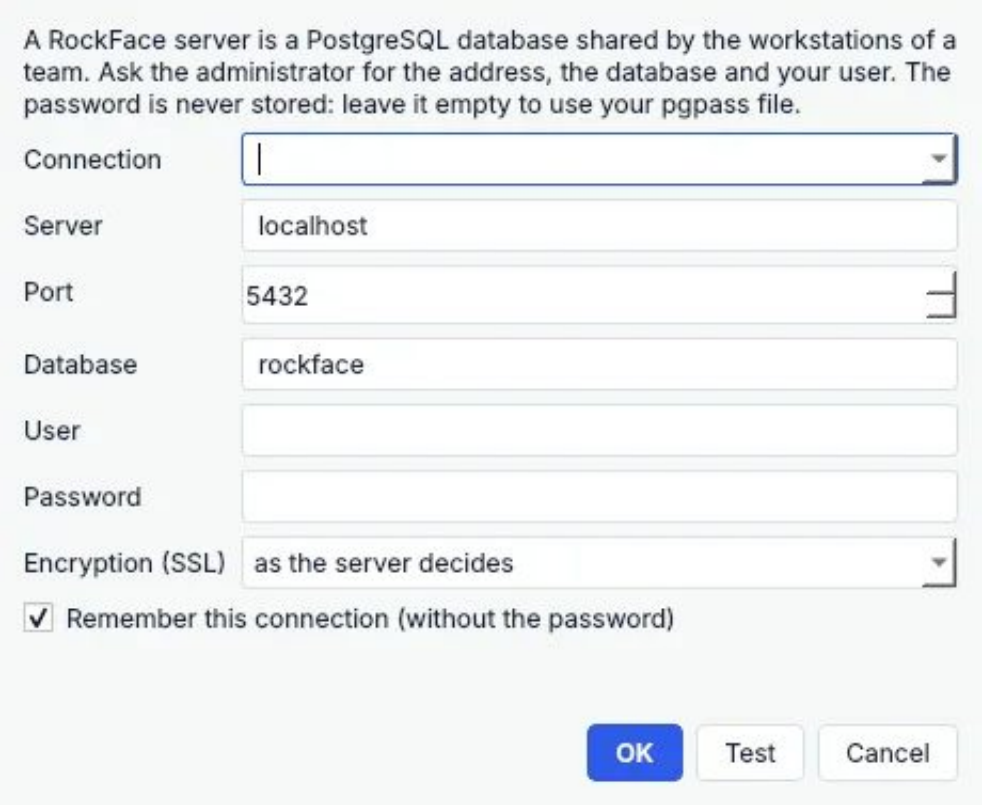
Coverage

O1 (Python scripting in every RockFace application).

Working together on a server

What it is for

A team of petrographers describes the same collection of samples at the same time. Instead of each person keeping a project file, everyone connects to one **server database**. Everyone sees what the others save, and nobody overwrites anyone else's work. RockFace Petro, Atlas, Explorer, Lab and MC all work the same on a server as on a file.



In the program: Working together on a server

Before you start

- The team's administrator has set up the server (see "Setting up a server" below) and given you:
  - the server address and port;
  - the database name;
  - your user name (login) and password.
- Your computer reaches the server over the network. On a laboratory network this usually works. Across networks, ask the administrator about SSL.

Step by step

Connecting

1. Open RockFace Petro. On the start page, or in **Home → Server**, choose **Connect to server**.
2. Fill in **Server**, **Port** (usually 5432), **Database**, **User** and **Password**.
   - Give the connection a name in **Connection** (for example "Laboratory").
   - Leave **Remember this connection** ticked, so next time you only type the password. RockFace never stores the password.
3. Optional: press **Test**. It shows your user and your **level**.
4. Press **OK**. The window title shows `database@server — user (level)`.

Your level

Your level decides what you can do. The administrator sets it.

Level	You can
viewer	read everything, run queries, figures and reports
petrographer	also create descriptions and change **your own**; add images, analyses, volumes, logs
manager	also change **anyone's** descriptions and the knowledge base, and remove locks
administrator	also manage users, levels and passwords, and back up

Editing together

- **Opening a description.** When you open a description for editing, RockFace **locks** it for you.
- **Someone else is editing it.** If another person opens the same description, they see a message like "ILL-02 is being edited by alice (LAB-PC3) since 10:42: opened read-only". Their Save button is off.
- **Another author's description.** It opens read-only for a petrographer, with a message saying who wrote it.
- **When the lock goes.** It is released when you open another description or close the program.
  - A computer that is switched off or loses the network stops renewing its lock. After 10 minutes the lock is free again.
  - A manager can remove a lock at once (Home → Server → Users, locks and audit → **Locks**).
- **Changed in between.** If a description you have open was changed by someone else in the meantime (for example a manager who took over a lock), your save is refused: "This record was changed on another workstation since you opened it". Open the description again, see the changes, and redo yours. RockFace never silently writes over someone else's work.
- **What the others do.** Every 15 seconds the status bar shows the last change made by someone else (for example "bob: create description:ILL-B2"), and the browser on the left is refreshed.

Your profile and password

Home → Server → Users, locks and audit → My profile: your name, institution and e-mail, and **Change my password** (at least 8 characters).

The audit

Users, locks and audit → Audit lists every change: when (UTC), who, what, on which record.

- Filter by user, action or part of the record name, then **Export CSV**.
- Nobody can change or delete the audit, not even a manager.

Publishing a project file to the server

To move a project from a file to the server, with its descriptions, images, analyses, volumes and logs:

1. Open the file in RockFace Petro.
2. Choose **Home → Server → Publish to server**.
3. Press **Server and database (as administrator)…** and give an administrator's connection to a **new, empty** database. It is created and prepared if it does not exist.
4. In the table, give each author found in the file the **login he or she has on the server**, so they keep editing their descriptions.
5. Press **OK**. RockFace opens the server copy.

Backing up

- **Save project as…** of a server project writes a complete project file (`.rockface`) with everything in it.
- The administrator can do the same from the command line (`rockface server backup`).

Setting up a server (administrator)

1. On the server computer, with Docker installed, copy `docker/.env.example` to `docker/.env` and choose a long password in `ROCKFACE_ADMIN_PASSWORD`.
2. Start the database: `docker compose up -d db` (in the `docker` folder).
3. Prepare it for RockFace:

   ```
   docker compose --profile admin build
   docker compose run --rm admin init "postgresql+psycopg://rockface_admin@db:5432/rockface"
   ```

4. Add the users:

   ```
   ROCKFACE_NEW_PASSWORD=… docker compose run --rm admin add-user "postgresql+psycopg://rockface_admin
   -role petrographer --name "Alice …"
   ```

   You can also add users from the program: **Users, locks and audit → Users → Add user**.

5. Back up regularly:

```
docker compose run --rm admin backup "postgresql+psycopg://rockface_admin@db:5432/rockface" /backup
YYYY-MM-DD.rockface
```

Without Docker: use any PostgreSQL 13 or later and a computer with RockFace installed (`pip install ".[server]"`). The commands are the same, without `docker compose run --rm admin`: for example `rockface server init postgresql+psycopg://admin@server:5432/rockface`.

Options

- **Encryption (SSL)**: "as the server decides" (default), *prefer*, *require* or *verify-full* (checks the server's certificate).
- **Empty password**: RockFace uses your `pgpass` file (`%APPDATA%\postgresql\pgpass.conf` on Windows, `~/.pgpass` elsewhere).

How it works

- **The server.** It is a PostgreSQL database. Your login is a database user. Your level is the group it belongs to: `rf_<database>_viewer`, `…_petrographer`, `…_manager` or `…_admin`.
- **Rights.** The program checks them before saving, and the database checks them again with its own privileges and row-level security policies. A description can only be changed by its author or by a manager, whatever program writes to the database.
- **Locks.** They are kept in the database (table `edit_lock`).
- **Versions.** Every description has a version number. A save carries the version that was read, and is refused if the database has a newer one.
- **More.** See `docs/SERVER.md` for the details and the acceptance test.

From a script

```
import rockface.api as rf
with rf.connect("postgresql+psycopg://alice@server:5432/rockface") as prj:
    print(prj.user, prj.role)
    for entry in prj.audit(limit=10):
        print(entry["when"], entry["user"], entry["action"], entry["target"])
    prj.backup("copy.rockface")
```

Limits and common problems

- **"Could not connect to the server"**: check the address, the port, the database name, your user and password, and that the server allows your computer (its `pg_hba.conf`).
- **"You can only read this database"**: your level is viewer; ask the administrator.
- **"... was written by ...: only its author or a manager may change it"**: it is a colleague's description. Ask a manager, or use **Save as...** (in the Description group) to make your own copy.
- Images, volumes and logs are shared records: when two people change the same one, the last save wins.

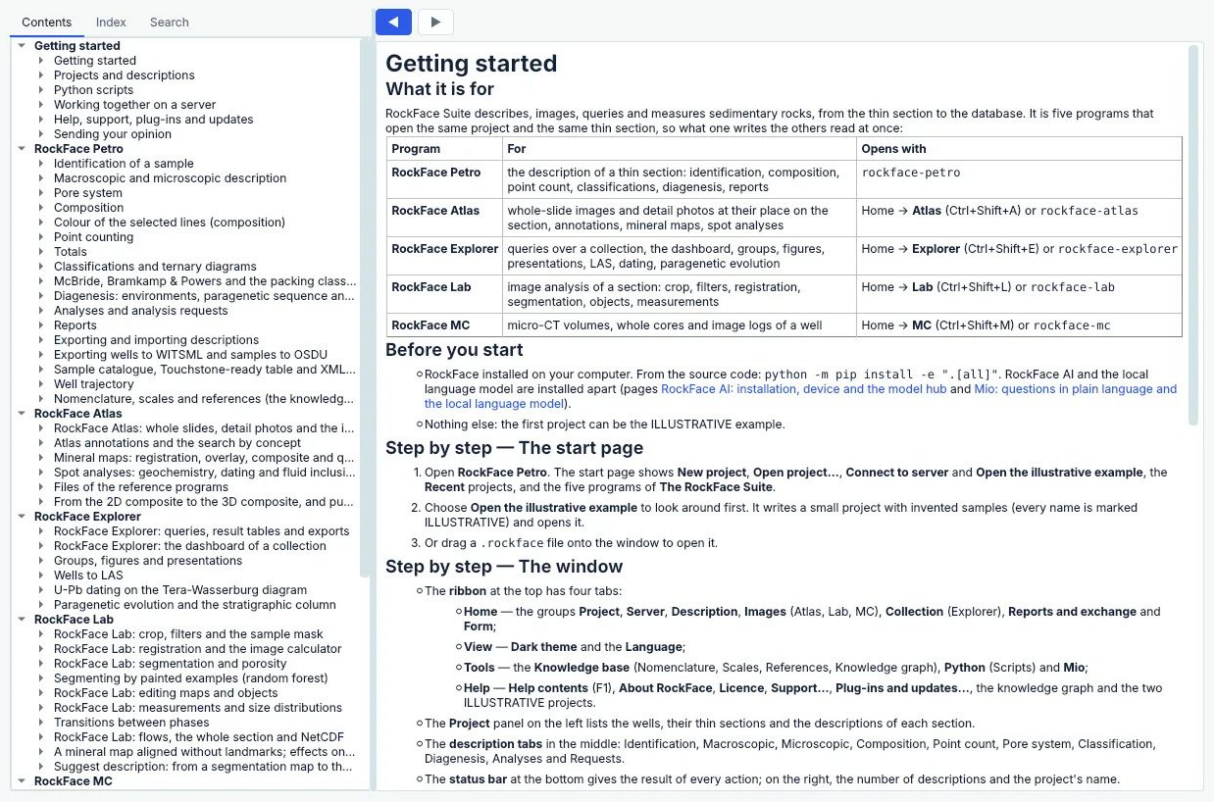
Coverage

P42 (server version), R4 (central storage), L13 (containers).

Help, support, plug-ins and updates

What it is for

Help contents (F1) opens this guide inside the program, with contents, index and search. **Support** prepares what to send when asking for help. **Plug-ins and updates** installs extensions written in Python and says when a newer RockFace exists.



In the program: Help, support, plug-ins and updates

Before you start

Nothing for the help. For plug-ins: a plug-in `.zip` (a folder with `plugin.json` and its code) or the address of a plug-in server.

Step by step

1. **Help → Help contents** (F1). *Contents*: pages and sections; *Index*: every title, with a filter; *Search*: words, then Enter — the best sections first.

2. **Help → Support...**: the versions of RockFace, Python, the libraries and the plug-ins, to copy or save; write to the support address your team set (preference `support/contact`).
3. **Help → Plug-ins and updates...**: *Install from a file*… shows the plug-in's name, author and the SHA-256 of the file before installing; *Install from a Git repository*… clones the repository (add `@branch` or `@tag` to the address for a version) and shows the commit before installing; tick *On* to enable it (at the next start); *Remove*. *Server*: blank for the RockFace catalogue or the address of another `index.json`; *List the server's plug-ins* and double-click one to install; *Check for a newer RockFace*.

Options

`plugin.json` field	Meaning
name, version, description, author, licence	shown in the list
requires	the oldest RockFace version it works with
module	the Python file with `register(app)`

How it works

- The guide is the same Markdown as the website, indexed when the program is built; the search ranks sections by BM25 over their words, as Mio does.
- Plug-ins live in `~/.rockface/plugins`; a plug-in starts disabled and is imported only after the user enables it; an archive with paths outside its folder is refused; a plug-in that fails at start is reported, not fatal.
- The server index lists the latest RockFace (version, installer, SHA-256, notes) and the plug-ins; downloads are kept only when their SHA-256 matches. RockFace never installs an update by itself.

From a script

```
rf.plugins.install_zip("tool.zip"); rf.plugins.enable("tool")
rf.plugins.install_git("https://example.org/team/tool.git", "v1.0")
rf.plugins.check_update("https://example.org/rockface/index.json")
```

Limits and common problems

- A plug-in runs with your rights: install only what you trust.
- The default catalogue is published with the installers (from 1.0); until then use a server of your team.

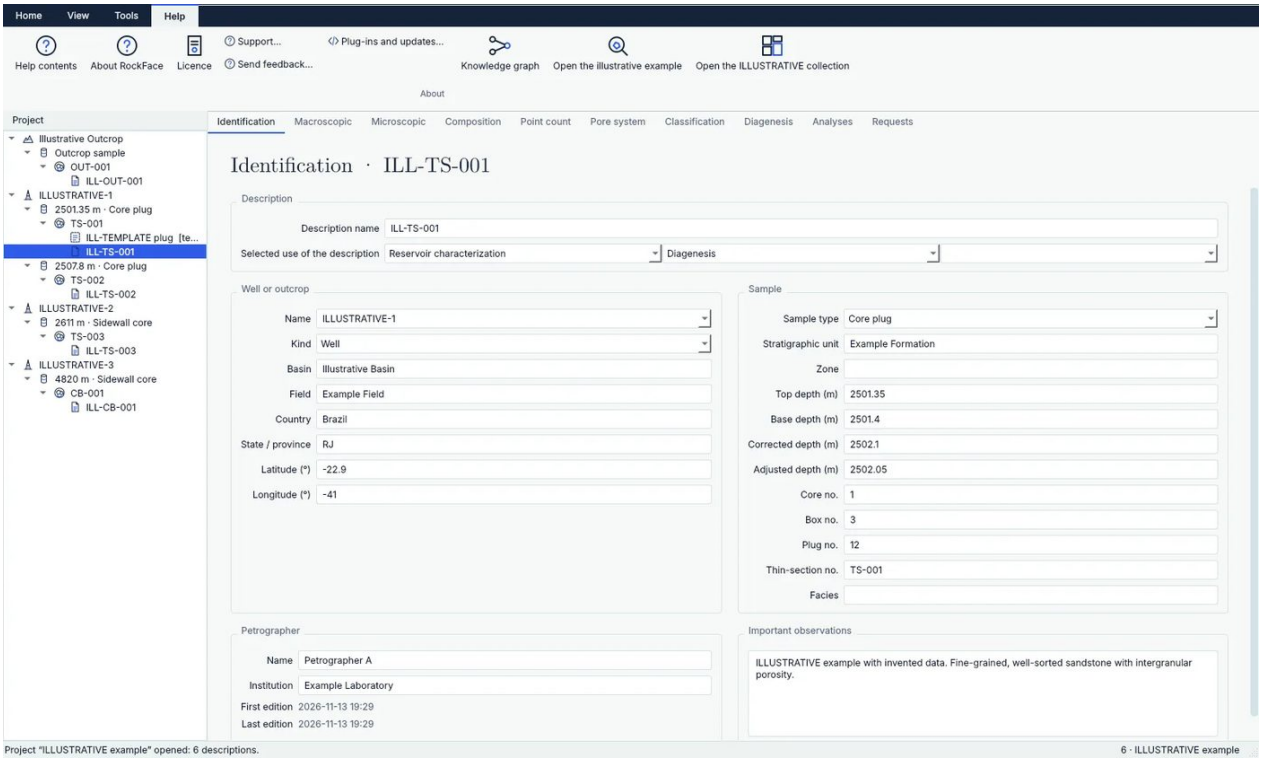
Coverage

P40 (help contents, support), P41 (plug-in manager and system updater); Counterledge C6.

Sending your opinion

What it is for

RockFace is tested by colleagues and students before it reaches every user. **Send feedback...** opens the feedback form of RockFace in your browser, where you say what works, what does not, and what is missing — for the program in general or for one function of this guide. The answers go to the RockFace team.



In the program: Sending your opinion

Before you start

- A web browser and a network connection.
- The form is a Microsoft Forms form of the RockFace team (University of Brasília). Until its address is set, the button says so (below).

Step by step

1. In RockFace Petro: **Help → Send feedback...**. In RockFace Atlas, Explorer, Lab and MC: the **Send feedback...** button of the toolbar.

2. Your browser opens the form. Say which program and which function you are writing about — the **Coverage** section at the end of each page of this guide gives the items of the function (for example *GT11*), which help the team find it.
3. Describe what you did, what you expected and what happened. Attach a screenshot if it helps; never attach confidential data.
4. Send the form.

When the form is not defined yet, RockFace shows "The feedback form is not defined yet." — write to your team's support address instead (**Help → Support...** prepares the versions to send).

Options

Where	What it changes
Preference `feedback/url`	The address of the form (set by the team when the form changes).
Environment variable `ROCKFACE_FEEDBACK_URL`	The same, for one computer or one session; it wins over the preference.

How it works

RockFace only opens the address of the form in the browser: nothing is sent by the program, and no file, project or personal data leaves the computer. The text of **Help → Support...** (versions only) may be pasted into the form.

From a script

```
print(rf.feedback_url())          # the address of the form, or "" when it is not defined yet
rf.send_feedback("lab")           # opens the form in the browser; returns the address, or "" when
↪  not defined
```

From a terminal: `rockface feedback` prints the address of the form (or that it is not defined yet) and opens it.

Limits and common problems

- "The feedback form is not defined yet." — the team has not set its address: use **Help → Support...**.
- Nothing opens: no browser is set as the default of the system; copy the address printed by `rockface feedback`.

Coverage

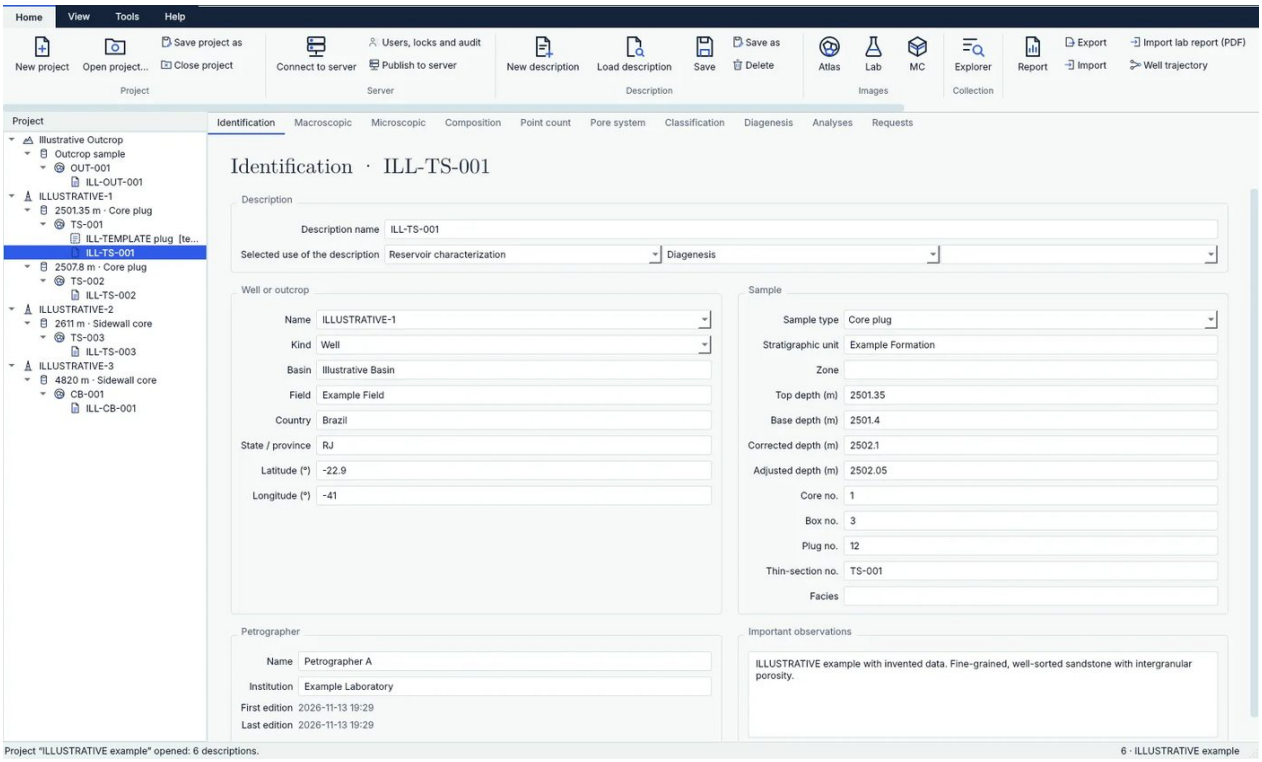
O2 (the testers' feedback form in the programs, after 1.0 and the site).

RockFace Petro

Identification of a sample

What it is for

The **Identification** tab says what the description is about: its name and uses, the well or outcrop, the sample and its depths, the thin section, the petrographer, and the important observations. Every query, report, figure and LAS file reads these fields.



In the program: Identification of a sample

Before you start

A description open or just created (page Projects and descriptions).

Step by step

1. Open the **Identification** tab (the first one).
2. **Description**
    - **Description name** — required and unique in the project. After the first save it is locked (use **Save as...** for another name).

- **Selected use of the description** — up to three uses (reservoir characterization, reservoir quality, diagenesis, provenance, depositional environment, petrophysical calibration, geochemistry and dating, teaching, other), without repeats.

3. **Well or outcrop**
   - **Name** — type a new one or choose one already in the project; choosing an existing well fills in its basin, field, country, state and position.
   - **Kind** (well or outcrop), **Basin**, **Field**, **Country**, **State / province**, **Latitude (°)**, **Longitude (°)**.
4. **Sample**
   - **Sample type** (core plug, whole core, sidewall core, cuttings, outcrop sample, other), **Stratigraphic unit**, **Zone**.
   - **Top depth (m)** and **Base depth (m)**; **Corrected depth (m)** (core-to-log correction) and **Adjusted depth (m)** (the final depth used in models).
   - **Core no.**, **Box no.**, **Plug no.**, **Thin-section no.** and **Facies** (the team's code or name).
5. **Petrographer** — **Name** and **Institution**. **First edition** and **Last edition** are filled in by RockFace.
6. **Important observations** — a free summary of what matters most in the sample.
7. **Save** (Ctrl+S).

Options

Field	Rule
Description name	Required; unique; locked after the first save.
Selected use	At most three, no repeats.
Depths	0 to 9999.99 m; the base cannot be shallower than the top.
Latitude, longitude	−90° to 90°; −180° to 180°.
Facies	Free text: the code or name the team uses (Explorer's dashboard groups by it).

How it works

Wells, samples and thin sections are shared records: two descriptions of the same thin section show the same well, depths and numbers, and changing them in one changes them for both. The values stored are language-neutral codes (for example the sample type), so the project reads the same in English, Spanish and Portuguese. The well's directional survey, when there is one, gives the true vertical depth and the position of each sample (page Well trajectory).

From a script

```
with rf.open("study.rockface") as prj:
    d = prj["TS-9"]
    print(d.identification.well_name, d.identification.top_depth)
    d.update(base_depth=2501.6, facies="F3", uses=["reservoir", "diagenesis"])   # checked as in
↪   the form, then saved
```

`d.update()` raises `rf.ScriptError` with the messages when a value breaks a rule, and nothing is written.

Limits and common problems

- “The description name is required.” / “… already exists in this project.”
- “Top depth (m) must be between 0 and 9999.99 m.”
- “The base depth (… m) is shallower than the top depth (… m).”
- “At most three uses can be selected.” / “A use of the description is selected twice.”
- Warnings (not errors): “No well or outcrop given.”, “The thin section has no number.” The description is saved, but queries by well or by thin section will not find it.

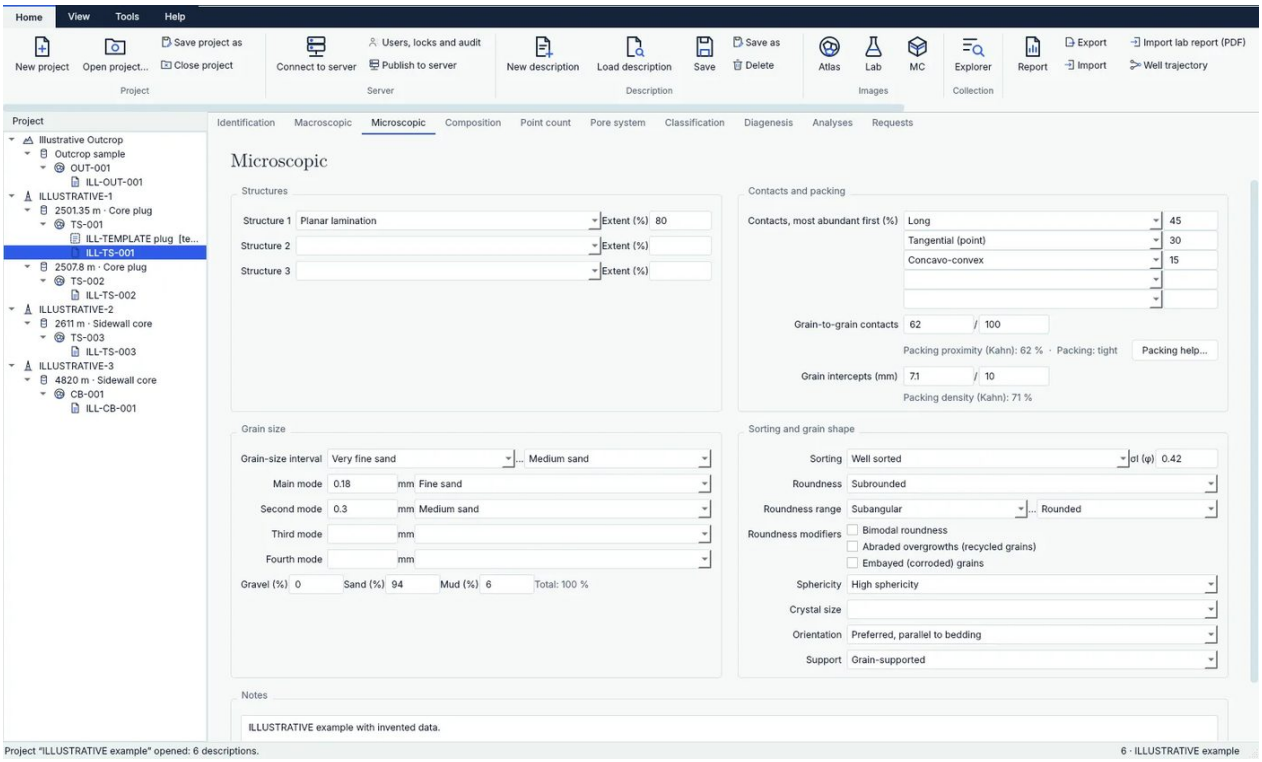
Coverage

P4 (identification fields), P5 (selected uses), P7 (important observations), P8 (position and depths).

Macroscopic and microscopic description

What it is for

The **Macroscopic** tab records what is seen in the hand sample or the core: structures, texture, colour and fabric. The **Microscopic** tab records what is seen in the thin section: structures, the grain-size interval and its modes, sorting, grain shape, fabric, the intergranular contacts and the packing. RockFace checks every value against its scale, so the text and the numbers of a description never disagree.



In the program: Macroscopic and microscopic description

Before you start

A description open (page Projects and descriptions). The lists come from the knowledge base (page Nomenclature, scales and references (the knowledge base)).

Step by step

Macroscopic

1. Open the **Macroscopic** tab.

2. **Structures** — up to three (**Structure 1–3**), each with its **Extent (%)** of the sample.
3. **Texture** — **Modal grain size** (Udden–Wentworth), **Crystal size** (Folk 1962, for crystalline rocks), **Sorting**, **Roundness** (Powers) and **Sphericity**.
4. **Colour** — the **Munsell code** (for example `5YR 6/4` or `N7`) and the **Colour name**.
5. **Fabric** — **Orientation** and **Support**.
6. **Notes**, then **Save** (Ctrl+S).

Microscopic

1. Open the **Microscopic** tab.
2. **Structures** — up to three, with their extent.
3. **Grain size**
   - **Grain-size interval**: the **Finest** and the **Coarsest** class.
   - **Modes**: the **Main mode** and up to three more (**Second**, **Third**, **Fourth mode**), each as a class or a size in **mm**. When a size is typed, its class is filled in.
   - **Gravel (%)**, **Sand (%)** and **Mud (%)**, with their total shown beside them.
4. **Sorting and grain shape** — **Sorting** as a class or as **σI (φ)** (Folk & Ward 1957); **Roundness**, the **Roundness range** and the **Roundness modifiers**; **Sphericity**; **Crystal size**; **Orientation** and **Support**.
5. **Contacts and packing**
   - **Contacts, most abundant first (%)** — up to five contact types (point, long, concavo-convex, sutured, floating) with their percentage.
   - For Kahn's (1956) packing: count, along a traverse, the **All contacts on the traverse** of the grains crossed and how many are **Grain-to-grain contacts**; RockFace shows the **Packing proximity (Kahn)** and its class (loose, normal, tight). Measure the **Grain intercepts (mm)** and the **Traverse length (mm)** for the **Packing density (Kahn)**.
   - **Packing help...** opens the visual help of the packing classes (page McBride, Bramkamp & Powers and the packing classes).
6. **Notes**, then **Save**.

Options

Field	Rule
Structures	At most three, no repeats; an extent needs a structure; 0–100 %.
Modes	At most four; the main mode first; each inside the grain-size interval.
Grain-size interval	The finest end first.
σI (φ), modal sizes	A number wins over the class: when they disagree, the class is updated, with a message.
Gravel + sand + mud	Should total 100 % (saved anyway, with a warning).
Contacts	At most five, no repeats, total at most 100 %.
Grain-to-grain contacts	Not more than all the contacts.

How it works

- Grain size follows the Udden–Wentworth scale (Wentworth 1922) with the φ of Krumbein (1934); sorting the inclusive graphic standard deviation of Folk & Ward (1957); roundness the classes of Powers (1953); crystal size Folk (1962).
- "Numbers win": a value in millimetres or φ is the measurement, and the class is derived from it.
- Packing proximity P = grain-to-grain contacts ÷ all contacts × 100; packing density = total grain intercept ÷ traverse length × 100 (Kahn 1956). Classes: loose below 40, normal 40–55, tight above 55.

From a script

```
d = prj["TS-9"]
d.micro.sorting_phi = 0.62              # Folk & Ward σI; the class follows on save
d.micro.grain_contacts, d.micro.total_contacts = 38, 80
print(d.micro.packing_proximity, d.micro.packing_class)       # 47.5 normal
d.macro.colour_code = "5Y 7/2"
d.save()                                # checked exactly as in the forms
```

Limits and common problems

- ""…" is not in Munsell notation (for example 5YR 6/4 or N7)."
- "Grain-size interval: the first end is coarser than the second."
- "Secondary modes were given without the main mode." / "… is outside the interval given."
- "Modal grain size: 0.3 corresponds to medium sand; the number wins and the class was updated." — a message, not an error.

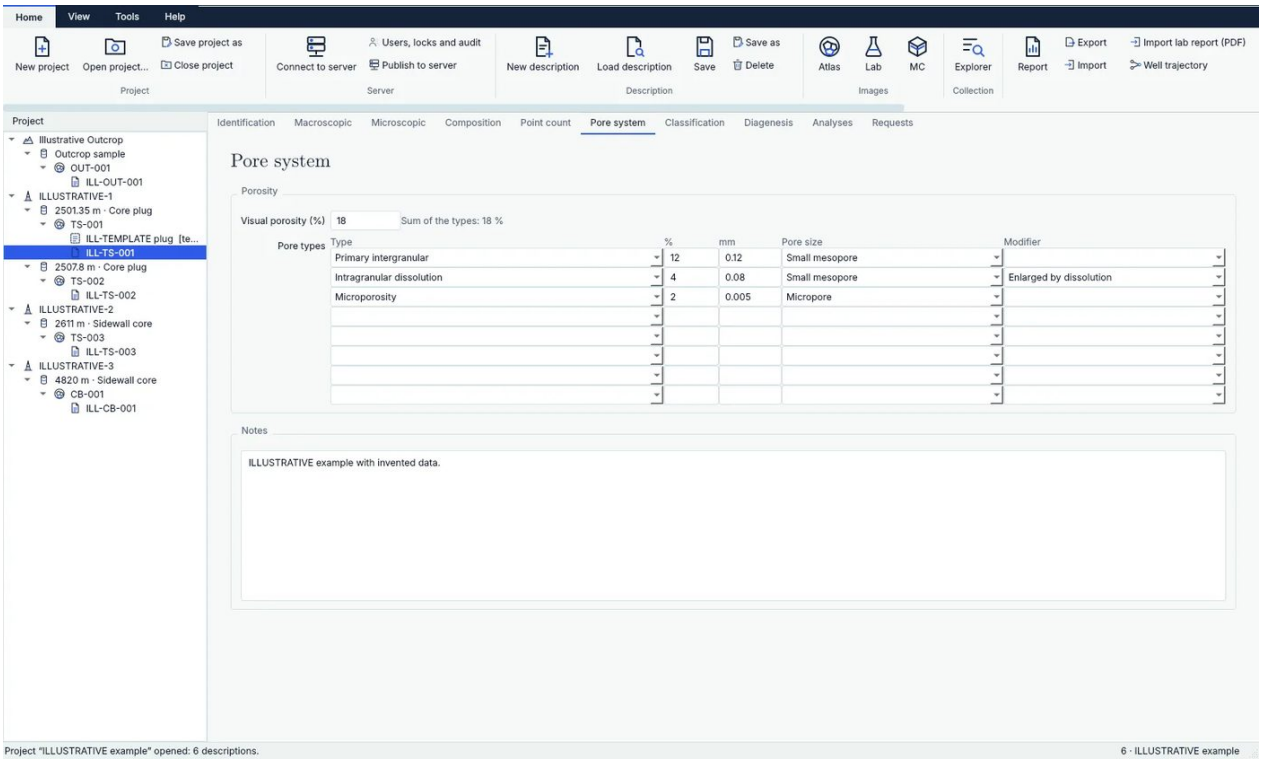
Coverage

P11 (macroscopic description), P12 (microscopic structures and grain size), P13 (sorting and shape), P14 (contacts and packing).

Pore system

What it is for

The **Pore system** tab records the visual porosity of the thin section and the pore types that make it up, each with its share, size and modifier. The pore types are those of Choquette & Pray (1970) for carbonates and of the sandstone pore classifications, from the knowledge base.



In the program: Pore system

Before you start

A description open. Blue-resin porosity measured on an image (RockFace Lab) or counted points (Point count tab) can be used as the visual porosity.

Step by step

1. Open the **Pore system** tab.
2. **Porosity** — type the **Visual porosity** (%). The **Sum of the types** is shown beside it.
3. **Pore types** — up to eight rows. For each:

- **Type** — the fabric-selective pores of Choquette & Pray (interparticle, intraparticle, intercrystalline, moldic, fenestral, shelter, growth-framework), the pores not fabric-selective (fracture, channel, vug, cavern) and those that may be either (breccia, boring, burrow, shrinkage), and the sandstone pores (primary and secondary intergranular, intragranular dissolution, oversized, microporosity);
- **%** — its share of the rock volume;
- **mm** — a typical size, or the **Pore size** class (the class follows a typed size);
- **Modifier** — a modifier of the knowledge base that applies to pores.

4. **Notes**, then **Save** (Ctrl+S).

Options

Field	Rule
Visual porosity	0–100 %.
Pore types	At most eight, no repeats; each 0–100 %.
Sum of the types	Compared with the visual porosity: more or less than it gives a warning.

How it works

The pore types and their modifiers are RockFace terms from the published literature (Choquette & Pray 1970 for carbonates; sandstone pore types after Schmidt & McDonald 1979 and Pettijohn et al. 1987). The pore-size classes are Choquette & Pray's micropores (< 1/16 mm), small and large mesopores (to 4 mm) and small and large megapores (to 256 mm). Pore types counted as points in the composition (kind *pore*) are a separate record: the Pore system is the petrographer's synthesis.

From a script

```
from rockface.core import texture as tx
d = prj["TS-9"]
d.pores.total_porosity = 18.0
d.pores.pores = [tx.Pore("interparticle", 12.0, size_mm=0.15), tx.Pore("moldic", 6.0)]
d.save()
```

Limits and common problems

- "The pore types add up to ... %, more than the visual porosity (... %)." — a warning: check the shares.
- "... must be between 0 and 100 %."
- An unknown pore type or modifier code (from a script or an old file) is refused with "Unknown value ...".

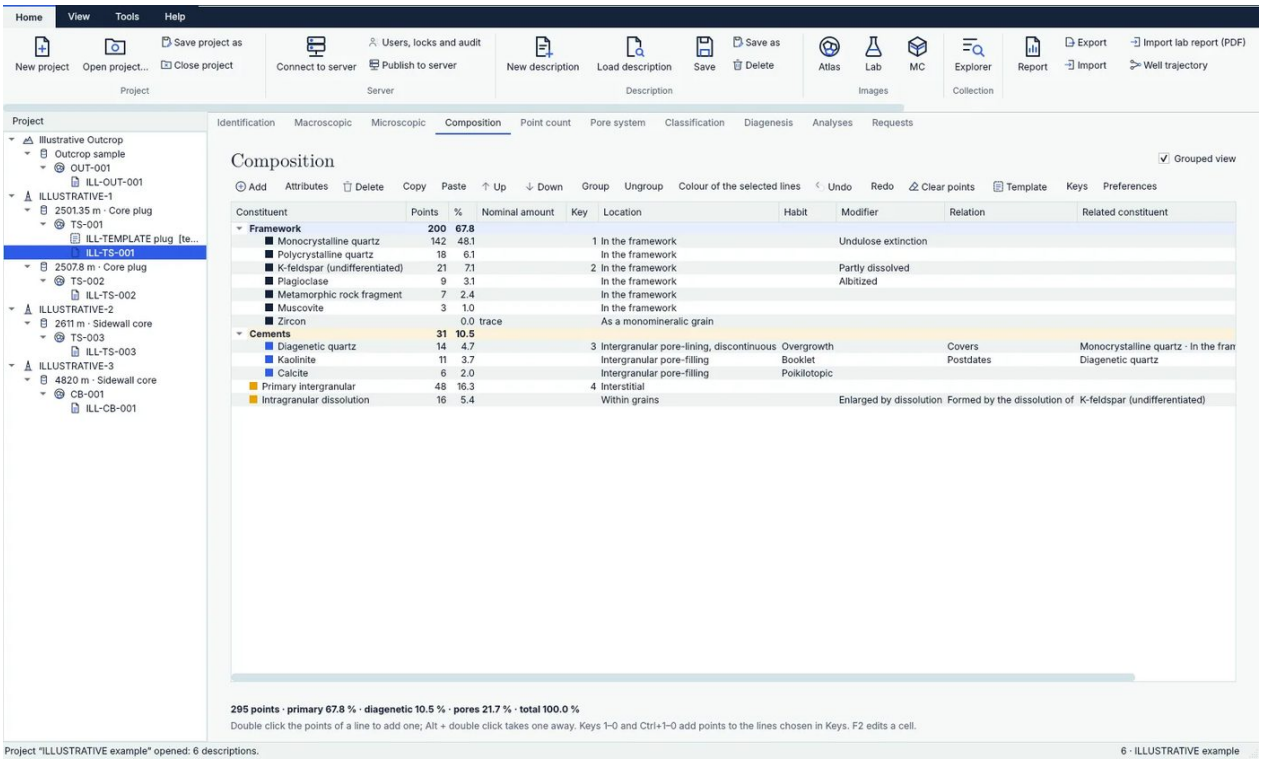
Coverage

P17 (pore-system description).

Composition

What it is for

The **Composition** tab is the table of what the rock is made of: primary constituents, diagenetic constituents and pore types, each with its attributes (location, habit, modifier, paragenetic relation) and its amount — counted points, a manual percentage or a visual estimate. Totals, classifications, the diagenetic interpretation, reports and queries are all worked out from it.



In the program: Composition

Before you start

A description open. The constituents and attributes come from the knowledge base (page Nomenclature, scales and references (the knowledge base)).

Step by step

Adding lines

1. Open the **Composition** tab.

2. **Add...** (Ins). In the **Composition line** dialog, choose the **Constituent** from the grouped lists (primary constituents, diagenetic constituents, pores); type in the filter to find it in any language.
3. Give the attributes its kind allows:
   - primary — **Location**, **Modifier**;
   - diagenetic — **Habit**, **Location**, **Modifier**, **Relation** (paragenetic), **Related constituent** and **Its location**;
   - pore — **Location**, **Modifier**, **Relation**, **Related constituent**.
4. Give the amount, one of:
   - **Points** — counted points;
   - **Manual %** — a percentage measured elsewhere;
   - **Nominal amount** — a visual estimate on the comparison-chart steps (trace, 1, 2, 3, 5, 7, 10, 15, 20, 25, 30, 40, 50 %).
5. **Observation** — a note about this line. **OK**.

The same constituent can appear on several lines with different attributes (for example calcite as a poikilotopic intergranular cement and as a replacement of feldspar).

Counting by clicks and keys

- A **double click** on the Points cell of a line adds a point; **Alt + double click** takes one away.
- **Keys...** gives lines to the keys 1–0 and Ctrl+1–0 (twenty keys): pressing a key adds a point to its line.
- The line under the table shows the points and the shares: "... points · primary ... % · diagenetic ... % · pores ... % · total ... %".
- **Clear points** sets every count to zero and keeps the lines.

For counting on the image of the section, with each point's position kept, use the **Point count** tab (page Point counting).

Editing the table

Button	Key	Does
Attributes...	Enter	Edits the selected line. F2 edits a single cell.
Delete	Del	Deletes the selected lines.
Copy, Paste	Ctrl+C, Ctrl+V	Copies lines, also into another description.
Up, Down	Ctrl+Up, Ctrl+Down	Moves the selected lines.
Group, Ungroup	Ctrl+G, Ctrl+U	Gathers lines under a named group, or undoes it; **Grouped view** shows the groups.
Colour of the selected lines...		A colour for the lines (also used in the point count and the reports).
Undo, Redo	Ctrl+Z, Ctrl+Y	Every change of the table.
Template...		Loads the composition of a template: replace the lines, or add the missing ones.

Button	Key	Does
Preferences…		**Double clicks per point** and **Remind me to save every** N points.

Save (Ctrl+S) when done; the reminder appears after the number of points set in the preferences.

Options

Option	What it changes
Double clicks per point	How many double clicks make one point (at least 1).
Remind me to save every	A reminder in the status bar after N points (0 = never).
Grouped view	Lines shown under their groups, or as one list.

How it works

- Lines with a manual % or a nominal amount keep that value (“fixed lines”); the counted lines share what is left, 100 − fixed, in proportion to their points. Without any counted point the percentages are the fixed values. A line with points and a % counts by its points (with a warning).
- The nominal steps follow the comparison charts of Terry & Chilingar (1955); a trace counts as 0 %.
- The attributes a constituent may take come from the knowledge base (“Applies to”), so a habit is never given to a primary grain.

From a script

```
d = prj["TS-9"]
d.add_line("quartz_mono", points=112, location="framework")
d.add_line("calcite_dia", points=24, habit="poikilotopic", location="ig_pore_filling")
d.add_line("kaolinite", nominal="3", habit="booklet", location="intragranular_replacive",
           relation="replaces", related="k_feldspar")
for line in d.lines():
    print(line["constituent"], round(line["share"], 2))      # quartz_mono 79.88, calcite_dia
    ↪  17.12, kaolinite 3.0
d.save()
```

Limits and common problems

- “… does not apply to …”: an attribute that the constituent’s kind does not take.
- “… appears twice with the same attributes (see line …)”: merge the two lines.
- “The manual and nominal percentages add up to … %, more than 100 %.”

- “…: no location given.” — a warning for diagenetic constituents.
- “A location of the related constituent was given without the constituent.”

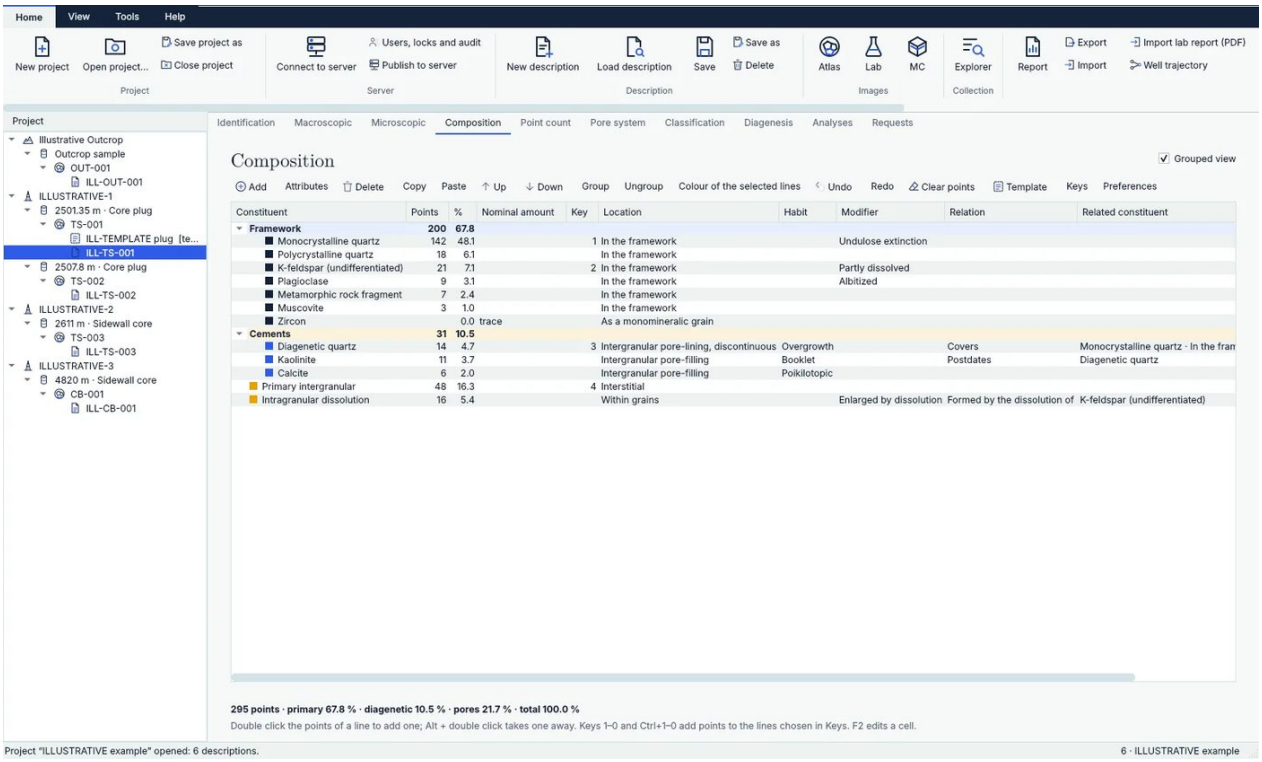
Coverage

P18 (composition table), P19 (attributes), P20 (points, manual %, nominal amount, observation), P21 (groups and colours), P22 (copy, paste, move, undo), P23 (keys), P24 (clear points, template, preferences, save reminder).

Colour of the selected lines (composition)

What it is for

Counterledge lets the petrographer colour lines of the composition table to find them at a glance while counting. In RockFace the selected lines take a colour through their group.



In the program: Colour of the selected lines (composition)

Step by step

1. **Petro → Composition**: select one or more lines.
2. **Colour of the selected lines...** and choose the colour.

How it works

The lines share a group (the one they already share, or a new one) and the group's colour paints them; ungrouping removes it. The colour is saved with the composition.

From a script

```
d = prj["ILL-TS-001"]; d.composition.groups   # {"g1": {"name": …, "colour": "#123456"}}
```

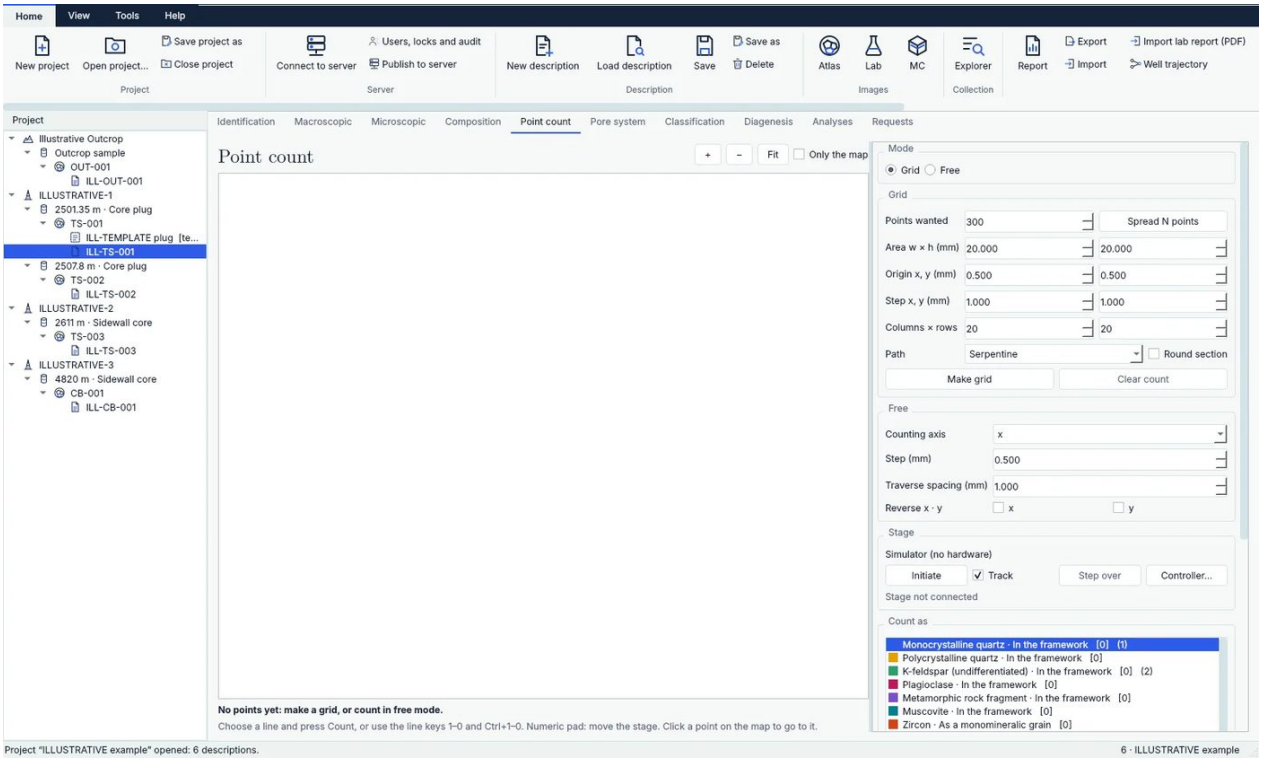
Coverage

Counterledge C2 (Configure Selected Row Color).

Point counting

What it is for

The **Point count** tab counts the composition on a grid of points laid over the thin section — for example 300 points — on the section image or with a motorized stage. Every point keeps its position in millimetres of the section frame, so a recount lands on the same points, any counted point can be visited again, and the map of the counted points shows the composition of the section. The counts fill the Points column of the composition.



In the program: Point counting

Before you start

- A description with its composition lines (page Composition): you count *as* one of the lines.
- Saved at least once: from then on every counted point is written to the project at once, so no point is ever lost ("Save the description to keep the count…").
- Optional: an image of the section (**Section image...**), or the whole-slide image of RockFace Atlas.

Step by step

The grid

1. Open the **Point count** tab. Leave **Mode** on **Grid**.
2. Either type **Points wanted** (e.g. 300) and the **Area w × h (mm)** and press **Spread N points**, or give the grid yourself: **Origin x, y (mm)**, **Step x, y (mm)** and **Columns × rows**.
3. **Path**: **Serpentine** (back and forth) or **Row by row**. Tick **Round section** to keep only the points inside a round section.
4. **Make grid**. The map on the left shows the points.

The section image

1. **Section image...** loads a picture of the section; under **Image and calibration** give its **µm per pixel** and the **Image origin x, y (px)** (the pixel at 0, 0 mm).
2. The points are drawn over the image. Without an image the points are drawn on their own.

Counting

1. **Initiate** goes to the first point. With **Track** on, each count moves to the next point not yet counted.
2. Choose the line in **Count as** and press **Count**, or press the line's key (1–0 and Ctrl+1–0, set in the Composition tab's **Keys...**).
3. **Step over** skips a point (on resin out of the section, a hole…); **Uncount** takes back the count of the current point; **Back** returns to the previous point.
4. **Observation at the point** and **Pictures at the point...** keep a note and photos for the point under the cross-hairs.
5. The counter shows "N of M points counted"; the composition's points follow at once.

Moving around

- Click a point on the map to go to it; **Go to point...** by its number.
- **Fit**, **Zoom in**, **Zoom out**, the mouse wheel; **Only the map** hides the panel.
- The path followed, the current point and the stage position are drawn on the map, the counted points coloured by their line.

With a motorized stage (free mode)

1. Choose the **Stage** (**Simulator (no hardware)** for practice) and press **Controller...** to move it with the numeric pad: 8, 2, 4, 6 and the diagonals 7, 9, 1, 3.
2. **Mode → Free**: give the **Counting axis**, the **Step (mm)**, the **Traverse spacing (mm)** and **Reverse x · y** for a stage mounted the other way. Each count moves the stage one step; at the end of the area it moves to the next traverse.

Coordinates

- **Coordinates...** lists the points counted as each constituent, with x and y in mm; **Export CSV...** writes them.
- **Import points...** reads points from a CSV file (x, y and the line label), in mm or in pixels with their size.
- **Clear count** forgets every counted point and keeps the grid.

Options

Field	What it changes
Points wanted, Area	Spread N points: square steps over the area, exactly N points.
Path	Serpentine or row by row.
Round section	Only the points inside the ellipse inscribed in the grid area.
Track	Move to the next point not counted after each count.
Counting axis, Step, Traverse spacing	Free mode with a stage.

How it works

The same grid parameters always give the same points (request L10: positions never lost, reproducible). A grid cannot be changed once points are counted ("... points are already counted: the grid stays"); clear the count first. The composition lines the count uses get exactly the number of points counted as them; other lines keep their own points. The counted points also appear in RockFace Atlas over the whole-slide image and in the figures of the section.

From a script

```
d = prj["TS-9"]
q = d.add_line("quartz_mono")
c = d.add_line("calcite_dia", habit="poikilotopic", location="ig_pore_filling")
d.count.set_grid(rf.ct.grid_for(300, 20.0, 30.0))       # 300 points on a 20 × 30 mm area
d.count.assign(q.uid)                                    # the current point counted as quartz;
↪  moves on
d.count.assign(c.uid)
rf.ct.sync_points(d.composition, d.count)               # the composition's points follow the count
d.save()
print(d.count.counted, d.count.coordinates_by_line())
```

Limits and common problems

- "No points yet: make a grid, or count in free mode."
- "Every point is counted or skipped." — the count is complete.
- RockFace ships the stage simulator; a motorized stage of a given make needs its driver, written on the `Stage` interface of `rockface.core.counting` (for example as a plug-in).

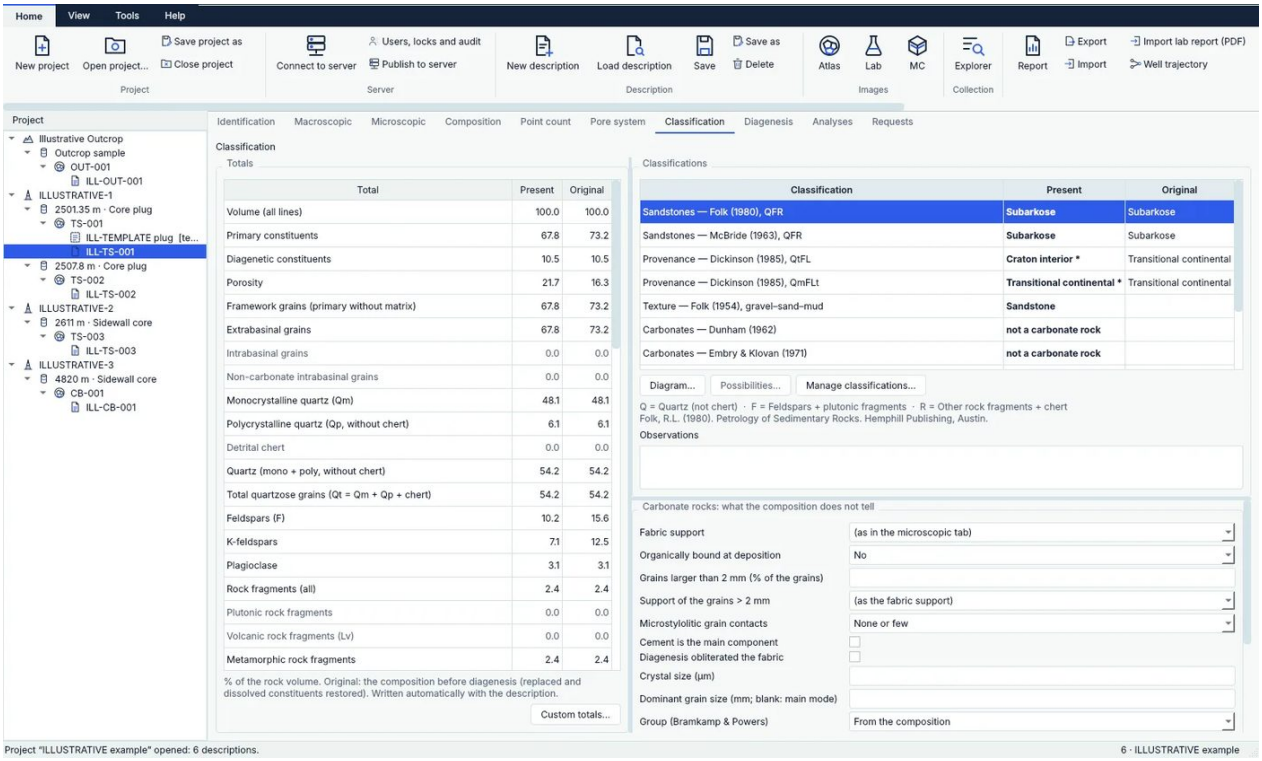
Coverage

P25 (stage buttons: Initiate, Track, Step over, Controller), S1 (free mode), S2 (movement log, return to points, go to a point, zoom, colours per constituent), S3 (map of the counted points, percentages, return to any point), L10 (a grid of points with coordinates on each thin section).

Totals

What it is for

A **total** adds up the composition lines of one kind: all the quartz, the feldspars, the intergranular cement, the dissolution porosity, the bioclasts… RockFace has 50 built-in totals and the compaction measures IGV, COPL, CEPL and ICOMPACT, worked out on the present composition and on the **original** one (before diagenesis). A project can add its own totals. The classifications, figures, queries and LAS curves use them.



In the program: Totals

Before you start

A description with its composition (page Composition).

Step by step

Reading the totals

1. Open the **Classification** tab. The **Totals** table is on the left.

2. Each row gives a total in % of the rock volume, in two columns:
   - **Present** — from the composition as described;
   - **Original** — from the composition before diagenesis: the share of each replacive or dissolution line is given back to the constituent it replaced or dissolved.
3. The totals are written automatically with the description: no button to press.

A custom total

1. **Custom totals...** opens **Custom totals of the project**.
2. **New**, then give the total a **Code** and a name.
3. Add **Conditions**: a line is picked when it matches every column of one condition — kind, group, constituent, location, habit, modifier, relation. Double-click a cell to choose codes: checked means *include*, partly checked means *exclude*; an empty cell (*any*) accepts anything. **Add condition**, **Remove condition**.
4. **In this description: ... %** shows the result at once.
5. **OK**. The total appears, marked *custom*, in every description of the project.

Options

The built-in totals include:

Group	Totals
Whole rock	volume, primary, diagenetic, porosity, framework grains
Origin	extrabasinal, intrabasinal, non-carbonate intrabasinal (Zuffa 1980)
Provenance	Qm, Qp, chert, quartz, Qt, feldspars, K-feldspars, plagioclase, rock fragments by kind, L, Lt (Dickinson 1985)
Others	micas, heavy minerals, matrix, depositional matrix
Carbonates	non-skeletal grains, bioclasts, pre-salt constituents, allochems, lime mud, dolomite, carbonates, evaporites
Diagenesis	cements (all, intergranular, silica, carbonate, clay, sulphate), diagenetic feldspar, replacive, displacive
Pores	intergranular, intragranular and moldic, dissolution, fracture, microporosity
Compaction	IGV, COPL, CEPL, ICOMPACT

How it works

- A total sums the percentages of the lines picked by any of its conditions.
- IGV = intergranular porosity + intergranular cement + depositional matrix (Paxton et al. 2002).
- COPL = Pi − ((100 − Pi) × IGV) / (100 − IGV) and CEPL = (Pi − COPL) × C / IGV, with Pi = 40 % (Lundegard 1992); ICOMPACT = COPL / (COPL + CEPL) (Ehrenberg 1995).
- The list of every total, with its conditions and sources, is in `docs/NOMENCLATURE.md` of the source code.

From a script

```
d = prj["TS-9"]
t = d.totals()                          # every total in % of the rock
print(t["qt"], t["intergranular_cement"], t["igv"], d.totals(original=True)["feldspar"])
prj.add_total("kaolinite_all", "Kaolinite (all)", rf.Selector(constituents=["kaolinite"]))
rows = prj.table(totals=["volume", "igv", "kaolinite_all"])
prj.to_csv("totals.csv", rows)
```

Limits and common problems

- A custom total with a code already used is refused.
- IGV, COPL and CEPL were defined for sandstones; RockFace works them out for any rock, so read them with care in carbonates.
- The original composition is only as good as the paragenetic relations given in the composition (replaces, dissolution of…).

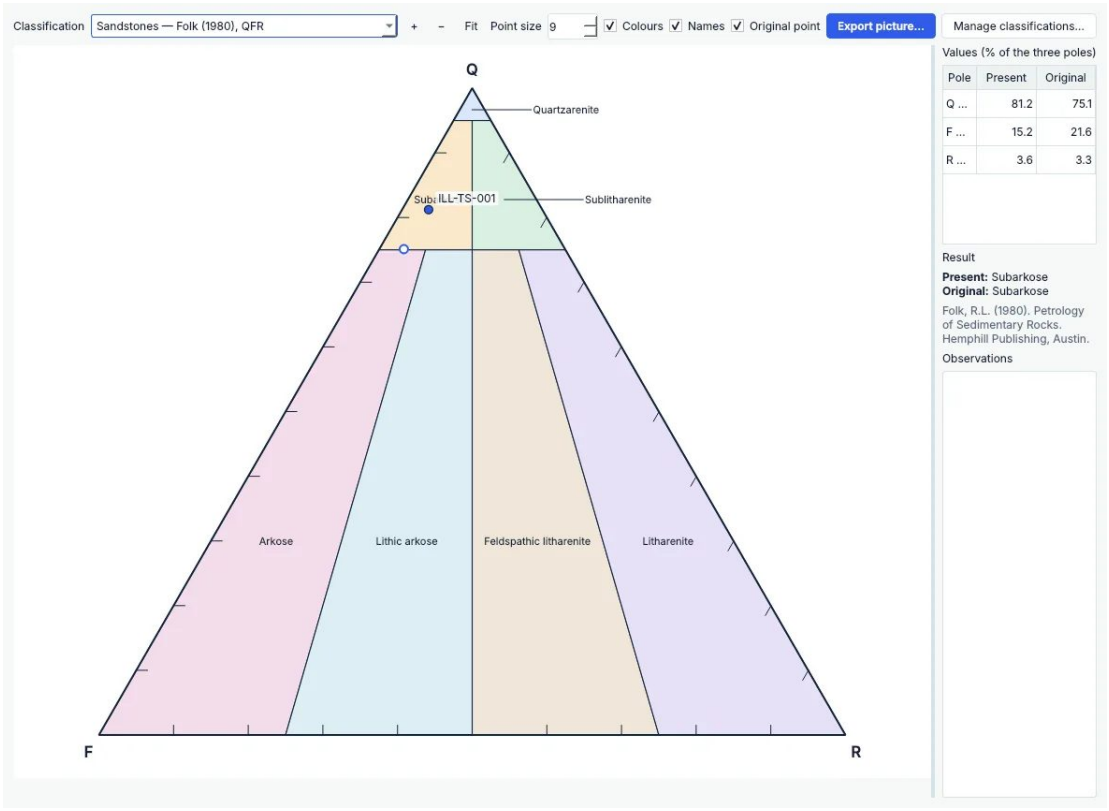
Coverage

P26 (total table, custom totals, saved automatically), P27 (more than 30 automatic totals), P28 (custom totals editor).

Classifications and ternary diagrams

What it is for

The **Classification** tab names the rock: sandstones by Folk (1980) and McBride (1963), provenance by Dickinson (QtFL and QmFLt), texture by Folk's gravel–sand–mud classes, carbonates by Dunham, Embry & Klovan, Wright, Grabau and Bramkamp & Powers. Each classification is worked out from the composition, on the present and on the original composition, and the ternary ones are drawn in a diagram window. A project can add its own ternary classifications.



In the program: Classifications and ternary diagrams

Before you start

- A description with its composition (and, for the texture, the gravel, sand and mud % of the Microscopic tab).
- For carbonates: the carbonate inputs that the composition cannot give (below).

Step by step

Reading the classifications

1. Open the **Classification** tab. **Classifications** lists every classification with its **Present** and **Original** result.
2. "no data" means the composition does not have what the classification needs; "outside the fields" that the point falls outside every field; "Near the boundary with: …" that it lies within the tolerance of another field.
3. **Observations** — a note on the classification.

The ternary diagram

1. Select a ternary classification and press **Diagram…**.
2. The **Ternary diagram** window shows the fields, the point of the description and, with **Original point**, the point of the original composition.
3. **Values (% of the three poles)** gives each **Pole** and the **Result**; **Under the cursor** names the field under the mouse.
4. **Fit**, **Zoom in**, **Zoom out**; **Point size**; **Colours** (fields coloured or not); **Names** (field names on or off).
5. **Export picture…** saves the diagram as PNG or SVG.

Carbonate rocks

1. When carbonates are more than half of the solids, fill in **Carbonate rocks: what the composition does not tell**:
   - **Fabric support** (as in the Microscopic tab, or set here); **Organically bound at deposition** (no; yes, mode not known; baffling; encrusting and binding; rigid framework);
   - **Grains larger than 2 mm (% of the grains)** and their **Support**;
   - **Microstylolitic grain contacts**, **Cement is the main component**, **Diagenesis obliterated the fabric**;
   - **Crystal size (µm)** and **Dominant grain size (mm)**;
   - the Bramkamp & Powers inputs (page McBride, Bramkamp & Powers and the packing classes).
2. **Possibilities…** lists every class of the selected classification with its criteria, the one found marked. **Missing: …** names the inputs still needed.

A classification of the project

1. **Manage classifications…** (in the tab or in the diagram window) opens **Ternary classifications of the project**.
2. **New**; give the **Code** and the name.
3. **Poles**: three poles, each with a **Symbol** and the **Totals added up** (codes of totals separated by commas).
4. **Fields**: **Add field**, its name and its **Vertices** — three percentages per vertex in the order of the poles, vertices separated by semicolons (for example `100 0 0; 50 50 0; 50 0 50`).
5. **Apply**. The classification appears in every description of the project, in Explorer's groups and in the figures.

Options

Classification	Inputs
Folk (1980), McBride (1963) — QFR	quartz, feldspars and rock fragments of the composition
Dickinson QtFL, QmFLt	Gazzi–Dickinson counting assumed; plutonic fragments are not plotted
Folk (1954) gravel–sand–mud	gravel, sand and mud % of the Microscopic tab
Dunham (1962), Embry & Klovan (1971)	support, grains, mud, organic binding, grains > 2 mm
Wright (1992)	also cement, microstylolites, obliteration, crystal size
Grabau (1904)	dominant grain size of the carbonate grains
Bramkamp & Powers (1958)	group, texture, porosity, alteration, dolomite

How it works

- A ternary point is the three poles' totals normalised to 100; the field is found by point-in-polygon on the published boundaries (Folk 1980; McBride 1963; Dickinson et al. 1983 and Dickinson 1985).
- The QmFLt recycled-orogen subfields come from one digitization of the published figure and are marked "Boundaries to be checked against the original figure."
- The carbonate classes follow the criteria of their papers, shown in **Possibilities...**. They apply when carbonates are above 50 % of the solids ("Carbonates are ... % of the solids; the carbonate classifications apply above 50 %").

From a script

```
d = prj["TS-9"]
print(d.classify("folk1980", label=True), d.point("folk1980"))
print(d.classify("dunham1962"), d.classify("folk1980", original=True))
d.carbonate.support = "grain_supported"              # carbonate inputs, then save
d.save()
rf.ternary_figure("qtfl.svg", "dickinson_qtfl", [d], original=False)   # several descriptions may
↪  be given
```

Limits and common problems

- "no data": the composition lacks the poles (e.g. no feldspar and no rock fragments is still a point; no framework grain at all is not).
- Custom fields with vertices that do not add up to 100: "Vertices not understood".
- A classification of the project with a code already used is refused.

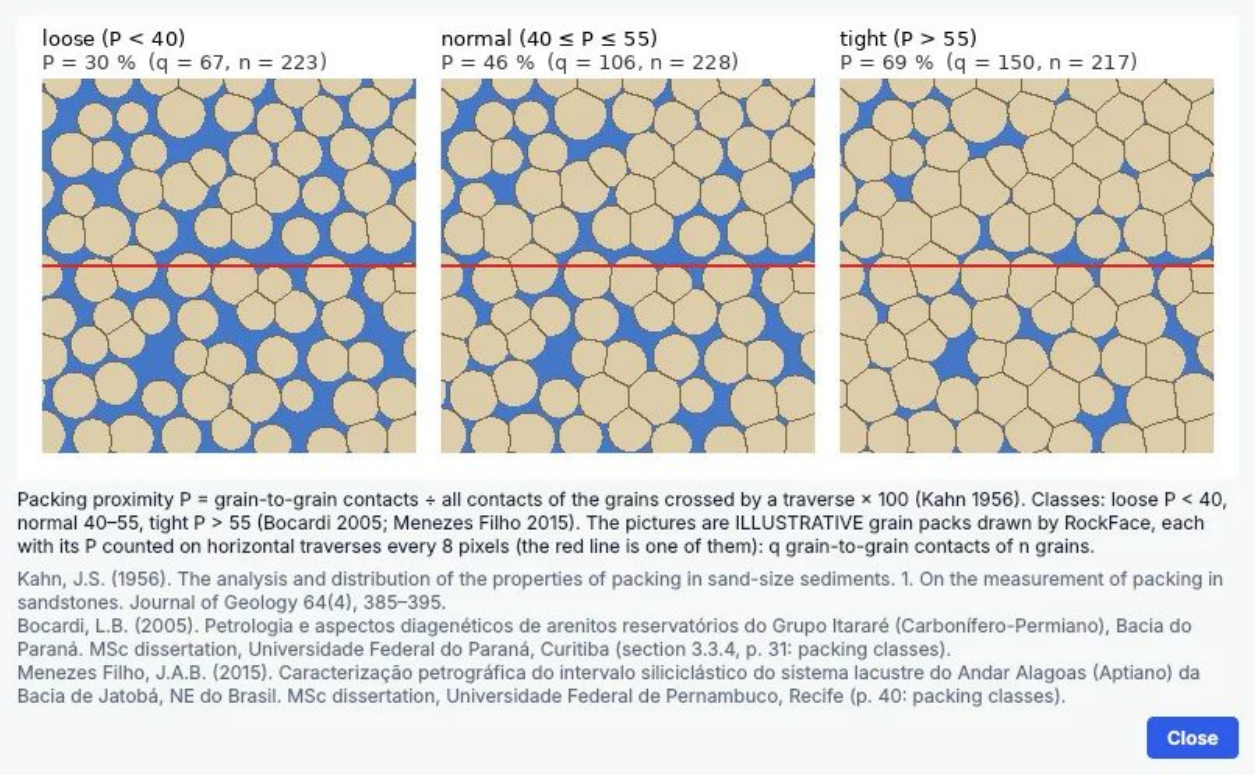
Coverage

P29 (siliciclastic classifications, present and original), P30 (Dickinson provenance), P31 (ternary diagram window), P32 (carbonate classifications and the table of possibilities), M3 (classifications recomputed from the latest data).

McBride, Bramkamp & Powers and the packing classes

What it is for

Three classifications that the reference programs offer and RockFace now has, each from its published source: **McBride (1963)** for sandstones, **Bramkamp & Powers (1958)** for carbonates — with its reservoir-rock symbol (for example *21BD*) — and the **packing classes** loose, normal and tight of the packing proximity of Kahn (1956).



In the program: McBride, Bramkamp & Powers and the packing classes

Before you start

- A description with its composition (Petro → Composition).
- For Bramkamp & Powers: the carbonate inputs that the composition cannot give (below).
- For the packing class: the grain contacts counted on a traverse, or the contact types in %.

Step by step

1. **Petro → Classification tab.** McBride appears beside Folk (1980) with its present and original field; **Diagram...** draws the triangle.
2. For a carbonate rock, fill in the **carbonate inputs** at the bottom right:
   - *Original texture* (*Bramkamp & Powers*): not visibly altered, moderately altered, strongly altered (still recognizable) or obliterated;
   - *Visual porosity*: A porous or poorly cemented, B compact, C dense;
   - *Altered by*: from the dolomite (default), anhydrite (A), silica (S) or changes not understood (X);
   - *Facies*, *Group* and *Dolomite %* come from the composition and can be overridden;
   - *Special rock type*: 14AX/14BX, 31BX or 51BX when the voids replace grains.
3. The row *Carbonates — Bramkamp & Powers (1958)* shows the name, the facies and the symbol; **Possibilities...** lists the 28 classes with their criteria.
4. **Petro → Microscopic tab → Contacts and packing.** Type the grain-to-grain and the total contacts of a traverse, or the contact types with their %: the proximity and the class appear at once. **Packing help...** shows three ILLUSTRATIVE packs, one per class, each with its proximity counted.

Options

Field	Values
McBride poles	Q = quartz, quartzite and chert; F = feldspars; R = other rock fragments (plutonic and carbonate fragments included)
Bramkamp & Powers group	1 fine-grained limestone, 2 calcarenitic limestone, 3 calcarenite, 5 coarse clastic carbonate; 4 crystalline dolomite (obliterated)
Symbol	group · degree (1 moderate, 2 strong, 3 relic) · porosity (A, B, C) · alteration (R, D, A, S, X)
Packing classes	loose P < 40 %, normal 40–55 % (both limits included), tight P > 55 %

How it works

- **McBride (1963)** places the point by the boundaries of Folk (1954): quartzarenite above 95 % Q; the lines F = 25 % and R = 25 % from the 75 % points of the sides, and F = 10 %, R = 10 % from the 10 % points of the base; lithic subarkose where both F and R are 10–25 %; F:R = 1:1 splits the rest (McBride 1963, fig. 1 and p. 666–667).
- **Bramkamp & Powers (1958)**: group 1 when the sand-size or coarser clastic carbonate fragments are below 10 %; group 2 above it with at least 10 % of the original fine matrix; groups 3 and 5 below 10 % matrix, split by the fragments coarser than sand. The alteration columns of their Table 2: < 10 % dolomite recrystallized, > 10 % porphyroblastic dolomite (moderate), 10–75 % dolomitic and > 75 % relic (strong); obliterated is group 4. The facies: impure with clay and silt, sandy with detrital grains (from 1 % of the solids). Percentages are of the solids.

- **Packing**: Kahn's packing proximity P = grain-to-grain contacts ÷ all contacts of the grains crossed × 100 (Kahn 1956); without counts, the same ratio from the contact types (every type but "floating"). The class limits are those of Bocardi (2005, p. 31) and Menezes Filho (2015, p. 40). The help pictures are computed by RockFace: a pack of round grains whose radii are scaled until the proximity, counted on horizontal traverses as Kahn describes, falls in the class.

From a script

```
d = prj["ILL-TS-001"]
d.classify("mcbride1963", label=True)                # 'Subarkose'
c = prj["ILL-CB-001"]
c.carbonate.bp_texture, c.carbonate.bp_porosity = "moderate", "B"
c.save()
c.classify("bramkamp_powers1958", label=True), c.symbol()   # (…, '21BD')
d.packing()                                          # {'proximity': 62.0, 'density': …, 'class':
↪  'tight'}
rf.packing_class(47)                                 # 'normal'
```

Limits and common problems

- McBride does not say where coarse plutonic fragments go: RockFace counts them as rock fragments (Folk 1980 counts them as feldspar), and says so beside the diagram.
- The visual porosity A/B/C of Bramkamp & Powers has no numbers in the paper: choose it by their criteria (the tooltip summarizes them). Until it is chosen the symbol shows "?".
- Bramkamp & Powers applies when carbonates are more than half of the solids.

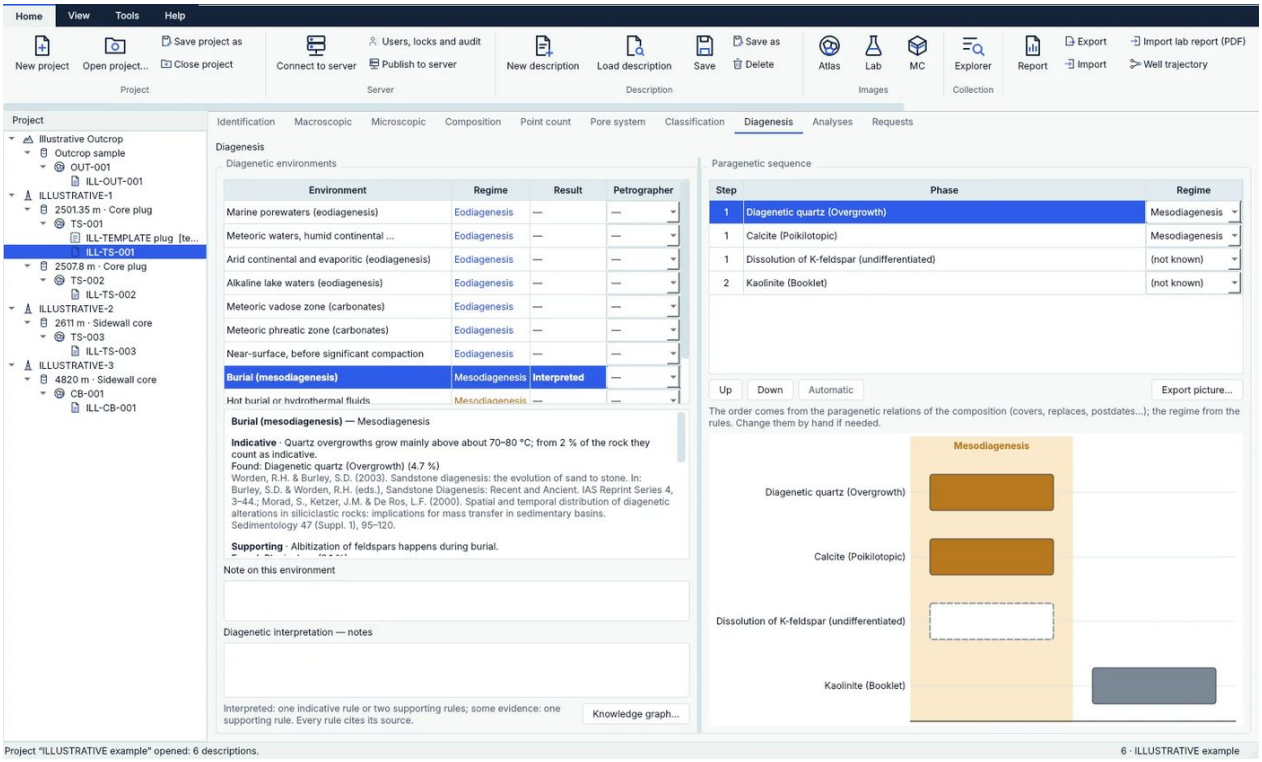
Coverage

P14 (packing classes and visual help), P29 and M3 (McBride), P32 (Bramkamp & Powers).

Diagenesis: environments, paragenetic sequence and knowledge graph

What it is for

The **Diagenesis** tab interprets the diagenetic environments the rock went through — marine, meteoric, arid, alkaline lacustrine, burial, hot fluids, uplift — from the features of its description, each conclusion with the rules and sources behind it. It also builds the **paragenetic sequence**: the diagenetic phases in the order they formed, with their regime (eo-, meso- or telodiagenesis). The petrographer confirms or rejects each environment, and can reorder the sequence by hand.



In the program: Diagenesis: environments, paragenetic sequence and knowledge graph

Before you start

A description whose composition has diagenetic constituents, ideally with their habits, locations and paragenetic relations (covers, replaces, postdates...) (page Composition).

Step by step

The environments

1. Open the **Diagenesis** tab.
2. **Diagenetic environments** lists every **Environment** with its **Regime** and the **Result**:
   - *interpreted* — one indicative rule or two different supporting rules were found;
   - *some evidence* — one supporting rule;
   - empty — "No evidence of this environment in the description."
3. Select an environment to see the rules **Found**: the feature of the description, the rule and its source.
4. In **Petrographer**, **confirm** or **reject** it; your decision wins over the automatic result. Write a **Note on this environment** if needed.
5. **Diagenetic interpretation — notes**: the overall interpretation in your words.

The paragenetic sequence

1. **Paragenetic sequence** shows the phases in order, each with its **Step**, **Phase** (precipitation or dissolution of a constituent) and **Regime**, and the chart of the sequence.
2. The order comes from the paragenetic relations of the composition; the regime from the rules. Contradictions are reported: "Contradictory relations between … and …: left out of the order", "… comes before … but belongs to a later regime".
3. **Up** and **Down** move a phase; the regime can be changed in its cell. **Automatic** brings back the order worked out by RockFace.
4. **Export picture...** saves the chart.
5. **Save** (Ctrl+S): the interpretation is part of the description.

The knowledge graph

1. **Knowledge graph...** (in the tab, or Tools → Knowledge base → **Knowledge graph**).
2. The graph links the features of a description → the rules → the environments → the regimes. Solid lines are indicative rules, dashed lines supporting rules; the rules found in the open description are in blue.
3. In **Environment**, choose one to see only its part of the graph, or **(all environments)**. **Export picture...** saves it.

Options

Field	What it changes
Petrographer	Confirmed or rejected: replaces the automatic result in reports, queries and figures.
Up, Down, regime cell	The sequence set by hand (kept with the description).
Automatic	Back to the order and regimes found by RockFace.

How it works

- The rules are RockFace's own, each from the published literature: Morad et al. (2000, 2010), Worden & Burley (2003), Longman (1980), Flügel (2010), Scholle & Ulmer-Scholle (2003), Radke & Mathis (1980), Tosca & Wright (2018), Paxton et al. (2002), Schmidt & McDonald (1979), Taylor (1950), Park & Schot (1968).
- An environment is *interpreted* with one indicative rule or two different supporting rules, and has *some evidence* with one supporting rule. The aggregation is stated in the program and in `docs/NOMENCLATURE.md`; the petrographer's decision always wins.
- The order of the phases is a topological sort of the relations; relations that contradict each other are left out and reported.
- RockFace Explorer draws the paragenetic evolution of several samples together (page Paragenetic evolution and the stratigraphic column).

From a script

```
d = prj["TS-9"]
for code, r in d.interpret().items():
    print(code, r["status"], r["regime"], r["rules"])          # e.g. meso_burial interpreted meso
    ↪ ['albitization']
d.diagenesis.decisions["eo_marine"] = "confirmed"
d.save()
for step in d.paragenesis():
    print(step["step"], step["phase"], step["kind"], step["regime"])
rf.paragenesis_figure("sequence.png", d)
rf.knowledge_graph("graph.svg", environment="meso_burial")      # also .png, or .dot for the whole
↪ graph
```

Limits and common problems

- "No diagenetic phases in the composition." — add diagenetic constituents first.
- Without paragenetic relations, the phases have no order: give the relations in the composition (covers, replaces, postdates, fills dissolution of...).
- The DOT file of the knowledge graph (for Graphviz) is written for the whole graph only.

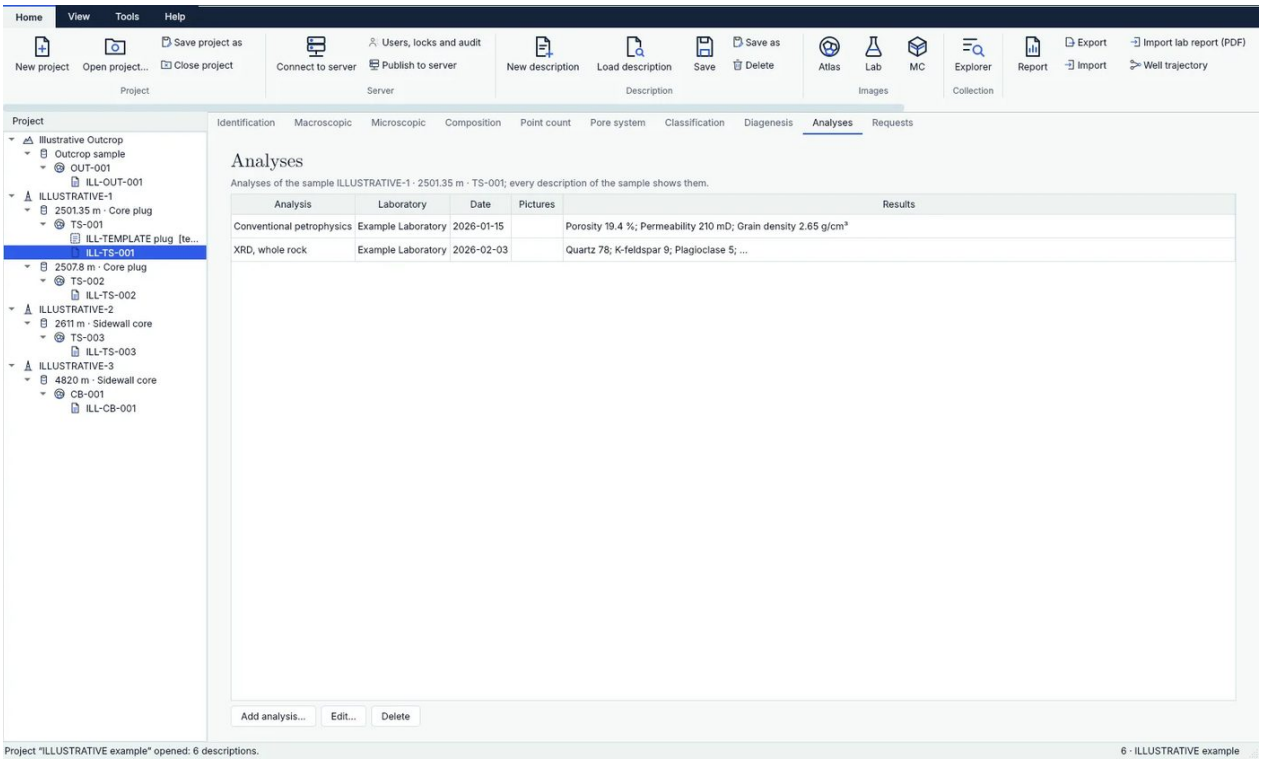
Coverage

P33 (diagenetic environment interpretation), P40 (knowledge graph), L8 (paragenetic sequence generator).

Analyses and analysis requests

What it is for

The **Analyses** tab keeps the laboratory results of the sample — petrophysics, XRD, SEM, isotopes, chemistry, biostratigraphy — with their pictures. The **Requests** tab asks the laboratory for new analyses and follows them until they are done. Both belong to the **sample**: every description of the sample's thin sections shows them.



In the program: Analyses and analysis requests

Before you start

A **saved** description: analyses and requests belong to its sample ("Save the description first: analyses and requests belong to its sample.").

Step by step

Adding an analysis

1. Open the **Analyses** tab. Its heading names the sample.

2. **Add analysis...**
3. Choose the **Analysis**: conventional petrophysics, special petrophysics, XRD (whole rock or clay fraction), SEM (secondary or backscattered electrons), stable isotopes (C, O), strontium isotopes, chemistry or biostratigraphy.
4. Fill in the **Laboratory**, the **Date (yyyy-mm-dd)** and the results. The fields follow the kind:
   - conventional petrophysics — **Porosity** (%), **Permeability** (mD), **Grain density** (g/cm³);
   - stable isotopes — **δ13C** and **δ18O** (‰ VPDB); strontium — **87Sr/86Sr**;
   - XRD — a table of **Mineral** and **%** (**Add row**, **Remove row**);
   - chemistry — a table of **Element or compound**, **Value** and **Unit**;
   - special petrophysics — the **Method**; biostratigraphy — **Biozone**, **Age** and **Fossils**;
   - **Notes** for anything else.
5. Under **Pictures and files**, **Add picture...** (SEM images, diffractograms, chromatograms); **Export...** saves one back to a file.
6. **OK**. The list shows the kind, laboratory, date, a summary of the results and the number of pictures. **Edit...** and **Delete** act on the selected analysis.

Requesting an analysis

1. Open the **Requests** tab, then **New request...**
2. Choose the **Analysis**. The **Tip** says what a good objective states for that kind.
3. Write the **Objective** (required): what question the analysis should answer.
4. **Status**: Requested, In progress, Done or Cancelled. Change it as the work goes on.
5. **Notes** and pictures (for example a photo of the area to sample), then **OK**.

The list shows the kind, objective, status, when it was requested and by whom.

Options

Field	Rule
Date	yyyy-mm-dd, yyyy-mm or yyyy.
Porosity, permeability, grain density	0–100 %, 0–100 000 mD, 1.5–5.0 g/cm³.
δ13C, δ18O	−50 to 50 ‰ VPDB; 87Sr/86Sr 0.69–0.80.
XRD rows	Each mineral named; the percentages should total 100 (saved anyway, with a warning).

How it works

Analyses and requests are stored with the sample, so a plug's porosity is entered once and shown in every description of that plug. They go into the reports, the exchange files, the queries of Explorer (petrophysics, isotopes, XRD) and its dashboard (porosity × permeability, grain density). Pictures are kept inside the project. Routine core analyses of a laboratory report in PDF can be read into the samples of a well by depth (Home → **Import lab report (PDF)...**, page Files of the reference programs).

From a script

```
d = prj["TS-9"]
d.add_analysis("conventional_petrophysics", laboratory="ILLUSTRATIVE lab", date="2026-03-02",
               values={"porosity": 18.2, "permeability": 245.0, "grain_density": 2.65})
d.add_analysis("xrd_whole_rock", rows=[{"mineral": "Quartz", "percent": 71}, {"mineral":
↪  "Calcite", "percent": 29}],
               pictures=["xrd.png"])
d.add_request("stable_isotopes", "δ13C and δ18O of the calcite cement, to date its water source")
for a in d.analyses():
    print(a["kind"], a["values"], a["rows"])
print([r["status"] for r in d.requests()])
```

Limits and common problems

- “Choose the kind of analysis.” / “State the objective of the request.”
- “... must be between ... and ...”: a value outside the plausible range of its kind.
- “The analysis has no results yet.” — a warning: the analysis is saved empty.
- Pictures larger than the project’s limit for attachments are refused.

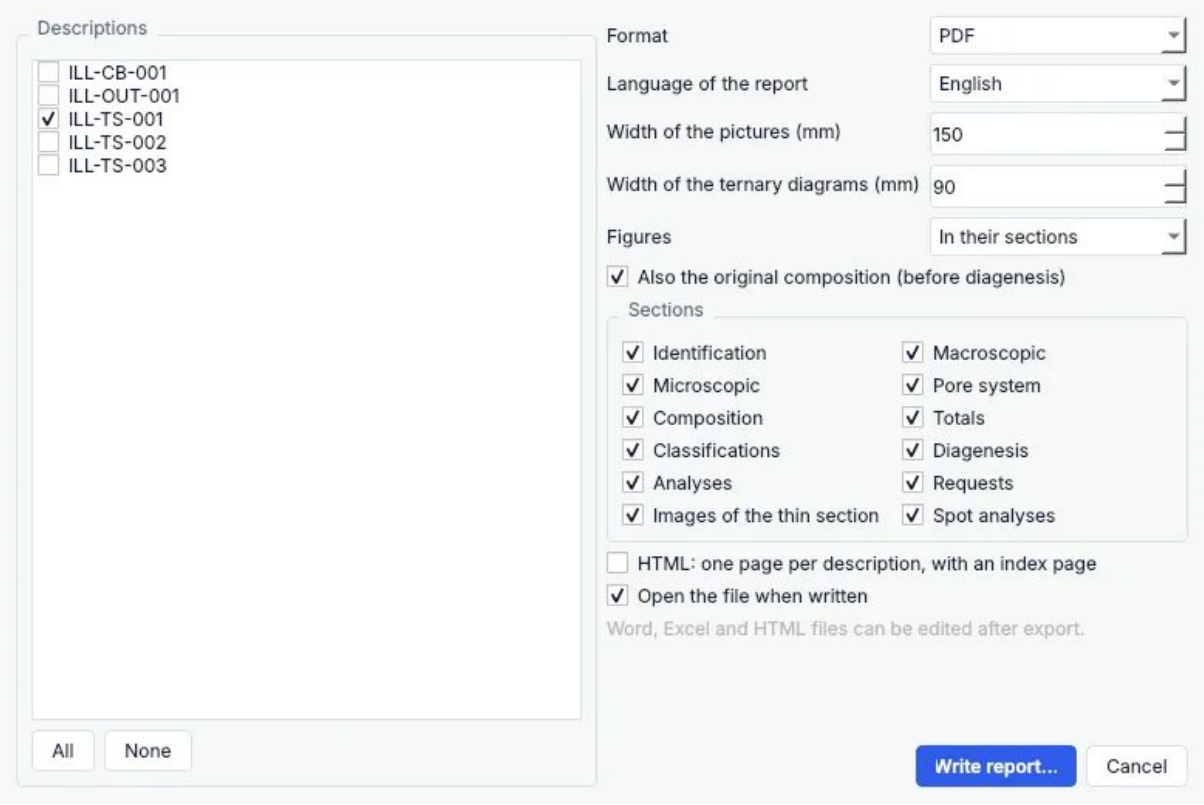
Coverage

P15 (additional analyses), P16 (analysis requests with pictures).

Reports

What it is for

A **report** writes one or many descriptions as a document: PDF to print or archive, Word (DOCX) or Excel (XLSX) to edit, or HTML pages to read in a browser — in English, Spanish or Portuguese, whatever the language of the interface. It holds the sections you choose: identification, description, composition, totals, classifications with their ternary diagrams, diagenesis with the paragenetic chart, analyses, requests, the images of the thin section and its spot analyses.



In the program: Reports

Before you start

The descriptions saved: unsaved changes are not in the report.

Step by step

1. **Home → Reports and exchange → Report...** (Ctrl+P).
2. **Descriptions**: tick those to write (**All**, **None**). The description open is ticked.

3. **Format**: PDF, Word, Excel or HTML.
4. **Language of the report**: English, Español or Português.
5. **Sections**: Identification, Macroscopic, Microscopic, Pore system, Composition, Totals, Classifications, Diagenesis, Analyses, Requests, Images of the thin section, Spot analyses.
6. Layout:
   - **Width of the pictures (mm)** and **Width of the ternary diagrams (mm)**;
   - **Figures**: **In their sections** or **All at the end**;
   - **Also the original composition (before diagenesis)**.
7. For HTML, tick **HTML: one page per description, with an index page** and choose the **Folder for the HTML pages**: each description gets its page, and `index.html` links them.
8. **Open the file when written**, then **Write report...** and choose where to save it.

"Word, Excel and HTML files can be edited after export."

Options

Option	What it changes
Format	PDF (A4), DOCX, XLSX (a sheet listing the descriptions and one sheet per description) or HTML.
Language of the report	Independent of the interface language.
Sections	Only the sections ticked are written.
Width of the pictures / ternary diagrams	Size of the figures in the document, in mm.
Figures	Inside their sections, or all gathered at the end.
Original composition	Adds the composition before diagenesis and its totals and classifications.

How it works

The report is built from the same records the forms show (codes translated into the language of the report); numbers are written with the decimal separator of that language. Ternary diagrams and the paragenetic chart are drawn as in the program. Explorer writes reports of the descriptions found by a query through the same dialog (page RockFace Explorer: queries, result tables and exports).

From a script

```
with rf.open("study.rockface") as prj:
    prj["TS-9"].report("TS-9.pdf", lang="pt")
    prj.report("well-1.docx", names=["TS-9", "TS-10"], lang="en",
               sections=["identification", "composition", "classification"], figure_mm=120,
↪   figures="end")
    prj.html_pages("site", lang="es")                # one page per description and index.html
```

Limits and common problems

- “No description chosen.”
- Word and Excel need the `reports` extra (python-docx, openpyxl), installed with `.[all]`.
- Very large whole-slide images are written reduced to the picture width chosen.

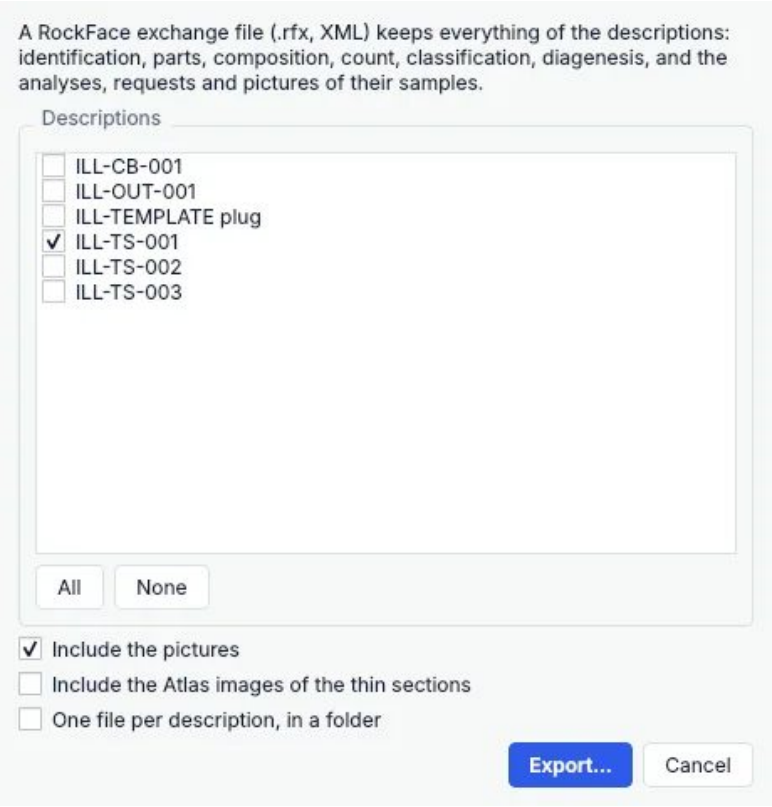
Coverage

P10 (reports in English, Portuguese and Spanish), P34 (reports: PDF, Excel, HTML, Word; language; configuration), P36 (HTML generator for batches).

Exporting and importing descriptions

What it is for

A **RockFace exchange file** (`.rfx`, XML) carries descriptions from one project to another: to send a study to a colleague, to gather the work of several petrographers, or to keep an archive in an open format. It keeps everything of a description — identification, parts, composition, point count, classification, diagenesis — and the analyses, requests and pictures of its sample; optionally the Atlas images of its thin section.



In the program: Exporting and importing descriptions

Before you start

- A project with saved descriptions (to export), or an exchange file (to import).
- On a server, importing needs the petrographer level or above.

Step by step

Exporting

1. **Home → Reports and exchange → Export...** (Ctrl+E).
2. Tick the **Descriptions** to export.
3. Choose:
   - **Include the pictures** — the pictures of analyses, requests and counted points;
   - **Include the Atlas images of the thin sections** — every tile of their pyramids goes into the file (whole-slide images of several gigabytes are better moved with **Save project as...**);
   - **One file per description, in a folder**.
4. **Export...** and choose the file (or the folder). "N descriptions exported to ...".

Importing

1. **Home → Reports and exchange → Import...** (Ctrl+I).
2. **Files...** or **Folder...** (every `.rfx` and `.xml` in it).
3. The list shows each **Description**, its **File** and whether it is **In this project** (*new* or *name already used*). Tick those to import.
4. **Names already in the project**:
   - **Import with a new name: (2), (3)...**
   - **Skip them**
   - **Replace the descriptions of the project** (asks for a confirmation).
5. **Import**. The result names what was imported, renamed or not imported, and why.

Options

Option	What it changes
Include the pictures	Attachments go into the file (base64).
Include the Atlas images	The images of the thin sections, with their annotations, maps and spots.
One file per description	A folder of files instead of one file.
Names already in the project	Rename, skip or replace.

How it works

The file is checked before anything is imported (format version, codes known to this RockFace, totals of the project). Wells, samples and thin sections are matched by name and numbers, so two descriptions of the same thin section stay together; identifiers are mapped to the new project; the summaries (totals, classifications) are worked out again. Files written by any earlier RockFace are read. Custom totals of the exporting project travel with the file.

XML of other programs is read through a mapping (Home → **Import XML with a mapping...**, page Sample catalogue, Touchstone-ready table and XML by mapping).

From a script

```
with rf.open("study.rockface") as prj:
    prj.export("for-the-lab.rfx", names=["TS-9", "TS-10"], images=True)
    prj.export_folder("archive", names=None)               # one file per description
with rf.create("copy.rockface") as other:
    for r in other.import_file("for-the-lab.rfx", policy="skip"):
        print(r["name"], r["action"], r["as"], r["messages"])
print(rf.read_exchange("for-the-lab.rfx").names())         # read without importing
```

Limits and common problems

- “No descriptions in the files chosen.”
- “…: not imported — …”: the reason (unknown code, a file written by a newer RockFace…).
- Replacing cannot be undone.

Coverage

P2 (Export, Import), P36 (XML generator for batches), P37 (XML importer: files or folders, name conflicts).

Exporting wells to WITSML and samples to OSDU

What it is for

- **WITSML** (Energistics) is the format of well data that reservoir-modelling and well-data programs read. It lets the petrography of the samples, and the image logs and curves of RockFace MC, sit beside the other logs of a well.
- **OSDU** is the open data platform of the energy industry. Its manifest lets the wells, samples, thin sections, descriptions, laboratory analyses and images of a project be ingested into an OSDU data platform, including the Rock and Fluid Sample service.

Before you start

- **For WITSML:** the samples need a well and a depth. The curves are those of Explorer's LAS export.
- **For OSDU:** ask the platform's data manager for the **data partition** and the **legal tag** to use.

Step by step

WITSML

1. In RockFace Petro, choose **Home → Reports and exchange → Export WITSML**.
2. Choose a folder. RockFace writes, for each well:
   - `<well>_well.xml` and `<well>_wellbore.xml`: the well and its wellbore;
   - `<well>_samples_log.xml`: one row per sample depth, with core porosity, permeability, grain density, visual porosity and facies;
   - one `<well>_<log>_log.xml` for each well log of RockFace MC. Image logs become array curves, one value per azimuth.
3. Load the files into the target program as WITSML 1.4.1.1 well, wellbore and log objects.

OSDU

1. Choose **Home → Reports and exchange → Export OSDU manifest**.
2. Choose the file name, then type the **data partition** and the **legal tag**. An empty legal tag writes a placeholder, to be replaced before ingestion.
3. Give the JSON file to the platform's manifest ingestion.

Options

From a script or the command line you can choose:

- the descriptions;
- the curves of the samples' log and the depth used (measured, corrected, adjusted);
- whether to include the MC logs and the image logs;
- for OSDU, the access groups and the countries.

How it works

- **WITSML.** RockFace writes WITSML 1.4.1.1 with its own code. The element order follows the WITSML 1.4.1.1 schema. The index is the measured depth in metres; missing values are left empty, with `-999.25` as the null value.
- **OSDU.** Wells and wellbores become `master-data--Well` and `master-data--Wellbore`. Samples and thin sections become `master-data--Sample` (a thin section is a child sample of its sample). Descriptions and laboratory analyses become `work-product-component--SamplesAnalysis`, and images `work-product-component--SampleImage`.
  - The petrography — composition with percentages, classifications, the codes of RockFace's nomenclature — goes in `ExtensionProperties.RockFace`, because OSDU has no petrography schema yet.
  - The mapping table is in `docs/SERVER.md`.

From a script

```
import rockface.api as rf
with rf.open("project.rockface") as prj:
    prj.export_witsml("witsml_folder")
    prj.export_osdu("manifest.json", partition="my-partition", legal_tag="my-partition-legal")
```

Or: `rockface export witsml project.rockface folder` and `rockface export osdu project.rockface manifest.json --partition my-partition --legal-tag my-partition-legal`.

Limits and common problems

- **WITSML validation.** The files were checked by reading them back, not against the official schema files. Please report any program that refuses them, using the feedback form.
- **OSDU schema versions.** The versions used are written in the manifest. If the platform uses others, the data manager may need to adjust them.
- **Images.** OSDU image records describe the images; the image files stay in RockFace. Export the images from Atlas to attach them as datasets.

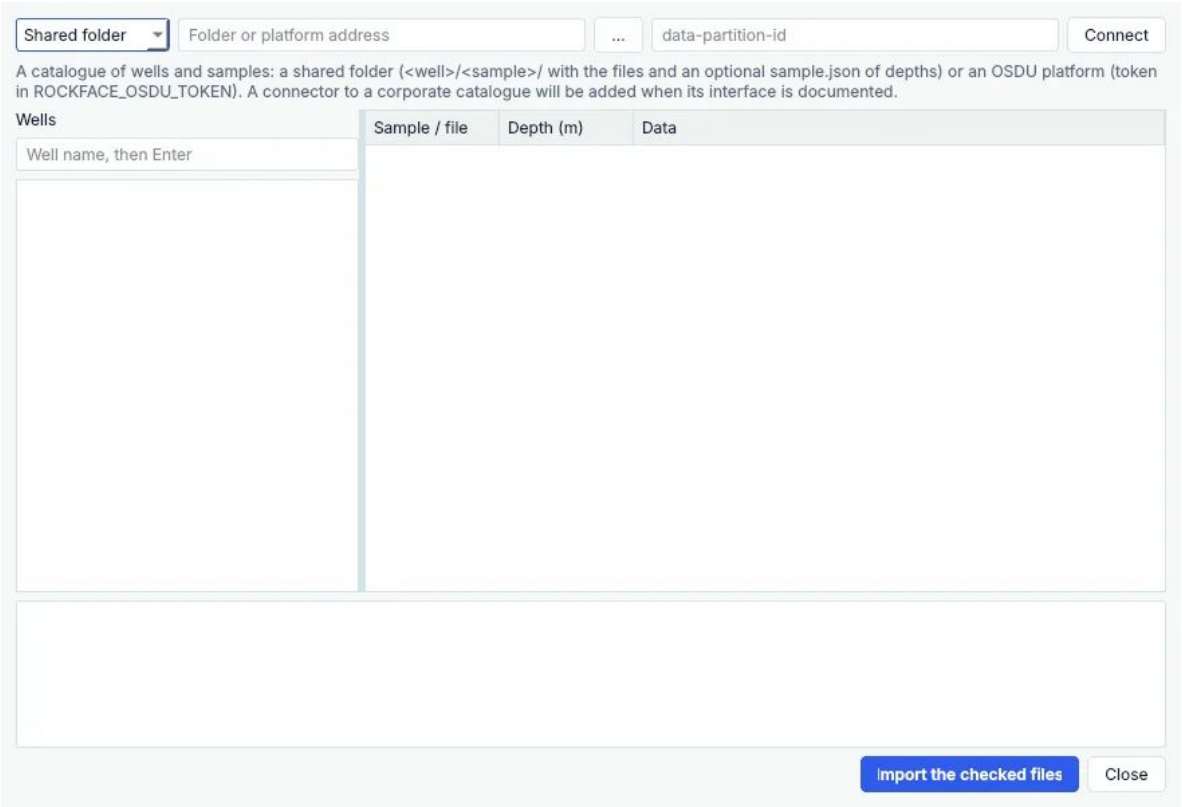
Coverage

P38 (WITSML export), L13 (OSDU).

Sample catalogue, Touchstone-ready table and XML by mapping

What it is for

Sample catalogue browses the wells and samples of a corporate catalogue and imports the chosen files. **Touchstone-ready table** writes the petrographic quantities that diagenetic modelling uses. **Import XML with a mapping** reads descriptions written by another program.



In the program: Sample catalogue, Touchstone-ready table and XML by mapping

Before you start

- Catalogue: a shared folder `<well>/<sample>/` (optionally a `sample.json` with `top_m`, `base_m`, `type`), or an OSDU platform with a token in `ROCKFACE_OSDU_TOKEN`.
- XML import: the XML file and a mapping (JSON) that says where things are in it.

Step by step

1. **Petro → Home → Sample catalogue…**: choose *Shared folder* or *OSDU platform*, the address, **Connect**; search a well; tick the files of its samples; **Import the checked files**. Pictures go to Atlas, volumes to MC, LAS files to the well's logs, PDF reports to the sample's analyses.
2. **Petro → Home → Touchstone-ready table…**: choose where to save the CSV.
3. **Petro → Home → Import XML with a mapping…**: choose the XML, then the mapping. Codes without a match are listed.

Options

Mapping key (XML)	Meaning
description	path of a description element (e.g. `.//description`)
name, well, top_depth, thin_section	paths inside it (`@attr`, `child`, `child/@attr`)
line, code, percent, points	the composition lines and their fields
codes	the other program's codes → RockFace constituents

The Touchstone mapping lists the columns: each a field (name, well, depth, grain size, sorting) or a sum of totals or constituents.

How it works

- The catalogue uses connectors with the same three questions (wells, samples, a file): a folder, or the OSDU Search service (`POST /api/search/v2/query`). The well and the sample are found or created by name and depth.
- The Touchstone input format is not published: the table carries the quantities of the published model (Lander & Walderhaug 1999; Lander et al. 2008) and says in its first line that it is not that format.
- Mapped XML: every code must match a constituent of the nomenclature; nothing is guessed.

From a script

```
cat = rf.catalogue.FolderConnector("//share/catalogue")
s = cat.samples("ILL-WELL-A")[0]
rf.catalogue.import_items(prj.raw, cat, s, s.items)
prj.touchstone_table("touchstone.csv")
prj.import_mapped_xml("descriptions.xml", mapping)
```

Limits and common problems

- A connector to a corporate sample catalogue will be added when its interface is documented to RockFace.
- An XML of another program and the "Touchstone format" will be named so only after real files have been read.

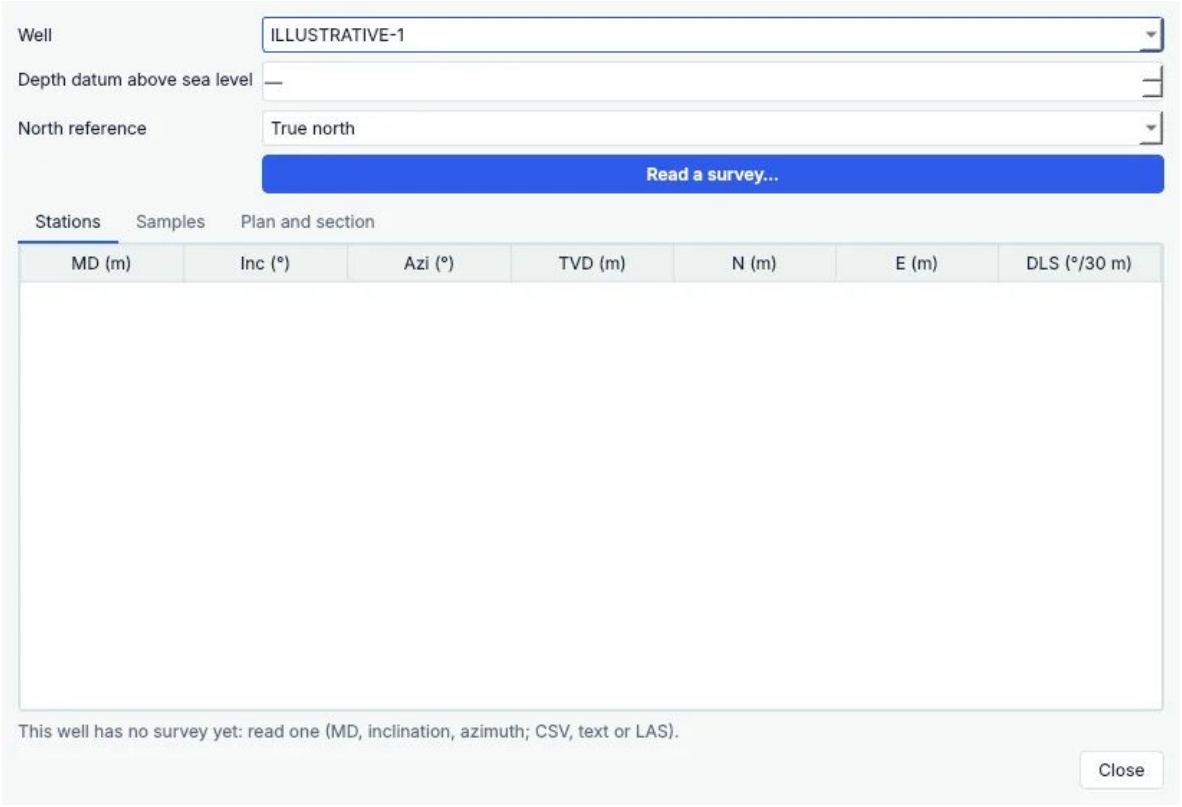
Coverage

GV16 (catalogue browser; corporate connector pending), P35 (Touchstone; final with a real file), P37 (XML importer).

Well trajectory

What it is for

A deviated well puts a sample at a measured depth (MD) that is not its true vertical depth (TVD), nor straight under the well head. **Well trajectory** reads the directional survey of the well and gives every sample its TVD, its depth below sea level, its offsets north and east and its latitude and longitude.



In the program: Well trajectory

Before you start

- The well in the project (Identification), with its head latitude and longitude for the geographic position.
- A survey file: measured depth, inclination and azimuth (CSV, text or LAS).

Step by step

1. **Petro → Home → Well trajectory...**

2. Choose the well; type the elevation of the depth datum above sea level and the north reference.
3. **Read a survey...** and choose the file. The stations appear with TVD, north, east and dogleg severity.
4. **Samples**: every sample of the well on the trajectory. **Plan and section**: the plan view and the vertical section, samples in red.

Options

Field	Meaning
Depth datum	elevation of the datum (kelly bushing, rotary table) above sea level: TVDSS = TVD − datum
North reference	true, grid or magnetic, as the survey gives the azimuths (recorded, not converted)
Columns	MD / Depth / Prof., Inc / Incl / Devi, Azi / Azim / Hazi, any order; comma, semicolon, tab or spaces

How it works

Minimum curvature (Bourgoyne et al. 1986; Sawaryn & Thorogood 2005): between two stations the well follows the arc of a circle; ΔN, ΔE and ΔTVD are the half sums of the direction cosines times ΔMD, times the ratio factor $RF = (2/\beta)\cdot\tan(\beta/2)$, β the dogleg. A sample between stations lies on the same arc. Latitude and longitude come from the head by the WGS 84 radii of curvature (local tangent plane).

From a script

```
st = prj.survey("ILL-WELL-1", "survey.csv", datum_elevation_m=25.0)
st[-1].tvd, st[-1].north, st[-1].east
rf.trajectory.sample_position(prj.raw, sample_id)     # {'tvd': …, 'tvdss': …, 'latitude': …, …}
```

Limits and common problems

- Above the first station the well is taken vertical; below the last it keeps the last direction (marked *extrapolated*).
- Magnetic azimuths are not corrected for declination: convert them first or choose the reference that matches.

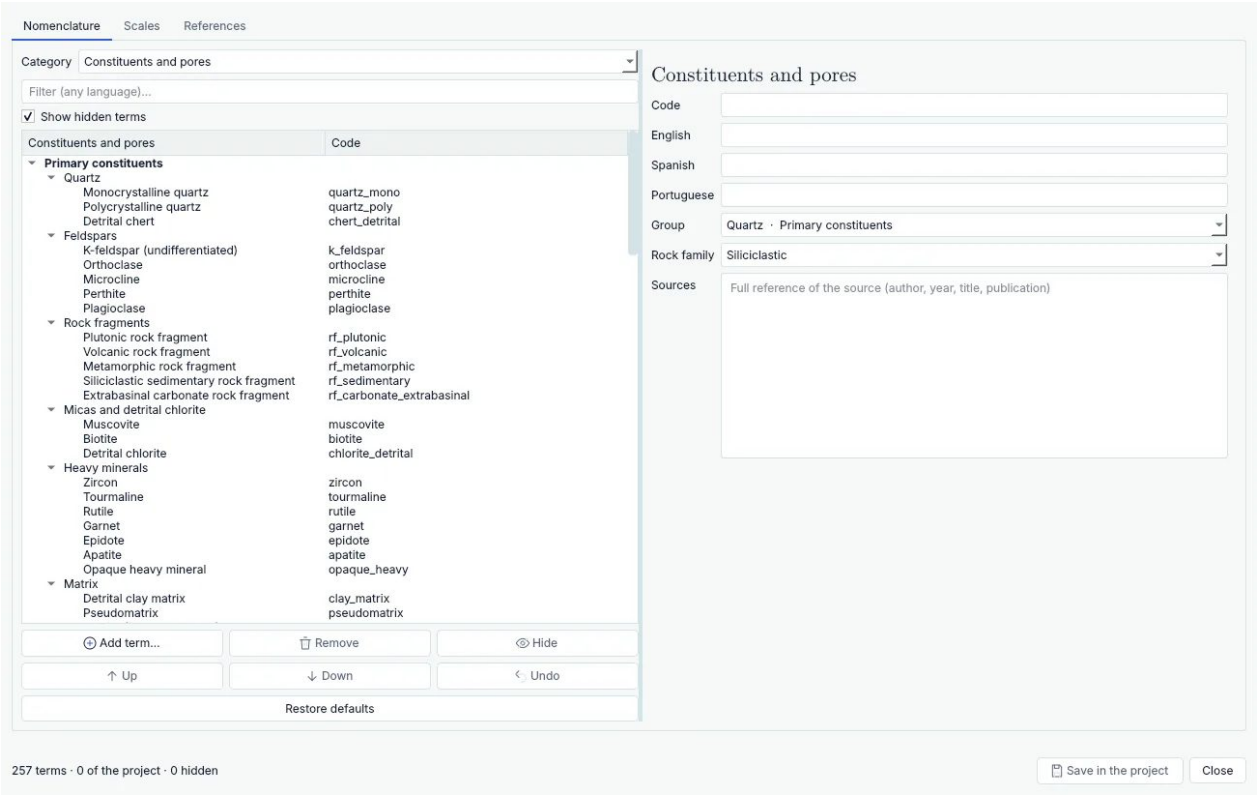
Coverage

P8 (sample location: geographic position, well depth and trajectory).

Nomenclature, scales and references (the knowledge base)

What it is for

Every list of RockFace — constituents, habits, locations, modifiers, paragenetic relations, structures, contacts, pore types — comes from its **knowledge base**: 257 built-in terms in English, Spanish and Portuguese, each with its published source. The knowledge-base editor shows them, and lets a project add its own terms, hide terms it does not use and put the lists in its preferred order. It also shows the numeric **scales** (grain size, sorting, roundness, crystal size, pore size) and the **references** cited.



In the program: Nomenclature, scales and references (the knowledge base)

Before you start

- A project open, to change anything. Without a project the built-in nomenclature is shown read-only.
- On a server, changing the knowledge base needs the manager level.

Step by step

Looking up a term

1. **Tools → Knowledge base → Nomenclature** (Ctrl+K).
2. Choose the **Category** (constituent, group, habit, location, modifier, relation, structure, contact…).
3. Type in **Filter (any language)…**: “quartzo”, “cuarzo” and “quartz” find the same term; accents are ignored.
4. Select a term to see its code, its labels in the three languages, its group, kind, rock family, what it applies to, and its **Sources**.

Adding a term of the project

1. Choose the category, then **Add term…**.
2. Fill in **Code** (lower case, letters, digits and _; it never changes afterwards), the labels in **English** (required), **Spanish** and **Portuguese** (a warning when missing), and, as the category asks, **Group** and **Rock family** (constituents), **Kind** (groups), **Applies to** (locations and modifiers) and **Inverse relation** (relations).
3. In **Sources**, give the full reference of the source (author, year, title, publication). Every built-in term carries its literature source; a project term without one is saved with a warning.
4. **Apply**, then **Save in the project**.

Hiding, ordering, undoing

- **Hide** takes a built-in term out of the lists of this project (descriptions that use it keep it). Tick **Show hidden terms** to see them, and **Show** to bring one back.
- **Up** and **Down** change the order in which the terms appear in the lists of the forms.
- **Remove** deletes a term of the project (built-in terms can only be hidden).
- **Undo** cancels the last change; **Restore defaults** brings back the built-in order and every hidden term (the project’s own terms are kept).
- **Save in the project** writes the changes, with a line in the audit; **Close** asks first if something is not saved.

Scales

1. **Tools → Knowledge base → Scales**.
2. Choose the **Scale**: grain size (Udden–Wentworth, with φ of Krumbein), sorting (Folk & Ward σI), roundness (Powers), crystal size (Folk 1962) or pore size. The table gives each class with **From**, **To (exclusive)** and φ.
3. **Check a value against a class**: type a **Value**, choose a **Class**; RockFace answers “Consistent” or “Not consistent: … is …, not …”. The description forms use the same check (page Macroscopic and microscopic description).

References

Tools → Knowledge base → References lists every reference with its **Key**, the full **Reference** and **Cited by** (how many terms, rules, totals and classifications cite it).

Options

Field	What it is
Code	The value stored in the project; never translated, never changed.
Group, Kind	Where the term goes in the composition lists (e.g. diagenetic carbonate).
Rock family	Siliciclastic, carbonate or both: the lists of a description show the family that applies.
Applies to	For locations and modifiers: the kinds of constituent they can describe.
Inverse relation	For paragenetic relations: the relation read the other way (e.g. *replaces* ↔ *replaced by*).

How it works

The built-in nomenclature is RockFace's own, built from the published literature; none of it comes from the ontology of a commercial program. The project keeps only its differences — its own terms, hidden terms and order — so a newer RockFace with more built-in terms still opens it. The full list, with the sources, is `docs/NOMENCLATURE.md` of the source code.

From a script

```
with rf.open("study.rockface") as prj:
    for t in prj.search("caulinita"):                # any language, accents ignored
        print(t.category, t.code, t.labels["en"], t.citations())
    print(prj.term("constituent", "kaolinite").labels)
from rockface.core import scales
print(scales.classify("grain_size", 0.3))            # 'medium_sand'
```

Limits and common problems

- "Open a project to add, hide or reorder terms; the built-in nomenclature is shown."
- A code already used, or a code with capitals or spaces, is refused; so is a constituent without its group and rock family.
- Hiding a term does not change the descriptions that already use it.

Coverage

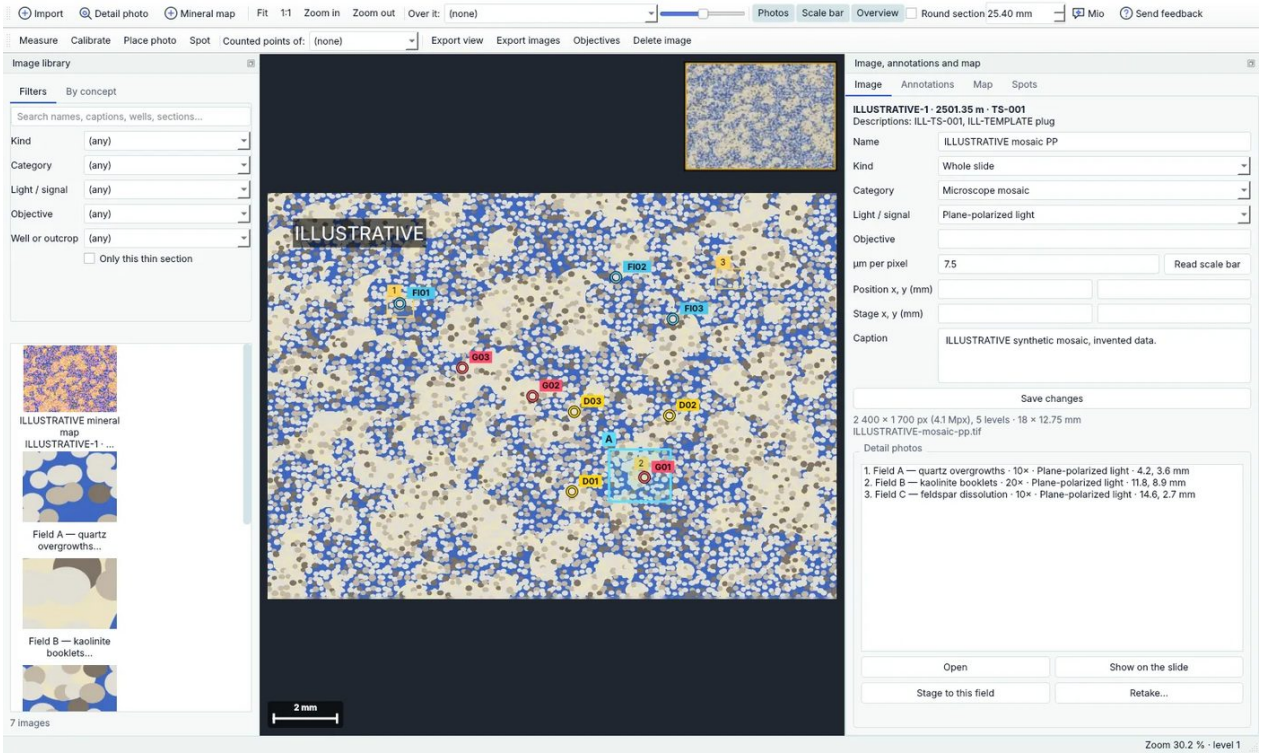
P39 (knowledge-base tools: attribute editors with and without consistency intervals, item order, add, update, remove, undo).

RockFace Atlas

RockFace Atlas: whole slides, detail photos and the image library

What it is for

RockFace Atlas keeps the images of each thin section: whole-slide images of any size (a scanner's multi-gigapixel slide, a microscope mosaic) in several lights, and detail photos placed at their exact position on the section, with the stage position where they were taken. The **image library** finds any image of the project by its name, caption, well, section, kind, light or objective, and the viewer zooms from the whole section to full resolution at once.



In the program: RockFace Atlas: whole slides, detail photos and the image library

Before you start

- A **saved** description: images belong to its thin section ("Save a description first: images belong to its thin section.").
- Image files: PNG, JPEG, TIFF, BigTIFF, OME-TIFF, BMP, GIF, WebP; scanner formats (SVS, NDPI, MRXS, SCN) with OpenSlide installed; Zeiss CZI with the `czi` extra.

Step by step

Opening Atlas

Home → Images → Atlas (Ctrl+Shift+A) in RockFace Petro opens Atlas on the same project and thin section; or `rockface atlas project.rockface`.

Importing a whole slide

1. **Import...** (Ctrl+I) and choose the files.
2. In **Import images**, check the **Thin section**, the **Kind** (whole slide), the **Category** (microscope mosaic, slide scanner, photomicrograph, macroscopic photo, other) and the **Light / signal** (plane-polarized, crossed polars, reflected, cathodoluminescence, ultraviolet, fluorescence, SEM, other).
3. Scale: **µm per pixel** — from the file when it carries it, else typed, else from the **Objective** and the table of **Objectives...**.
4. **Tiles**: **JPEG** (photographs; smaller) or **PNG** (lossless; maps).
5. **OK**. "Building the image pyramid…" shows the progress; a very large image takes a while, once.

Detail photos

1. **Detail photo...**: choose the photo, its **Objective**, light, and, when known, its **Position x, y (mm)** on the section and the **Stage x, y (mm)** where it was taken.
2. Or place it afterwards: select it under **Detail photos**, choose the tool **Place photo**, and click where it is on the whole slide.
3. **Show on the slide** frames the photo's field; **Open** shows the photo itself; **Stage to this field** sends the stage (or the simulator) back to where it was taken.
4. **Retake...** adds a new photo of the same field, with the same position, stage position and objective, linked to the first ("N photos of this field").

The viewer

- Drag to pan; the mouse wheel zooms. **Fit**, **1:1**, **Zoom in**, **Zoom out**.
- **Over it:** lays another image of the section over the one open (XP over PP, CL, a mineral map), with the **Opacity of the image laid over**.
- Show or hide **Photos** (the fields of the detail photos, numbered), the **Scale bar**, the **Overview** map and the **Round section** outline.
- **Counted points of:** shows the points counted in a description, coloured by constituent (page Point counting).
- **Tools**: **Measure** (click two points: the distance in µm), **Calibrate** (click two points and give their distance: sets the scale), **Place photo**, **Spot** (page Spot analyses: geochemistry, dating and fluid inclusions).
- **Export view...** saves what is shown, or the whole image, as PNG, JPEG or TIFF.

The image library

1. The **Image library** lists every image of the project. Type in **Search names, captions, wells, sections...**.
2. **Filters**: kind, category, light, objective and **Well or outcrop**; **Only this thin section**.
3. **By concept** finds images by what their annotations show (page Atlas annotations and the search by concept).
4. Select an image to see and edit its **Name**, kind, category, light, objective, scale and **Caption**; **Save changes**. **Read scale bar** finds the scale bar burnt into a picture and reads its length (OCR).
5. **Delete image** removes the image and its tiles from the project.

Options

Field	What it changes
Objectives...	µm per pixel of the camera for each objective, measured once on a stage micrometer.
Tiles	JPEG (smaller, for photographs) or PNG (lossless, for maps and labels).
Position x, y (mm)	Where the photo lies on the section (its centre), in the section frame.
Stage x, y (mm)	Where the stage was when the photo was taken (to come back to it).

How it works

Each image is stored in the project as a pyramid of tiles: only the tiles on the screen are read, at the level that matches the zoom, so a 2-gigapixel section opens in a fraction of a second and pans without waiting. Every image has its place in the section frame (millimetres), shared with the point count, the annotations, the mineral maps and the spot analyses: what is at a point on one image is at the same point on the others.

From a script

```
d = prj["TS-9"]
slide = d.add_image("TS-9_PP.tif", category="scanner", modality="pp")
photo = d.add_photo("field1.jpg", x_mm=4.2, y_mm=3.6, magnification=10, stage_x_mm=31.2,
↪  stage_y_mm=24.6)
slide.overview("TS-9_fields.png", width=2400)              # the slide with the photo fields
↪  numbered
crop = slide.region(*slide.to_px(4.0, 3.4), 400, 300)      # a NumPy array at full resolution
for img in prj.images(kind="photo", modality="xp", well="ILLUSTRATIVE W-1"):
    print(img.name, img.um_per_px, img.x_mm, img.y_mm)
prj.objectives = {"10": 0.65, "20": 0.325}                 # µm per pixel for each objective
```

Limits and common problems

- “This thin section has no whole-slide image.” — photos can still be imported, and placed later.
- “The stage position of this photo is not known.” — Stage to this field needs the stage position.
- “A scale needs two different points and a length above zero.”
- Moving several gigabytes of images to another project: **Save project as...** is faster than an exchange file.

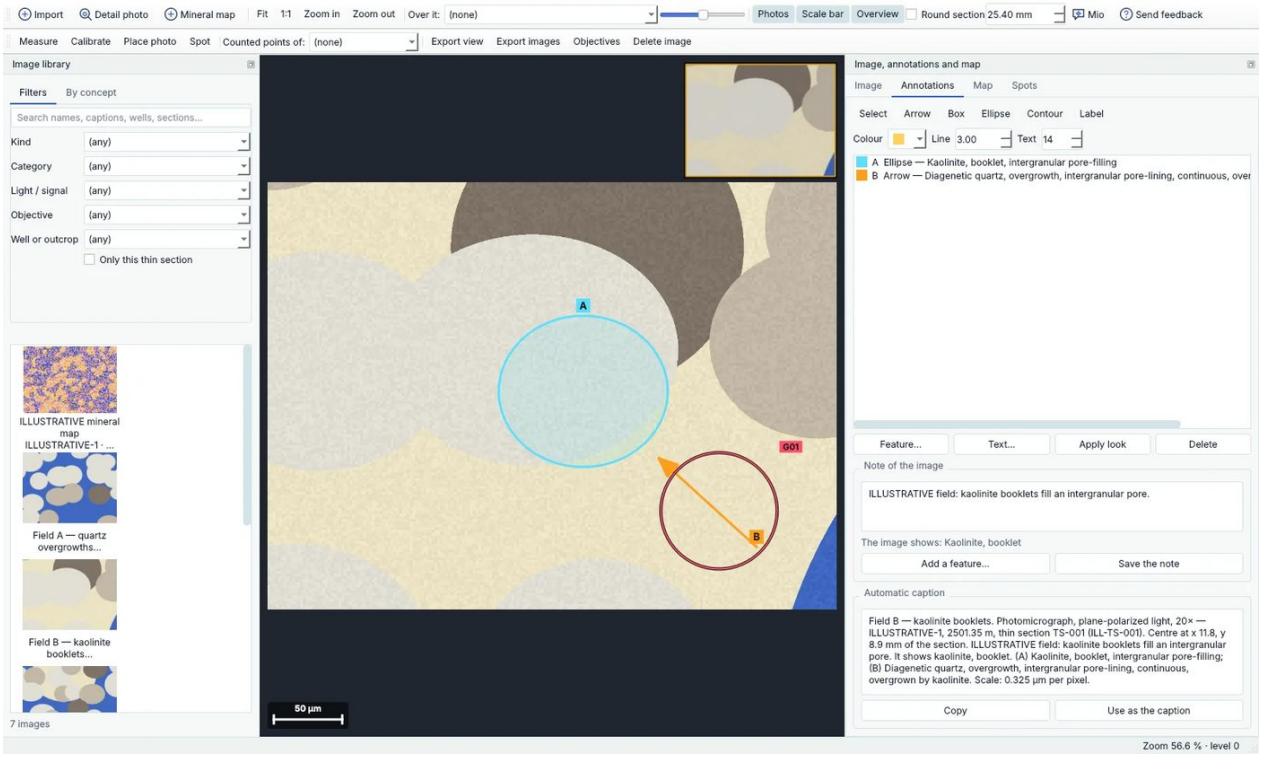
Coverage

R1 (database of photomicrographs with fast queries), GT1 (thin-section images with scale), GT21 (CZI and the scale bar), L6 (photos taken again at the same referenced position).

Atlas annotations and the search by concept

What it is for

Annotations mark what an image shows — an arrow on a kaolinite booklet, a box around a dissolved feldspar, a contour of a pore, a label — and tie each mark to the nomenclature as a **feature**: a constituent with its habit, location, modifier and relation, as in the composition. RockFace writes the **automatic caption** of the image from them, exports figures with or without the marks, and finds images by what they show: rock type → concept → attribute → value.



In the program: Atlas annotations and the search by concept

Before you start

An image open in RockFace Atlas (page RockFace Atlas: whole slides, detail photos and the image library).

Step by step

Drawing

1. Open the **Annotations** tab of the panel **Image, annotations and map**.

2. Choose a tool:
   - **Arrow** — click where it starts, then where it points;
   - **Box** — click two opposite corners; **Ellipse** — two corners of the box around it;
   - **Contour** — click the points; double-click, Enter or the first point closes it;
   - **Label** — click where the text goes, then type it.
3. **Select** — click an annotation to choose it, drag it to move it, Delete removes it.
4. **Colour**, **Line** width and **Text** size; **Apply look** gives them to the selected annotation.

Saying what it shows

1. Select an annotation and press **Feature...**. In **What the annotation shows**, choose the constituent and its habit, location, modifier and relation (the same lists as the composition).
2. **Text...** changes the text of a label or adds a text to any annotation.
3. Annotations that say something are lettered A, B, C… in the order they were drawn; the letters are drawn on the image and used by the caption.

The note and the caption

1. **Note of the image** — a text about the whole image, with **Add a feature...** for what the whole image shows; **Save the note**.
2. **Automatic caption** — the name, kind and light, objective, sample, depth and thin section, the position on the section, the note, "(A) …; (B) …" and the scale. **Copy** it, or **Use as the caption** of the image.

Exporting figures

1. **Export images...**
2. **Images**: **The image open**, **Every image of its thin section** or **The images listed in the library**.
3. **Annotations**: **With annotations**, **Without annotations** or **Both (two files each)**.
4. **Longer side at most** (pixels) and **A caption file (.txt) next to each image**.
5. Choose the folder.

Finding images by concept

1. In the **Image library**, open **By concept**.
2. **Look in**: **the annotations of the images**, **the compositions of the descriptions**, or **annotations and compositions**.
3. Open the tree **Rock type › concept › attribute › value** — for example *Subarkose › Kaolinite › Habit › Booklet* — and select a node: the library shows the images found, with **Found in: ...** (the letters, the note or the description). **Show every image** clears the search.

Option	What it changes

Options

Option	What it changes
Colour, Line, Text	Look of the annotation (also in exported figures).
Feature	What the annotation shows, with the nomenclature of RockFace.
Look in	Search the annotations, the compositions of the thin sections, or both.

How it works

Annotations are stored in the full-resolution pixels of their image, so they stay in place at any zoom and in any export. The features use the codes of the knowledge base, so a caption is written in the language of the interface (or of the export) and the search finds an image whatever language it was annotated in. The rock type of an image is the classification of the descriptions of its thin section.

From a script

```
img = prj.image("Field B — kaolinite booklets")
img.annotate("rect", [[820, 410], [1260, 760]], features=[rf.Feature("kaolinite",
↪  habit="booklet")])
img.annotate("label", [[1300, 420]], text="Kln")
img.note("Kaolinite booklets fill the pores.", [{"constituent": "kaolinite"}])
print(img.caption_auto("en"))
for found, reasons in prj.search_images(concept="kaolinite", attribute="habit", value="booklet"):
    found.export(f"{found.name}.png", max_side=3000, annotated=True, caption_file=True)
prj.export_images("figures", annotated=True, caption_file=True, lang="pt")
```

Limits and common problems

- A feature with a code unknown to the knowledge base is refused.
- Only one note per image: a new note replaces it.
- “Delete this annotation?” — deleting cannot be undone; the letters of the others follow.

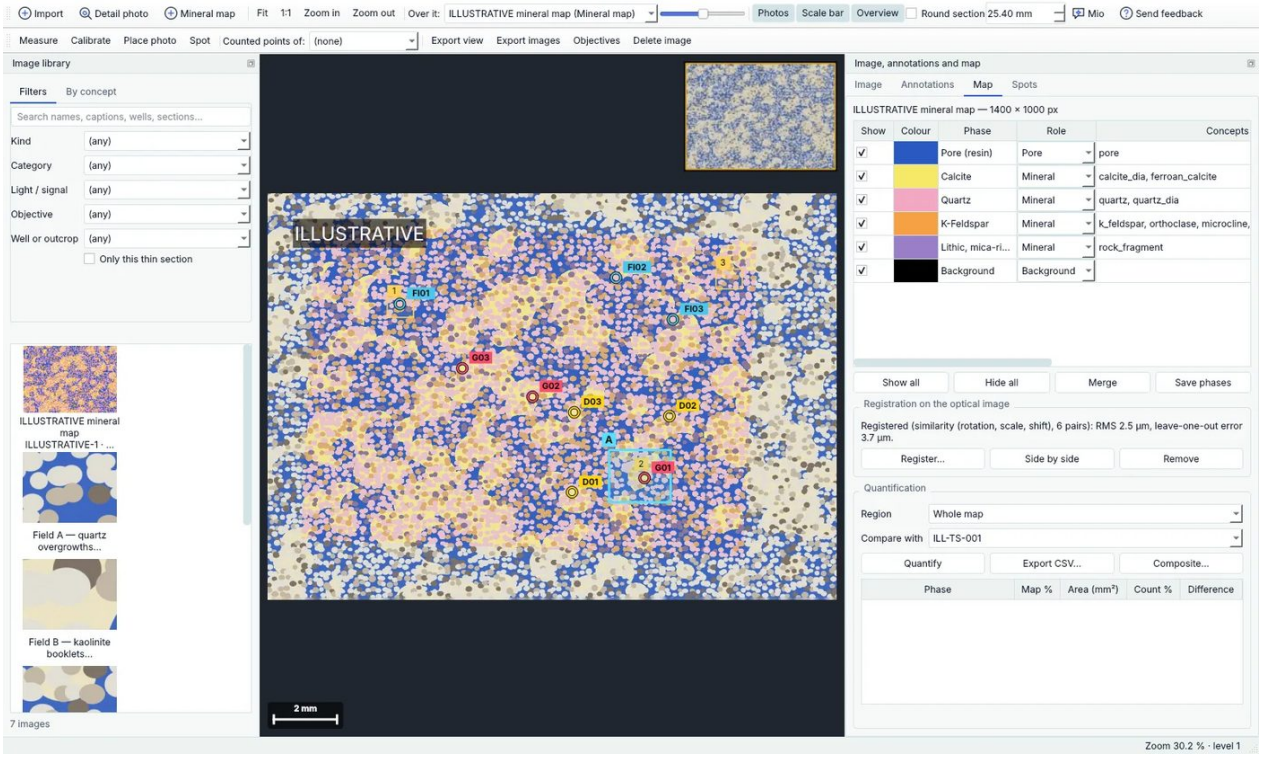
Coverage

R2 (hierarchical search: rock type → concept → attribute → value), R3 (image editor: arrows, contours, letters, notes; features with the nomenclature).

Mineral maps: registration, overlay, composite and quantification

What it is for

A mineral map from QEMSCAN, MLA, TIMA, a µXRF phase map or any classified image tells which mineral is at each pixel. RockFace Atlas registers it on the optical image of the same thin section, lays it over the image with any opacity, makes a **composite image** (the map blended over the photograph, with its legend and scale), and **quantifies** each phase over the whole map or over any region — comparing the map with the point count.



In the program: Mineral maps: registration, overlay, composite and quantification

Before you start

- A saved description whose thin section has (ideally) a whole-slide optical image in Atlas.
- The map as PNG or TIFF (a palette image, a grey image of phase numbers or an RGB image of phase colours) and its legend: CSV or TXT (with or without a header, e.g. `Mineral,R,G,B`) or a 3D Slicer colour table.

Step by step

Importing the map

1. In RockFace Atlas, **Mineral map...**.
2. In **Import a mineral map**, choose the map, its **Legend** (by default the legend file found beside the map), the **Colour tolerance** (how far a colour of an RGB map may be from the legend's) and the **µm per pixel**.
3. RockFace reads the phases ("... pixels, N phases"). In the **Map** tab, check every phase:
   - **Phase** name and **Colour**;
   - **Role**: Mineral, Pore, Background or Unclassified;
   - **Concepts**: codes of constituents, groups or kinds of the nomenclature, separated by commas (e.g. `quartz, quartz_dia`; `pore`) — they link the map to the nomenclature and to the point count;
   - **Show** to hide or show a phase; **Show all**, **Hide all**; **Merge** joins selected phases.
4. **Save phases**.

Registering the map on the optical image

1. **Register...** opens the two images side by side.
2. Click a point on the optical image (left), then the same point on the map (right): a grain corner, a pore, a scratch. Repeat. Two pairs are enough for a **similarity** (rotation, scale, shift), three for an **affine** transform; spread them over the map and add a few more.
3. The table gives each **Pair** with its **Residual**; **Remove pair**, **Remove all**.
4. Choose the **Transform** and **Apply**. RockFace reports the RMS of the pairs and the **leave-one-out error** (each pair predicted by the others), the scale and the rotation.
5. The map is now drawn in the frame of the section, over every image of it: in the viewer, choose it in **Over it:** and set the opacity. **Side by side** shows the two images with **Move both views together**. **Remove** undoes the registration (the map goes back to its scale at the origin of the section).

Quantifying

1. In **Quantification**, choose the **Region**: **Whole map**, **Area on screen**, or an annotation (a box or a contour drawn on any image of the section).
2. Choose a description in **Compare with** to put its point count beside the map.
3. **Quantify**. The table gives each phase with its **Map %**, **Area (mm²)**, **Count %** and **Difference**, and the summary: area analysed, the share of minerals and pores, porosity. Background and unclassified pixels are left out of the %.
4. **Export CSV...** writes the table.

The composite image

1. **Composite...** opens **Composite image**.
2. **Optical image** (by default the plane-polarized slide), **Map drawn as** (colours blended over the image, or phase outlines), opacity, **Width**, **Region** (whole image or the area on screen).
3. **Legend with the % of each phase, and scale bar**; **Also add it to the image library**.
4. **OK** and save the picture.

Options

Option	What it changes
Colour tolerance	RGB distance within which a map colour is taken as a legend colour.
Role	Background and unclassified are left out of the %; pores give the porosity.
Concepts	The link to the composition lines for the comparison with the count.
Transform	Similarity (≥ 2 pairs) or affine (≥ 3 pairs).

How it works

The registration is a least-squares fit of the landmark pairs; the leave-one-out error is a conservative estimate of the real error (about twice it, on test cases with a known answer: six pairs picked to the nearest pixel give 2.9 µm RMS on a 10 µm map). The map's tiles hold phase numbers drawn through a colour table, so phases are shown, hidden or recoloured at once. A map can also be aligned on a segmentation without landmarks (page A mineral map aligned without landmarks; effects on big volumes).

From a script

```
d = prj["TS-9"]
qem = d.add_map("TS-9_qemscan.tif", legend="TS-9_qemscan.csv", um_per_px=10)
pp = prj.image("TS-9 PP")
fit = qem.register([((140, 120), (380, 288)), ((1250, 90), (1860, 314)), ((700, 520), (1102, 854)),
                    ((180, 880), (390, 1304))], model="similarity", target=pp)
print(f"RMS {fit.rms_um:.1f} µm, leave-one-out {fit.loo_um:.1f} µm")
for row in qem.compare(d):
    print(row["phase"], round(row["map"], 1), row["count"])
qem.composite("TS-9_composite.png", base=pp, opacity=0.5)
```

Limits and common problems

- "No mineral map on this thin section. Import one with Mineral map..."
- "N pairs: at least ... are needed."
- "Not registered and without scale: register it on the optical image."
- An RGB map with colours far from the legend: raise the colour tolerance, or give a legend with the exact colours.

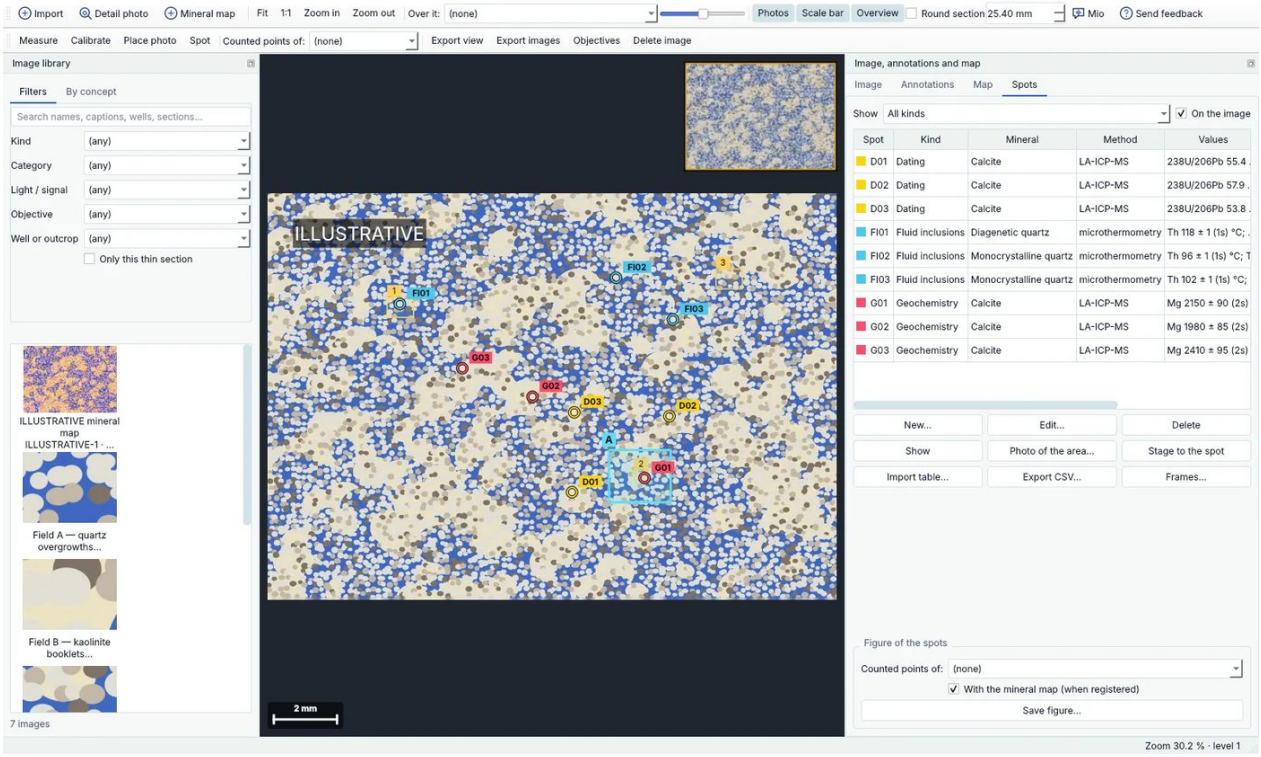
Coverage

GT2 (QEMSCAN maps), GT5 (registration with landmark pairs), L9 (overlay of the optical image and the mineralogical map → composite image).

Spot analyses: geochemistry, dating and fluid inclusions

What it is for

A **spot analysis** is a measurement made at one point of the thin section: a laser-ablation or microprobe analysis, a U-Pb date, a fluid-inclusion microthermometry. RockFace Atlas keeps each spot at its place in the section frame, with its values and uncertainties, the mineral analysed and a photo of the area, so the spot is drawn at the same place on every image of the section, can be found again under the instrument, and is reached from a query or a figure.



In the program: Spot analyses: geochemistry, dating and fluid inclusions

Before you start

- A saved description whose thin section is in Atlas, with a calibrated image.
- For laboratory tables: a CSV file with a spot name, x and y, and the value columns.

Step by step

Adding a spot by hand

1. In RockFace Atlas, choose the tool **Spot** and click where the analysis was made — or **New...** in the **Spots** tab.
2. In **Spot analysis**, give the **Kind** (geochemistry, dating, fluid inclusion, other), the **Spot** name (given in order — G01, D01, FI01... — when left empty), the **Position (x, y)**, the **Spot size**, **Method**, **Laboratory**, **Date** and **Analyst**.
3. The **Mineral** analysed and its **Location**, from the nomenclature.
4. **Values**: **Add value** for each — **Name**, **Value**, **±**, **Level** (1s, 2s), **Unit**, **< DL** (below the detection limit). **Usual fields of the kind** fills in the usual names (U-Pb ratios, Th and Tm ice...).
5. **Metadata** (**Add metadata**) for anything else; **Photo of the area** links a detail photo; **Notes**. **OK**.

Importing a laboratory table

1. In the **Spots** tab, **Import table...**.
2. Choose the CSV and the **Coordinates in the table**: section frame (mm or µm), pixels of a calibrated image, or an instrument frame (below).
3. **Kind** (**when the table has none**).
4. RockFace reads the rows ("N rows; M with errors will be left out") and shows each **Line** with its **Problems**. Value columns such as `Sr (ppm)` take their unit from the header; the uncertainty follows in a column `±`, `2s`, `err` or named `Sr 2s`; `<0.5` is below the detection limit; text columns become metadata.
5. **OK**.

Instrument frames: finding a spot again

1. **Frames...** opens **Instrument frames of the section**. Give the **Instrument** name and **Its unit**.
2. Read the same marks — fiducials, scratches, grain corners — on the instrument (**Instrument x, y**, or **Read the stage**) and on the image of the section (**Section x, y (mm)**). **Add mark** for each; two marks are enough for a similarity, three for an affine transform, a fourth checks the others.
3. **Fit**: the RMS of the marks and the leave-one-out error.
4. Then **Stage to the spot** sends the stage to a selected spot, in the instrument's own coordinates. Calibrate again after the slide is put back in the holder.

Looking at the spots

- The **Spots** tab lists them by kind (**Show**: **All kinds** or one), with their values. **Show** centres the viewer on a spot; **On the image** draws them; **Edit...**, **Delete**.
- **Photo of the area...** adds or opens the photo of the spot's area.
- **Export CSV...** — one row per spot, one column per value with its uncertainty.
- **Figure of the spots** — the section with the spots labelled, the **Counted points of:** a description, and **With the mineral map** (**when registered**); **Save figure...**.

Options

Field	What it is
Level	The uncertainty's level: 1s or 2s.
< DL	Below the detection limit (the value is the limit).
Coordinates in the table	mm, µm, pixels of an image, or an instrument frame.

How it works

Spots are stored in millimetres of the section frame, never only in the instrument's coordinates: an instrument frame is a transform (similarity or affine) fitted on marks, so the same spot is found on any instrument that has its frame. Some values are derived and shown (never stored), each with its source: the salinity of a fluid inclusion from Tm ice (Bodnar 1993) and the $^{206}Pb/^{238}U$ age from $^{238}U/^{206}Pb$ (Jaffey et al. 1971 decay constant, no common-Pb correction). U-Pb spots are dated together in RockFace Explorer (page U-Pb dating on the Tera-Wasserburg diagram).

From a script

```
d = prj["TS-9"]
d.calibrate_frame("LA-ICP-MS stage", [((41250, 18630), (1.0, 1.0)), ((57240, 18965), (17.0, 1.0)),
                                      ((41020, 29380), (1.0, 11.75))], unit="um")
d.import_spots("TS-9_U-Pb.csv", coords="frame", frame="LA-ICP-MS stage", kind="dating")
fi = d.add_spot("fluid_inclusion", 4.184, 3.488, values=[rf.Value("Th", 118.5, 1.0, "°C", "1s"),
                                                         rf.Value("Tm ice", -12.3, 0.2, "°C", "1s")],
                method="microthermometry", constituent="quartz_dia")
print(rf.spots.derived(fi))                              # salinity 16.24 wt% NaCl eq. (Bodnar 1993)
print(d.to_frame("LA-ICP-MS stage", fi.x_mm, fi.y_mm))   # where it is on the instrument
d.spot_figure("TS-9_spots.png")
```

Limits and common problems

- "No instrument frame for this section: add one with Frames... (fiducial marks)."
- A name already used in the section is refused; implausible values (a negative ppm, rho outside −1...1) only warn.
- The derived U-Pb age has no common-lead correction: use the dating of Explorer for ages.

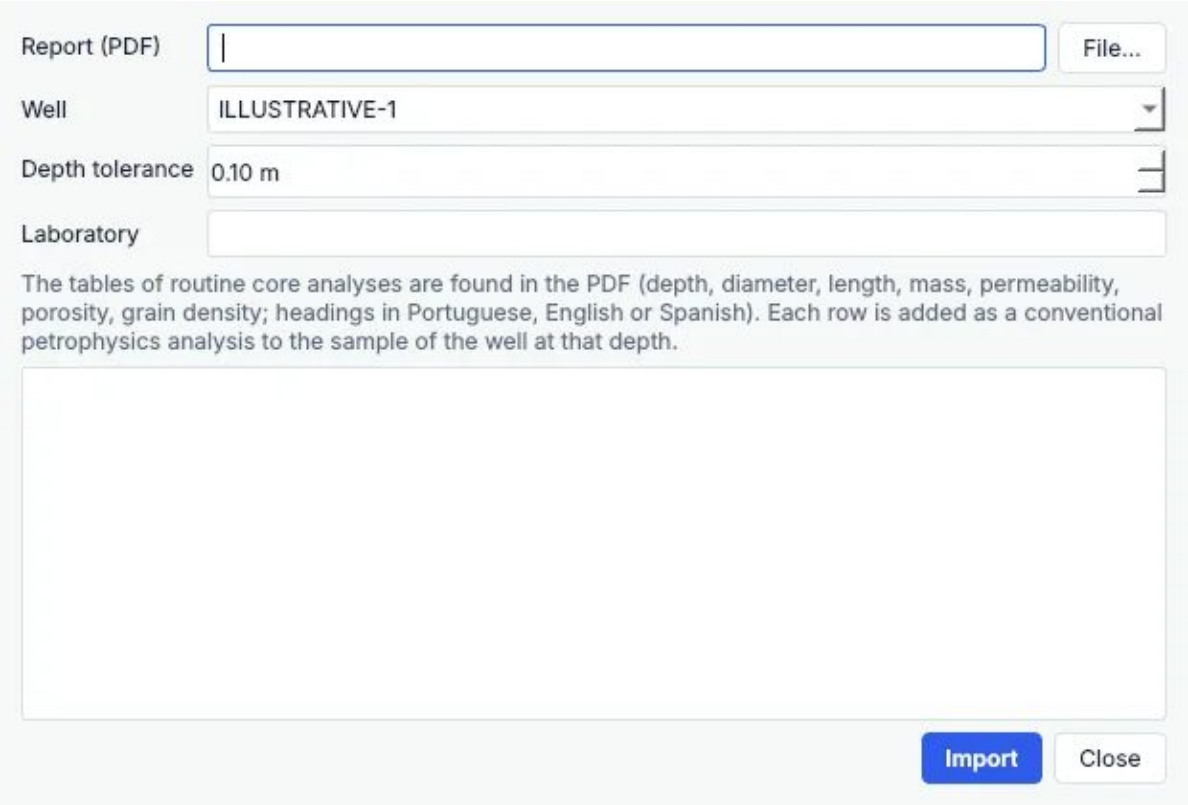
Coverage

L3 (links to spot analyses with photos of the analysed areas), L12 in part (spot analyses with detail images and dating).

Files of the reference programs

What it is for

RockFace opens the files that GeoSlicer and the laboratories hand over, so nothing has to be converted first: micro-CT volumes from Amira/Avizo, HDF5, DICOM series and TESCAN reconstructions; Zeiss CZI thin-section scans; GeoSlicer scenes and segmentations; the tables of laboratory reports in PDF; and the scale bar burnt into a microscope picture.



In the program: Files of the reference programs

Before you start

- A project with a sample (volumes) or a thin section (images).
- Optional packages, installed once: `pip install rockface[formats]` (HDF5, DICOM, PDF, OCR) and, for CZI, `pip install rockface[czi]`. OCR also needs the Tesseract program.

Step by step

1. **Volumes — MC → Import.** Choose the file or folder: `.am`, `.h5`/`.hdf5`, a `.dcm` file or a DICOM folder, a TESCAN folder (with `*.acq.1.xml` and `recon/`), or a GeoSlicer scene `.mrml` or segmentation `.seg.nrrd`. Click **Import**.
2. **Volumes out — MC → Volume → Export** now also writes `.am`, `.h5` and, for a segmentation, `.seg.nrrd`.
3. **CZI — Atlas → Import** a `.czi` like any slide.
4. **Lab report — Petro → Home → Import lab report (PDF)...**: choose the PDF, the well and the depth tolerance; each row becomes a conventional petrophysics analysis of the sample at that depth.
5. **Scale bar — Atlas → image properties → Read scale bar**: the bar is found and its length read; if the text cannot be read, type the length.

Options

Format	What RockFace reads
Amira `.am`	binary (little or big endian) or ASCII; byte to double; raw, HxByteRLE or HxZip; voxel from BoundingBox and Units
HDF5	any 3D dataset (the largest by default); voxel from `voxel_um`, `spacing` + `units`, `element_size_um`
DICOM	the largest series of a folder, slices ordered along their normal, Hounsfield units; no patient fields kept
TESCAN	the slices of `recon/`, voxel = VoxelSize (mm) of `*.rec.1.xml`, origin from `*.acq.1.xml`
`_NNNNNnm` in a name	voxel size in nanometres when the file gives none
CZI	the first scene, RGB or grey, µm per pixel from the scaling
`.mrml`, `.seg.nrrd`	volumes, label maps and segmentations (3D or layered) with their segment names and colours
Lab report PDF	depth, diameter, length, mass, permeability, porosity, grain density (PT, EN, ES headings; comma or point)

How it works

- Each reader turns the file into a volume source read slab by slab, so large files stay out of memory; the voxel size comes from the file, else from the name, else from the import form.
- Lab reports: ruled tables are read by their lines; tables of aligned text by the columns of the numbers; the headings are matched by their words (*Profundidade*, *Permeabilidade*, *Massa específica*...).
- Scale bar: the longest solid horizontal bar that contrasts with its surroundings, then the text next to it read by Tesseract ("500 µm", "0.5 mm"); µm per pixel = length ÷ bar pixels.

From a script

```
src = rf.volume.open_volume("scan.am")                          # or .h5, a DICOM or TESCAN folder
v = rf.volume.add_volume(prj.raw, src, sample_id)
rf.volume.export(prj.raw, v.id, "out.seg.nrrd")                 # a labels volume for GeoSlicer
rf.geoslicer.import_scene(prj.raw, "project.mrml", sample_id)
prj.import_lab_report("rca.pdf", "ILL-WELL-1", tolerance_m=0.1)
rf.atlas.read_scale_bar(prj.raw, image_id)                    # Scale(bar, text, value_um, um_per_px)
```

Limits and common problems

- CZI needs pylibCZIrw (LGPL-3.0, optional, like OpenSlide for other slide formats).
- A lab report whose table is an image (a scan) has no text: run OCR on the PDF first.
- Without units, an Amira bounding box is guessed from the voxel size (metres, millimetres or micrometres); the guess is written in the volume's history.

Coverage

GV1 and GT1 formats (0.28), GC1 DICOM, GI lab-report tables, GT scale bar; GeoSlicer audit of 0.28.

From the 2D composite to the 3D composite, and publishing

What it is for

The mineral map of a thin section says what every grain is; the micro-CT of the plug shows the grains in 3D but not what they are. **3D composite** carries the map's phases into the whole volume, and gives each phase its properties. **Publication** sends the project — samples, descriptions, analyses, images — to the corporate data environment on a schedule, only what changed.

Before you start

- A micro-CT volume of the sample and a mineral map (or a segmentation) of its thin section.
- The map located in the volume: **MC → Register → Locate the image** (or both images calibrated in the section frame when another image of the same section was located).
- For publishing: a folder watched by the platform's ingestion, or the address of an OSDU platform and a token.

Step by step

1. Locate the map (or the optical picture) in the volume; save the placement.
2. In a script or the script window: `prj.composite_3d(map, volume)` — the map's phases on the slice train a model on the grey volume, a quarter of the slice is held out to measure the agreement, and the whole volume is classified.
3. Give the phases their properties: `rf.composite3d.property_volume(...)` makes a float volume per property.
4. Publishing: `rockface publish project.rockface --folder <path> --period 24` (or `--url <platform>`), then `rockface publish project.rockface --if-due` from the system's scheduler (`--schedule` prints the lines).

Options

Setting	Meaning
hold_out	share of the slice kept out of training to measure the agreement (default 0.25)
method	random forest or Gaussian Bayesian model
target	`folder` (manifest files) or `rafs` (OSDU Storage + RAFS DDMS)
period_hours	how often `--if-due` publishes
token_env	the environment variable with the access token (ROCKFACE_OSDU_TOKEN); never stored

Setting	Meaning
analysis_type, content_version	RAFS content type and schema version of the routine core analyses

How it works

- The map is sampled at every voxel of the placed slice through the placement matrix; background phases do not train. The model is the random forest of MC with the features of the grey volume.
- Publication builds the OSDU manifest of the project, compares each record with the SHA-256 of what was sent, and sends only the changed ones: to a folder (a numbered JSON written atomically) or to the platform — wells and wellbores by `PUT /api/storage/v2/records`, samples by `POST /api/rafs-ddms/v2/masterdata`, descriptions and analyses by `POST /api/rafs-ddms/v2/samplesanalysis`, and the routine core analyses as content data (`/samplesanalysis/{id}/data/{type}`), with the headers `Authorization` and `data-partition-id`.

From a script

```
c = prj.composite_3d(map_img, vol)                    # Composite3D(labels_id,
↪  held_out={'accuracy': …}, …)
pv = rf.composite3d.property_volume(prj.raw, c.labels_id, {"Calcite": 2.71, "Quartz": 2.65},
↪  "Grain density",
                                    "g/cm³")
prj.publication(target="folder", folder="//share/osdu-inbox", period_hours=24)
prj.publish()                                         # {'sent': …, 'unchanged': …, 'file': …}
```

Limits and common problems

- The 3D composite is as good as the contrast of the phases in the CT: phases with the same attenuation (calcite and dolomite at low energy) are told apart only with a second volume (another energy) as an extra channel.
- Check the RAFS content schema of the platform (`GET /api/rafs-ddms/v2/samplesanalysis/{type}/data/sch` before the first publication; the petrography travels in ExtensionProperties until RAFS has a schema for it.

Coverage

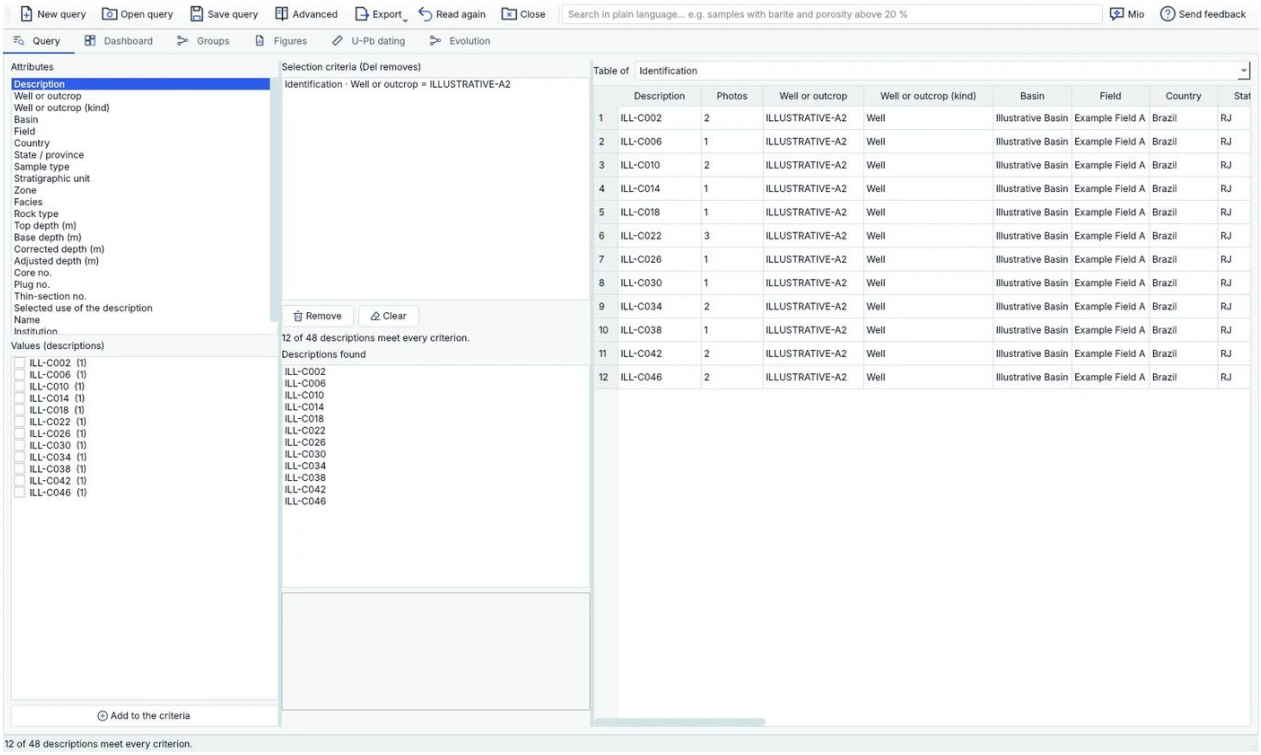
L13 (2D composite → tomography → 3D composite with properties; containers, model hub, periodic incorporation, OSDU RAFS DDMS).

RockFace Explorer

RockFace Explorer: queries, result tables and exports

What it is for

RockFace Explorer asks questions of a whole collection: *which samples have barite?*, *which samples of well X have U-Pb dating?*, *the grainstones with porosity above 15 % and permeability above 100 mD*. A query can use any concept of a description — identification, texture, composition with its attributes, pore system, petrophysics, analyses, classifications, totals, the diagenetic interpretation, images, spot analyses, mineral maps — and gives the descriptions found, a table of any concept for them with their photos, and exports.



In the program: RockFace Explorer: queries, result tables and exports

Before you start

- A project with descriptions. To try Explorer: Help → **Open the ILLUSTRATIVE collection** (48 descriptions).
- A question can also be typed in plain language in the search box at the top (page Mio: questions in plain language and the local language model).

Step by step

Opening Explorer

Home → Collection → Explorer (Ctrl+Shift+E), or `rockface explorer project.rockface`. The **Query** tab opens.

A simple query

1. **Attributes**: an attribute of the identification (well, field, formation, zone, facies, rock type, depths, sample type, uses…).
2. **Values (descriptions)** lists its values with the number of descriptions that have each. Tick or select one or more.
3. For a number, give the **Numeric range**: **From** and **To** (either end may stay empty); "Found: … to …" shows the range in the collection.
4. **Add to the criteria**. The criterion appears in **Selection criteria (Del removes)**.
5. Add more criteria: a description must meet all of them. "N of M descriptions meet every criterion."

An advanced query

1. **Advanced** shows the **Concepts**: Identification, Macroscopic and Microscopic texture, Primary composition, Diagenetic composition, Pores in the composition, Pore system, Petrophysics, Laboratory analyses, Classifications, Totals, Diagenetic interpretation, Images, Spot analyses, Mineral maps.
2. Choose a concept, then an attribute and its values or range, and **Add to the criteria**.
3. Within one concept, the criteria must be met by **the same row**: "diagenetic constituent = calcite" and "habit = poikilotopic" find the samples with poikilotopic calcite, not those with some calcite and something poikilotopic.

The result table

1. **Descriptions found** lists them; pick some to see only them in the table and to export them (none picked: all).
2. **Table of**: the concept to show — for example Petrophysics, or Diagenetic composition with each line's share. **Every row** shows all the rows of the descriptions found, not only those that meet the criteria.
3. The **Photos** column gives the number of images of each thin section; select a row to see the photo, double-click it to open the description in RockFace Petro.
4. Click a column header to sort.

Saving and exporting

- **Save query…** (give it a name), **Open query…**, **New query**; the **Saved queries** dialog also **Load**s and **Delete**s them. Queries are kept in the project.
- **Export**:
  - **Result table to Excel or CSV…** — numbers stay numbers;

  - **Descriptions found (HTML, Excel, PDF, Word)...** — through the report dialog (page Reports);
  - **Descriptions selected in the table to HTML...** or **to XML...** (an exchange file, page Exporting and importing descriptions);
  - **Wells to LAS (Petrel)...** and **Presentation of the query (.pptx)...** (page Groups, figures and presentations).

- **Read again** reloads the descriptions after changes made in Petro.

Options

Option	What it changes
Advanced	Every concept instead of the identification only.
Every row	All rows of the descriptions found, not only the rows that meet the criteria.
Descriptions found (picked)	Limit the table and the exports to them.

How it works

Every description is read into *facts*, by concept: one row per composition line, per analysis, per spot, per map phase... A description meets the query when, for each concept named, one and the same row meets all the criteria of that concept, and every concept is met. Facts are read in about 8 ms per description and a query is answered in about 1 ms on 480 descriptions. Templates are left out.

From a script

```
with rf.open("collection.rockface") as prj:
    print([d.name for d in prj.query(("diagenetic", "constituent", "barite"))])          # which
    ↪ samples have barite?
    print([d.name for d in prj.query(("spots", "system", "U-Pb"), well="ILLUSTRATIVE-A2")])   #
    ↪ dated, in well A2
    good = prj.query(("petrophysics", "porosity", 15, None), ("petrophysics", "permeability", 100,
↪   None))
    prj.save_query("Good reservoir", ("petrophysics", "porosity", 15, None))
    header, rows = prj.query_table("petrophysics", facies="Grainstone")
    prj.export_table("good.xlsx", "petrophysics", ("petrophysics", "porosity", 15, None))
```

Limits and common problems

- “Tick or select at least one value.” / “Type numbers for the range (either end may stay empty).”
- “The query has no criteria yet.” / “Give the query a name.”
- “Nothing to export: no description found or selected.”

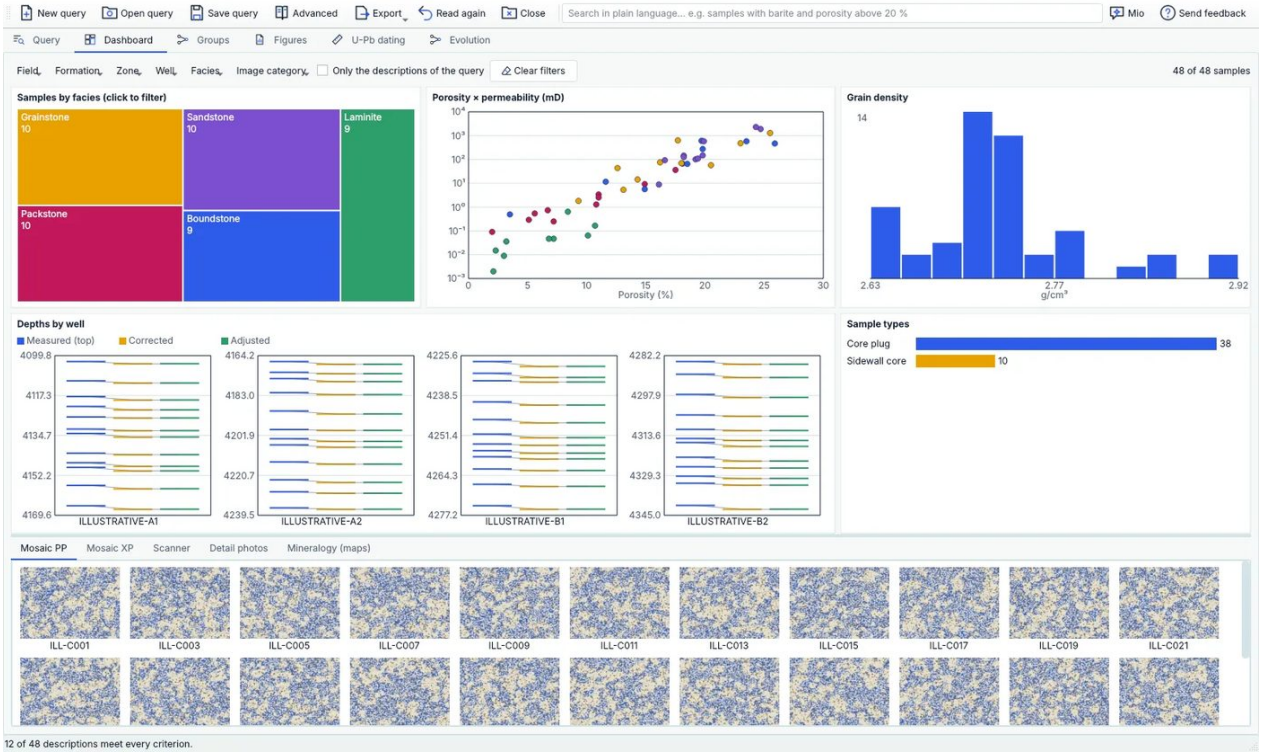
Coverage

Q1 (simple query), Q2 (advanced query on every concept), Q3 (numeric ranges), Q4 (criteria list), Q5 (result table), Q6 (saved queries), Q7 (export to Excel), Q8 (descriptions found to HTML, Excel and PDF), Q9 (selected descriptions to HTML and XML), Q10 (export configuration), Q11 (photos column), L1 (search: “which samples have barite?”).

RockFace Explorer: the dashboard of a collection

What it is for

The **Dashboard** tab shows a collection at a glance: the samples by facies, porosity against permeability, the grain density, the depths sampled in each well, the sample types, and galleries of the images (PP and XP mosaics, scanner slides, detail photos, mineral maps). Filters by field, formation, zone, well, facies and image category narrow every chart at once.



In the program: RockFace Explorer: the dashboard of a collection

Before you start

A project with descriptions, ideally with petrophysics (Analyses tab, page Analyses and analysis requests) and images (RockFace Atlas).

Step by step

1. In RockFace Explorer, open the **Dashboard** tab.
2. The charts:

- **Samples by facies (click to filter)** — a treemap; click a facies to keep only it;
- **Porosity × permeability (mD)** — one point per sample, coloured by facies, permeability on a log axis;
- **Grain density** — a histogram;
- **Depths by well** — the **Measured (top)**, **Corrected** and **Adjusted** depths of the samples of each well;
- **Sample types** — plugs, whole cores, sidewall cores, cuttings, outcrop samples.

3. **Filters**: tick values of **Field**, **Formation**, **Zone**, **Well**, **Facies** and **Image category**; every chart and gallery follows. "N of M samples". **Clear filters** removes them.
4. **Only the descriptions of the query** limits the dashboard to the descriptions found in the Query tab.
5. **Galleries**: **Mosaic PP**, **Mosaic XP**, **Scanner**, **Detail photos**, **Mineralogy (maps)**. Double-click an image to open it in RockFace Atlas.

Options

Option	What it changes
Filters	Every chart and gallery shows only the samples that match.
Only the descriptions of the query	The dashboard of the query's result.

How it works

The dashboard reads the same facts as the queries: petrophysics comes from the conventional-petrophysics analyses of each sample, the facies, field, formation, zone and well from the identification, the galleries from the Atlas images of each thin section (by category and light). The treemap is squarified (each rectangle's area is the number of samples).

From a script

```
board = prj.dashboard(field="Example Field A")
print(board.facies())                    # samples by facies
for phi, k, facies, did in board.phi_k():
    print(phi, k, facies)
print(board.grain_density(10), board.depths(), board.sample_types())
print(board.gallery("mosaic_pp"))        # image ids
```

Limits and common problems

- "No data for the descriptions shown." — no petrophysics or no images among the samples filtered.
- A sample without facies is shown under "—".

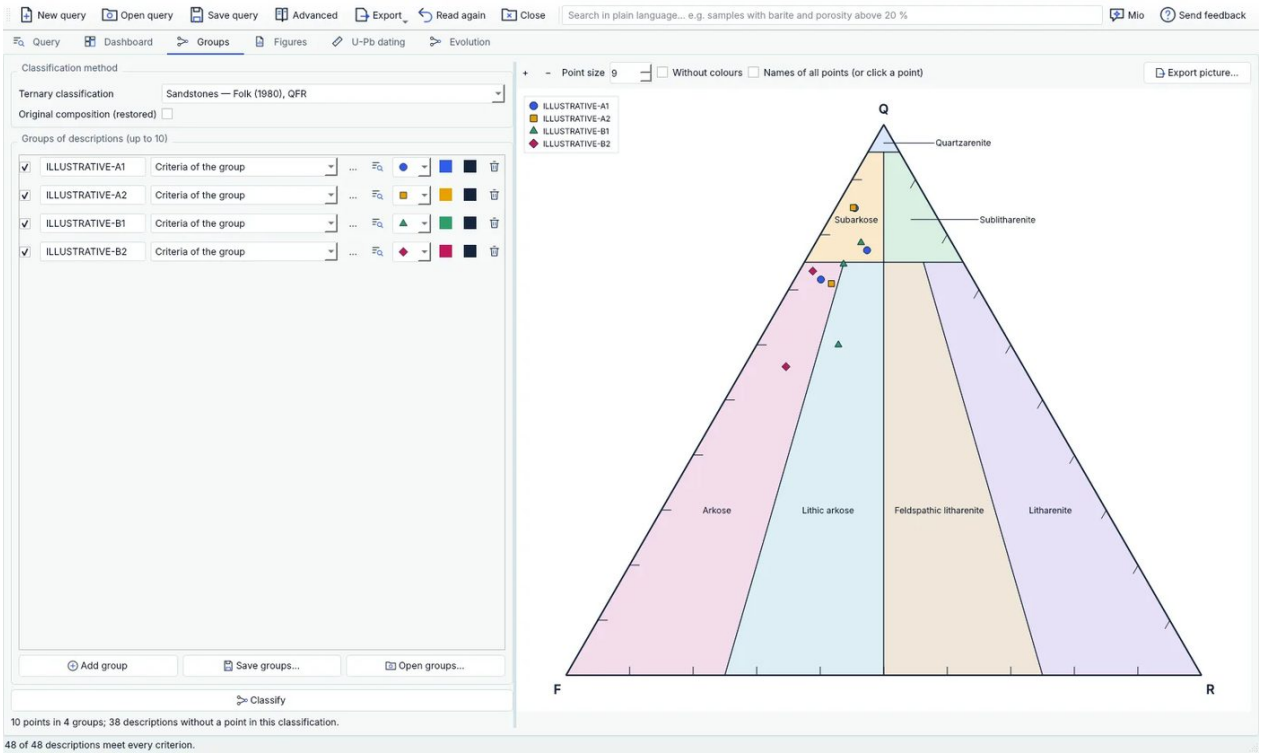
Coverage

L1 (dashboards), Q2 (concepts of petrophysics in the collection).

Groups, figures and presentations

What it is for

RockFace Explorer draws several **groups** of samples together on a ternary classification — the samples of each well on Folk's QFR, each facies on Dickinson's QtFL — with their markers and colours. Its **figure generator** makes publication figures from any attribute of the collection (crossplots, histograms, bars, box plots, curves against depth by well, ternary diagrams), sized for a journal, and exports them also as a SciVelyx Graphs project. A **presentation** gathers figures, tables and image galleries into a PowerPoint file.



In the program: Groups, figures and presentations

Before you start

A project with descriptions; saved queries help (page RockFace Explorer: queries, result tables and exports).

Step by step

Groups on a ternary classification

1. Open the **Groups** tab of Explorer.
2. **Classification method**: the **Ternary classification** (Folk 1980, McBride 1963, Dickinson QtFL and QmFLt, or one of the project) and, if wanted, **Original composition (restored)**.
3. **Add group** (up to ten). For each group, in **Groups of descriptions**:
   - the descriptions: **Query of the Query tab**, **Criteria of the group**, or a saved query; **Picked by hand** adds others ("Those found by the query are fixed; tick others to add them by hand");
   - **Point marker**, **Fill colour**, **Border colour**; **Show the group**; **Remove the group**.
4. **Classify** draws every group, with the legend. "N points in G groups; M descriptions without a point in this classification."
5. In the diagram: **Point size**, **Without colours**, **Names of all points (or click a point)**, **Zoom in**, **Zoom out**; the hover box gives the name and the vertex values; "Field under the pointer: …".
6. **Export picture...**; **Save groups...** and **Open groups...** keep the sets in the project.

A figure

1. Open the **Figures** tab.
2. **Kind**: **Crossplot**, **Histogram**, **Bars (number of samples)**, **Box plot by category**, **Against depth, by well** or **Ternary with groups**.
3. **Attribute (X)** and **Attribute (Y)** — any attribute of any concept (for example Petrophysics › Porosity, Totals › Qt); **Colour by** an attribute or **Groups (Groups tab)**; **Bins** for histograms; **Depth** (measured, corrected, adjusted) for curves against depth.
4. **Logarithmic X**, **Logarithmic Y**, **Names of the samples**, **Legend**, **Only the descriptions of the query**.
5. **Title**; **Journal size** (Nature, Elsevier, AAPG, ACS one or two columns, a 16:9 slide) or **Free size** with **Width (mm)**, **Height (mm)** and **Font size (pt)**.
6. **Export...** writes PNG, TIFF, JPEG, SVG, PDF, the plotted data as CSV, or a SciVelyx Graphs project (`.scivelyx.json`) to finish the figure there.
7. **Save figure...** keeps it in the project; **Open figure...** draws a saved one again from the latest records.

A presentation

1. Under **Presentation**, add slides: **+ Figure** (the figure shown), **+ Table** (the result table of a concept), **+ Images** (a gallery of the thin sections).
2. **Write presentation (.pptx)...**. The deck is 16:9, opens with a title slide and the query's criteria.
3. Or, in the Query tab, **Export → Presentation of the query (.pptx)...** writes one at once.

Options

Option	What it changes
Original composition (restored)	The groups are placed with the composition before diagenesis.
Journal size	Width, height and fonts of the journal's column (or of a slide).
Where a description has several rows	The mean is used, except when X and Y are of the same concept (one point per row).

How it works

Groups and figures are recomputed from the saved descriptions every time they are drawn, so a saved figure always shows the latest data. Figures are drawn with Matplotlib at 600 dpi by default; the SciVelyx Graphs project carries the same data and styles, so the figure can be finished in SciVelyx Graphs.

From a script

```
wells = [rf.Group(w, rf.Query([rf.Criterion("identification", "well", [w])])) for w in
↪  ("ILLUSTRATIVE-A1", "ILLUSTRATIVE-A2")]
prj.figure("qfr.pdf", kind="ternary", scheme="folk1980", groups=wells, colour_by="groups")
phik = prj.figure("phi_k.png", kind="crossplot", x="petrophysics.porosity",
↪  y="petrophysics.permeability",
                  log_y=True, colour_by="identification.facies", preset="elsevier-1col")
prj.figure("phi_k.scivelyx.json", phik)                    # open it in SciVelyx Graphs
prj.save_figure("Porosity and permeability", phik)
prj.presentation("barite.pptx", ("diagenetic", "constituent", "barite"), figures_=[phik])
```

Limits and common problems

- "At most 10 groups."
- "No figures saved in this project yet." / "No groups saved in this project yet."
- "Add slides first." — a presentation needs at least one slide.
- PowerPoint presentations need the `explorer` extra (python-pptx), installed with `.[all]`.

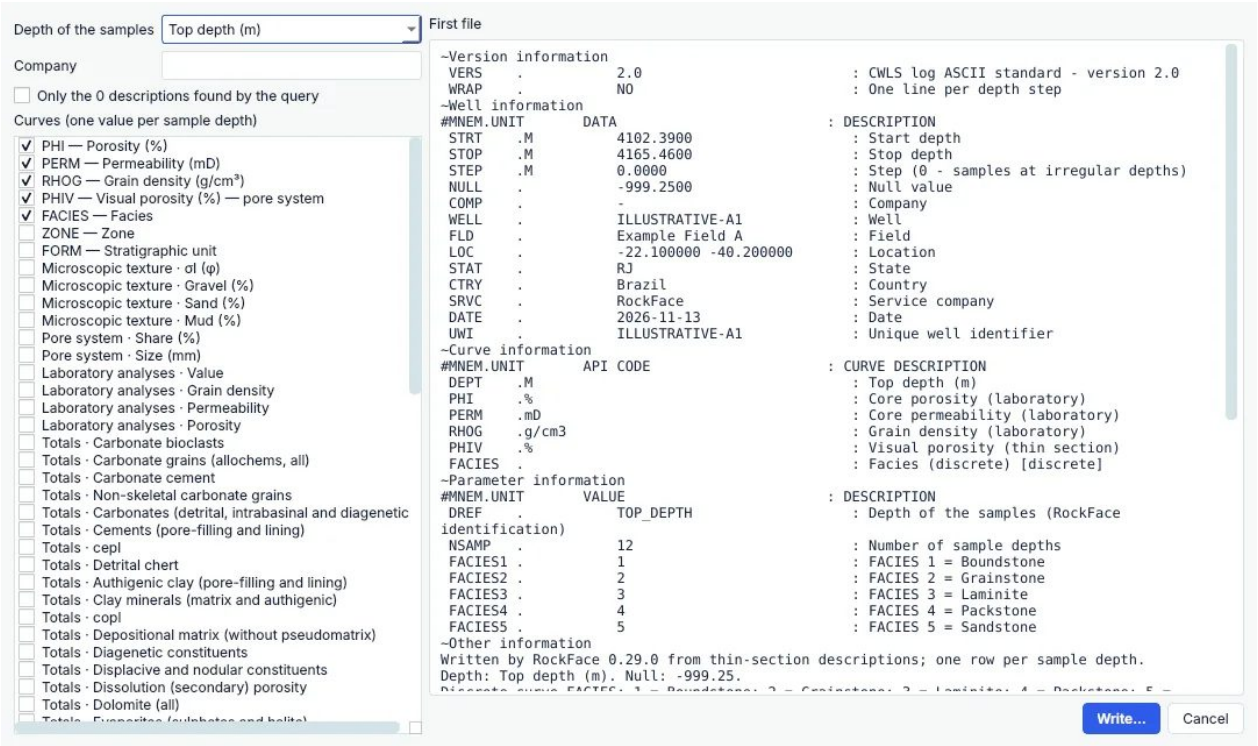
Coverage

M1 (up to 10 groups from saved queries), M2 (marker, fill and border colour per group), M3 (classification methods recomputed from the latest data), M4 (legend, hover box, labels, zoom, export), L5 (figure generator and presentations).

Wells to LAS

What it is for

The descriptions of a well, written as a **LAS 2.0** file — one per well — so that reservoir-modelling programs such as Petrel, Techlog or Geolog read the petrography as curves against depth: porosity and permeability, totals, classifications, facies… Text attributes become discrete curves with their code tables.



In the program: Wells to LAS

Before you start

- Descriptions with a well and a depth.
- Optional: a query in Explorer, to write only the descriptions found.

Step by step

1. In RockFace Explorer, **Export → Wells to LAS (Petrel)…**. The **Wells to LAS 2.0** dialog opens.
2. **Depth of the samples**: measured (top), corrected or adjusted.
3. **Company** — written in the header.
4. **Only the N descriptions found by the query**, or every description.

5. **Curves (one value per sample depth)**: tick the attributes to write (porosity, permeability, grain density, totals, classifications, facies, zone…). Each has its mnemonic and unit.
6. **First file** shows the beginning of the first file, to check.
7. **Write...** and choose the folder. "N LAS files written in …".

Options

Option	What it changes
Depth of the samples	Which depth of the identification is the DEPT curve.
Curves	Numeric curves, or discrete curves for text attributes (integer codes with their table).

How it works

- One LAS 2.0 file per well, with an irregular depth step (`STEP 0`) and `NULL -999.25`.
- Text attributes (facies, a classification) become discrete curves of integer codes; the code table is written in the Parameter and Other sections.
- Two samples at the same depth are merged.
- The files were checked with two independent readers (lasio and W. Weckesser's LASReader): every value equals the record, the required header items are present and lasio gives no warning.

From a script

```
prj.export_las("las", ("identification", "formation", "Example Carbonate Unit"),
               curves=["petrophysics.porosity", "petrophysics.permeability", rf.Curve("totals.qt",
↪  "QT", "%"),
                       "identification.facies"],
               depth="corrected_depth", company="ILLUSTRATIVE")
las = rf.las.read("las/ILLUSTRATIVE-A1.las")            # read a LAS 2.0 file back
```

Limits and common problems

- "No sample with this depth." — the depth chosen is empty for these samples (for example no adjusted depth).
- Samples without a well are left out.
- For image logs and curves of RockFace MC, see RockFace MC: image logs (LAS, CSV, DLIS, NetCDF) and the WITSML export (page Exporting wells to WITSML and samples to OSDU).

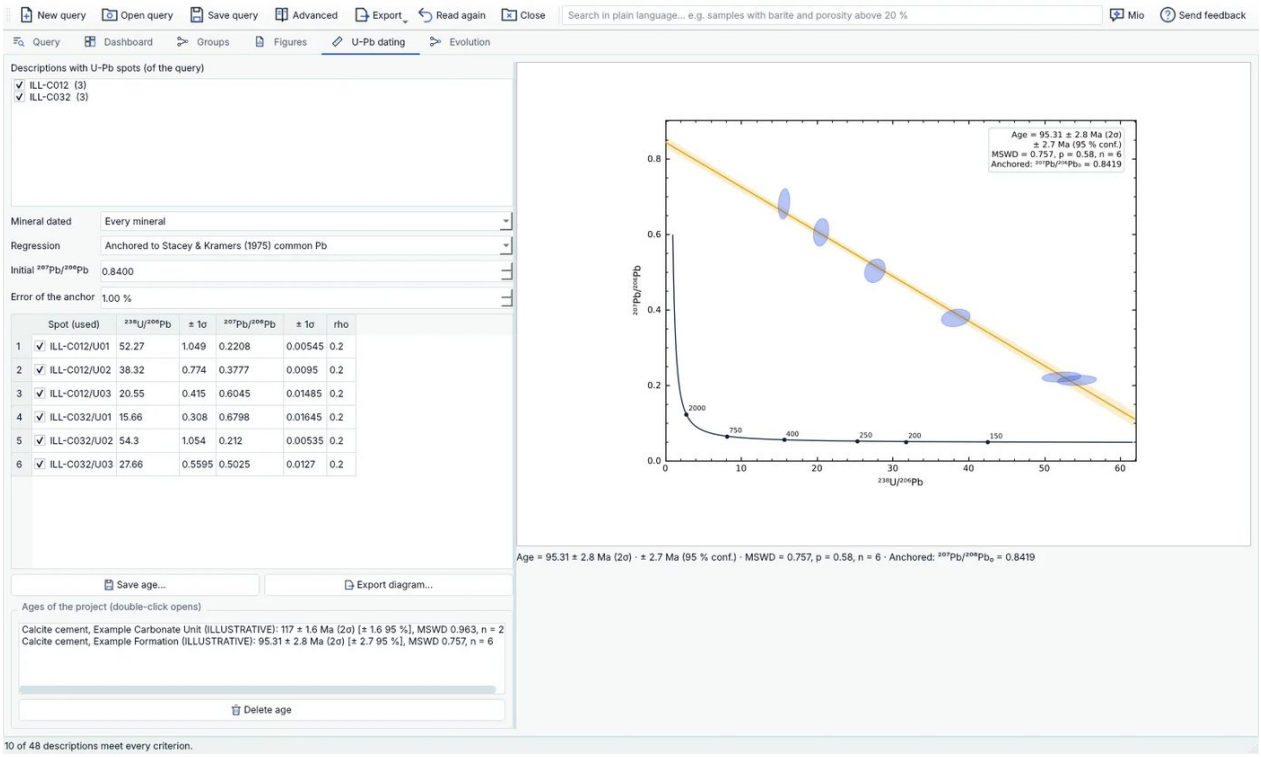
Coverage

P38 (export to LAS for reservoir-modelling software), L4 (LAS exporter to Petrel).

U-Pb dating on the Tera-Wasserburg diagram

What it is for

The U-Pb spots of a diagenetic mineral (a calcite cement, a dolomite) measured on several samples are dated together: RockFace Explorer plots them on the **Tera-Wasserburg diagram**, fits the York regression line, and gives the age at its lower intercept with the concordia, with its uncertainty, the MSWD and the probability of fit. The age is kept in the project with the phase it dates, and placed in the paragenetic evolution and in the stratigraphic column.



In the program: U-Pb dating on the Tera-Wasserburg diagram

Before you start

- U-Pb spot analyses on the thin sections (page Spot analyses: geochemistry, dating and fluid inclusions): kind *dating* (or system "U-Pb") with $^{238}U/^{206}Pb$ and $^{207}Pb/^{206}Pb$, their uncertainties and the error correlation (rho).
- A query in Explorer that finds the samples to date together (for example one formation).

Step by step

1. In RockFace Explorer, open the **U-Pb dating** tab.
2. **Descriptions with U-Pb spots (of the query)** lists the samples found that have U-Pb spots.
3. **Mineral dated**: the constituent (or group) whose spots are used, or **Every mineral**.
4. **Regression**:
   - **Free (York)** — the line through the points;
   - **Anchored to Stacey & Kramers (1975) common Pb** — the initial $^{207}Pb/^{206}Pb$ from the model at the age found;
   - **Anchored to a given initial $^{207}Pb/^{206}Pb$** — type the **Initial $^{207}Pb/^{206}Pb$** and the **Error of the anchor**.
5. The table lists every spot; untick **Spot (used)** to leave a spot out of the regression (it stays on the diagram, marked).
6. The diagram shows the points with their error ellipses, the concordia with its ages, the regression line and the result: "Age = ... ± ... Ma (2σ)", "± ... Ma (95 % conf.)", the MSWD, n, and the initial $^{207}Pb/^{206}Pb$.
7. **Export diagram...** saves it (PNG, SVG, PDF).
8. **Save age...**: give it a name; it is kept with the phase it dates (mineral and habit). **Ages of the project (double-click opens)** lists the saved ages; **Delete age**.

Options

Option	What it changes
Mineral dated	Which spots are used.
Regression	Free, anchored on Stacey & Kramers common lead, or anchored on a given value with its error.
Spot (used)	Leaves an outlier out of the fit.

How it works

- York regression with correlated errors in both coordinates (York et al. 2004); the age is the lower intercept of the line with the Tera-Wasserburg concordia (decay constants of Jaffey et al. 1971, $^{238}U/^{235}U$ of Hiess et al. 2012). The anchored fit forces the line through the initial $^{207}Pb/^{206}Pb$ (Stacey & Kramers 1975, or the value given).
- The uncertainty is given as 2σ (analytical) and as the 95 % interval: $t(n-2) \times \sigma \times \sqrt{MSWD}$ when MSWD > 1, else 1.96σ (Ludwig 2003); decay-constant errors are not included. MSWD $= \chi^2/(n-2)$ (Wendt & Carl 1991), with the probability of fit.
- Checked against published values: Pearson's data with York's weights give the published slope, intercept and errors; on 800 synthetic data sets the 95 % interval holds the true age 97.2 % (free) and 97.5 % (anchored) of the time.

From a script

```
d = prj.date(formation="Example Formation", constituent="calcite_dia", anchor="stacey_kramers",
↪  figure="tw.pdf")
print(f"{d.age:.1f} ± {d.age_2s:.1f} Ma (2σ), MSWD {d.mswd:.2f}, n = {d.n}")
prj.save_age("Sandstone calcite", ("identification", "formation", "Example Formation"),
             constituent="calcite_dia", habit="poikilotopic", anchor="stacey_kramers")
print(prj.ages())
```

Limits and common problems

- “No age: …” — too few spots, or “The line does not cut the concordia.”
- “No age to save: date some spots first.”
- Spots without rho are taken as uncorrelated (rho = 0).

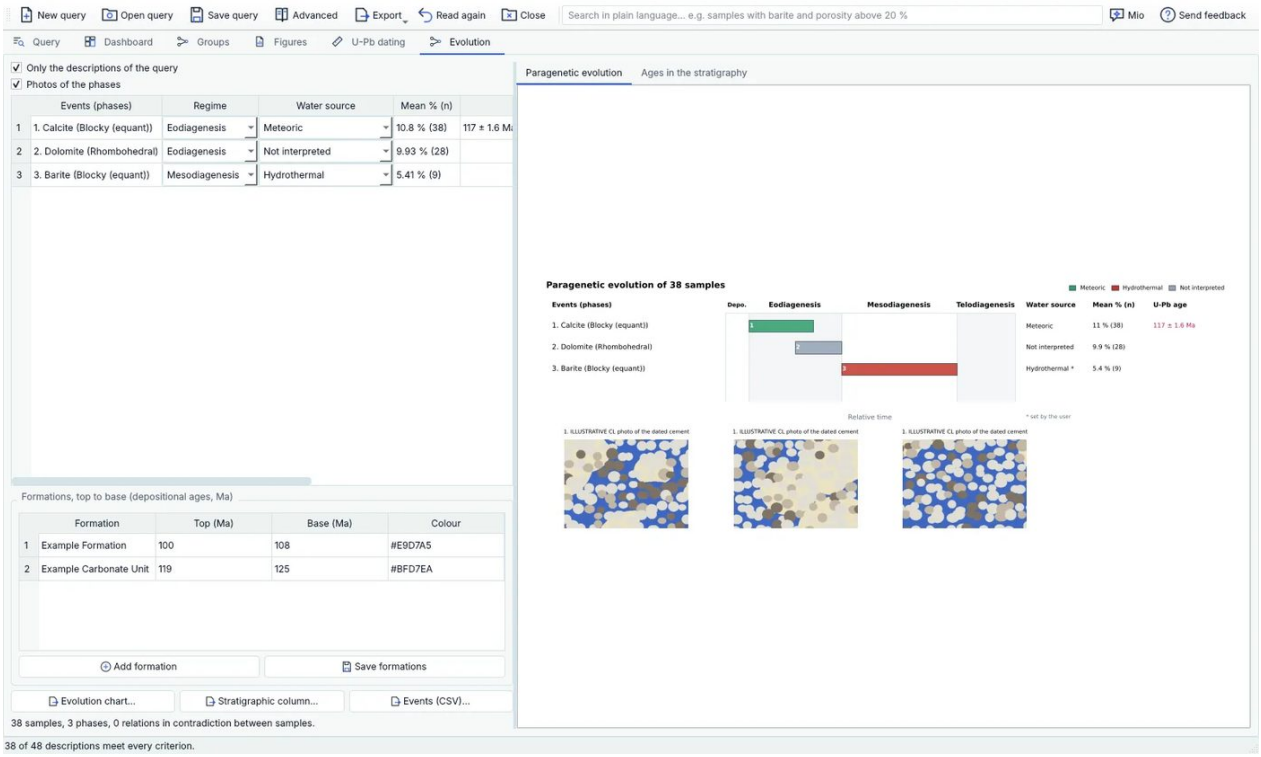
Coverage

L12 (dating U-Pb, Tera-Wasserburg; ages in the stratigraphy).

Paragenetic evolution and the stratigraphic column

What it is for

The **paragenetic evolution** puts the diagenetic phases of several samples — a formation, a field, a query — in one chart: the events in order, their regime (eo-, meso-, telodiagenesis), the water source of each, the mean amount and the number of samples, the U-Pb ages, and photos of the phases. The **stratigraphic column** places the formations with their depositional ages and the U-Pb ages of the samples, to show that each cement is younger than the rock that holds it.



In the program: Paragenetic evolution and the stratigraphic column

Before you start

- Descriptions with their composition and paragenetic relations (page Diagenesis: environments, paragenetic sequence and knowledge graph).
- For the ages: dated phases (page U-Pb dating on the Tera-Wasserburg diagram).
- For the column: the formations with their depositional ages (below).

Step by step

The evolution

1. In RockFace Explorer, open the **Evolution** tab, then **Paragenetic evolution**.
2. The descriptions are those of the query of the Query tab (**Only the descriptions of the query**), or all.
3. The table lists the **Events (phases)** in order, with their **Regime**, **Water source**, **Mean % (n)** and **U-Pb age**. "N samples, P phases, C relations in contradiction between samples."
4. Change the **Regime** or the **Water source** of a phase in its cell (Marine, Meteoric, Evaporitic, Alkaline lake, Near-surface pore water, Basinal (burial), Hydrothermal, Not interpreted): your choice is kept in the project.
5. **Photos of the phases** adds photos of each phase to the chart: the photos of its dated spots, then the Atlas images annotated with its constituent.
6. **Evolution chart...** saves the chart (PNG, SVG, PDF); **Events (CSV)...** the table.

The stratigraphic column

1. Open **Ages in the stratigraphy**.
2. In **Formations, top to base (depositional ages, Ma)**, **Add formation** and give its **Formation** name, **Top (Ma)**, **Base (Ma)** and **Colour**. **Save formations**.
3. The column shows each formation with its age range and, beside it, the U-Pb ages saved for its samples.
4. **Stratigraphic column...** saves it.

Options

Option	What it changes
Only the descriptions of the query	The evolution of the samples found, or of all.
Regime, Water source	Set by hand for a phase (cleared with an empty value).
Photos of the phases	Photos in the chart.

How it works

Each phase is a constituent with its habit and its kind (precipitation or dissolution), gathered across the samples. The events are ordered by regime, then by the paragenetic relations of all the samples together, then by their place in each sample's own sequence; relations that contradict each other between samples are counted and left out. The regime comes from the diagenetic rules (a phase without one takes the regime of the phases before it), the water source from the environments interpreted, unless set by hand. Ages are placed on the phase they date (mineral and habit of the saved age).

From a script

```
evo = prj.evolution(formation="Example Formation", path="evolution.png")
for ph in evo.phases:
    print(ph.event, ph.label, ph.regime, ph.water, ph.share_mean, ph.ages)
prj.set_phase(evo.phases[-1].key, water="hydrothermal")
prj.stratigraphy = [{"name": "Example Formation", "top": 100.0, "base": 108.0, "colour":
↪  "#E9D7A5"}]
prj.strat_column("column.pdf")
```

Limits and common problems

- A phase without paragenetic relations in any sample is placed by its regime and its place in each sample's sequence only.
- The column needs the depositional ages of the formations (base older than top).

Coverage

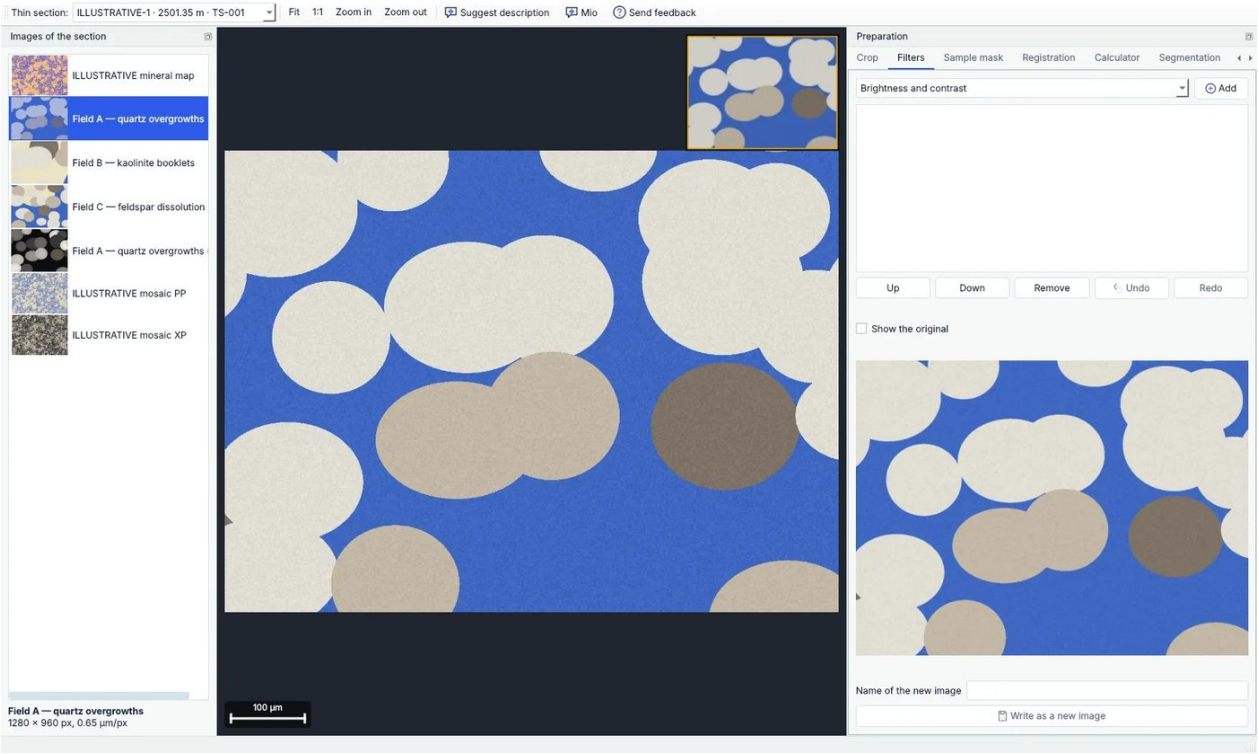
L8 (paragenetic sequence of several samples), L12 (ages in the stratigraphy; paragenetic evolution of several samples with quantification and photos).

RockFace Lab

RockFace Lab: crop, filters and the sample mask

What it is for

RockFace Lab prepares the images of a thin section for analysis. **Crop** cuts every image of the section to the same rectangle, keeping its place in the section. **Filters** improve an image — brightness and contrast, saturation, gamma, levels, histogram equalization, shading correction, smoothing, sharpening — with a preview. The **Sample mask** finds the useful area of the section (the rock, without the glass and the borders) and splits it into fragments.



In the program: RockFace Lab: crop, filters and the sample mask

Before you start

- A thin section with images in RockFace Atlas (page RockFace Atlas: whole slides, detail photos and the image library).
- Open Lab: **Home → Images → Lab** (Ctrl+Shift+L) in RockFace Petro, or `rockface lab project.rockface`.

Step by step

The window

- **Thin section:** at the top chooses the section; **Images of the section** lists its images (originals and those made by Lab, with what made them).
- The viewer shows the image selected: **Fit**, **1:1**, **Zoom in**, **Zoom out**.
- The tabs on the right: **Crop**, **Filters**, **Sample mask**, **Registration**, **Calculator**, **Segmentation**, **Edit and objects**, **Measurements**, **Flows**, **AI**.

Every tool writes a **new image** of the section; the original is never changed.

Crop

1. Open **Crop**. Give the rectangle in the section frame: **Left (mm)**, **Top (mm)**, **Right (mm)**, **Bottom (mm)**, or:
   - **Draw on the image** — drag a rectangle;
   - **Whole image**; **Box of the sample mask**;
   - a rectangle from **Saved regions** or **Last used**.
2. **Name of the region** and **Save region** keep it for the next sections; **Delete region**.
3. In **Images to crop**, check the images (PP, XP, a map...). Each is cropped to the part it covers.
4. **Crop the checked images**. "N cropped image(s) added to the section."

Filters

1. Open **Filters** and choose the image.
2. **Add** a filter to the list:
   - **Brightness and contrast**, **Saturation**, **Gamma**;
   - **Automatic levels** (**Low** and **High percentile**, channels joint or each);
   - **Histogram equalization**;
   - **Shading correction** (uneven lighting: **Reference** resin, bright, all, mask or flat; **Surface degree**; **Largest gain**);
   - **Gaussian smoothing** (**Sigma (px)**), **Median filter** (**Size (px)**), **Unsharp mask (sharpen)** (**Sigma**, **Amount**);
   - **Paint outside the sample** (with a mask **Image** and its **Fill**).
3. The filters run in order: **Up**, **Down**, **Remove**, **Undo**, **Redo**. The viewer shows the result at once; **Show the original** compares.
4. **Name of the new image**, then **Write as a new image**. "Filtered image added: ...".

Sample mask

1. Open **Sample mask**. The mask is found **by colour (blue resin)** or **by samples drawn on the image**: **Draw rock sample** and **Draw resin sample** on the image ("Samples: N rock pixels, M resin pixels").
2. **Split into fragments** by the resin between them: **Resin wider than**, **Keep the largest** (N fragments, or all), **Smoothing**, **Smallest area**.
3. **Find the sample**. "Sample: ...% of the image, N fragment(s)". "No glass or border found: the whole image is the sample." when the image is all rock.

4. **Name of the mask**, then **Keep the mask**. The mask is used by Crop (**Box of the sample mask**), by **Paint outside the sample**, by the segmentation and by the measurements.

Options

Option	What it changes
Shading correction — Reference	What should be even: the resin, the bright areas, all of the image, a mask, or a flat-field image.
Automatic levels — Channels	Stretch the three channels together (keeps the colours) or each apart.
Resin wider than	The narrowest resin gap that separates two fragments (fraction of the shorter side).

How it works

- Brightness, contrast and saturation give exactly what Pillow's ImageEnhance gives, and the histogram equalization of the luminance exactly what OpenCV gives (checked on 26 images, 20 of them real thin-section patches).
- Filters are applied block by block, so a whole section of any size is filtered with little memory, and the result is the same as filtering the whole image at once.
- The shading correction fits a polynomial surface to the reference (the resin by default) and divides it out: an unevenness of 0.086–0.130 was brought back to 0.021 on test sections.
- The sample mask finds the glass and the borders by colour and texture; on PP and XP sections it matched the true mask with IoU 0.97–0.9996.

From a script

```
pp, xp = prj.image("ILLUSTRATIVE mosaic PP"), prj.image("ILLUSTRATIVE mosaic XP")
pp_c, xp_c = prj.crop([pp, xp], roi=(2.0, 2.0, 8.0, 6.0))          # mm of the section frame
better = pp_c.filter(["shading", ("brightness_contrast", {"contrast": 20}), ("unsharp", {"sigma":
↪  1.5})])
mask = pp.sample_mask(split=True, n_largest=2)
print(mask.areas_mm2())
prj.save_roi("Centre", (2.0, 2.0, 8.0, 6.0))
```

Limits and common problems

- "Give a rectangle to crop." / "Check at least one image."
- "The section has no sample mask yet (Sample mask tab)."
- On a section without blue resin, find the mask **by samples drawn on the image**.

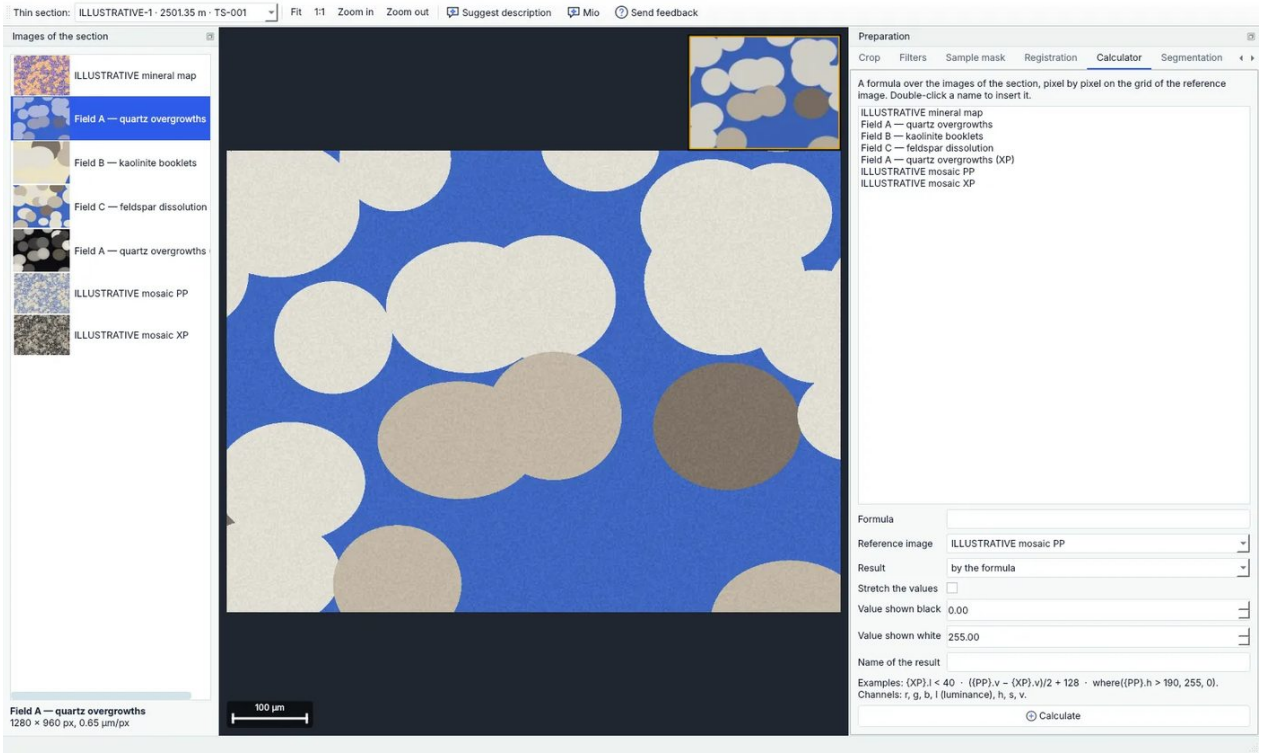
Coverage

GT3 (crop), GT4 (enhancement filters), GT6 (sample foreground: useful area, fragments, keep the N largest).

RockFace Lab: registration and the image calculator

What it is for

Registration aligns one image of a section on another — the crossed-polars image on the plane-polarized one, a cathodoluminescence image, a second scan — by landmark pairs and/or automatically, and writes it on the pixel grid of the fixed image, so the two can be used together pixel by pixel. The **Calculator** combines the images of a section with a formula, pixel by pixel: a mask of the dark pixels of XP, the difference of two lights, a threshold of the hue.



In the program: RockFace Lab: registration and the image calculator

Before you start

Two images of the same thin section in RockFace Lab (page RockFace Lab: crop, filters and the sample mask).

Step by step

Registration

1. Open the **Registration** tab.
2. **Fixed image** (usually PP) and **Moving image** (XP, CL...).
3. **Transform**: **translation**, **similarity (turn, scale, shift)**, **affine** or **thin-plate spline**.
4. Either:
   - **Landmark pairs...** — click a point on the fixed image (left), then the same point on the moving image (right); the fit is made after every pair, with the **Residual** of each and "RMS ... px, leave-one-out ... px"; **Remove pair**, **Remove all**; or
   - **Automatic** — the correlation of the grain outlines (ECC), started from the section frames or from the pairs. **Search (automatic)**: from the section frames, **shift**, **any rotation**, or **rotation and scale**.
5. The result: "...: rotation ...°, scale ..., shift ... px", the correlation of the outlines, the landmark errors.
6. Check it: **Moving over fixed** blends the two; **Flicker** alternates them.
7. **Save the registration** keeps it on the moving image; **Write on the grid of the fixed image** writes the moving image resampled pixel for pixel on the fixed one ("Image added on the fixed grid: ...").

Calculator

1. Open the **Calculator** tab. The images of the section are listed; double-click a name to insert it.
2. Write the **Formula**, with `{image name}` and a channel: `.r .g .b`, `.l` (luminance), `.h .s .v`. Examples:
   - `{XP}.l < 40` — the dark pixels of XP (a mask);
   - `({PP}.v - {XP}.v)/2 + 128` — a grey image;
   - `where({PP}.h > 190, 255, 0)`. Numbers, `+ - * / **`, comparisons, `& | ~`, and `abs sqrt log log10 exp min max where clip round`.
3. **Reference image** — the grid of the result (the other images are taken on it through their registrations).
4. **Result**: **by the formula** (a comparison gives a mask, a number a grey image), **mask** or **grey image**. **Stretch the values** with the **Value shown black** and **Value shown white**.
5. **Name of the result**, then **Calculate**. "Uses: ..." lists the images used. "Result added: ...".

Options

Option	What it changes
Transform	Translation, similarity, affine (≥ 3 pairs) or thin-plate spline (non-linear, many pairs).
Search (automatic)	How far the automatic search looks: none, shift, any rotation, rotation and scale.
Result	Mask (a map of 0/1) or grey image.

How it works

- Landmark fits are least squares; the leave-one-out error predicts each pair from the others. The automatic registration maximizes the enhanced correlation coefficient (ECC) of the grain outlines, after a search of shift, rotation and scale.
- Checks: XP on PP recovered within 0.017 px for any rotation and ±8 % scale; a scan placed 6 px off corrected to 0.15 px; a thin-plate spline 0.06 px on a non-linear warp; on real thin-section pairs the common offset was found in 52 and 59 of 60 patches.
- The calculator works block by block on the grid of the reference image.

From a script

```
pp_c, xp_c = prj.image("ILLUSTRATIVE mosaic PP (crop)"), prj.image("ILLUSTRATIVE mosaic XP (crop)")
t = xp_c.register_on(pp_c, search="rotation")            # automatic, from the section frames
print(t.rotation_deg, t.scale, t.ecc)
xp_on_pp = xp_c.resample_on(pp_c)                        # on the grid of the PP image
dark = prj.calculate("{%s}.l < 60" % xp_on_pp.name, pp_c, name="dark in XP")
print(dark.areas_mm2())
```

Limits and common problems

- “Choose two different images of the section.”
- “N pairs: at least ... are needed.” — 1 for a translation, 2 for a similarity, 3 for an affine transform, 4 for a thin-plate spline.
- The automatic registration needs images with grains in common; on very different lights (CL on PP), give a few landmark pairs first.

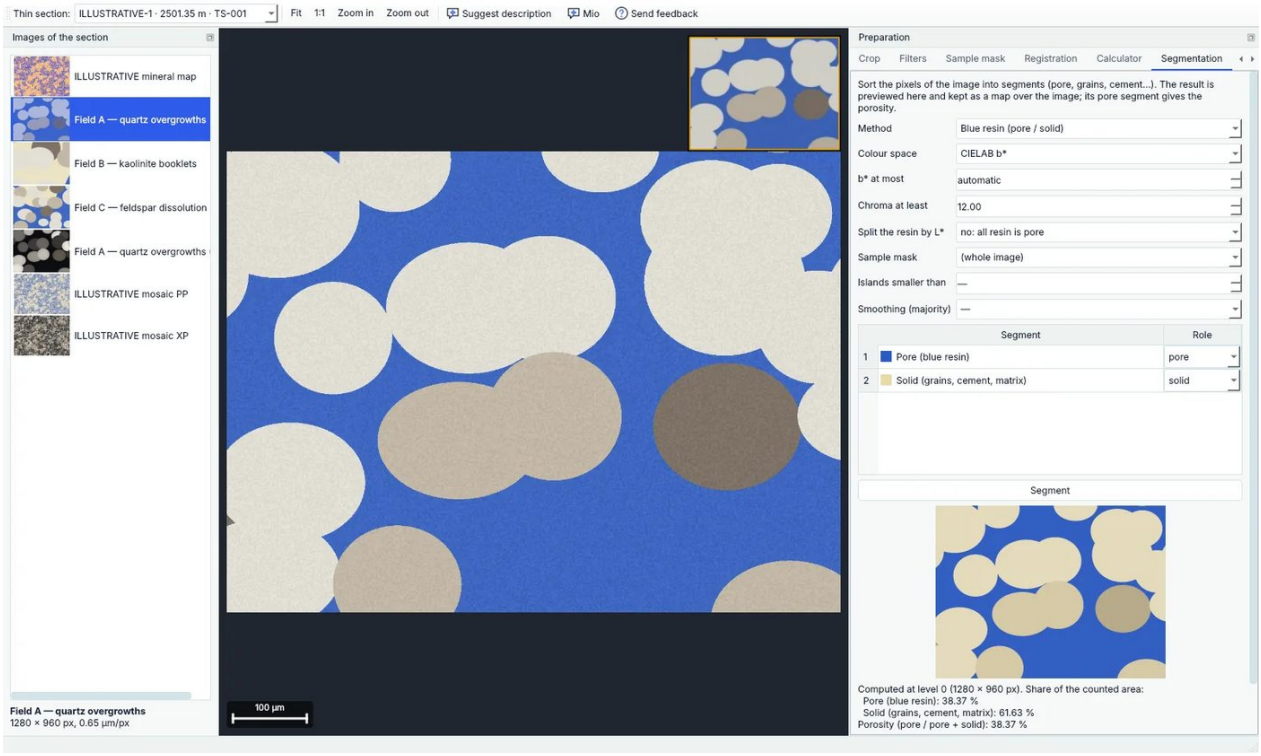
Coverage

GT5 (registration PP ↔ XP with landmark pairs, automatic registration), GT19 (Volume Calculator of the “More tools”).

RockFace Lab: segmentation and porosity

What it is for

Segmentation sorts the pixels of a thin-section image into segments — pore, grains, cement, matrix… — and keeps the result as a **map** over the image. Its pore segment gives the **porosity**. RockFace Lab offers the classical methods: the blue resin, thresholds of a channel (also multi-level Otsu), a colour range, a Bayesian classifier learnt from samples drawn on the image, k-means (also on superpixels), seeds grown by watershed, and a random forest trained on painted examples.



In the program: RockFace Lab: segmentation and porosity

Before you start

An image of the section in RockFace Lab, ideally cropped and filtered (page RockFace Lab: crop, filters and the sample mask). A sample mask limits the segmentation to the rock.

Step by step

1. Open the **Segmentation** tab and select a picture (not a map).
2. **Method**:

- **Blue resin (pore / solid)** — the blue epoxy is pore. In CIELAB the resin is found by its blue (b*), the cut chosen automatically (Otsu) or typed, with **Chroma at least**; or by the **CIELAB hue angle**. The resin can be split by its lightness: **no: all resin is pore**, **open pore = bright resin (Otsu)**, or **open pore = L* at least…** — the dark, resin-tinted part is then microporosity.
- **Thresholds of a channel** — choose the **Channel** (luminance, red, green, blue, HSV hue, saturation, value, CIELAB L, *a*, b*, chroma, hue angle) and the cuts; empty: automatic (multi-level Otsu).
- **Colour range (HSV)** — the ranges of hue, saturation and value of a segment.
- **Bayesian classifier (samples)** — **Draw a sample of the selected segment** for every segment (at least 8 pixels each); the classifier learns their colours.
- **K-means (unsupervised)** — the number of **Clusters**, on pixels or on **Superpixels**.
- **Grow from seeds (watershed)** — **Paint seeds of the selected segment** with the **Brush radius**.
- **Random forest (painted examples)** — page Segmenting by painted examples (random forest).

3. **Segments**: **Add segment**, **Remove segment**; each with its name, colour and **Role** (pore, solid, unclassified).
4. Region: the whole image, or a sample mask.
5. **Segment**. The preview shows the map over the image and "Share of the counted area" of each segment, the **Porosity (pore / pore + solid)** and the thresholds used.
6. Clean the map: **Islands smaller than** (pixels given to their surroundings) and **Smoothing (majority)**.
7. **Name of the map**, then **Keep the segmentation as a map**. "Segmentation kept: … (porosity … %)."

The map can be corrected by hand and split into objects (page RockFace Lab: editing maps and objects), then measured (page RockFace Lab: measurements and size distributions).

Options

Option	What it changes
Blue resin — the split by L*	All resin is pore, or only the bright resin (open pore); the dark resin is microporosity.
Channel, cuts	Which channel is cut, and where (empty: Otsu).
Superpixels (k-means)	Cluster SLIC superpixels instead of pixels: smoother segments.
Role	Pore and solid make the porosity; unclassified is left out of it.

How it works

- Colours are converted to CIELAB (within 0.0049 of scikit-image on 100 000 colours) and to HSV; 12 channels are offered.
- On three ILLUSTRATIVE sections of exact truth (22.3–25.6 % porosity), the blue resin, Otsu of b* and the Bayesian classifier were exact on clean images and within 0.5 points (IoU ≥ 0.95) when blurred, noisy and with thinner resin together.

- On the 20 carbonate sections of GO-Forward D2.1 (CC BY 4.0), whose published values are the open porosity of a SLIC + k-means segmentation, the hue-angle resin with the L* split follows them with a correlation of 0.97; counting all the resin gives the total optical porosity.
- The map is a tiled image like the others, in the frame of the section, so it is laid over every image of the section in Atlas and Lab.

From a script

```
pp = prj.image("ILLUSTRATIVE mosaic PP")
seg = pp.segment("resin", post=[("islands", {"min_px": 20})])
print(f"porosity {seg.porosity():.2f} %")
km = pp.segment("kmeans", k=4, superpixels=5000, save=False)          # a look, nothing written
otsu = pp.segment("thresholds", channel="b", classes=3, name="Otsu of b*")
```

Limits and common problems

- “Choose a picture of the section (not a map) to segment.”
- “Draw a sample (of at least 8 pixels) for every segment.” / “Paint seeds of at least one segment.”
- The preview is computed at a reduced level on large images (“Computed at level ...”); the map kept is at full resolution.
- Sections without blue resin: use thresholds, a colour range, the classifier or the random forest.

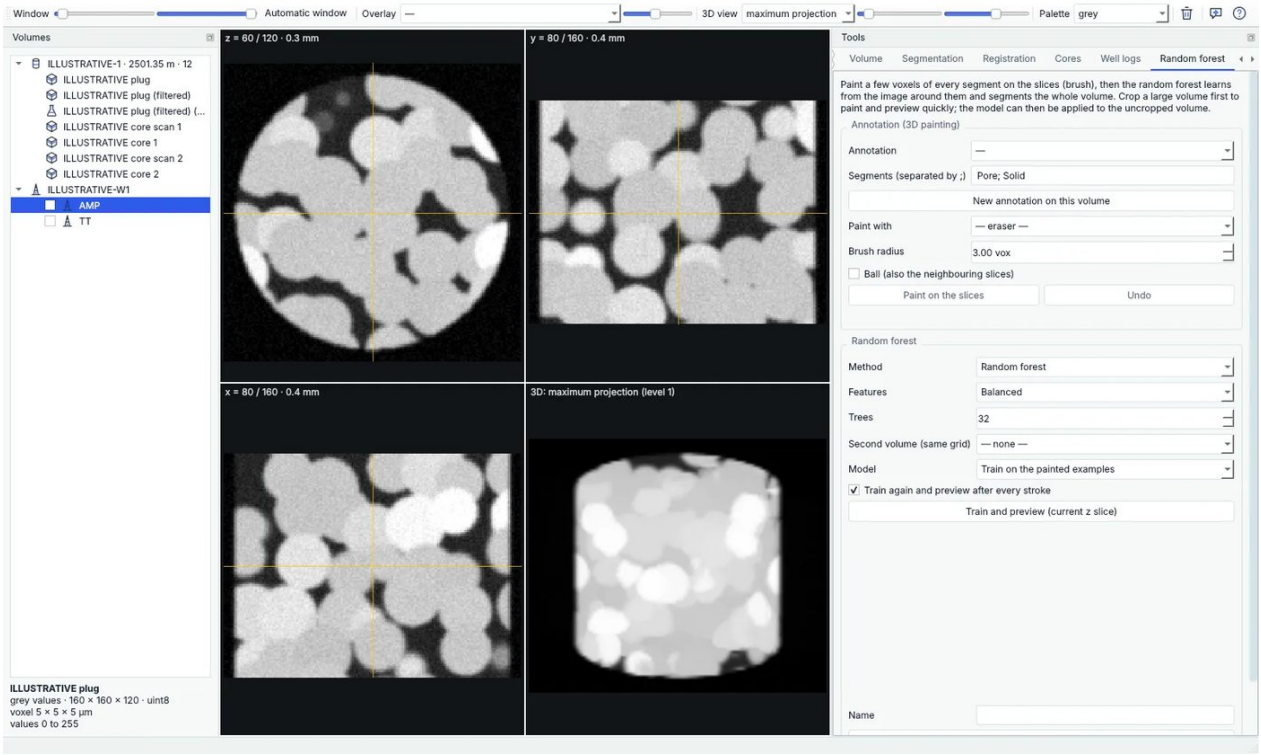
Coverage

GT7 (manual and threshold segmentation), GT8 (pore / non-pore by Bayesian inference), L7 in part (quantification of the whole section, classical).

Segmenting by painted examples (random forest)

What it is for

Some phases cannot be told apart by a colour or a grey value: a cement with the colour of the grains, vugs with the mean grey of the matrix. You paint a few pixels (or voxels) of every segment, and the program learns from the image **around** them — smoothness, texture, edges — and classifies the whole picture or volume. Each new stroke trains it again and the preview changes at once, so you correct it where it is wrong.



In the program: Segmenting by painted examples (random forest)

It works in **RockFace Lab** on thin-section pictures and in **RockFace MC** on micro-CT volumes.

Before you start

- Lab: a picture of a thin section (a crossed-polarized picture of the same section can be added as a second picture, if it is registered on the first or both have the section frame).
- MC: a grey volume. For a large volume, crop a representative part first (**Volume → Crop**) to paint and preview quickly; the model can then be applied to the whole volume.

Step by step

In Lab (pictures)

1. Open the picture and go to the **Segmentation** tab.
2. In **Method**, choose **Random forest (painted examples)**. Two segments are proposed (pore and solid): rename them, add more with **Add segment**.
3. Zoom on a representative part of the picture: the preview is computed on the part on screen.
4. Select a segment in the table, press **Paint examples of the selected segment** and paint a few strokes on it. Do the same for every segment. Paint the boundaries and the places that are easy to confuse.
5. With **Update the preview after every stroke** ticked, the preview follows each stroke. Look where it is wrong and paint there.
6. Press **Apply to the whole image** to classify the whole picture at the pixel size you trained on, block by block, or **Keep the segmentation as a map** to keep only the part computed.

In MC (volumes)

1. Open the grey volume and go to the **Random forest** tab.
2. Type the segments in **Segments (separated by ;)** (for example `Vug; Matrix`) and press **New annotation on this volume**. The annotation is shown over the slices.
3. Choose the segment in **Paint with**, the **Brush radius**, and press **Paint on the slices**. Drag the mouse on any of the three slice views. **— eraser —** removes paint; **Undo** takes back the last stroke. Tick **Ball** to paint the neighbouring slices too.
4. Paint a few voxels of every segment, on several slices.
5. **Train and preview (current z slice)** shows the result on a window of the current slice; with **Train again and preview after every stroke**, it follows each stroke.
6. Press **Apply to the whole volume**. The result is a new segmented volume, with its porosity.

Options

- **Features** — what the forest looks at, as in GeoSlicer's Interactive Segmenter:
  - *Sharp*: the value, Gaussian σ 1 and 2, local variance 5×5 — fine details;
  - *Balanced*: the value, Gaussian 1, 2, 4, variance 5 and 9 — a good start;
  - *Smooth* and *Extra smooth*: larger scales, smoother segments;
  - *Complete*: all of them, plus the gradient and the Laplacian.
- **Trees** — more trees, a steadier result (32 is enough most of the time).
- **Second picture / Second volume** — more channels on the same grid (the XP picture of a section).
- **Method** (MC) — *Random forest* or *Bayesian inference* (*Gaussian*) (each segment a normal distribution of the features).
- **Model** — *Train on the painted examples*, or a model kept in the project. Every model applied is kept in the project; **Export model...** writes it to a `.rfmodel` file, **Import model...** reads one (from a colleague, say).

How it works

- For every pixel or voxel, the features of the chosen set are computed on each channel (L*, a*, b* of a colour picture; the grey value of a volume). A random forest (Breiman 2001) of decision trees learns from the painted points which features tell the segments apart; every point gets the segment most trees vote for.
- A feature depends only on the points within a margin around it, so the image is classified block by block with that margin: the result is the same as the whole image at once, however large.
- A model is kept as plain numbers (the nodes of the trees), never as program code: a model received from someone else is safe to open.
- Source: L. Breiman (2001), *Random forests*, Machine Learning 45, 5–32; the presets follow the GeoSlicer manual.

From a script

```
import rockface.api as rf
prj = rf.open("project.rockface")
pp = prj.image("ILLUSTRATIVE mosaic PP")
seg, forest = pp.forest(strokes=[(1, [(120, 80), (140, 90)], 6), (2, [(600, 420)], 6)],
                        segments=["Pore", "Solid"], preset="balanced")
whole, fractions, porosity = pp.apply_forest(forest)
vol = prj.volume("Plug A")
ann = vol.new_annotation(["Vug", "Matrix"])
ann.paint("z", 40, [(30, 30), (36, 30)], 2.0, 1)
f3 = vol.train_forest(ann, name="Vugs")
seg3 = vol.apply_forest(f3)
```

Limits and common problems

- **“Paint examples of at least two segments.”** Every segment needs a few strokes.
- **The preview is good but the whole image is not.** The strokes came from one part only; paint other parts too.
- **“The model was trained on ... channels”**: a model trained with a second picture needs one now.
- In Lab, the preview and the training use the strokes on screen; strokes outside the part on screen are kept for later.
- The pretrained networks of GeoSlicer (U-Net) come in stage 0.26.

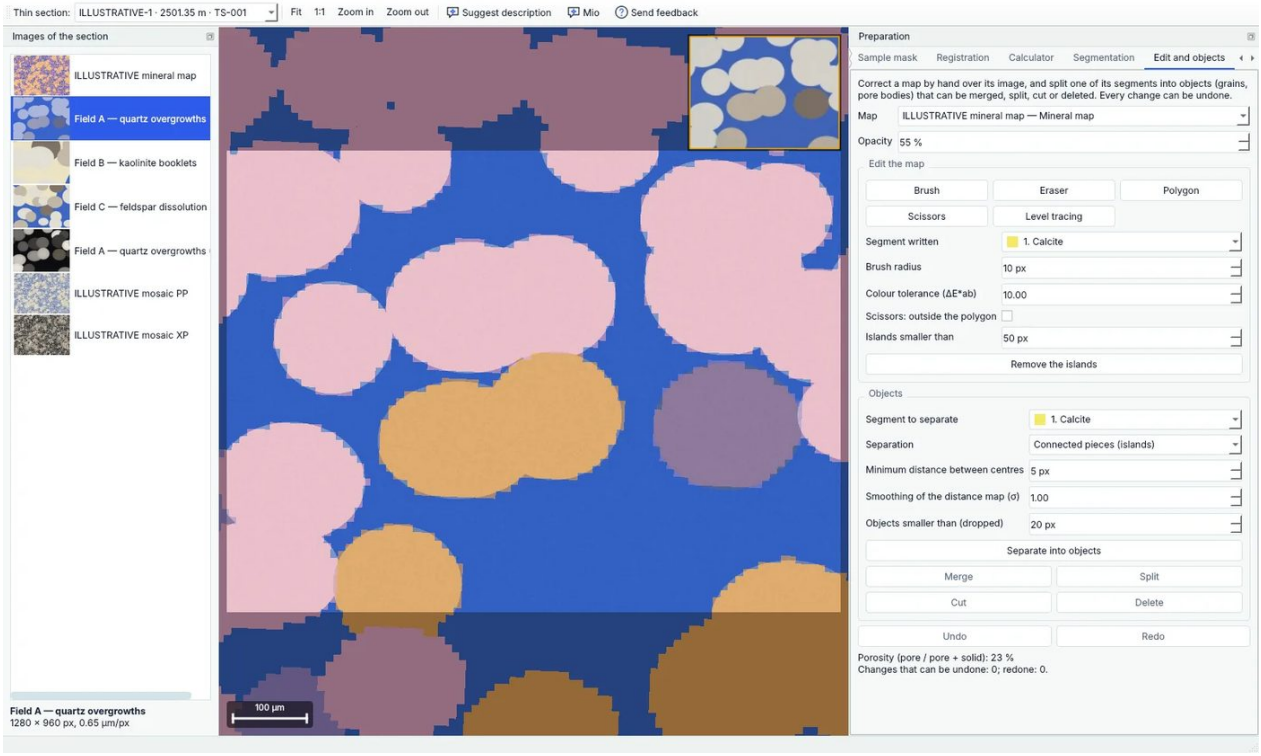
Coverage

GV6, GT7, GT8 (Interactive Segmenter, Segmenter — random forest and Bayesian methods).

RockFace Lab: editing maps and objects

What it is for

A segmentation is rarely perfect: a grain painted as cement, resin bubbles taken as pores, a scratch. The **Edit and objects** tab corrects a map by hand over its image — brush, eraser, polygon, scissors, level tracing, islands — and splits one of its segments into **objects**: each grain or pore body on its own, so that it can be measured. Objects can be merged, split, cut and deleted. Every change can be undone.



In the program: RockFace Lab: editing maps and objects

Before you start

A map of the section: a segmentation (page RockFace Lab: segmentation and porosity) or a mineral map ("The section has no map yet: segment an image first.").

Step by step

Correcting a map

1. Open **Edit and objects** and choose the **Map**; set its **Opacity** over the image.

2. Tick **Edit the map** and choose the **Segment written**.
3. Choose a tool:
    - **Brush** — drag over the image to paint the chosen segment (**Brush radius**);
    - **Eraser** — drag to erase (not classified);
    - **Polygon** — click the corners; double-click or Enter fills it;
    - **Scissors** — draw a polygon: its inside (or, with **Scissors: outside the polygon**, its outside) is set to the chosen segment;
    - **Level tracing** — click a point: the connected pixels of a similar colour (**Colour tolerance (ΔE*ab)**) take the chosen segment.
4. **Islands smaller than** and **Remove the islands**: small pieces are given to their surroundings.
5. **Undo** and **Redo** — "Changes that can be undone: …; redone: …".

Every edit is written into the map at full resolution and at every zoom level at once.

Objects

1. Under **Objects**, choose the **Segment to separate** (the grains, or the pores).
2. **Separation**:
    - islands — each connected piece is an object;
    - watershed — touching grains are split at their necks: **Minimum distance between centres** and **Smoothing of the distance map** (σ);
    - SNOW — the over-segmented watershed of Gostick (2017), for pore bodies.
3. **Objects smaller than** (**dropped**).
4. **Separate into objects**. "N objects kept: …". The objects are drawn each in its colour.
5. Edit them with the tools:
    - **Merge** — click the objects, then **Merge the objects clicked** ("Objects clicked: n (at least 2)");
    - **Split** — click an object to split it by watershed;
    - **Cut** — click the two ends of a cutting line;
    - **Delete** — click an object to delete it. **Undo** and **Redo** as for the map.

Options

Option	What it changes
Colour tolerance (ΔE*ab)	How different a colour may be and still be traced.
Minimum distance between centres	Larger: fewer, bigger objects.
Smoothing of the distance map	Larger: less over-splitting of irregular grains.
Objects smaller than	Small objects are dropped (noise).

How it works

Edits are written in place in the tiles of the map, every level of the pyramid rebuilt; each edit returns an undo patch, so undoing gives back exactly the map before (checked against maps written anew). Objects are kept as an object map: every pixel holds the number of its object. On sections of exact truth, grains separated by watershed and SNOW were found with recall 0.78–0.83 and precision 0.96–0.98; with the XP colours the recall rose to 0.87–0.89 (page RockFace Lab: measurements and size distributions).

From a script

```
seg = prj.image("ILLUSTRATIVE mosaic PP").segment("resin")      # a segmentation map
patch = seg.paint([(100, 100), (300, 120)], 15, 1)      # brush along a line, radius 15 px, segment
↪ 1
seg.undo(patch)
seg.level_trace(420, 380, tolerance=8, phase=2)
seg.remove_islands(30)
grains = seg.objects(2, "watershed", min_distance=6, sigma=1.0, min_px=40)
print(grains.object_count())
grains.merge_objects([(1421, 842), (1422, 842)])       # a point in each of two touching objects
grains.cut_objects((600, 300), (640, 360))
```

Limits and common problems

- “Objects clicked: 1 (at least 2).” — click a second object before merging.
- Separating a very large map takes time: separate a crop first to choose the parameters.
- The scissors are reached from scripts for any polygon (`img.scissors`).

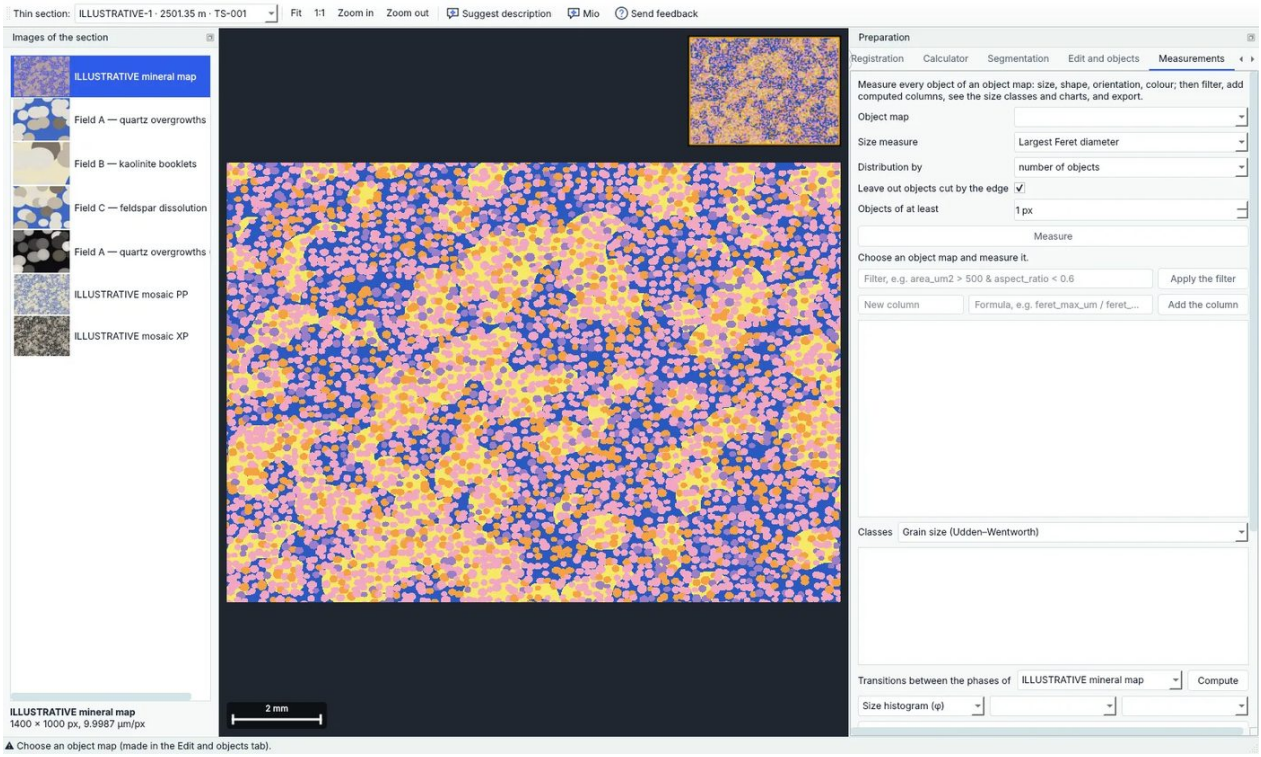
Coverage

GT7 (manual segmentation: paint, erase, scissors, level tracing, islands), GT9 (partitioning: islands, watershed, SNOW), GT10 (object editor: merge, split, cut, delete).

RockFace Lab: measurements and size distributions

What it is for

The **Measurements** tab measures every object of an object map — each grain or pore: area, perimeter, Feret diameters, the fitted ellipse, shape factors, orientation, brightness and colour — and, for pores, the throats between them and the coordination number. It gives the size classes (Udden–Wentworth for grains, Choquette & Pray for pores), the Folk & Ward statistics, tables with computed columns and filters, charts, and the exports.



In the program: RockFace Lab: measurements and size distributions

Before you start

An object map (page RockFace Lab: editing maps and objects), ideally with its scale.

Step by step

Measuring

1. Open **Measurements** and choose the **Object map**.
2. **Size measure** (the column used for the sizes: largest Feret diameter, equivalent diameter…) and **Distribution by**: **number of objects** or **area (as a point count)**.
3. **Leave out objects cut by the edge**; **Objects of at least** N pixels.
4. **Measure**. "N objects measured: …".
5. The summary: "Size: mean … µm (class); D10, D50, D90", "Folk & Ward (φ): mean, sorting (class), skewness, kurtosis", "Shape: mean aspect ratio, mean circularity", and for pores "Mean coordination number … (N throats)".

The table

- One row per object with its **Area**, **Perimeter**, **Perimeter / area**, **Equivalent diameter**, **Largest**, **Smallest** and **Mean Feret diameter**, **Angle of the largest Feret**, **Aspect ratio**, **Elongation**, **Ellipse major** and **minor axis**, **Orientation (ellipse)**, **Eccentricity**, **Form factor**, **Circularity**, **Solidity**, **Convexity**, **γ (perimeter / circle)**, **Centre x, y** (px and mm), **On the edge**, the **Coordination number** of pores, and the brightness and colour of the picture under the object.
- **Filter**, e.g. `area_um2 > 500 & aspect_ratio < 0.6`, then **Apply the filter** ("Objects shown: n of N").
- **New column** with a **Formula**, e.g. `feret_max_um / feret_min_um`, then **Add the column**.
- The table can be sorted, filtered by column, exported to CSV or XLSX, and shown as a histogram.

Classes and charts

1. **Classes**: **Grain size (Udden–Wentworth)** (14 classes from clay to boulder), **Pore size (Choquette & Pray)**, or **Groups of similar area** — with the count, % by number and % by area of each.
2. Chart: **Size histogram (φ)** with its cumulative curve, **Crossplot with histograms**, **Rose diagram of orientations**, **Size classes**, or **Transitions between phases (stacked bars)** (page Transitions between phases). **Draw the chart**; save it as PNG.
3. Export the tables (CSV, XLSX) and the objects in colour (PNG).

Options

Option	What it changes
Size measure	The diameter used for classes and statistics.
Distribution by area	Each object weighs its area, as a point count would.
Leave out objects cut by the edge	Objects touching the image border are not counted.

How it works

- Perimeter by the Crofton formula; Feret diameters from the convex hull of the sub-pixel outline; the ellipse of the same second moments; shape factors after ISO 9276-6.
- Throats: the length of the boundary between two touching pores; the coordination number is the number of pores each touches.
- Checked on shapes of known size at random positions and angles: from 20 px across, area, perimeter, Feret diameters and ellipse within about 2 %, orientation within a fraction of a degree; coordination 4 and 6 in square and hexagonal packings; the Folk & Ward measures of 517 discs of a known log-normal law equal to those of the drawn diameters.

From a script

```
grains = prj.image("ILLUSTRATIVE mosaic PP").segment("resin").objects(2, "watershed", min_px=40)
m = grains.measure()
print(m.summary(exclude_edge=True))                        # Folk & Ward, D10/D50/D90, shape,
↪  coordination
t = m.table.filter("touches_edge == 0")
t.add_column("ratio", "feret_max_um / feret_min_um")
print(rf.measure.class_table(t, "feret_max_um", "wentworth"))
rf.measure.chart("rose", t, "orientations.png", x="ellipse_angle")
rf.measure.to_xlsx("grains.xlsx", {"grains": t})
```

Limits and common problems

- “The map has no scale: lengths are in pixels.” — give the image its scale in Atlas.
- Objects of one or two pixels have meaningless shapes: raise **Objects of at least**.
- The Crofton perimeter is 5–7 % low on long straight sides.

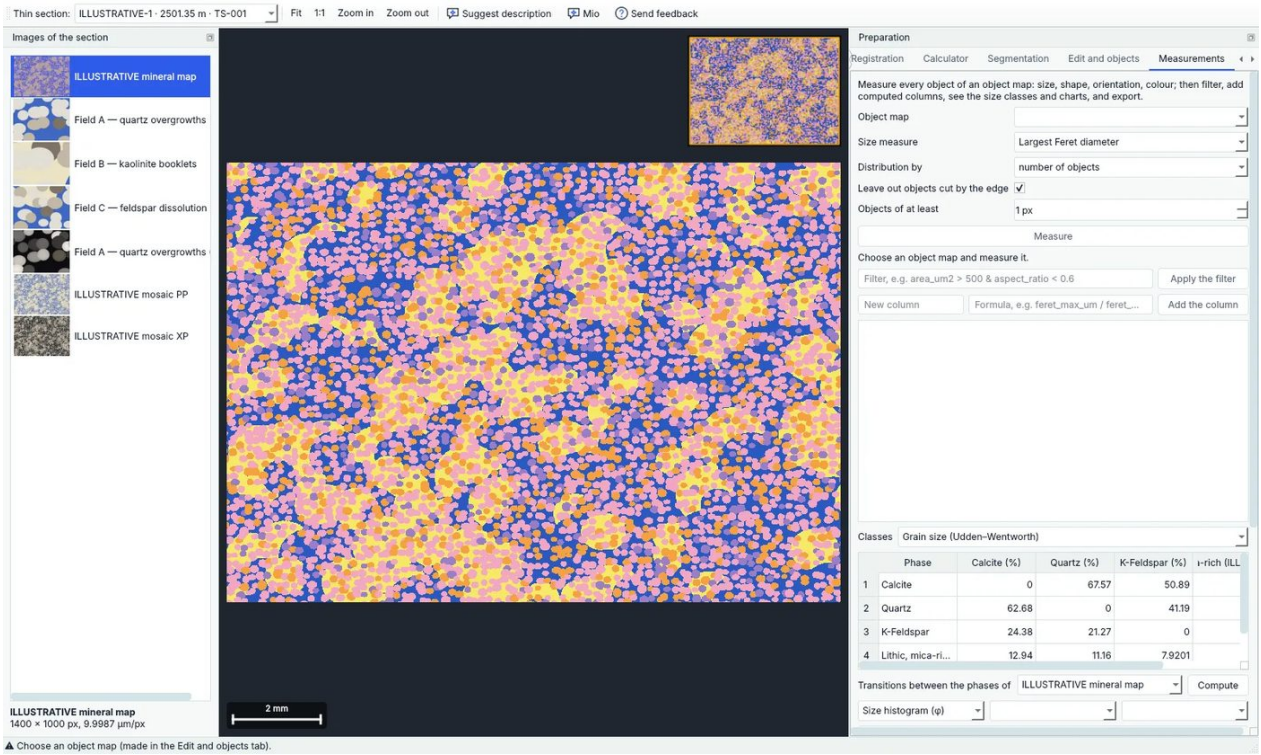
Coverage

GT11 (per-object metrics), GT12 (2D throats and coordination), GT13 (size classes; groups by similar area), GT14 (outputs: XLSX tables, coloured objects), GT19 (tables, filter, charts).

Transitions between phases

What it is for

Which minerals touch which: does the dolomite border the pores, is the quartz surrounded by calcite cement? The transition table of a mineral map or a segmentation says, for every phase, how its neighbouring pixels are shared among the other phases. The stacked bar chart shows it at a glance.



In the program: Transitions between phases

Before you start

A mineral map or a segmentation of a thin section in RockFace Lab.

Step by step

1. Open the section in RockFace Lab and go to the **Measurements** tab.
2. In **Transitions between the phases of**, choose the map and press **Compute**.
3. The table (in place of the size classes) has one column per phase: the % of its neighbouring pixels that belong to each phase (the rows). The chart shows one bar per phase, stacked by the phases it touches.

4. **Chart (PNG)** keeps the chart as a PNG file.

How it works

Each pixel is compared with its 8 neighbours over the whole map at full resolution; the contacts of a phase with itself are left out; each column is divided by its total (GeoSlicer's "transition analysis" of the Segment Inspector).

From a script

```
seg = prj.image("ILLUSTRATIVE mosaic PP (segmentation)")
table = seg.transitions()                # rf.Table: "phase" and one column per phase, in %
print(table.rows())
```

Limits and common problems

- "This needs a segmentation or a mineral map": the transitions are computed on maps, not on pictures.
- Phases that do not occur in the map are left out.

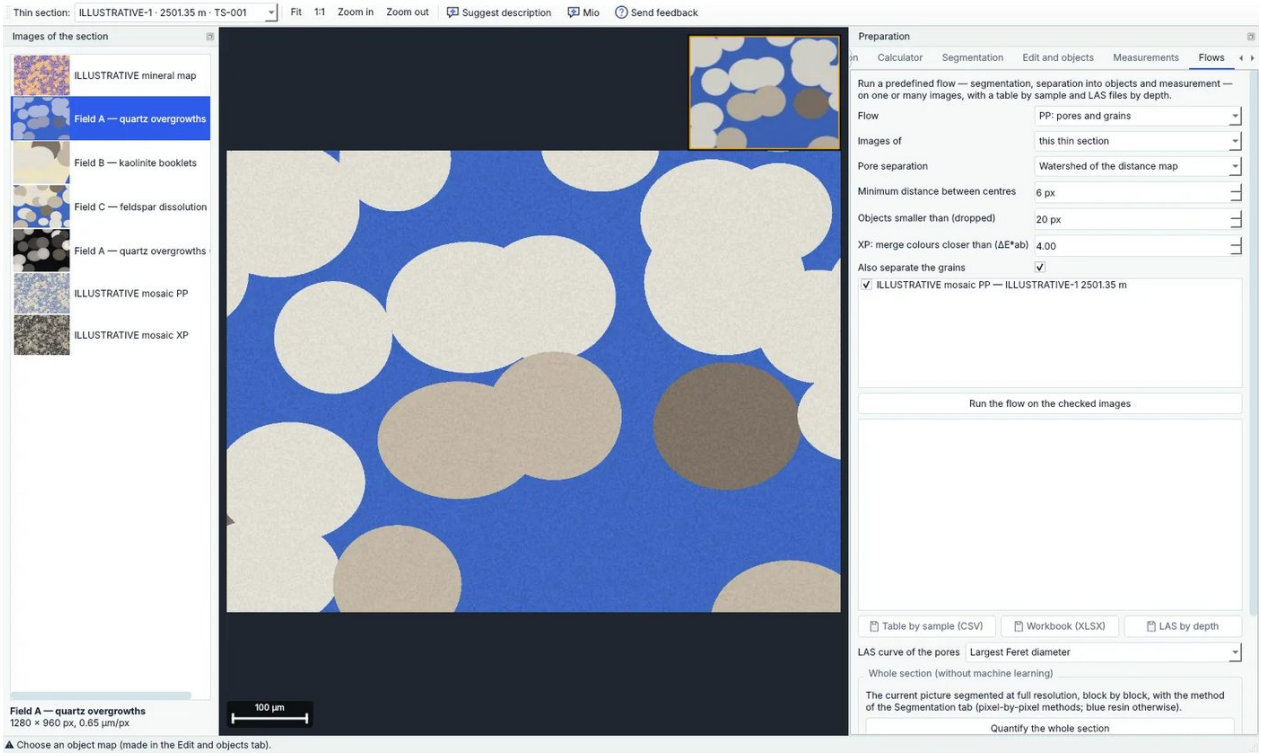
Coverage

GT19 (Segment Inspector: transition analysis; Charts: transition).

RockFace Lab: flows, the whole section and NetCDF

What it is for

A **flow** runs the whole chain — segmentation, separation into objects, measurement — on one image or on many (every image of a section, or of the project) in one go, and gives a table with one row per sample and LAS files of the pore sizes by depth for each well. **Whole section** segments a full thin-section image at full resolution, block by block. **Import and export** reads and writes label images and NetCDF files to exchange maps with other programs.



In the program: RockFace Lab: flows, the whole section and NetCDF

Before you start

Images of the sections in RockFace Lab; for the PP/XP flow, the XP image of each section; for LAS, descriptions with a well and a depth.

Step by step

A flow

1. Open the **Flows** tab.
2. **Flow**:
   - **PP: pores and grains** — blue-resin segmentation, pores and grains separated and measured;
   - **PP/XP: grains separated by the XP colours** — grains told apart by their extinction colours in crossed polars (**XP: merge colours closer than (ΔE*ab)**);
   - **Mineral map (QEMSCAN...): objects of every phase**.
3. **Images of**: **this thin section** or **the whole project**; check the images to use.
4. **Pore separation** (watershed, SNOW, islands), **Minimum distance between centres**, **Objects smaller than (dropped)**, **Also separate the grains**.
5. **Run the flow on the checked images**. "Running the flow: image k of n...", then "Flow done on N images (M with errors)".
6. The table has one row per sample: well, depth, thin section, porosity, the fractions of the phases, and the size statistics. Export it (CSV, XLSX).
7. **LAS curve of the pores** (the size column) and **LAS by depth**: one LAS file per well with the maximum, mean, median, minimum and standard deviation of the column and the porosity at each sample depth.

The whole section

1. Under **Whole section (without machine learning)**, select a picture.
2. **Quantify the whole section** segments it at full resolution, block by block, with the method of the Segmentation tab (pixel-by-pixel methods; blue resin otherwise). "...: N pixels counted", with the porosity and the share of each segment.

For the whole section segmented by a neural network, see RockFace AI: segmenting with models, training, fine-tuning and validating.

Import and export

- **Import a label image...** (PNG, TIFF) — **Labels as a segmentation map** or **Labels as an object map**.
- **Export the map as labels (TIFF)...**
- **Import NetCDF...** and **Export the section as NetCDF...** — the images and maps of the section with their scale, origin and phases (classic NetCDF; NetCDF-4 read).

Options

Option	What it changes
Flow	PP, PP/XP or mineral map.
Images of	One thin section or the whole project.
XP: merge colours closer than	Grains whose XP colours differ less than this are one grain.
LAS curve of the pores	The size column written to the LAS files.

How it works

- The flows use the same functions as the tabs, so a flow gives the result the tabs would give one step at a time.
- Grains separated with the XP colours were found with recall 0.87–0.89, against 0.46–0.51 from PP alone, on sections of known grains.
- The whole section is processed block by block with overlapping margins: on a 27 Mpx section the result was identical to the whole image at once, and the porosity equal to the truth.
- NetCDF and label TIFF round trips are identical.

From a script

```
pp = prj.image("ILLUSTRATIVE mosaic PP")
r = pp.run_flow("pp")
pores, grains = r.measurements
print(r.porosity, grains.summary(exclude_edge=True)["d50_mm"])
results = prj.run_flow(prj.images(kind="slide", modality="pp"), flow="pp")
prj.flow_table(results).to_csv("by_sample.csv")
prj.las_by_depth(results, "las", column="feret_max_um")
seg, fractions, porosity = pp.quantify_whole("resin")
prj.export_netcdf([pp, seg], "section.nc")
```

Limits and common problems

- “Check at least one image.”
- “The section has no XP picture for this image.” — the PP/XP flow needs both lights.
- Samples without a well or a depth are in the table but not in the LAS files.

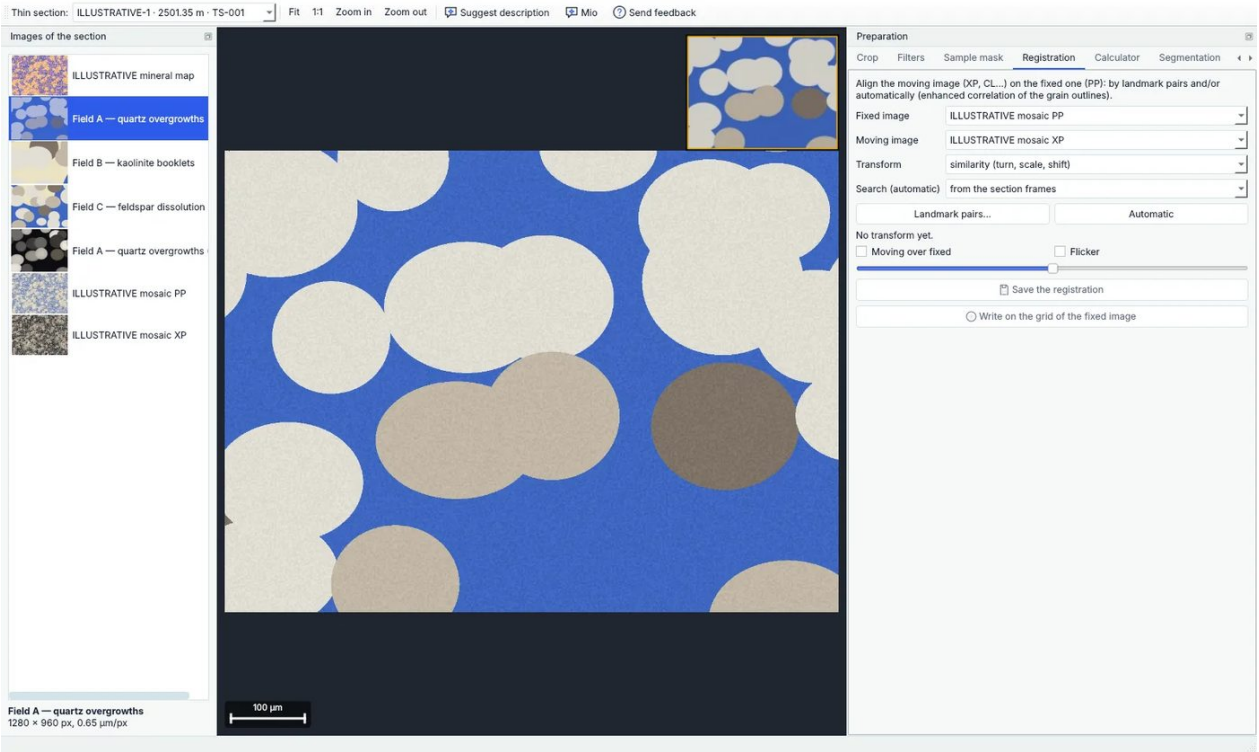
Coverage

GT14 (LAS per depth, NetCDF), GT15 (predefined flows PP, PP/XP, QEMSCAN), GT16 (multi-image analysis), GT17 (export and import of standard sources), L7 (quantification of the whole thin section, classical).

A mineral map aligned without landmarks; effects on big volumes

What it is for

Two details found by the 0.23 audit and closed in 0.28. **A mineral map aligned on a segmentation of the picture** without clicking landmark pairs (GeoSlicer's Thin Section Auto Registration). **The effects of the segment editor on big volumes** — margin, smoothing, boundary removal and expand — at full resolution, slab by slab, instead of on a reduced level (GeoSlicer's Big Image effects).



In the program: A mineral map aligned without landmarks; effects on big volumes

Before you start

- Alignment: a segmentation of the picture (Lab → Segmentation) and the mineral map of the same thin section, both with a pore phase (phase role "Pore"), or with phases that show the same boundaries.
- Effects: a segmented volume in MC.

Step by step

1. **Lab → Registration**: choose the segmentation as *Fixed image* and the mineral map (marked "map") as *Moving image*; **Automatic** (with *Search* (*automatic*) "shift", "any rotation" or "rotation and scale" when the two have no common section frame); **Save the registration**. The map is then laid over the picture in Atlas and Lab.
2. **MC → Segmentation → Effects on the segmented volume**: nothing changes for the user — a volume too big to be held whole is now edited at full resolution, a few planes at a time.

How it works

- Alignment: the pores of each map (or, without pores, the boundaries between any two phases) become a grey image; the two are aligned by ECC on their gradients over a pyramid (Evangelidis & Psarakis 2008), starting from the section frames or from the search, as Lab aligns two pictures (0.17).
- Effects: each slab of planes is read with the margin in z the effect needs (the margin or the width plus one plane, the smoothing size plus one) and cut back after the effect — the same result as on the whole volume. For *expand* the nearest segment is looked for within the slab and 16 planes around it (`halo`).

From a script

```
t = prj.image("Mineral map").register_on("Segmentation", search="shift")    # two maps: by their
↪  pores
rf.lab.register_map(prj.raw, seg_id, map_id, fixed_phases=[1], map_phases=[3])
rf.vol_tools.edit_labels(prj.raw, labels_id, "boundary_removal", width=2)    # big volumes: slab
↪  by slab
```

Limits and common problems

- The phases chosen must be the same thing in both maps (pores of the picture and the porosity of the map); with very different phase sets use landmark pairs.
- Islands, logical operations, grow from seeds and level tracing still work on a reduced level of very big volumes: they need the whole volume at once.

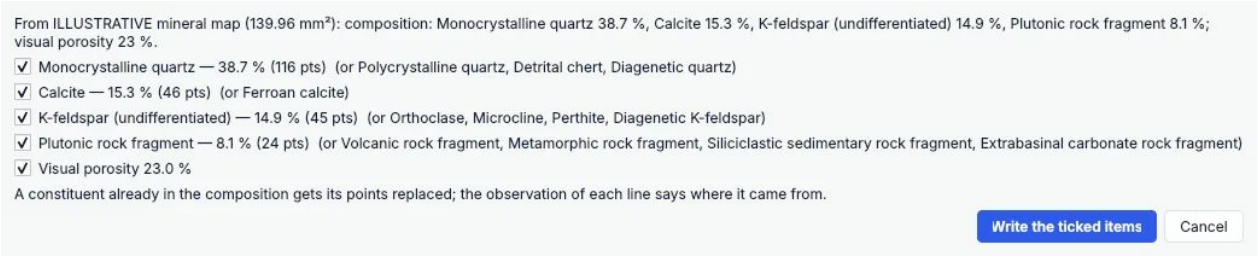
Coverage

0.23 audit rows ThinSectionAutoRegistration (GT5), BoundaryRemovalBigImage and ExpandSegmentsBigImage (GV6, GV8).

Suggest description: from a segmentation map to the description

What it is for

A segmentation map already says how much of the section each mineral and the pores cover. **Suggest description** turns it into proposals for the description of the thin section — a composition line for each constituent with its points, the visual porosity, and, from an object map of grains, the main grain-size mode and the sorting — and writes only what you tick. It saves typing what the map already measured and keeps the petrographer in charge.



In the program: Suggest description: from a segmentation map to the description

Before you start

- In Lab, a thin section with a segmentation map: made in **Lab → Segmentation**, by an AI model in **Lab → AI**, or imported as a mineral map in Atlas.
- A description of that thin section (created in Petro).
- The phases of the map named with concepts of the nomenclature (the map's legend; Atlas → mineral map → phases).

Step by step

1. **Lab** → choose the thin section; open the map (or keep any image: the last map of the section is used).
2. Click **Suggest description** in the toolbar. If the section has several descriptions, choose one.
3. The dialog shows the suggestion as a sentence and one checkbox per item:
   - each constituent with its share of the counted area and its points of a 300-point count — when the map's phase allows several constituents (a phase named "quartz"), the others are listed (*or* …) to check;
   - the visual porosity (the pore phases);
   - the grain-size mode and the sorting, when an object map of grains is given (script).
4. Untick what you do not want and click **Write the ticked items**. Each line written says in its observation that it was suggested from the map.

In the Mio panel, `@mio sugira a descrição de ILL-NL-003` (or the skill *sample report*) shows the same suggestion as text, without writing anything.

Options

Item	Written into
Composition line	the composition: a new line with the points, or the points of the same constituent replaced
Visual porosity	the pore system: total porosity
Grain size	the thin-section texture: main mode (mm and Wentworth class)
Sorting	the thin-section texture: sorting class and σI

How it works

- The map is quantified with the background and unclassified phases left out (as in Atlas → mineral map); the share of each phase becomes points of a 300-point count, rounded (a share below one point is listed but not applicable).
- A phase's concepts give the constituent: the first constituent named, or the first constituent of a group named; its kind (primary, diagenetic, pore) comes from the knowledge base.
- Grain size and sorting are the graphic measures of Folk & Ward (1957) of the maximum Feret diameters of the grain objects, classed by Wentworth (1922) and Folk & Ward's sorting terms.
- The sentence is written from a template in English, Portuguese or Spanish, so every number is a number of the map; a language model may rephrase it, and the rephrasing is kept only with exactly the same numbers.

From a script

```
d = prj["ILL-NL-003"]
s = d.suggestion()                                  # the last map of its thin section
print(rf.suggest.text(s, "pt", prj.kb))
print([(i.key, i.percent, i.points) for i in s.items])
d.apply_suggestion(s, accept=["porosity", "line:calcite_dia"])
s = prj.suggest_description("my map", objects="my grains")      # with grain size and sorting
```

Limits and common problems

- "This thin section has no segmentation map": make one in Lab → Segmentation or Lab → AI.
- A phase named only by a kind ("Cement") is listed but not applicable: name it with a constituent in the map's legend.
- The points replace those of the same constituent in the composition (without habit or location); lines with a habit or a location are left as they are.

Coverage

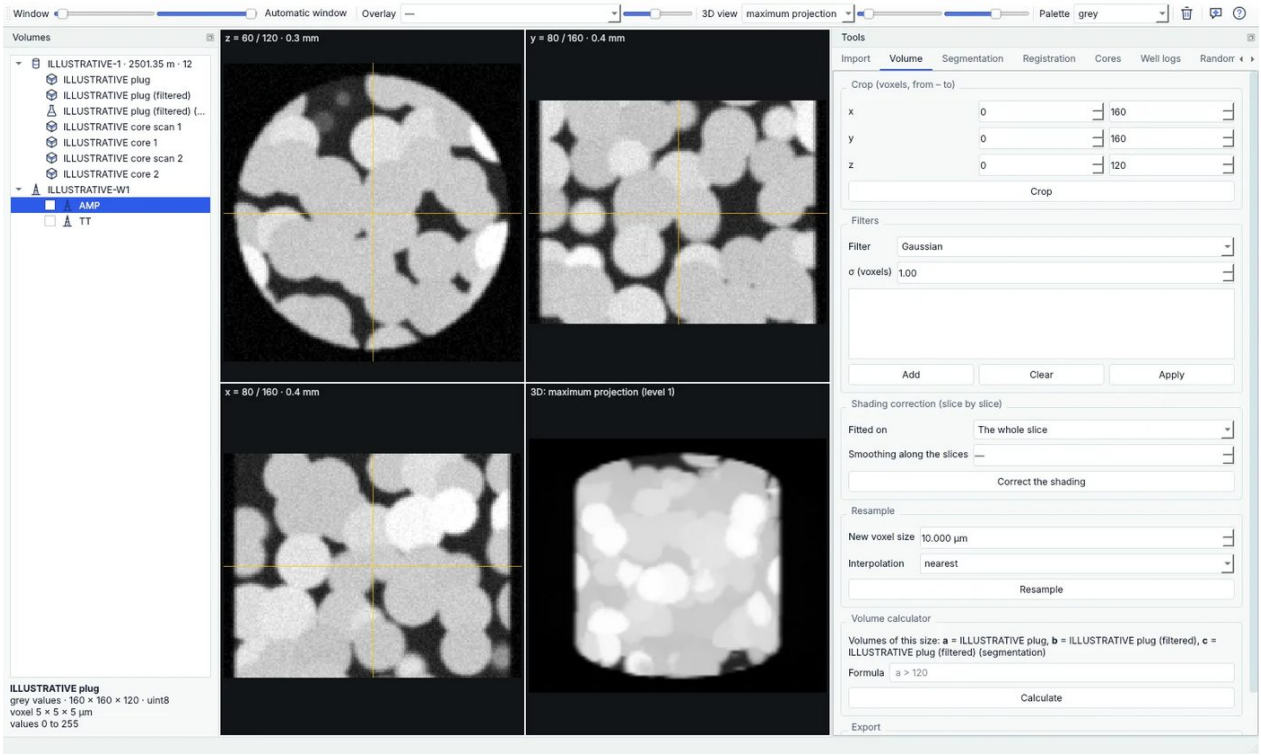
“Segmentation turned into description suggestions” of the ROADMAP stage 0.27; L7 (whole-section quantification) carried into the description.

RockFace MC

RockFace MC: micro-CT volumes — import, views and processing

What it is for

RockFace MC keeps the micro-CT volumes of each sample — a plug, a whole core, a cuttings pot — next to its thin sections and descriptions. It imports the usual formats, stores volumes of any size out of core, shows them in three slices and in 3D, and crops, filters, resamples, combines and exports them. Every operation writes a new volume with its history; the original is never changed.



In the program: RockFace MC: micro-CT volumes — import, views and processing

Before you start

- A saved description: a volume belongs to the description's sample (and may be tied to one of its thin sections).
- Open MC: **Home → Images → MC** (Ctrl+Shift+M) in RockFace Petro, or `rockface mc project.rockface`.

Step by step

The window

- **Volumes** on the left, by sample (and the wells with their logs, page RockFace MC: image logs); select one to show it.
- Three slice views (x, y, z) and the **3D view** (**maximum projection**, **mean projection**, or **off**). Click in a slice to move the others; the status shows "Voxel x, y, z: value".
- **Window** (grey levels shown) and **Automatic window**; **Overlay** lays a segmented volume over the grey one.
- The tabs: **Import**, **Volume**, **Segmentation**, **Registration**, **Cores**, **Well logs**, **Random forest**, **Analyses**, **Pore network**, **Microporosity**, **Multiscale**, **AI**, **More**.

Importing a volume

1. Open **Import**. **File...** or **Folder...**: a folder of slices, a TESCAN or DICOM folder, a multi-page TIFF, RAW, MetaImage (.mhd/.mha), NRRD, NetCDF, HDF5, Amira (.am) or Imperial ASCII (.dat).
2. For RAW: **Size (RAW)** in voxels, **Type (RAW)**, **Byte order (RAW)** (little- or big-endian) and **Header to skip (RAW)**. The other formats say it in their header.
3. **Voxel size** (from the file, or typed in µm).
4. **Content**: **grey values** or **segmented**; for a segmented volume, **Values and segments** maps each value of the file to a segment.
5. **Sample** and, optionally, the **Thin section**; the **Name**.
6. **Import**. "... imported: nx × ny × nz voxels."

Public micro-CT images can be downloaded with their licence: **Open rock data (public images)...** (page Open rock data: public micro-CT images).

Processing (the Volume tab)

- **Crop (voxels, from – to)** for x, y and z, then **Crop**.
- **Filters**: **Filter** (Gaussian, median, mean, unsharp mask, window to 8 bits, invert, anisotropic diffusion) with its parameters, **Add** to the list, **Clear**, **Apply**. Filters run block by block.
- **Shading correction (slice by slice)** — page Volume tools: filters, shading, segment editor effects, objects, registration, analyses.
- **Resample**: **New voxel size**, **Interpolation** (nearest, linear, cubic), **Resample**.
- **Volume calculator**: a **Formula** over the volumes of the same size (listed in "Volumes of this size:"), e.g. `(a - b) / 2` or `a > 120` (a condition gives a segmented 0/1 volume); **Calculate**.
- **Export**: **Export...** to .mhd (+ .raw), .mha, .nrrd, .tif or .nc, with the voxel size.
- **Delete volume** (it cannot be undone).

Options

Option	What it changes
Content	Grey values, or segmented (labels with their segments).
Interpolation	Nearest for segmented volumes; linear or cubic for grey values.

Option	What it changes
Window	Only the display; the stored values are not changed.

How it works

Volumes are stored in the project as compressed chunks of 64^3 voxels with a pyramid of halved levels, so the views read only what they show and a volume larger than the memory is processed a block at a time: 403 million voxels were imported and segmented with about 1 GB of memory. Filters block by block equal the whole volume at once; five formats were read back identical.

From a script

```
d = prj["TS-9"]
ct = d.import_volume("plug.mhd")                                  # or prj.import_volume(path, d,
↪  voxel_um=5.0)
small = ct.crop((0, 400, 0, 400, 0, 400)).resample(factor=0.5)  # box (z0, z1, y0, y1, x0, x1) in
↪  voxels
smooth = ct.filter(("median", {"size": 3}), ("gaussian", {"sigma": 0.8}))
diff = prj.calculate_volumes("a - b", {"a": ct, "b": smooth}, name="residual")
print(ct.shape, ct.voxel_um)                                       # (nz, ny, nx), µm
smooth.export("plug_smooth.nrrd")
```

Limits and common problems

- “Choose a file or folder and a sample.”
- A RAW file read with the wrong size or type looks like noise: check the size, type and byte order.
- Very large imports take time; the progress shows and the work can be cancelled.

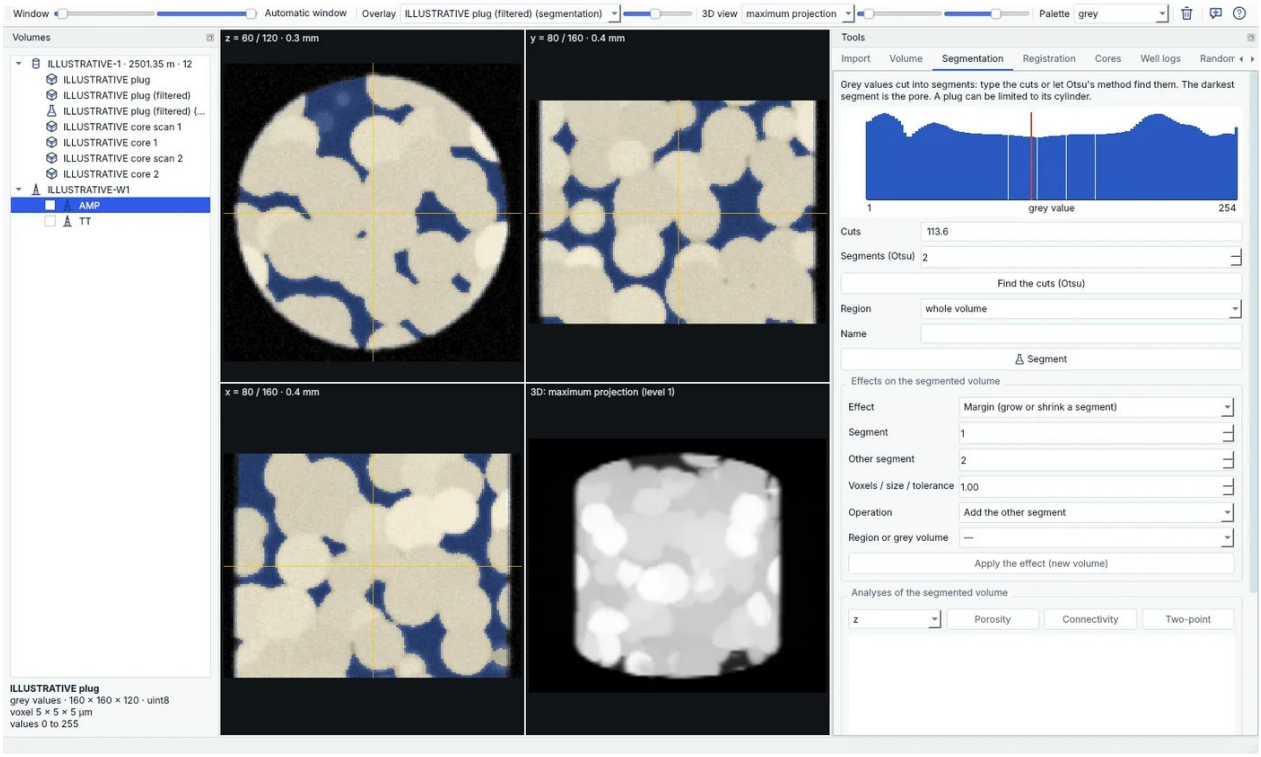
Coverage

GV1 (import), GV2 (export), GV3 (crop), GV4 (filters), GV5 (resample), GV8 (large images out of core), GV14 (volume calculator), L2 (links from samples to tomography).

RockFace MC: segmentation, porosity and registration of volumes

What it is for

The **Segmentation** tab cuts the grey values of a micro-CT volume into segments — pore and solid, or more phases — by typed cuts or by Otsu's method, limited to the cylinder of a plug when needed, and measures the porosity, its profile along an axis, the connectivity of the pore space and its two-point correlation. The **Registration** tab aligns two volumes of the same sample, and finds where a thin-section image lies in a volume, adding the matching slice to the thin section.



In the program: RockFace MC: segmentation, porosity and registration of volumes

Before you start

A volume in RockFace MC (page RockFace MC: micro-CT volumes — import, views and processing), filtered if noisy.

Step by step

Segmenting

1. Open **Segmentation** and select the grey volume.
2. **Cuts**: type them, or leave empty for Otsu (**empty: Otsu**); **Segments (Otsu)** gives the number of segments; **Find the cuts (Otsu)** shows them on the histogram.
3. **Region**: the **whole volume**, the **cylinder of the plug** (found automatically), or **inside** another segmented volume.
4. **Segment**. "Segmented: porosity ... %." The darkest segment is the pore. The segmented volume appears under the sample, and can be laid over the grey one with **Overlay**.

Analyses of the segmented volume

- **Porosity** — the porosity of the region, and the porosity of every slice along an axis (**slice along x, y, z**).
- **Connectivity** — along an axis: connected and isolated porosity, the number of pore clusters, and whether the pore space crosses the volume.
- **Two-point** — the two-point correlation of the pores along x, y and z, and the correlation length of each.

Effects on the segments (grow, shrink, smooth, logical operations with another segment) are in **Effects on the segmented volume** (page Volume tools: filters, shading, segment editor effects, objects, registration, analyses).

Registering two volumes

1. Open **Registration**. In **Two volumes of the sample**, choose the **Fixed** and the **Moving** volume.
2. **Find the shift (3D phase correlation)**: "Shift found: x, y, z voxels", kept on the moving volume. Tick **Find the rotation too (no landmarks)** for a rotated scan (page Volume tools: filters, shading, segment editor effects, objects, registration, analyses).
3. **Resample the moving volume on the fixed one** writes it on the fixed grid, voxel for voxel.

A thin-section image in the volume

1. Under **A thin-section image in the volume**, choose the **Image** (a photo or slide of one of the sample's thin sections) and **Slices across** (the axis).
2. **Locate the image**. RockFace looks for the slice that shows the image, with its rotation, scale and shift: "Slice z = ...; rotation ...°, ... voxels per pixel; ECC ...".
3. **Add the matching slice to the thin section**: the slice of the volume is added to the section's images in Atlas, laid over the image — the link between the thin section and the tomography.

Options

Option	What it changes
Cuts / Segments (Otsu)	Where the grey values are cut, typed or by multi-level Otsu.

Option	What it changes
Region	Whole volume, the plug's cylinder, or inside a segmented volume.
Slices across	The axis along which the thin section was cut from the plug.

How it works

- The porosity of the Berea sandstone of Imperial College London (400^3 voxels, 5.345 µm) is 19.645 % against the published 19.6 %; a grey scan simulated from it and segmented by Otsu gave 19.54–21.06 %.
- The shift between two volumes is found by 3D phase correlation (a known shift found exactly); landmarks fit a rigid, similarity or affine transform.
- A thin-section image is located by a search of slice, angle and scale, refined by the enhanced correlation coefficient: pictures of a slice at other pixel sizes and angles (20° to 137°) were placed on the right slice within 0.01–0.83 voxel.

From a script

```
ct = prj.volume("ILLUSTRATIVE plug")
pores = ct.filter(("median", {"size": 3})).segment(roi="cylinder")          # Otsu, two segments
print(pores.porosity()["porosity"], pores.connectivity("z")["percolates"])
print(pores.profile("z")[:5], pores.two_point()["length_x"])
other = prj.volume("ILLUSTRATIVE plug, second scan")
other.register(ct)                                                           # the automatic shift
aligned = other.resample_on(ct)
found = prj["TS-9"].images("photo")[0].locate_in(ct, axis="z", add_slice=True)
```

Limits and common problems

- "Choose two different volumes."
- A thin-section image is found only if the volume holds that slice: give the axis across which the section was cut.
- A plug's cylinder is found from the air around it: on a crop without air, use the whole volume or a region.

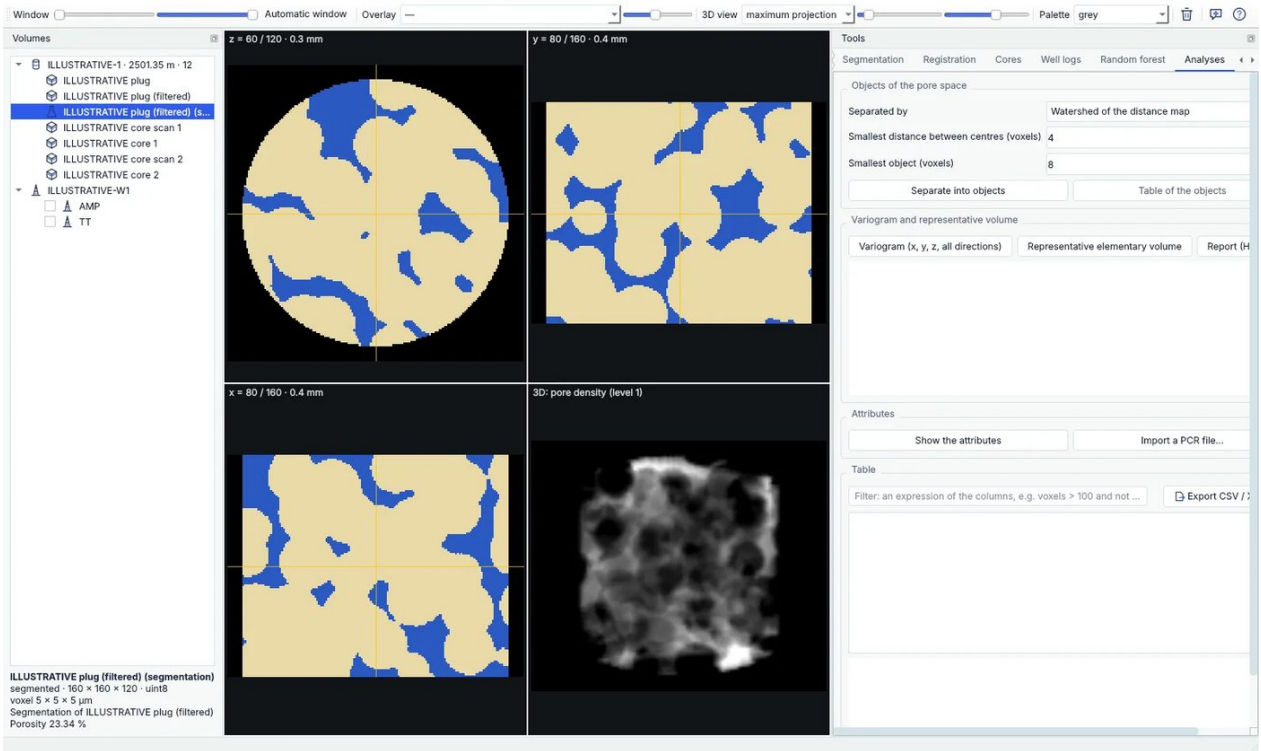
Coverage

GV6 (segmentation), GV7 (registration), L2 (links from samples to tomography), L13 in part (2D image → tomography).

Volume tools: filters, shading, segment editor effects, objects, registration, analyses

What it is for

The tools of RockFace MC that prepare a micro-CT volume, correct its segmentation and measure it: an edge-preserving filter, the correction of the shading (beam hardening), the effects of a segment editor, the separation of the pore space into objects, the registration of two scans turned with respect to each other, and the variogram, the representative elementary volume and the attributes of a volume.



In the program: Volume tools: filters, shading, segment editor effects, objects, registration, analyses

Before you start

A volume open in RockFace MC. Every tool writes a **new** volume; the original is never changed.

Step by step

Anisotropic diffusion (Volume tab)

1. **Volume → Filters**, choose **Anisotropic diffusion** (**edge-preserving**).
2. Set **Conductance** (lower keeps more edges) and **Iterations** (more smooths more), press **Add**, then **Apply**.

Shading correction (Volume tab)

1. **Shading correction** (**slice by slice**): choose where the background is fitted — **The whole slice**, the cylinder of the plug, or inside a segmented volume.
2. Optional: **Smoothing along the slices** (in slices).
3. Press **Correct the shading**.

Effects on the segmented volume (Segmentation tab)

Open a segmented volume. In **Effects on the segmented volume** choose the **Effect**, the **Segment** and, when asked, the **Other segment**, the **Voxels / size / tolerance** or the **Region or grey volume**, then **Apply the effect** (**new volume**):

- **Margin** grows (positive voxels) or shrinks (negative) a segment;
- **Smoothing** removes the roughness of a segment;
- **Islands** removes the pieces smaller than the size given;
- **Logical operators** add, subtract, intersect, copy, invert or clear segments;
- **Mask** keeps only what is inside another segmented volume;
- **Expand segments** fills the voxels at 0 with the nearest segment;
- **Boundary removal** empties the voxels where two segments touch;
- **Grow from seeds** grows the painted voxels of an annotation over a grey volume;
- **Level tracing from the crosshair** takes the connected voxels of a grey volume close to the value under the crosshair.

Objects of the pore space (Analyses tab)

1. Open a segmented volume. In **Objects of the pore space** choose **Separated by**: *Watershed of the distance map* (touching pores are split) or *Connected pieces*.
2. Set **Smallest distance between centres** and **Smallest object**, press **Separate into objects**.
3. The table lists every object: voxels, volume, equivalent diameter, axes, centroid, the radius of the largest inscribed sphere, and whether it touches the border.

Registration with rotation (Registration tab)

Choose the **Fixed** and **Moving** volumes, tick **Find the rotation too** (**no landmarks**) and press **Find the shift** (**3D phase correlation**). The rotation (about x, y, z) and the shift are kept on the moving volume; **Resample the moving volume on the fixed one** writes it on the fixed grid.

Variogram, REV and attributes (Analyses tab)

- **Variogram (x, y, z, all directions)**: the semivariogram of a grey volume, or of the pores of a segmented one, with the fitted model (range, sill, nugget) in the table.
- **Representative elementary volume**: the standard deviation of the mean of random cubes of growing edge; the REV is where the curve levels off.
- **Report (HTML)...** writes the plots and tables in one page.
- **Show the attributes** lists size, voxel, values, history; **Import a PCR file...** adds the reconstruction parameters of the scanner (the Min and Max of the grey scale).

The **Table** of the tab is filtered by an expression (`voxels > 100 and not border`), exported to CSV or Excel, and shows the histogram of the chosen column.

How it works

- Anisotropic diffusion: Perona & Malik (1990); the conductance is scaled by the root mean square of the gradient, as in ITK's gradient anisotropic diffusion. Computed block by block, the same as the whole volume.
- Shading: on each slice, a second-degree polynomial f(x, y) is fitted to random points of the mask and the slice becomes s / f · M (M the mean of the mask).
- Objects: the watershed (Vincent & Soille 1991) of the smoothed distance map, one marker per local maximum.
- Rotation: a search of the angle about z with the 3D phase correlation of the shift, then the six parameters refined by the normalized cross-correlation.
- Variogram: Matheron (1963); spherical model; with a region, only the pairs with both ends inside it count.

From a script

```
vol = prj.volume("Plug A")
smooth = vol.filter(("anisotropic", {"conductance": 1.0, "iterations": 5}))
flat = vol.shading_correction("cylinder")
seg = flat.segment()
objects = seg.partition(min_distance=4)
rows = objects.objects()
t = prj.volume("Plug A, second scan").register_rigid(vol)
v = seg.variogram()
seg.variogram_report("variogram.html")
bigger = seg.edit("margin", segment=1, voxels=2)
```

Limits and common problems

- The effects and the objects work on the whole volume in memory: above 64 million voxels they work at a reduced level of the volume.
- The rotation search assumes the same voxel size in both volumes.
- The Gabor, Minkowski and variogram-range features of GeoSlicer's segmenter are not offered.

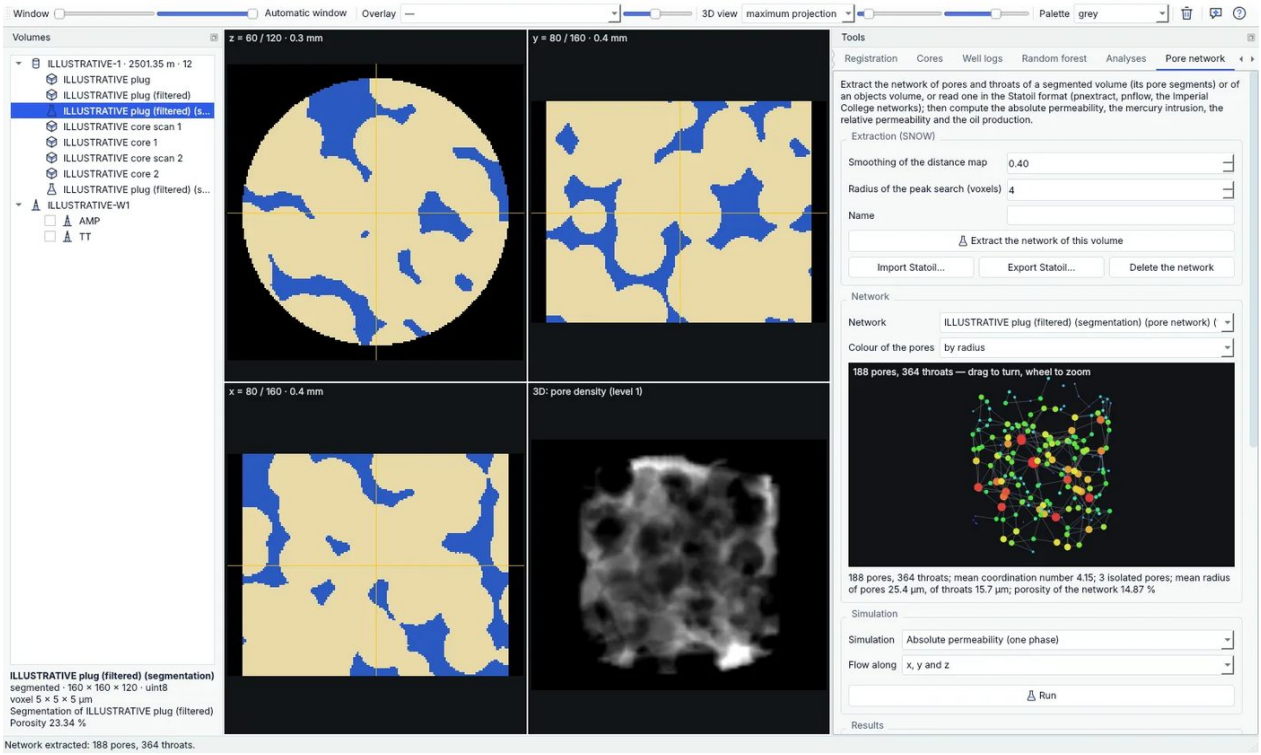
Coverage

GV4, GV6, GV7, GV14 (Filters, Polynomial Shading Correction, Segment Editor, Segment Inspector, Register, Variogram Analysis, Attribute Tools, Tables).

Pore network: extraction, permeability, mercury intrusion, relative permeability, production

What it is for

The pore space of a micro-CT volume turned into a network of pores (the bodies) joined by throats (the constrictions), on which the flow is computed: the absolute permeability along x, y and z, the mercury intrusion curve (MICP) to compare with the laboratory, the relative permeability curves of oil and water (drainage then imbibition), the oil recovery of a waterflood, and how all of these change with the wettability.



In the program: Pore network: extraction, permeability, mercury intrusion, relative permeability, production

Before you start

- A segmented volume of the sample in RockFace MC (tab Segmentation), with the pore segments marked with the role “pore” — or an objects volume (tab Analyses → Objects of the pore space), whose objects become the pores.
- Or a network in the Statoil format (the four files `*_node1.dat`, `*_node2.dat`, `*_link1.dat`, `*_link2.dat` of pnextract and pnflow, as the Imperial College networks).

- A large volume is extracted at the finest level of its pyramid under 64 million voxels; crop it first (Volume → Crop) to work at full resolution.

Step by step

Extract the network

1. Open the segmented volume and go to the **Pore network** tab.
2. In **Extraction (SNOW)**, keep **Smoothing of the distance map** 0.40 and **Radius of the peak search** 4 voxels (the values of the SNOW algorithm), give a **Name** if you wish, and click **Extract the network of this volume**.
3. The network appears in **Network** with its numbers (pores, throats, mean coordination number, isolated pores, mean radii, porosity of the network), drawn in 3D: drag to turn, wheel to zoom; **Colour of the pores** by radius or by coordination number. The regions of the pores are kept as an objects volume.
4. **Import Statoil...** reads a network made elsewhere; **Export Statoil...** writes the current one (flow along the axis chosen in **Flow along**); **Delete the network** removes it from the project.

Run a simulation

Choose the **Simulation**, its parameters, and click **Run**. The chart and the table show the result; every result is kept with the network.

- **Absolute permeability (one phase)** — K along x, y, z (or one axis) in mD and m².
- **Mercury intrusion (MICP)** — the mercury saturation against the pressure (log), with the throat radius of each step in the table.
- **Relative permeability (two phases)** — drainage (oil into the water-filled rock) then imbibition (water back): krw and kro against Sw, Pc of each step; Swi, Sor and the end points in the status bar.
- **Sensitivity test** — the two-phase run repeated for every value of one parameter (**Vary**, values separated by ;): the cloud of curves and its mean, the table of the results of each run and the correlations of the parameter with each result.
- **Kabs REV** — the volume cut into centred cubes of growing edge, each extracted again, and the permeability of each size: where it stops changing, the volume is representative.

After a two-phase run

- **Animate the cycles** — the pores coloured by their fluid at each step (oil red, water blue); the slider chooses the step.
- **Production prediction** — the waterflood of the imbibition curve (Buckley–Leverett): oil recovered against the pore volumes of water injected; after a sensitivity test, every curve and the pessimistic, neutral and optimistic predictions (P10, P50, P90).
- **Reference curve (CSV)...** — a measured krel curve (columns `Sw`, `krw`, `kro`) drawn with the simulated one.
- **Report (HTML)...** — one page with the network, the permeability, MICP, krel, production and REV.

Options

- **Mercury**: surface tension 0.485 N/m and contact angle 140° by default.
- **Oil–water interfacial tension** (0.03 N/m), **viscosities** of water and oil.
- **Receding contact angle** (drainage) — its centre and **range** (the angles of the elements spread uniformly over the range); **Advancing − receding angle**: the advancing angle of imbibition. Below 90° the rock is water-wet, above 90° oil-wet.
- **Sw step** — the saturation between two computed points; **Minimum Swi** — where drainage stops.
- **Lower / upper plane of the box** — the saturation and the flow are computed between these planes (0.1 and 0.9 of the length by default, as pnflow's calculation box): the pores beyond them hold the pressures.

How it works

- **Extraction**: the distance map of the pore space, smoothed, its maxima as markers (a marker nearer to a higher one than to the solid dropped), the watershed: each region a pore, each contact of two regions a throat (Gostick 2017). Pore radius: the largest distance in the region; throat radius: the largest on the contact; shape factor of a throat $G = r^2 / (4\,A)$ from its area A.
- **Flow**: each conduit (half pore, throat, half pore) three ducts in series, $g = k\,G\,A^2 / \mu$ ($k = 3/5$ triangle, 0.5623 square, 1/2 circle; Patzek & Silin 2001); the pressures of the pores solved; $K = Q\,\mu\,L / (A\,\Delta P)$.
- **MICP**: mercury enters from every face, an element at $P = -2\,\sigma\cos\theta / r$ (Washburn 1921) if a neighbour is already filled.
- **Two phases**: drainage by invasion percolation from the inlet face, water kept in the corners; imbibition by snap-off in throats, $\sigma\,(\cos\theta - \sin\theta\tan\beta) / r$, and piston-like filling, $2\,\sigma\cos\theta / r$, oil trapped when it loses its path to the outlet; kr from the conductances of each phase (Valvatne & Blunt 2004, simplified).
- **Production**: the fractional flow, the Welge tangent and the recovery (Buckley & Leverett 1942; Welge 1952).

From a script

```
seg = prj.volume("ILLUSTRATIVE sphere pack")
net = seg.extract_network(name="ILLUSTRATIVE network")
print(net.stats()["coordination_mean"])
k = net.kabs()                          # {"x": {"K_mD": …}, …, "mean_mD"}
m = net.micp()
kr = net.two_phase(theta_r=30, separation=20, swi_min=0.2)
pr = net.production(kr, mu_w=1e-3, mu_o=5e-3)
runs = net.sensitivity({"theta_r": [20, 40, 60]})
net.export_statoil("pack")              # pack_node1.dat …
net.report("network.html")
berea = prj.import_statoil("Berea_node1.dat")
```

Limits and common problems

- **“Nothing flows from face to face”** — the pores do not connect the two faces along that axis: try another axis, or the segmentation misses the small pores.
- The two-phase model is a simplified one: no cooperative pore filling, no oil layers, no film swelling; in a strongly water-wet rock snap-off dominates and kro falls early. Compare with measured curves (Reference curve) before using the values.
- The extracted radii come from the distance map at voxel centres: throats one voxel wide are not resolved.
- Extracting 64 million voxels (400^3) takes about 1.5 minutes and 3.5 GB of memory.
- Microporosity (unresolved pores) and multiscale networks come in stage 0.25.

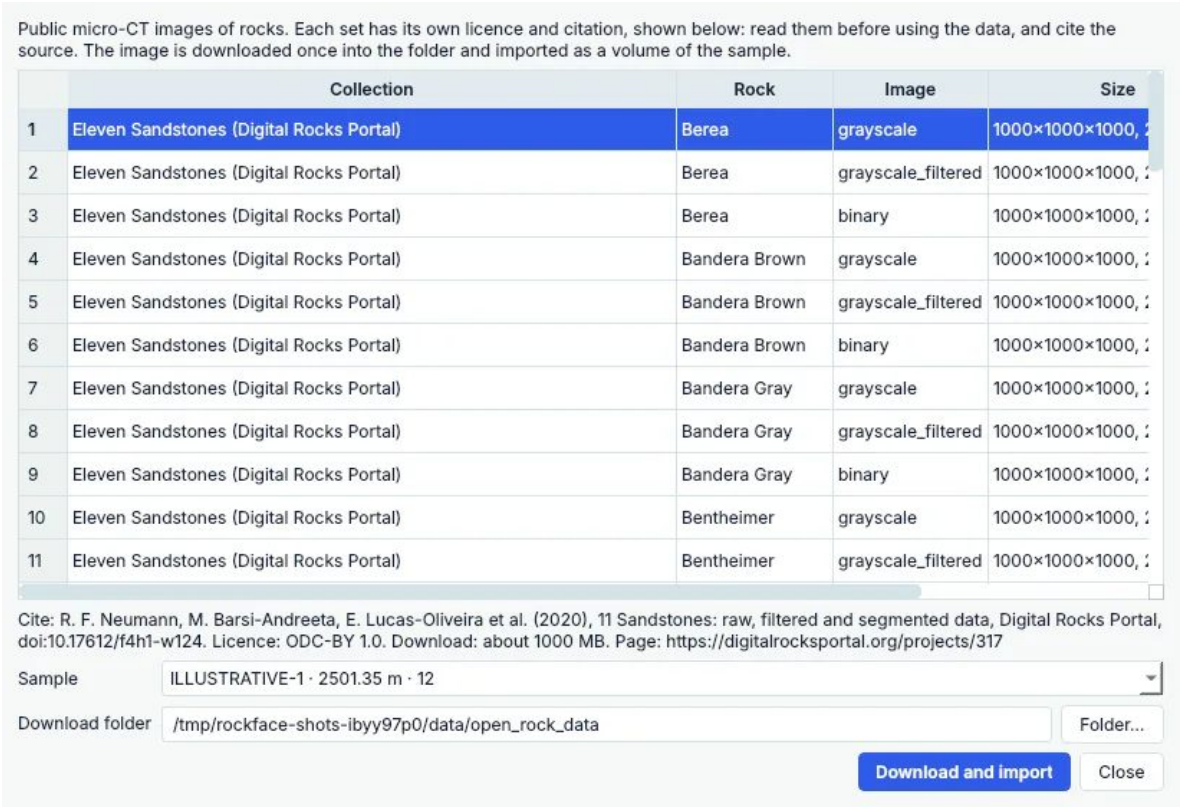
Coverage

GV12 (Pore Network: Extraction, Simulation — one phase, two phases, mercury injection —, Kabs REV, Krel EDA, Production Prediction, Cycles Visualization, PNM report).

Open rock data: public micro-CT images

What it is for

To try RockFace MC, compare methods or teach, without images of your own: public micro-CT images of sandstones and carbonates, downloaded and imported into a sample with their licence and citation.



In the program: Open rock data: public micro-CT images

Before you start

- A project with at least one sample (a description), and an Internet connection.
- Room on the disk: an image of 1000^3 voxels takes about 1 GB, and the 2015 Imperial College archive several GB.

Step by step

1. In RockFace MC, tab **Import**, click **Open rock data (public images)...**.

2. Choose an image in the table (collection, rock, image, size, licence). Below the table: the citation, the licence, the size of the download and the page of the source. **Read the licence before using the data, and cite the source.**
3. Choose the **Sample** that receives the volume and the **Download folder**.
4. Click **Download and import**. The file is downloaded once (a second import reuses it) and imported as a volume of the sample; its caption holds the citation and the licence.

Options

- **Eleven Sandstones** (Digital Rocks Portal): eleven sandstones at 2.25 µm, 1000^3 voxels, in three forms — grey, filtered and binary. Licence ODC-BY 1.0.
- **ICL sandstones and carbonates 2009** (Imperial College London): binary images of Berea, the S and LV60 sandstones and carbonates, in 7-Zip archives (the package `py7zr` is needed to open them).
- **ICL micro-CT 2015** (Imperial College London): Bentheimer, Doddington, Estaillades and Ketton, 1000^3, grey, in one archive of several gigabytes.

How it works

The list of images, addresses, sizes and voxel sizes follows the `drd` library of L. Mosser (Apache 2.0), which GeoSlicer uses; the files are downloaded through a temporary `.part` file and imported out of core, a block at a time.

From a script

```
for e in rf.ProjectAPI.rock_data()[:3]:
    print(e.id, e.rock, e.licence)
v = prj.download_rock_data("eleven/Berea/binary", prj.descriptions()[0], folder="data")
```

Limits and common problems

- **"The download ... failed"** — no connection, or the source moved the file; try again later or download it from the page of the source and import it with **Import** (RAW, 1000 × 1000 × 1000, 8 bits).
- **"This archive is in the 7-Zip format"** — install `py7zr` (`pip install py7zr`).
- The binary images are imported as grey volumes with two values: segment them with a threshold (tab Segmentation).

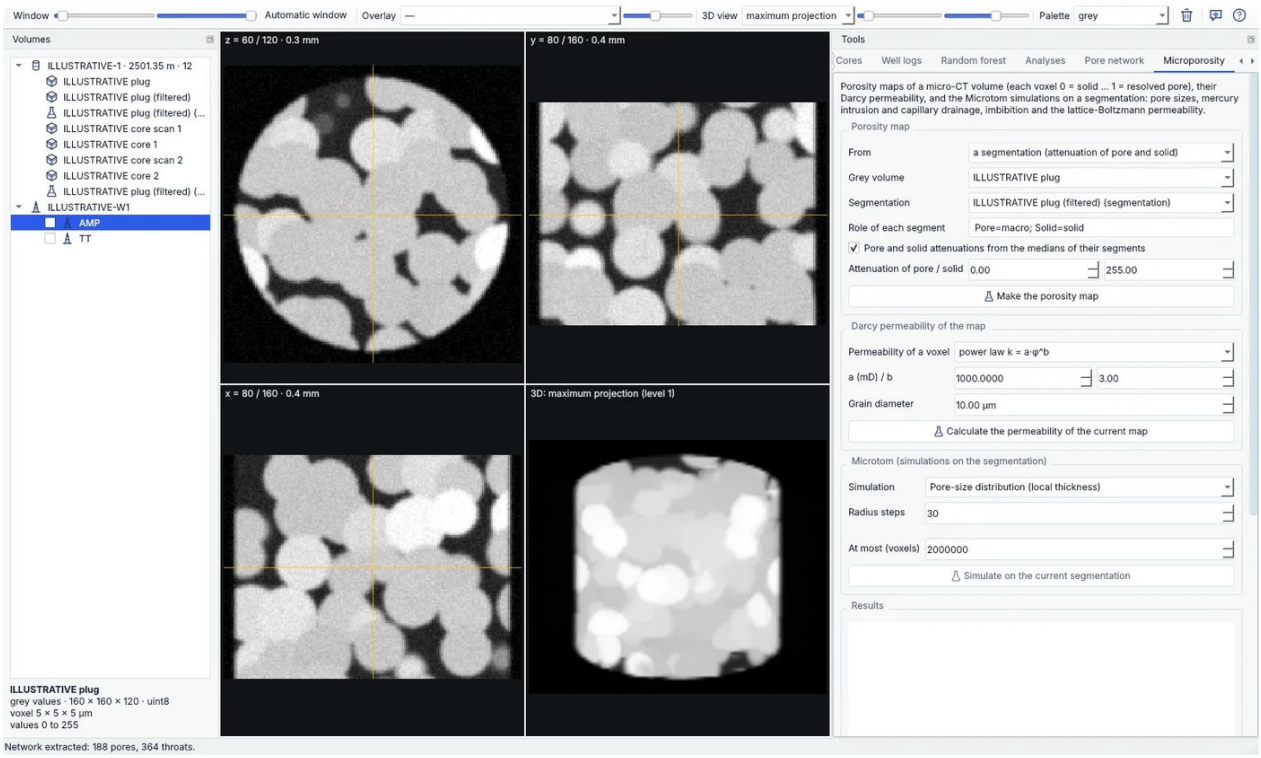
Coverage

GV17 (Open Rock Data).

Microporosity: porosity maps, Darcy permeability and the Microtom simulations

What it is for

A micro-CT volume resolves the large pores but not the pores smaller than its voxel: a voxel of microporous cement or clay is grey, between the pore and the grain. The **Microporosity** tab of RockFace MC turns the volume into a **porosity map** (each voxel 0 = solid … 1 = resolved pore, the microporosity in between), computes the **Darcy permeability** of that map, and runs the **Microtom** simulations on a segmentation: pore-size distributions, mercury intrusion and capillary drainage, imbibition, and the absolute permeability by lattice Boltzmann.



In the program: Microporosity: porosity maps, Darcy permeability and the Microtom simulations

Before you start

- A grey micro-CT volume and its segmentation on the same grid (tab Segmentation), with a segment for the resolved pores, one for the microporous phase and one for the grains.
- For the method from saturation: a dry scan and a scan of the same plug saturated with a contrast fluid, registered on the same grid (tab Registration), and the segmentation.
- The Microtom simulations work on a segmentation with the role “pore” on its pore segments.

Step by step

Make a porosity map

1. Open the grey volume and go to the **Microporosity** tab.
2. In **Porosity map**, choose **From**: *a segmentation* or *a dry and a saturated scan*.
3. Choose the **Grey volume** (or the **Dry scan** and the **Saturated scan**) and the **Segmentation**.
4. Check the **Role of each segment**, written as `Segment=role; …`: `macro` (resolved pore, porosity 1), `micro` (microporous: porosity from the attenuation), `solid` (porosity 0) and `high` (dense minerals, porosity 0). The proposal comes from the segment names and roles; type over it to change it.
5. Keep **Pore and solid attenuations from the medians of their segments**, or untick it and type the **Attenuation of pore / solid**.
6. Click **Make the porosity map**. The map is a new volume (category *porosity*); the table gives the total porosity, the resolved part and the microporosity.

Darcy permeability of the map

1. Open the porosity map.
2. In **Darcy permeability of the map**, choose the **Permeability of a voxel**: *power law* $k = a \cdot \varphi^{b}$ (type a in mD and b) or *Kozeny–Carman* (type the **Grain diameter**). The resolved pores get the permeability of a tube as wide as the voxel.
3. Click **Calculate the permeability of the current map**: K along x, y and z.

Microtom simulations

1. Open the segmentation.
2. Choose the **Simulation** and its options, and click **Simulate on the current segmentation**. A map made by the simulation (pore radius, entry radius) is kept as a new volume; the chart and the table show the curve.

Options

Field	What it changes
Simulation	*Pore-size distribution* (*local thickness*): the radius of the largest ball that holds each pore voxel, as a cumulative curve. *Pore-size distribution* (*non-overlapping spheres*): balls placed largest first without overlap (a hierarchical count). *Mercury intrusion* (*MICP*): σ 0.485 N/m, θ 140°, from one face. *Capillary drainage*: the same with your σ and θ. *Imbibition* and *Imbibition with trapping* (*Sor*): the wetting fluid comes back from the small pores; with trapping, the oil cut off from the outlet stays (Sor). *Absolute permeability* (*lattice Boltzmann*) and its *REV* on cubes of growing edge
Direction of invasion	The face the fluid enters from (x, y or z)
Radius steps	Number of radii of the curves
Interfacial tension / contact angle	σ and θ of drainage and imbibition

Field	What it changes
Relaxation time / tolerance	τ of the lattice-Boltzmann model (1.5 by default) and the relative change of the mean velocity at which the flow is steady
At most (voxels)	The simulation runs on the finest level of the pyramid with at most this number of voxels

How it works

- **Porosity from the attenuation**: in the microporous phase $\varphi = (I_{solid} - I) / (I_{solid} - I_{pore})$, clipped to 0...1; the resolved pores are 1, the solid 0. From a dry and a saturated scan, $\varphi = (\Delta - \Delta_{solid}) / (\Delta_{pore} - \Delta_{solid})$ with $\Delta = I_{saturated} - I_{dry}$, the difference of the two scans relative to those of a resolved pore and of the solid. Both are linear mixing models of the attenuation.
- **Darcy permeability of the map**: each voxel gets $k(\varphi)$ and the pressure equation $\nabla\cdot(k\,\nabla p) = 0$ is solved with a finite-volume scheme (harmonic means between voxels), pressure fixed on two opposite faces; $K = Q\,L / (A\,\Delta p)$. Exact on layered maps (arithmetic and harmonic means).
- **Pore sizes**: local thickness (Hildebrand & Rüegsegger 1997) from the Euclidean distance map.
- **Drainage and MICP**: morphological invasion (Hilpert & Miller 2001): at the radius r the non-wetting fluid fills the balls of radius r whose centres connect to the inlet face; $P = 2\sigma|\cos\theta| / r$ (Washburn 1921).
- **Lattice Boltzmann**: D3Q19, two-relaxation-time collision (Ginzburg, Verhaeghe & d'Humières 2008) with the magic parameter 1/8 that places the bounce-back wall halfway between the nodes; a body force drives the flow and $K = \nu\,\langle u\rangle / g$. The sample is joined to its mirror image along the flow, so the boundaries are periodic. Checked on plane Poiseuille flow (exact) and on arrays of spheres against Sangani & Acrivos (1982).

From a script

```
grey, seg = prj.volume("plug grey"), prj.volume("plug segmentation")
phi, info = grey.porosity_map(seg, {1: "macro", 2: "micro", 3: "solid"})
k = phi.darcy(model="power", a_md=50, b=3)
psd = seg.microtom("psd")
lbm = seg.lbm_permeability(axes=("x", "y", "z"))
```

Limits and common problems

- The attenuation of the microporous phase must lie between those of the pore and the solid; a dense mineral in the microporous segment reads as solid. Give such minerals their own segment with the role `high`.
- The lattice-Boltzmann run grows with the number of pore voxels: a 128^3 cube of a sandstone takes about an hour on two processor cores. Send it as a job (Multiscale tab → Jobs) or reduce **At most (voxels)**.
- Below about 4 voxels across, a throat is not resolved by the lattice Boltzmann model; the voxel size sets the smallest pore that flows.

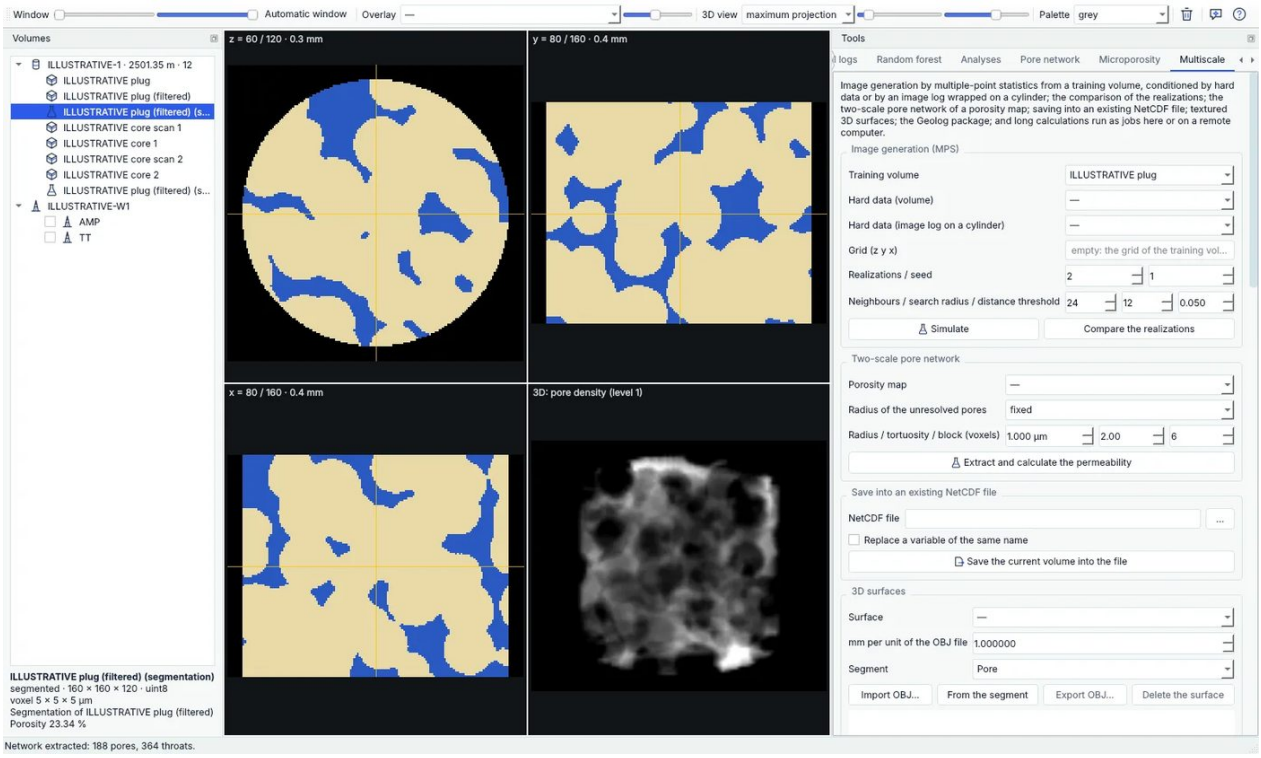
Coverage

GV9 (microporosity), GV10 (Microtom), GV11 (flow modelling and permeability).

Multiscale: image generation (MPS), comparison of the realizations and the two-scale pore network

What it is for

A plug scanned at high resolution covers a few millimetres; an image log or a core scan covers metres at a coarser resolution. The **Multiscale** tab of RockFace MC joins the scales: it **generates 3D images** of the fine scale with the statistics of a training volume, conditioned by what is known at the coarse scale (hard data, or an image log wrapped on the borehole wall), **compares** the realizations with the training image, and extracts a **two-scale pore network** from a porosity map, where the microporous voxels become unresolved pores.



In the program: Multiscale: image generation (MPS), comparison of the realizations and the two-scale pore network

Before you start

- For MPS: a training volume (a segmentation, for a categorical simulation, or a grey or porosity volume, for a continuous one). Optionally a hard-data volume on the grid of the simulation (0 or empty = no datum) or an image log of the well.
- For the two-scale network: a porosity map (tab Microporosity).

Step by step

Generate images (MPS)

1. Go to the **Multiscale** tab; in **Image generation (MPS)** choose the **Training volume**.
2. Optionally choose **Hard data (volume)** or **Hard data (image log on a cylinder)**: the log's columns are laid on the wall of a vertical cylinder of the grid, one row per slice.
3. Type the **Grid (z y x)** of the simulation (empty: the grid of the training volume), the number of **Realizations** and the **seed**.
4. Set the **Neighbours / search radius / distance threshold** of the direct sampling, and click **Simulate**. Each realization is a new volume of the sample (category *simulation*).

Compare the realizations

Click **Compare the realizations**: the chart shows the porosity of every slice of each realization against the training image, and the table the mean pore radius of each (pore-size distribution by local thickness).

Two-scale pore network

1. In **Two-scale pore network** choose the **Porosity map**.
2. Choose the **Radius of the unresolved pores**: *fixed* (type it), *Leverett J* (*from k*), or *from a MICP curve*, and the **tortuosity** and the **block** size (voxels of microporosity gathered into one unresolved pore).
3. Click **Extract and calculate the permeability**. The network is kept in the project (tab Pore network, where all the simulations of 0.24 run on it) with its permeability along x, y and z.

Options

Field	What it changes
Neighbours	Number of informed nodes nearest to the node simulated that make its pattern (24 by default)
Search radius	How far the neighbours are looked for (voxels)
Distance threshold	The training image is scanned until a pattern differs by less than this fraction (0.05: at most one neighbour of twenty in another class)
Radius of the unresolved pores	*fixed*: one radius; *Leverett*: $r = \sqrt{(8k/\varphi)}$ from a permeability k of the microporous phase; *MICP*: radii drawn from a mercury curve, the largest to the most porous blocks
Tortuosity / block	τ of the capillary-bundle model of the microporous plugs; the edge of the cubes of microporous voxels that make one unresolved pore

How it works

- **Direct sampling** (Mariethoz, Renard & Straubhaar 2010): the nodes are visited in a random order; for each one the pattern of its informed neighbours is compared with patterns at random places of the training image, and the value of the first place close enough (or of the closest one scanned) is pasted. Hard data are informed from the start and kept.
- **The two-scale network** (after Bultreys et al. 2015): the resolved pores ($\varphi = 1$) are extracted by SNOW as in 0.24; the microporous voxels are gathered into cubes that become unresolved pores, of volume $\Sigma\varphi$ and radius given by the subscale model. A conduit that touches an unresolved pore is a porous plug of permeability $k = \varphi\, r^2 / (8\,\tau)$ (a bundle of capillaries of radius r and tortuosity τ), conductance $k\,A / L$. A binary map gives exactly the network of 0.24; a uniform microporous block gives exactly k of the plug.

From a script

```
seg = prj.volume("plug segmentation")
reals = seg.mps(realizations=5, shape=(64, 128, 128), log=prj.log("AMP image log"), seed=3)
net = prj.volume("plug porosity map").multiscale_network({"kind": "fixed", "radius_um": 0.5},
↪  tortuosity=2.0)
print(net.kabs())
```

Limits and common problems

- Direct sampling slows down with large training images and strict thresholds: start with a small grid, a threshold of 0.1 and the scanned fraction 0.05 (`fraction` in a script), then refine. Long runs can go as jobs.
- The generative network (RockSinGAN) of the reference program is an AI model; it comes with the AI stage (0.26).
- The subscale models are simple: the microporous plugs are capillary bundles; use a MICP curve of the microporous phase when the laboratory has one.

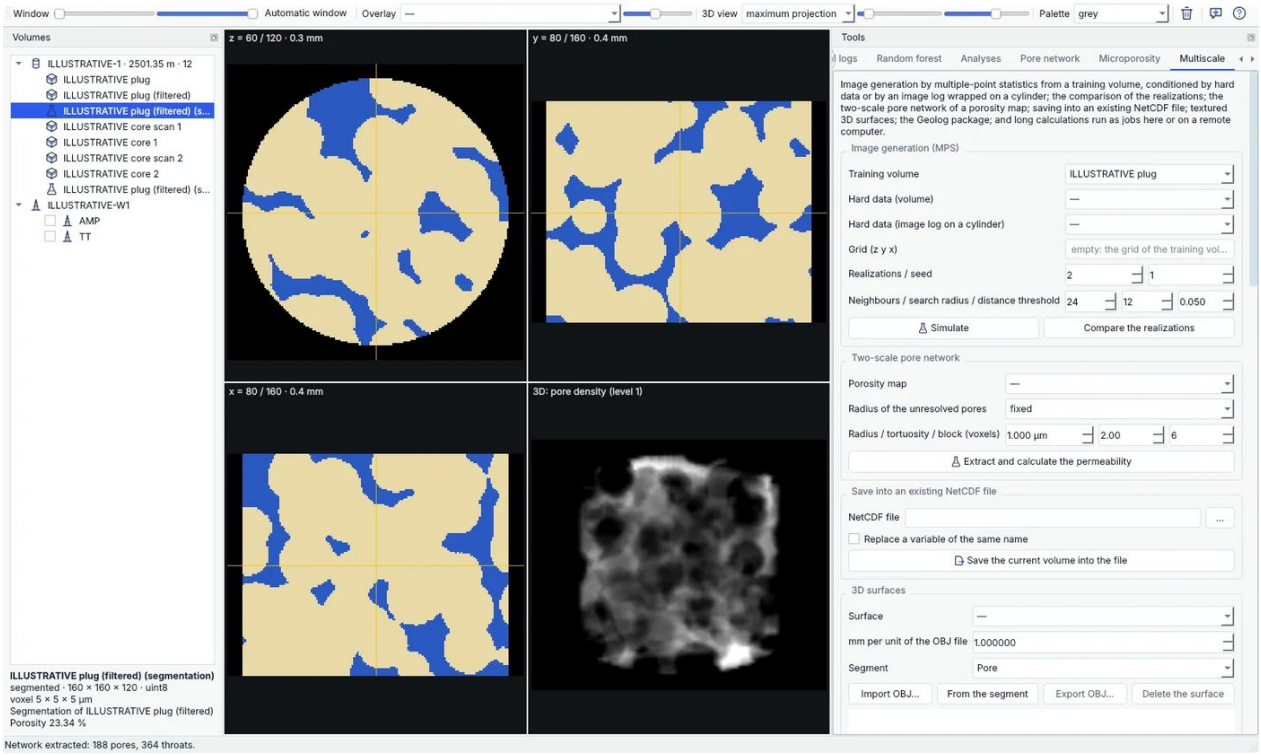
Coverage

GV13 (multiscale), GM5 (image generation — MPS; the generative network in 0.26), GM6 (post-processing), GM7 (pore network across scales), GM3 (image-log preprocessing: the log on a cylinder), GM4 and GM8 (preprocessing and segmentation of the scales, with the tools of 0.20–0.23).

3D surfaces, saving into an existing NetCDF file and the Geolog package

What it is for

Three exchange tools of the **Multiscale** tab of RockFace MC:



In the program: 3D surfaces, saving into an existing NetCDF file and the Geolog package

- **3D surfaces**: the photogrammetry models of cores, plugs and outcrops (Wavefront OBJ with its MTL materials and texture pictures) kept in the project and shown in 3D, and the surface of a segment of a volume made and written as OBJ.
- **Save into an existing NetCDF file**: the volumes made in RockFace written as new variables of the NetCDF file the data came from, which keeps everything it held (its attributes, dimensions and other variables).
- **Geolog (files)**: the logs of a well written as a package that Geolog loads with its own LAS and DLIS loaders, and the files that Geolog writes read back, with their null values.

Before you start

- A surface: an `.obj` file (with its `.mtl` and pictures in the same folder), or a segmented volume.

- NetCDF: an existing classic NetCDF file (or a NetCDF-4 file, with h5py installed) and the volume to save.
- Geolog: the logs of a well (Well logs tab); DLIS files are written when `dliswriter` is installed and read when `dlisio` is installed; LAS always.

Step by step

3D surfaces

1. In **3D surfaces**, type the **mm per unit of the OBJ file** (1 when the model is in mm; 1000 when in metres) and click **Import OBJ...**. The surface is kept with the current sample and shown: drag to turn, wheel to zoom.
2. To make one from a volume, open a segmentation, choose the **Segment** and click **From the segment**.
3. **Export OBJ...** writes the OBJ, its MTL and the textures as PNG; **Delete the surface** removes it.

Save into a NetCDF file

1. Open the volume to save, and in **Save into an existing NetCDF file** choose the **NetCDF file**.
2. Tick **Replace a variable of the same name** to overwrite one; otherwise a new name is given (`_2`, `_3`...).
3. Click **Save the current volume into the file**. A volume of the same shape as a variable of the file takes its dimensions, so it lines up with the image it came from.

Geolog

1. In **Geolog (files)** choose the **Well** and click **Write a package for Geolog...**, then a folder: one LAS per image log, one LAS per depth grid for the curves, a DLIS with every log, `rockface_geolog.json` and `README_geolog.txt`.
2. To read Geolog's files, type the **Null values** (−999.25; −999; −9999; −99999 by default) and the hole **Diameter**, and click **Read files from Geolog...** on the folder (or a file).

Options

Field	What it changes
mm per unit of the OBJ file	Scale of the vertices to mm
Replace a variable of the same name	Overwrite instead of adding a suffix
Null values	Values read as missing in the LAS and DLIS files, besides the NULL of each LAS file
Diameter	Hole diameter set on the image logs read

How it works

- **OBJ/MTL**: vertices (with optional per-vertex colours), texture coordinates, normals, faces of any number of sides split into triangles, negative indices, `usemtl` and `mtllib`; materials with `Kd`, `d` and `map_Kd`. A vertex used with two texture coordinates becomes two vertices. The view draws the faces from the back (painter's algorithm) with the colour of the texture at their centre and Lambert shading; heavy meshes are reduced for the view by vertex clustering (Rossignac & Borrel 1993). From a volume, the surface is made by marching cubes (Lorensen & Cline 1987) after a light Gaussian smoothing of the mask (area of a sphere within 0.5 %).
- **NetCDF**: a classic file is rewritten through a temporary file with all it held plus the new variables, then replaced; a NetCDF-4 file is appended in place and the dimension scales of the matching variable are attached.
- **Geolog**: mnemonics upper case, letters, digits and underscore, at most 32 characters, unique; the manifest keeps what LAS cannot (kinds, segment legends, azimuths, diameter) so the package comes back to RockFace whole.

From a script

```
s = prj.import_surface("core_box.obj", sample, scale_mm=1000)
s.render("core_box.png")
prj.volume("plug segmentation").save_into_netcdf("plug.nc")
prj.geolog_export("Well A", "to_geolog")
logs = prj.geolog_import("from_geolog", "Well A", nulls=(-999.25, -9999))
```

Limits and common problems

- The view shows one colour per face; very fine textures on large faces look flat (the OBJ written keeps the texture whole).
- Classic NetCDF files above 2 GB need the 64-bit offset format, which the rewrite keeps; NetCDF-4 needs h5py.
- RockFace does not open a Geolog project itself: the exchange is by files, which Geolog loads and writes.

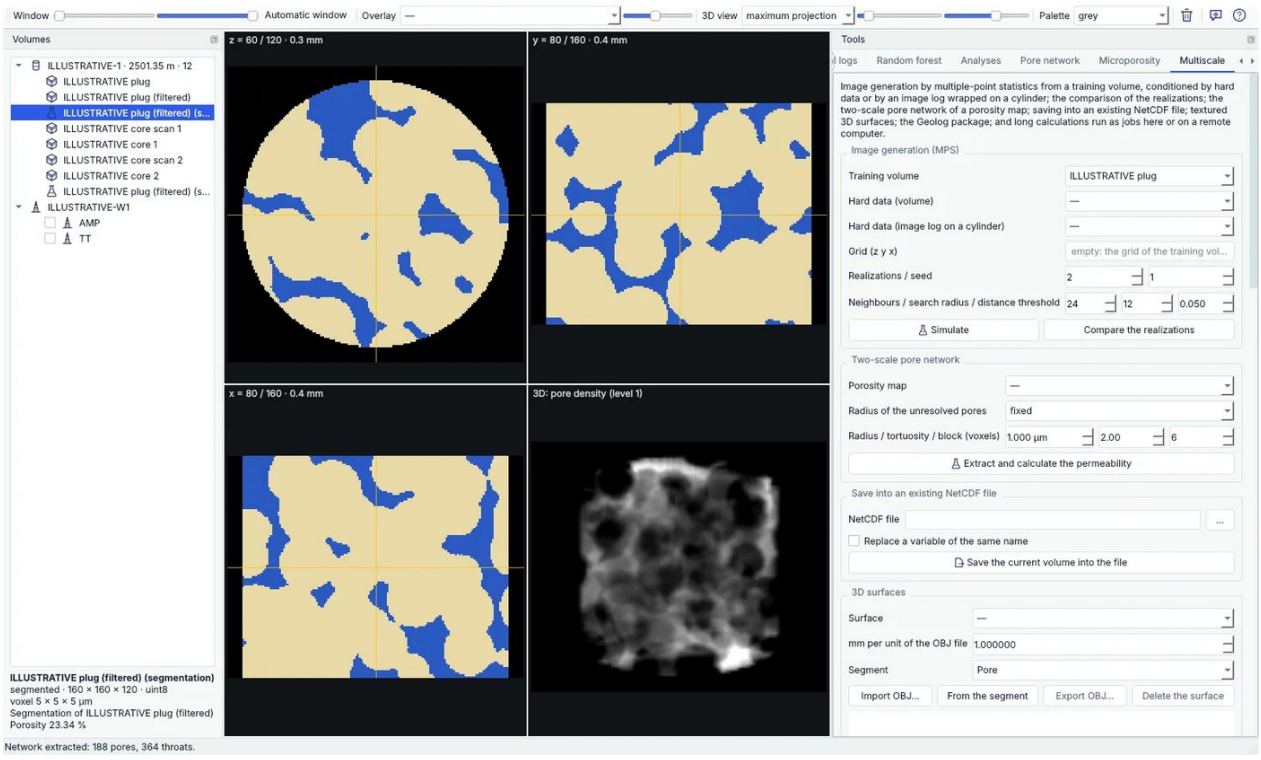
Coverage

GV18 (textured 3D surfaces), GV15 (saving into an existing NetCDF file), GM1 (Geolog integration), GM2 (import and export tools).

Jobs: batch and remote computing

What it is for

The long calculations of RockFace MC — lattice-Boltzmann permeability, Microtom curves on large volumes, MPS realizations, pore-network extractions, porosity maps — can run as **jobs**: in other processes of this computer, several at a time and over many volumes (batch), or on a **remote computer** (a workstation or a cluster with SLURM), while the program stays free. When a job is done, its results are brought back into the project.



In the program: Jobs: batch and remote computing

Before you start

- For remote jobs: an SSH account on the remote computer that opens with your keys (no password prompt), and RockFace installed there (the same version, or its source folder given as the Python path). RockFace keeps no password.

Step by step

1. Open the volume (or the log) the calculation works on and go to the **Multiscale** tab, group **Jobs**.

2. Choose the **Calculation** and type its **Parameters (JSON)**, e.g. `{"method": "permeability", "tol": 1e-5}` for a Microtom simulation. The volume or log open is the input.
3. Tick **One job per volume of the sample of the same kind (batch)** to run it over every volume like this one.
4. To run here: set **Jobs at a time here** and click **Run here**.
5. To run remotely: type the **Remote computer** (`user@host` or an alias of your SSH configuration), the **Folder there / Python there**, an optional **Setup line** (e.g. `module load python`), the **Scheduler / options** (*none*: a background process; *SLURM*: `sbatch` with options such as `--time=02:00:00 --mem=16G`), and click **Send to the remote computer**.
6. The table follows the jobs (state, progress, time, error); **Update** reads the states again (remote jobs are copied back when done). Click **Bring the results back** to add what the newest finished job made to the project; **Cancel the job** stops it.

Options

Field	What it changes
Calculation	One of: Microtom simulation, porosity maps (segmentation, saturation), Darcy permeability of a map, MPS realizations, pore network extraction, two-scale network, permeability REV of a network, permeability model of an image log
Parameters (JSON)	The arguments of the calculation, as in a script
Jobs at a time here	Processes running together; the others wait
Scheduler / options	Background process, or SLURM with `#SBATCH` options

How it works

A job is a folder next to the project (`<project>.jobs`): `job.json` (the calculation, its parameters, its inputs), `input.rockface` (a small project with only the objects needed, with their samples and wells), then `status.json` and `result.json` written by the run. The calculation is one of a fixed list of RockFace functions — never code sent with the job. Remote jobs are copied with `scp`, started with `ssh` (or `sbatch`), followed by reading `status.json` and copied back. Bringing back copies every object the job made (volumes, logs, networks, surfaces) into the project, tied to the same samples and wells, with the job recorded in its history.

From a script

```
seg = prj.volume("plug segmentation")
job = prj.prepare_job("microtom", volume_id=seg.id, method="permeability")
prj.run_jobs([job], workers=2)
print(prj.bring_back(job))
host = rf.jobs.Host("me@cluster", workdir="/scratch/me/rf", scheduler="slurm",
 ↪  slurm="--time=04:00:00")
runner = rf.jobs.RemoteRunner(host)
runner.submit(prj.prepare_job("pnm_extract", volume_id=seg.id))
```

Limits and common problems

- “Permission denied (publickey)”: the SSH key is not accepted; open a terminal and check that `ssh user@host` works without a password.
- A remote job fails at once when RockFace is missing there: set **Python there** to the Python that has it, or the source folder in the Python path of a script (`Host(pythonpath=…)`).
- Results are brought back on demand, never written into the project behind your back.

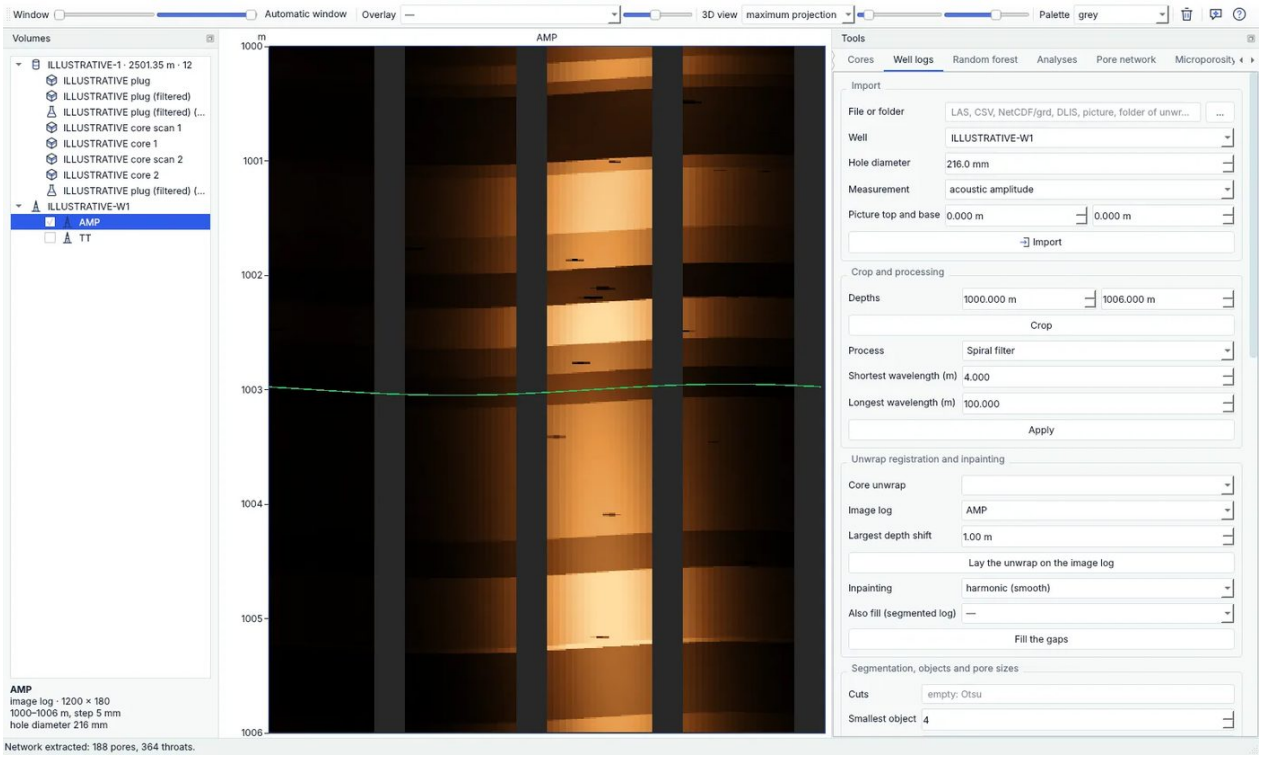
Coverage

GM10 (remote computing), GM9 (multiscale flows), and the remote workflow of the pore network (GV12).

Permeability model of image logs

What it is for

A continuous permeability curve along the well from a **segmented image log** (e.g. vugs, microporous matrix and tight matrix on an acoustic amplitude log), a **porosity curve** and the **plug permeabilities** measured in the laboratory. It is meant for dual-porosity carbonates, where the vugs seen on the image carry much of the flow.



In the program: Permeability model of image logs

Before you start

- A segmented image log (Well logs tab → Segmentation), with a segment for the macropores (vugs).
- A porosity curve of the same well (imported from LAS, CSV…; in fraction or in %).
- The plug permeabilities: depth (m) and K (mD), at least two inside the image.

Step by step

1. Open the segmented image log and go to the **Well logs** tab, group **Permeability model (image log)**.

2. Choose the **Segmented image log** and the **Porosity curve**.
3. Choose the **Macropore segment** (its porosity exponent is fixed at 1) and, if needed, an **Ignored segment** (an uncertainty class, or a bad-image class).
4. Set the **Depth window** over which the fractions of the segments are counted (0: every row of the image).
5. Paste the **Plugs**, one per line: depth and K (spaces, tabs or semicolons).
6. Optionally type **Forced points (Kds)** the same way: depths and K the curve must pass near, to correct a zone of interest.
7. Click **Fit and write the permeability curve**. The curve (unit mD) is added to the well, and the fit is shown: number of plugs, misfit of log10 K, R^2, and A and B of each segment.

Options

Field	What it changes
Macropore segment	The segment whose term is A·F·Φ (exponent 1)
Ignored segment	Pixels left out of the fractions
Depth window	Rows gathered into one depth of the curve
Forced points (Kds)	Extra points of the fit with ten times the weight of a plug

How it works

At every depth (Menezes de Jesus, Compan & Surmas 2016):

$K = A1 \cdot F1 \cdot \Phi^{B1} + A2 \cdot F2 \cdot \Phi^{B2} + \ldots + An \cdot Fn \cdot \Phi^{Bn} + Am \cdot Fm \cdot \Phi$

where Fn is the fraction of segment n in the rows at that depth, Φ the porosity curve interpolated to the image depths, Fm the fraction of the macropore segment. $A \geq 0$ and $0 \leq B \leq 8$ are fitted to the plugs by least squares on log10 K (permeability spans decades), from several starting exponents (trust-region reflective). Synthetic plugs made with known A and B are recovered exactly.

From a script

```
seg = prj.log("AMP (segmentation)")
k, fit = seg.permeability_model("PHIT", [(2510.35, 120.0), (2512.10, 8.5), (2515.80, 300.0)],
↪  macro="Pore",
                                window_m=0.1)
print(fit["A"], fit["B"], fit["rms_log10"])
```

Limits and common problems

- With few plugs, the terms of the classes with little influence can come out as 0; check the A and B shown and their misfit.
- The model uses only the image and the porosity; the plugs must sample the classes of the image for the fit to hold elsewhere.

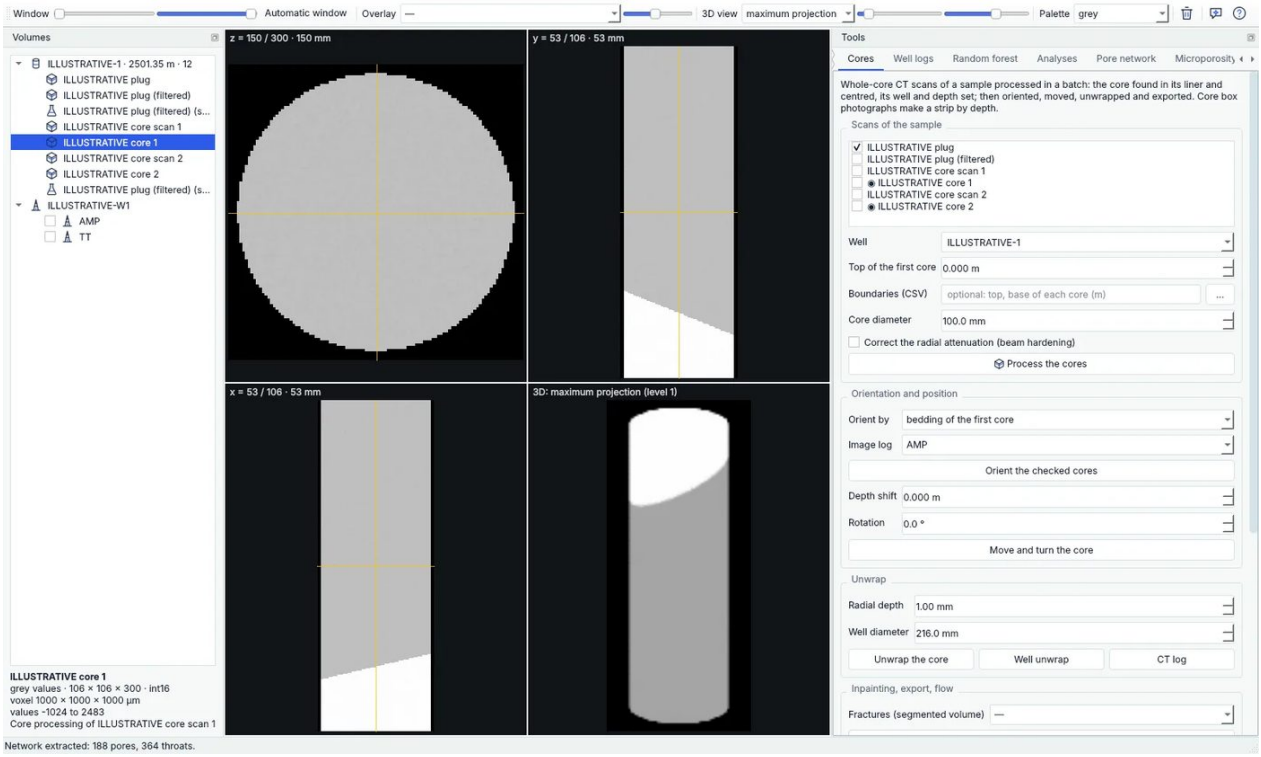
Coverage

GI10 (permeability modelling).

RockFace MC: whole cores and core-box photographs

What it is for

The **Cores** tab processes the whole-core CT scans of a well in a batch: each core is found in its liner and centred, given its well and depth, oriented (by the bedding of the first core or by an image log of the well), moved and turned, unwrapped as an image of its outer surface, laid as one well unwrap and as a CT log, its fractures filled, and exported. Core-box photographs are cut into their cores and laid by depth as one strip of the well.



In the program: RockFace MC: whole cores and core-box photographs

Before you start

- The CT scans imported as grey volumes of a sample of the well (page RockFace MC: micro-CT volumes — import, views and processing).
- The top depth of the first core and the core diameter; optionally a table of the top and base of each core (CSV).
- For orientation by an image log: the image log of the well (page RockFace MC: image logs).

Step by step

Processing the scans

1. Open **Cores**. Under **Scans of the sample**, check the scans.
2. **Well**, **Top of the first core**, **Core diameter**; **Boundaries (CSV)** (optional: top and base of each core, in m).
3. **Correct the radial attenuation (beam hardening)** if the core is brighter at its rim.
4. **Process the cores**. "Finding and centring the cores…", then "N core(s) processed". Each core shows "Core of …: top–base m, … mm, rotation …°, shift … m".

Orientation and position

1. **Orient by**: **bedding of the first core** (the others turned to follow it), or **an image log of the well** (choose the **Image log**).
2. **Orient the checked cores**. "Rotations: …".
3. By hand: **Depth shift** and **Rotation**, then **Move and turn the core**.

Unwrap and logs

1. **Radial depth** (how far inside the surface the unwrap is read), then **Unwrap the core**: an image of the outer surface, depth down, azimuth across, laid in the well's logs.
2. **Well diameter** and **Well unwrap**: the checked cores as one unwrap of the well, at the well's diameter.
3. **CT log**: the mean CT value by depth, as a curve of the well.

Inpainting, export, flow

- **Fractures (segmented volume)** and **Fill the fractures**: the fractures of a segmented core filled slice by slice.
- **Export the checked cores…**: an HTML summary, the CSV matrix and the Techlog form with depth, the PNG unwrap, the TIFF volume. "N files written in …".
- **Core flow: core → segmentation**: import → core → crop → segmentation and porosity, in one go.

Core-box photographs

1. Under **Core box photographs**, **Add photographs…** and give each its **Top (m)** and **Base (m)**.
2. **Channels per box** (rows of cores in a box), or **automatic**.
3. **Make the strip**: the rows of cores are found by the background colour, cut, and laid end to end by depth as one strip of the well.

Options

Option	What it changes
Correct the radial attenuation	Removes the beam-hardening brightening of the rim.
Orient by	The bedding of the first core, or an image log.
Radial depth	Depth inside the surface at which the unwrap is read (mm).
Channels per box	The number of rows of cores in each box photograph.

How it works

- The liner and the core are found as circles in the slices; on the ILLUSTRATIVE well the core radius was found at 49.6–49.7 mm for 50 mm, and on a real whole-core CT (internal test) the core inside the liner at 130.4 mm.
- Orientation by the bedding fits the dominant sinusoid of each core's unwrap; by an image log, it correlates the depth gradients of the unwrap and the log: within 0.1° and 2 mm on the ILLUSTRATIVE well.
- Core-box photographs made a strip exactly on the ILLUSTRATIVE well.

From a script

```
scans = prj["TS-9"].volumes()                                # the CT scans of the sample, top to base
cores = [prj.process_core(v, 2500.0 + 0.9 * k, 100.0, well="W-1") for k, v in enumerate(scans)]
print(cores[0].core["diameter_mm"], cores[0].core["top_m"])
prj.orient_cores(cores, "sinusoid")                              # or "image_log", log=prj.log("AMP")
unwrap = cores[0].unwrap(radial_depth_mm=1.0)
well_unwrap = prj.well_unwrap(cores, well_diameter_mm=216)
ct = prj.ct_log(cores)
prj.export_cores(cores, "cores_out")
strip = prj.load_core_photos("W-1", [("box1.jpg", 2500.0, 2503.0), ("box2.jpg", 2503.0, 2506.0)])
```

Limits and common problems

- "Check at least one scan or core."
- "Give the top and base depth of every photograph."
- A core touching its liner may be found as the liner: check the diameter shown, and give the core diameter.

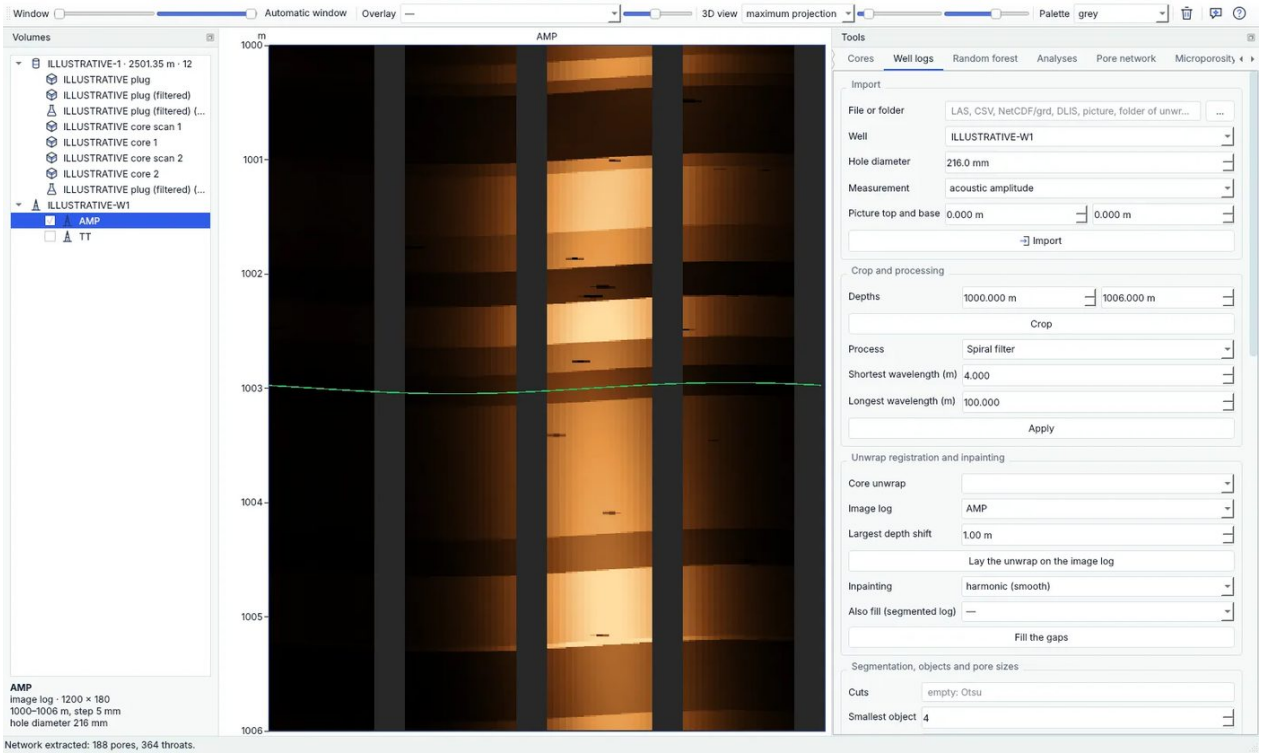
Coverage

GC1 (multicore), GC2 (core photograph loader), GC3 (crop), GC4 (multicore transformations), GC5 (segmentation), GC6 (core inpainting), GC7 (export), GC8 (flows), GI2 (unwraps of processed cores).

RockFace MC: image logs

What it is for

The **Well logs** tab keeps the image logs of a well — acoustic amplitude and transit time, resistivity, conductivity, core unwraps, CT logs — with its curves, on one depth axis. It imports and exports the usual formats, crops and processes them (eccentricity, spiral filter, quality and heterogeneity, azimuth shift, CLAHE), fills their gaps (inpainting), segments them into vugs with their sizes by depth, fits the sinusoids of planar features, and lays core unwraps on the image log.



In the program: RockFace MC: image logs

Before you start

- The log files: LAS 2.0 (with stacked image curves), CSV (matrix, Techlog, long form; also gzipped), NetCDF or GMT grids, DLIS (with the optional `dlis` extra), a picture with its top and base depths, or a folder of core unwraps.
- The well's name; the hole diameter, for lengths in millimetres.

Step by step

Importing

1. Open **Well logs**. **File or folder** (LAS, CSV, NetCDF/grd, DLIS, picture, folder of unwraps).
2. **Well**, **Hole diameter**, **Measurement** (acoustic amplitude, transit time, resistivity, conductivity, CT, photograph, other); for a picture, its **Picture top and base**.
3. **Import**. "N log(s) imported." The wells and their logs appear in the left panel; the tracks are shown side by side on one depth axis, with a **Palette**.

Crop and processing

1. **Depths** and **Crop**.
2. **Process** — choose one, with its parameters, then **Apply**:
   - **Eccentricity** — corrects the amplitude of a tool off the centre of the hole, using the **Transit-time log**;
   - **Spiral filter** — removes the spiral pattern of the tool (**Shortest** and **Longest wavelength (m)**); "Filtered: ...% of the image changed";
   - **Quality indicator** and **Heterogeneity index** — curves by depth (**Window (m)**);
   - **Azimuth shift** — turns the image by an **Azimuth deviation curve**;
   - **CLAHE** — local contrast (**Clip limit**).

Unwrap registration and inpainting

1. **Core unwrap** and **Largest depth shift**, then **Lay the unwrap on the image log**: "Unwrap moved ... m and turned ...° (correlation ...)".
2. **Inpainting**: **linear (row by row)**, **harmonic (smooth)** or **biharmonic**; **Also fill (segmented log)** for a mask; **Fill the gaps** — the pad gaps of the tool and the masked areas are filled.

Segmentation, objects and pore sizes

1. **Cuts** (empty: Otsu), then **Segment**.
2. **Objects**: the vugs as objects across the 0/360° seam (**Smallest object**): "N objects; equivalent diameter mean ... mm, largest ... mm", with their table.
3. **Pore sizes**: the local thickness of the pore space in mm and its histograms by **Depth interval**.

Planar features

1. **Pick points**: click at least three points of a bedding plane or fracture on the image track.
2. **Fit the sinusoid**: "Plane at ... m: dip ...° toward ...°".
3. **Dominant fabric**: the dominant sinusoid of the depths on screen.

Export

Export... writes LAS, CSV (matrix or Techlog), NetCDF, PNG/TIFF or DLIS (with the optional dliswriter).

Options

Option	What it changes
Measurement	What the log is; amplitude and transit time go together for the eccentricity.
Inpainting	Linear (fast), harmonic (smooth), biharmonic (follows gradients).
Smallest object	Vugs smaller than this are left out.

How it works

- Logs are stored by depth with a pyramid, so long wells are shown at once.
- On the ILLUSTRATIVE well (dipping beds and vugs of known truth): dominant fabric 25.0°/120.0° exact; a hidden pad filled with RMSE 0.043–0.050 against 0.083 for the row mean; 38 of 40 vugs found.
- On a real FMI log (internal test): a hidden pad filled with RMSE 14.97 (linear) against 29.07 for the row mean; two passes matched within 0.8 mm.
- Editing log segmentations by brush and reviewing objects: page Editing a log segmentation and its objects; permeability model of segmented logs: page Permeability model of image logs.

From a script

```
amp, tt = prj.import_log("amplitude.las", "W-1", diameter_mm=216)[:2]
(fixed, tau, _), = amp.eccentricity(tt)
clean, changed = fixed.inpaint(method="biharmonic").spiral_filter(4, 20)
print(clean.dominant_sinusoid(2501, 2510).dip_deg)
vugs, table = clean.segment().objects(min_px=6)
table.to_csv("vugs.csv")
clean.export("clean.las")
```

Limits and common problems

- “Choose a file or folder.”
- “No log to show.” — the well has no log yet, or none in the depths shown.
- DLIS needs the `dlis` extra (dlisio, LGPL, loaded only when used).

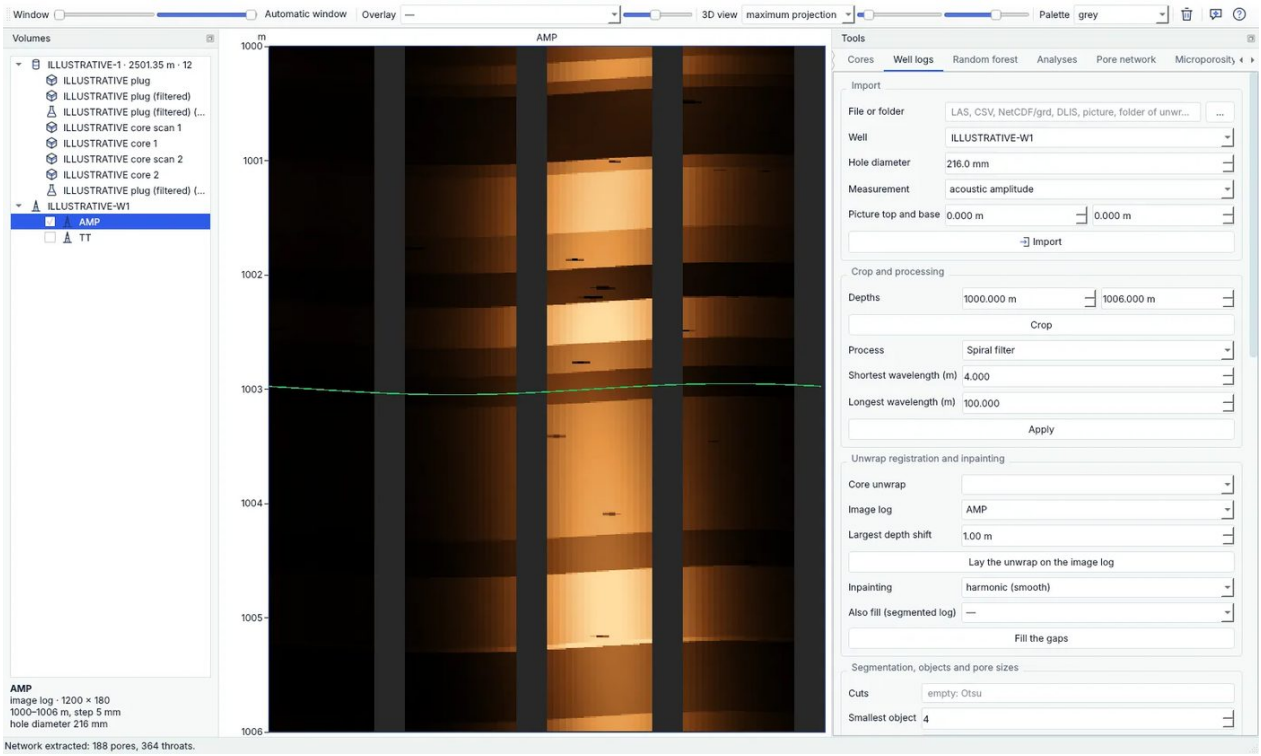
Coverage

GI1 (import), GI2 (unwrap), GI3 (export), GI4 (crop), GI5 (processing), GI6 (unwrap registration), GI7 (inpainting), GI8 (segmentation, planar features), GI9 (pore-size distribution).

Editing a log segmentation and its objects

What it is for

An automatic segmentation of an image log is never perfect: a vug is missed, a pad edge is taken for a vug, two vugs are joined. Here you correct the segmentation with a brush, and review the objects (vugs, spots) one by one: decline the false ones, merge the pieces of one object, keep only those whose measurements make sense.



In the program: Editing a log segmentation and its objects

Before you start

A segmented log (Well logs tab → Segmentation, objects and pore sizes → Segment) or a log of objects (... → Objects), open in the tracks of RockFace MC.

Step by step

Correcting a segmentation

1. Open the segmented log. In **Brush and objects**, choose what to **Paint** (a segment, or **— eraser —**) and the **Brush radius** in millimetres of the borehole wall.

2. Press **Paint on the track** and drag the mouse on the track. A stroke crosses the 0/360° seam the short way.
3. **Undo** takes back the last stroke.

Reviewing the objects

1. Open the log of objects. The table lists every object: depth (top, base, centre), azimuth, area, equivalent diameter, height, width, and whether it touches the top or base of the log.
2. Type a filter (`area_mm2 > 50 and not touches_edge`): the table shows the objects that pass; the histogram shows the chosen column.
3. Select objects in the table and press **Decline selected** or **Merge selected**, or press **Keep only the rows shown** to decline every object the filter hides.
4. To add an object, choose **A new object** in **Paint** and paint it; **Grow the selected object** and **Trim the selected object** change the object selected in the table.

How it works

The brush is a disc of the given radius on the unrolled wall (depth step × column width in mm). The rows touched are rewritten in place with their pyramid of levels. The measurements are recomputed from the objects after every change.

From a script

```
log = prj.log("Vugs (ILLUSTRATIVE)")
patch = log.paint([(120.0, 2510.30), (126.0, 2510.32)], radius_mm=8, value=2)
log.undo(patch)
objects = prj.log("Vugs (ILLUSTRATIVE) (objects)")
table = objects.instances()
declined = objects.keep_only("area_mm2 > 50 & touches_edge == 0")
new_label = objects.paint_instance([(200.0, 2511.0)], radius_mm=10)
```

Limits and common problems

- **“Select an object in the table first.”** Grow and trim need an object selected.
- New paint never covers another object; trim only removes pixels of the selected object.
- The trained instance segmenter of GeoSlicer comes in stage 0.26.

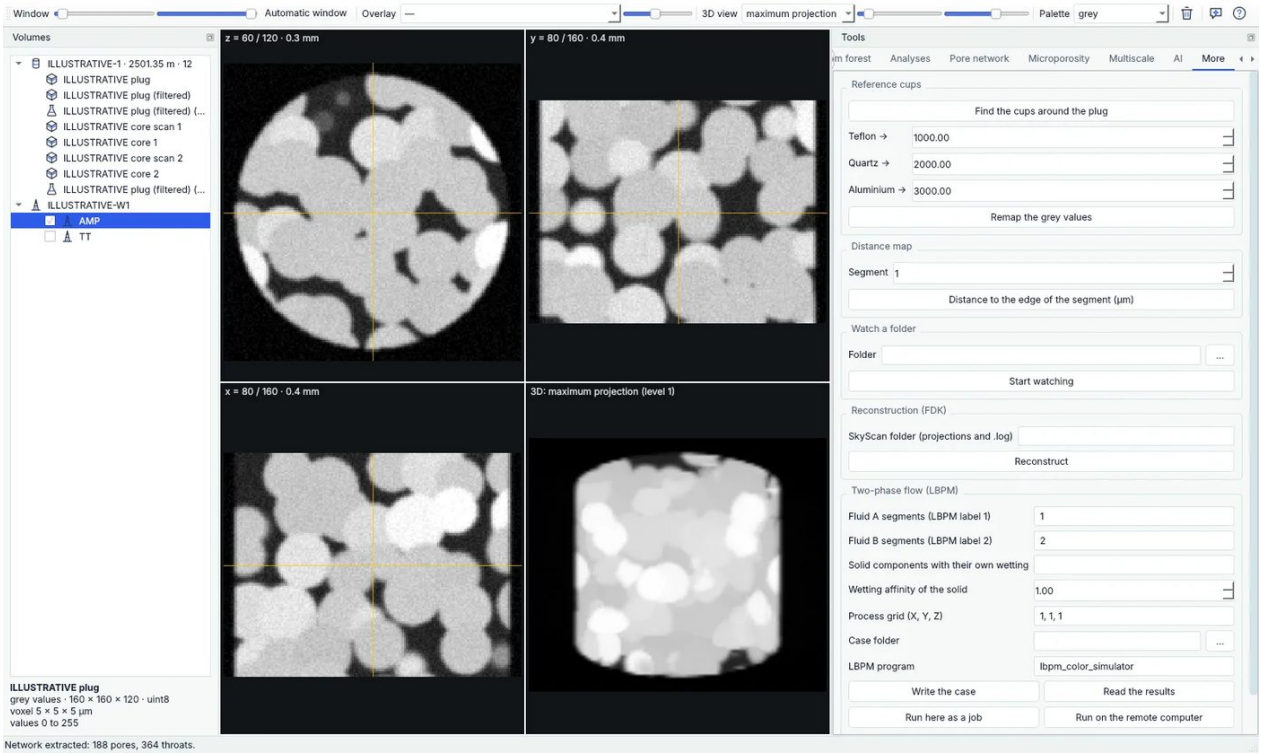
Coverage

GI8 (Segment Editor of image logs, Instance Segmenter Editor).

MC → More: cups, distance map, folder watch, reconstruction, upscaling and two-phase flow

What it is for

Tools found by the audit of the GeoSlicer source: calibrate grey values with **reference cups**, measure the **distance** to the edge of a segment, import scans from a **folder** while the scanner writes them, **reconstruct** a SkyScan acquisition, carry core measurements to the **LAS** depth samples and plug measurements **along the core**; on image logs, segment a **depth range** and trace a level across the 0/360° seam; simulate **two-phase flow** on a segmented volume with LBPM.



In the program: MC → More: cups, distance map, folder watch, reconstruction, upscaling and two-phase flow

Before you start

A sample with volumes (MC), or a well with image logs.

Step by step

1. **MC → More → Reference cups**: open the scan with its plug and the Teflon, quartz and aluminium cups; **Find the cups around the plug**; type the values they should have; **Remap the grey values** (a new volume).
2. **Distance map**: on a segmented volume choose the segment; **Distance to the edge of the segment (µm)**.
3. **Watch a folder**: choose the scanner's output folder; **Start watching**: every scan that stopped growing is imported; numbered names make a time sequence.
4. **Reconstruction (FDK)**: a SkyScan folder (projections and `.log`); **Reconstruct**.
5. **Two-phase flow (LBPM)**: open a segmented volume; give the segments of fluid A (LBPM label 1) and fluid B (label 2), the solid components that wet differently, the wetting affinity of the solid and the process grid; **Write the case** (the folder, the command and a SLURM script `run_lbpm.sh`), or **Run here as a job** / **Run on the remote computer** (the computer of MC → Multiscale → Jobs (batch and remote)); **Read the results** shows the end points and the curves.
6. Upscaling, plug to core and the log tools are in the API (below).

How it works

- Cups: the plug's cylinder is found, the voxels around it split by a four-class Otsu (air and three references) and 1-D k-means; the remap is the least-squares line through the references.
- Distance: the Euclidean distance transform with the voxel size of each axis.
- Folder watch: a dataset is imported when its size and time are the same on two looks.
- Reconstruction: Feldkamp–Davis–Kress for a cone beam on a circular orbit (Feldkamp et al. 1984; Kak & Slaney

  1988) and filtered back-projection for a parallel beam, in NumPy (GeoSlicer's uses ASTRA, GPL, which RockFace cannot ship); attenuation in 1/mm.

- Upscaling to LAS: the mean of the core values inside each LAS sample window; plug to core: a random forest (or a linear fit) of the plug values on the core's CT features, checked leave-one-out before use.
- Logs: a depth range written as a segment; level tracing grows the region of similar values from a seed, connected across the seam of a full-turn log.
- Two-phase flow: LBPM (McClure et al. 2021; GPL-3.0, so RockFace does not contain it — it runs the user's own installation). The case: the volume as LBPM's 8-bit RAW (x fastest) and `input.db`, its `Domain` written to match — size, process grid and sub-domains, voxel length in µm, the segments mapped to 0 (solid), 1, 2 and negative labels for the solid components with their `ComponentAffinity`; the other sections from LBPM's documentation (colour model, fractional-flow protocol). The results: `timelog.csv` (relative permeability against the saturation) and `subphase.csv` (saturation from the connected and disconnected phase volumes, capillary pressure pn − pw, interfacial area, Euler characteristic), as GeoSlicer's Pore Scale Modelling reads them.

From a script

```
c = rf.gs_tools.cups(prj.raw, vol_id); rf.gs_tools.remap(prj.raw, vol_id, c.medians, {"teflon":
↪  1000, "aluminium": 3000})
rf.gs_tools.distance_map(prj.raw, labels_id, 1)
vol, voxel_um = rf.gs_tools.reconstruct_skyscan("acquisition_folder")
rf.gs_tools.upscale_to_las(depth, porosity, "well.las", "out.las", compare="PHIT")
rf.gs_tools.plug_to_core(core_depth, features, plug_depth, plug_porosity)
rf.log_edit.fill_depth_range(prj.raw, log_id, 1001.0, 1001.2, 2)
rf.log_edit.level_trace(prj.raw, grey_id, labels_id, azimuth=10, depth=1000.5, tolerance=10,
↪  value=3)
res = prj.lbpm(labels_id, {1: "fluid_a", 2: "fluid_b"}, "lbpm-case", nproc=(2, 2, 1))   # needs
↪  LBPM installed
prj.lbpm(labels_id, {1: "fluid_a"}, "lbpm-case", run=False)       # only the case and its command
```

`rockface notebook project.rockface` writes a Jupyter notebook that opens the project with the API.

Limits and common problems

- The reconstruction expects flat-corrected projections (as SkyScan writes them) and a centred rotation axis.
- Plug to core needs plugs that span the range of the core; the leave-one-out error says how far to trust it.
- LBPM must be installed where the job runs (here, the remote computer or the cluster); each size of the volume must divide by the process grid. The pressures are in lattice units.

Coverage

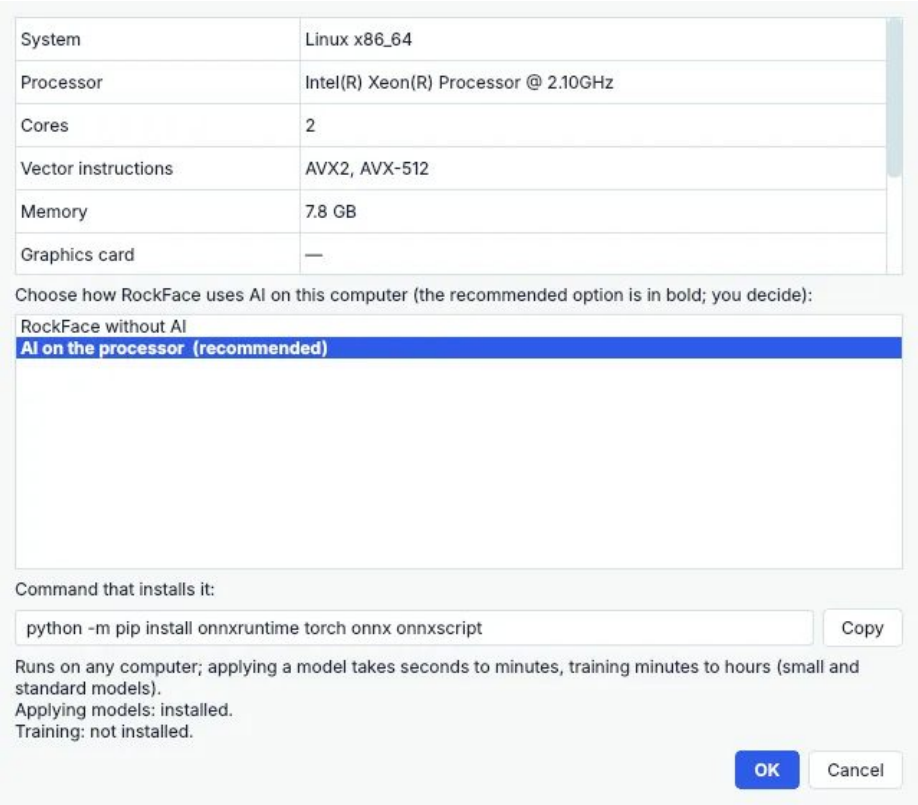
GeoSlicer audit of 0.28 (GV20–GV28, GI12–GI13).

RockFace AI and Mio

RockFace AI: installation, device and the model hub

What it is for

RockFace AI adds neural networks to Lab and MC: models that segment pictures, whole thin sections, micro-CT volumes and image logs, that you can train and fine-tune on your own annotations, and a generative model of rock images. It is an **option**: RockFace without AI has every other function, and nothing else changes when you add it. This page shows how to install it for the computer you have, how to choose where the models run (the processor or a graphics card), and how the models are kept, shared and compared in the **model hub**.



In the program: RockFace AI: installation, device and the model hub

Before you start

- RockFace installed (0.26 or later).
- For a graphics card: the card's own driver (NVIDIA, AMD or Intel). The rest comes with the packages.

Step by step

Installing: the computer is detected, you choose

1. In a terminal, run `rockface ai setup`. RockFace lists what it found — system, processor, cores, vector instructions, memory, graphics cards, and the AI packages already installed — and the options for this computer:

Option	When
RockFace without AI	Nothing more to install
AI on the processor	Any computer; applying a model takes seconds to minutes, training minutes to hours
AI on the NVIDIA graphics card (CUDA)	An NVIDIA card: the fastest for applying and for training
AI on the graphics card (DirectML)	Windows with any DirectX 12 card (NVIDIA, AMD, Intel): models are applied on the card, training runs on the processor
AI on Apple silicon (Core ML and MPS)	A Mac with an M processor

   The recommended option is marked; **you decide**. Type its number (or press Enter for the recommended one).

2. RockFace shows the installation command (`python -m pip install …`) and asks before running it. With `--option cuda --yes` it chooses and installs without asking; with `--dry-run` it only shows the command.

3. The same dialog is in the program: **Lab → AI** or **MC → AI**, button **Hardware and device...**. It shows the hardware, the options with the recommended one in bold, and the command that installs the one you select (with a **Copy** button). **OK** keeps your choice.

`rockface ai info` shows the hardware and the device in use; `rockface ai models [project]` lists the models.

The model hub

The table **Models** of the AI tab lists every model RockFace knows for that tab, from three places:

- **RockFace** — the models shipped with the program (see *How it works*);
- **Hub folder** — a folder shared by the team (a network drive, a synchronised folder), chosen the first time you click **Publish to hub**, or given by the variable `ROCKFACE_AI_HUB`;
- **Project** — the models trained in, or imported into, the project.

Select a row and use:

- **Card...** — what the model is: task, inputs, classes, the data it was trained on and their licences, the metrics on held-out data, its parent model (when it was fine-tuned) and its lineage, and the SHA-256 of the network;
- **Import... / Export...** — a model as one `.rfai` file, to send to a colleague or keep;
- **Publish to hub** — the model written to the hub folder as `<name>/<version>.rfai`, with an `index.json` of the hub; a version is never overwritten;
- **Delete** — a model of the project (with a line in the audit);
- **Compare** — the cards of the selected rows side by side (metrics, data, parent).

Options

Field	What it changes
Option (setup)	Which packages are installed and which device runs the models
`ROCKFACE_AI_DEVICE`	The device without the interface (`cpu`, `cuda`, `directml`, `coreml`) — for scripts and jobs
`ROCKFACE_AI_HUB`	The hub folder without the interface

How it works

- **Two layers, installed apart.** Applying a model uses ONNX Runtime (MIT licence; extra `ai`), whose execution providers run the same network on the processor, on CUDA, on DirectML or on Core ML; the processor is always the fallback. Training uses PyTorch (BSD licence; extra `ai-train`) on the processor, CUDA or Apple's MPS.
- **A model is data, never code.** An `.rfai` file is a zip with `model.onnx` (the network, ONNX format) and `card.json` (its card). The card holds the SHA-256 of the network, checked on every load: a changed file is refused. Models are never pickled Python objects.
- **Versions and lineage.** Each model has a name and a version; fine-tuning a model makes the next version with the model it came from as its parent; the same name and version cannot be kept twice.
- **The models shipped** were trained only on data whose licence allows it — RockFace's synthetic data (exact truth) and open data under CC BY 4.0 or ODC-BY — and each card states the data and the metrics measured on data the model did not see. Data under non-commercial or share-alike licences are used only for internal tests, never to train a model that ships.

From a script

```
info = prj.ai_hardware()              # hardware, options (recommended one marked), device,
↪  packages installed
for m in prj.ai_models():            # every model: source, name, version, task, licence, held-out
↪  metrics
    print(m["name"], m["version"], m["metrics"])
m = prj.ai_model("rockface-thinsection-pores")      # the newest version, wherever it is
prj.ai_export(m, "pores.rfai"); rid = prj.ai_import("pores.rfai")
prj.ai_publish(rid, "//server/rockface-hub"); print(prj.ai_lineage(rid))
```

Limits and common problems

- "RockFace AI is not installed": run `rockface ai setup` (or `pip install -e ".[ai]"` from the source).
- "Training needs the extra ai-train": applying models works; training needs PyTorch (`pip install -e ".[ai-train]"`).
- On Linux, the PyTorch package from PyPI brings the CUDA libraries (about 3 GB). For a processor-only computer, install it from the PyTorch CPU index (`pip install torch --index-url https://download.pytorch.org/whl/cpu`).

- A card not listed: update its driver; on Windows the DirectML option works with any DirectX 12 card.

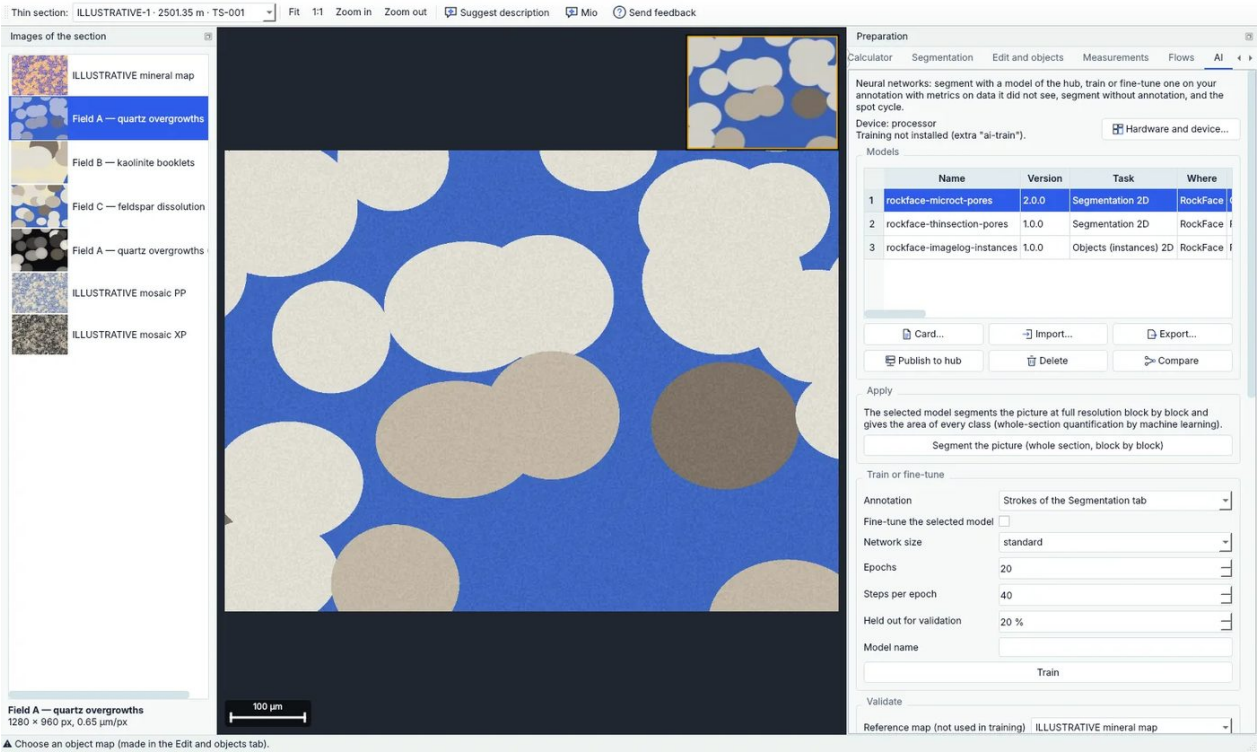
Coverage

L13 (model hub), GT18 / GV19 (the pretrained models and the choice of the model for the hardware found), and the AI installation of the ROADMAP stage 0.26.

RockFace AI: segmenting with models, training, fine-tuning and validating

What it is for

A neural network trained on many examples segments a picture, a whole thin section, a micro-CT volume or an image log in one click, and gives the area of every class with how confident it is. When a model does not fit your material — another resin, another stain, another scanner — you **paint a few examples** and **fine-tune** it: the new version learns your material, and its accuracy is measured on part of your annotation it never saw. This is the annotation → training → validation → application → retraining cycle. The **unsupervised** segmentation splits a picture into its textures with no annotation at all, and the **instance segmenter** of image logs finds the holes of sidewall coring and the vugs as separate objects.



In the program: RockFace AI: segmenting with models, training, fine-tuning and validating

Before you start

- RockFace AI installed (see RockFace AI: installation, device and the model hub). Training also needs the extra `ai-train`.
- A thin section with a picture (Lab), or a grey volume or an image log (MC).

Step by step

Lab — a picture or a whole thin section

1. Open the section in **Lab**, open the picture and go to the **AI** tab.
2. **Apply**: select a model of the task *Segmentation 2D* (e.g. `rockface-thinsection-pores`) and click **Segment the picture (whole section, block by block)**. The picture is classified at full resolution, block by block, and kept as a segmentation map; the result shows the area of every class, the porosity and the mean confidence. A model of the task *Objects (instances) 2D* trained on pictures gives an object map instead, with one number per grain, measured in the **Measurements** tab.
3. **Train or fine-tune**:
   - **Annotation**: *Strokes of the Segmentation tab* (paint examples of each segment there, as for the random forest) or a segmentation map of the picture (made by any method, corrected by hand), or *Objects: …* — an object map of the picture (grains or pores split in the **Edit and objects** tab and corrected by hand) to train an **instance model** that separates touching grains (the textural instance model);
   - tick **Fine-tune the selected model** to start from the model selected in the table (the classes must match);
   - choose the **Network size** (small, standard, large), **Epochs**, **Steps per epoch**, the share **Held out for validation** and a **Model name**, and click **Train**. The result shows the accuracy, mean IoU and Dice of each class on the held-out part; the model is kept in the project.
4. **Validate**: choose a **Reference map** that was not used in training (of this or another picture) and click **Validate**: the metrics of the selected model against it. Classes are matched by name.
5. **Without annotation**: set **At most / At least segments**, **Iterations** and the **Working size**, and click **Segment without annotation**. The map's segments are named *Segment 1…k* by decreasing area; rename and merge them in the **Edit and objects** tab.

MC — a volume or an image log

1. Open a grey volume and go to the **AI** tab.
2. **Apply**: select a model (e.g. `rockface-microct-pores`; a 2D model runs slice by slice) and click **Segment the volume**: a labels volume with the fractions, the porosity and the mean confidence.
3. **Train, fine-tune and validate**: choose the **Grey volume** and the **Annotation (labels volume)** — painted in the **Random forest** tab or a segmentation corrected by hand — the **Network** (2D slice by slice, or 3D), and click **Train**. In 2D, a share of the annotated slices is held out; in 3D, blocks of the volume. **Validate** measures the selected model against a segmented volume of the same grid.
4. **Image logs**: open an amplitude log; select a model of the task *Objects (instances) 2D* (e.g. `rockface-imagelog-instances`), choose the **Transit time** log of the same well when the model takes it, and click **Apply to the image log**: an object log with the table of the objects (sizes, depths, azimuths), editable with the log editor of 0.23. A *Segmentation 2D* model gives a labels log instead.

Options

Field	What it changes
Network size	small (8 filters in the first layer), standard (16), large (32): larger learns more, needs more memory and time
Epochs, Steps per epoch	How long the training lasts; the network with the best held-out accuracy is kept
Held out for validation	The share of the annotation kept out of training to measure the model (blocks of 64 px in 2D, slices or blocks of 32 voxels in MC)
Fine-tune the selected model	Start from that model (learning rate ×0.3); the new version has it as parent
At most / At least segments	The bounds of the unsupervised segmentation
Smallest object	Objects of the instance segmenter below this area are dropped

How it works

- **Network**: a U-Net (Ronneberger, Fischer & Brox 2015) of four levels with batch normalisation, in 2D or 3D; loss = class-weighted cross-entropy plus Dice on the annotated pixels only (Milletari, Navab & Ahmadi 2016); AdamW (Loshchilov & Hutter 2019) with a cosine schedule; random flips and rotations of 90°.
- **Metrics** (on held-out data): accuracy, intersection over union and Dice per class, the confusion matrix.
- **Large pictures and volumes**: tiles with a margin, reflected at the borders, classified in batches; the margin limits the seams between tiles.
- **Unsupervised**: a small network trained on the picture itself, its classes made constant on SLIC superpixels and spatially continuous (Kanezaki 2018; Kim, Kanezaki & Tanaka 2020; Achanta et al. 2012); the number of classes falls as the network merges them.
- **Instances**: the network classifies background, inside and edge of objects; the insides are the seeds of a watershed over the objects, across the 0°/360° seam of the log.

From a script

```
pic = prj.image("ILLUSTRATIVE mosaic PP")
seg, fr, por, conf = pic.ai_segment("rockface-thinsection-pores")          # whole section, block
↪  by block
model, met = pic.ai_train(map=seg, base="rockface-thinsection-pores", epochs=10)   # fine-tune,
↪  held-out metrics
print(pic.ai_validate(model, "reference map"))
seg2, fr2 = pic.ai_unsupervised(max_segments=8)
grains, n = pic.ai_objects(pic.ai_train_objects("grain objects")[0])          # the textural
↪  instance model
vol = prj.volume("plug scan")
labels, fr, por, conf = vol.ai_segment("rockface-microct-pores")
model, met = vol.ai_train("painted slices", base="rockface-microct-pores")
objects, table, n = prj.log("AMP").ai_instances("rockface-imagelog-instances", transit_time="TT")
```

Limits and common problems

- "The annotation has k classes and the model m": fine-tuning keeps the model's classes; paint the same segments.
- "No class of the reference matches a class of the model by name": rename the segments of the reference map.
- A model applied to pictures of another magnification: the shipped thin-section model was trained at about the resolution of the GO-Forward images; fine-tune it on a few strokes of your pictures.
- Training on a processor takes minutes for a picture and can take hours for a 3D network; run long trainings on a graphics card or as a job (MC → Multiscale → Jobs: *AI: train a volume segmenter*).

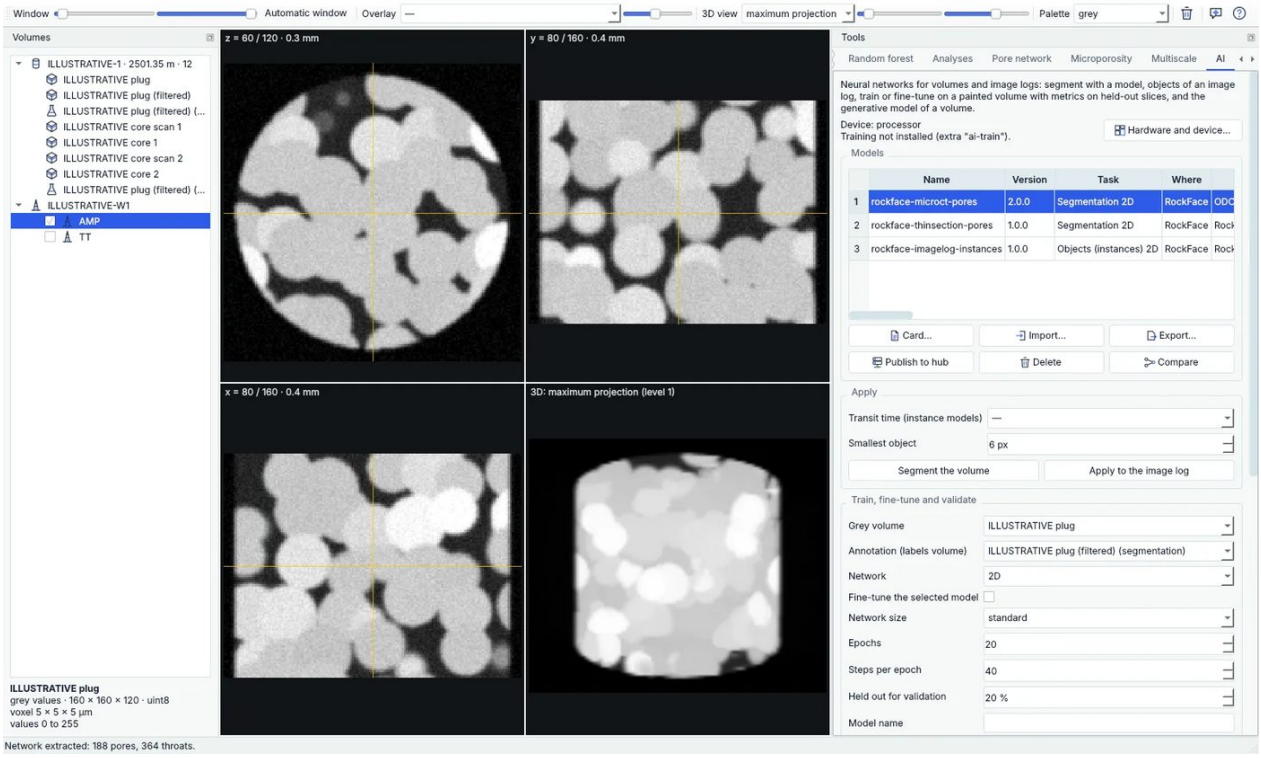
Coverage

GT18 (pretrained segmenters, thin section), GV19 (pretrained volume segmenters), GI11 (instance segmenter of image logs), GT20 (unsupervised segmentation), L7 (whole-section quantification by ML), L11 (annotation → ML → validation → extrapolation → retraining, for mineralogy and texture).

RockFace AI: generative model of a volume (SinGAN) and the spot cycle

What it is for

Generating rock images. From one training volume — a segmented micro-CT volume or a grey one — a generative network learns its texture and makes as many new volumes as you want, of the same size or larger, with the same porosity and pore sizes, for simulations that need several realizations or a volume larger than the scan. It sits beside the MPS generation of 0.25 (Multiscale tab).



In the program: RockFace AI: generative model of a volume (SinGAN) and the spot cycle

The spot cycle. When some spot analyses of a project (microprobe, LA-ICP-MS, EDS...) have their constituent named by the petrographer, a classifier learns the constituent from the measured values, tells how often it is right on thin sections it did not see, and proposes a constituent with its probability for the spots not yet named. You accept, correct or reject each proposal; the next training uses them.

Before you start

- SinGAN: RockFace AI with training (extra `ai-train`); a graphics card for 3D training of useful size.
- Spot cycle: spot analyses with values (Atlas → Spots, or imported from CSV), at least 6 named spots of at least 2 constituents; the constituents are nomenclature terms.

Step by step

Generative model (MC → AI)

1. Open the training volume (segmented or grey).
2. In **Generative model (SinGAN)**, set the **Iterations** per scale and the **Training volume up to** size (the volume is read at the level that fits it), and click **Train on the volume**. The result compares 4 volumes generated from new noise with the training volume: porosity, two-point probability (S2) and correlation length. The model is kept in the project with its card.
3. Select the model in the table (task *Generation 3D*), set **Volumes**, **Size × training** (1.5 makes each side 1.5 times longer) and **Seed**, and click **Generate volumes**: they are added to the sample, as labels volumes with the legend of the training volume when it was segmented.
4. For large volumes run the training as a job: **Multiscale → Jobs**, calculation *AI: train a generative model* (*SinGAN*), here or on a remote computer with a graphics card; **Bring the results back** adds the model.

Spot cycle (Lab → AI)

1. Choose the **Spots** kind and **From** *this thin section* or *the whole project*, and click **Learn**. The result shows the accuracy and the per-class scores by cross-validation in folds by thin section (a section is never both in training and in validation).
2. Choose the **Classifier** and **Propose from probability** (e.g. 0.6), and click **Propose**: the table lists the unnamed spots with the proposed constituent and its probability.
3. Untick what you reject, type over a constituent to correct it, and click **Accept checked**: the constituents are written into the spots, each with the model and the probability in its record (so you can always tell a name given by hand from an accepted proposal).
4. Learn again: the new classifier uses the accepted spots too.

Options

Field	What it changes
Iterations	Training steps per scale of the pyramid
Training volume up to	The largest training volume (voxels per side); the level of the volume that fits is used
Volumes, Size × training, Seed	How many volumes, how large, and which noise
Propose from probability	Proposals below it are not listed

How it works

- **SinGAN** (Shaham, Dekel & Michaeli 2019), in 3D as in GeoSlicer's RockSinGAN module: a pyramid of small convolutional generators and patch discriminators trained coarse to fine on the one training volume; each scale is trained with the Wasserstein loss and gradient penalty (Gulrajani et al. 2017) plus the reconstruction of the training volume from a fixed noise; the noise of each finer scale is scaled by the reconstruction error of the scale below. The whole pyramid is one ONNX graph, so generation runs on any device.
- **Comparison**: porosity, the two-point probability S2(h) along x and the length where S2 has fallen 1/e of the way to φ^2 (Torquato 2002).
- **Spot classifier**: the random forest of 0.23 on the values (below the detection limit = 0; missing = median of the training spots), cross-validated in folds grouped by thin section.

From a script

```
vol = prj.volume("plug segmentation")
model, comparison = vol.ai_generator(iterations=300, max_voxels=32 ** 3)
new = vol.ai_generate(model, n=4, scale=1.5)
rid, metrics = prj.ai_spots_train(kind="geochemistry")
props = prj.ai_spots_propose(rid, kind="geochemistry", min_probability=0.6)
prj.ai_spots_accept(props, rid)
```

Limits and common problems

- A 3D generator trains in minutes on a graphics card and in hours on a processor; start with a small training size and few iterations to check the result, then run the full training as a job.
- A generator learns one texture: a training volume with a trend (a layer, a fracture) is reproduced as texture, not as a trend; use MPS with hard data when the layout matters.
- Spot proposals are only as good as the named spots: name examples of every constituent you expect, from more than one thin section.

Coverage

GM5 (image generation: the generative network RockSinGAN), L11 (annotation → ML → validation → extrapolation → retraining, for spot geochemistry).

Mio: questions in plain language and the local language model

What it is for

Mio is RockFace's assistant. You ask her about the descriptions of the project in your own words — "which samples have barite?", "quantas amostras do poço B têm porosidade acima de 20 %?", "average permeability of the samples with kaolinite", "what is missing before the study?", "how do I segment a section with AI?" — and she answers from the database: the descriptions that match, how many, minimum, mean and maximum of a number, the values of a field, the main facts of one sample, the descriptions without a value, the page of this guide that explains a task. Every answer shows the conditions she read, so you see exactly what was asked of the database.



In the program: Mio: questions in plain language and the local language model

In the Explorer the same reading works as a **plain-language search**: type the question in the search box of the toolbar and the query appears in the criteria list, ready to refine, save or export.

Mio works without a language model (her rules understand the words of RockFace's nomenclature in English, Portuguese and Spanish, numbers — also written in words — and comparisons). A **local language model** — a small open model that runs on your computer, on the processor or the graphics card — lets her understand freer questions: other spellings ("baryte", "kaolin"), synonyms ("dolomite rhombs"), words outside the nomenclature.

Before you start

- A project open with descriptions.
- For the language model: RockFace with the extra `llm` (`pip install "rockface-suite[llm]"`, which installs llama-cpp-python), or a local server already running (Ollama, LM Studio or llama-server). No graphics card is needed.

Step by step

Asking Mio

1. Click **Mio** in the toolbar (in Petro: **Tools → Mio**), or press **Ctrl+Alt+M**. The Mio panel opens on the right.
2. Type the question and press **Enter** (or **Ask**). `@mio` at the start is optional in the panel.
3. Read the answer and the line **Conditions read**. **Words not used** lists words she did not turn into a condition — rephrase if one of them mattered.
4. When the answer is a search, **Open in Explorer** shows it as an Explorer query.

In any search box that talks to Mio (the Explorer's), a text that starts with **@mio** goes to the panel.

Plain-language search in the Explorer

1. **Explorer** → type in the box *Search in plain language…* (for example *samples with barite and porosity above 20 %*) and press **Enter**.
2. The criteria appear in the list and the descriptions found below them. A question that is not a single query ("calcite or dolomite") selects the descriptions in the list instead.

Choosing the language model

1. In the Mio panel, click **Language model…**.
2. The dialog shows this computer (memory, threads, graphics cards, whether llama.cpp can use the card) and the models that fit it, with size, licence, where it would run and the **recommended** one marked. You choose:
   - **The model selected in the list** — downloaded once (it asks first, with the size and licence) and kept in `~/.rockface/llm`;
   - **A GGUF file on this computer** — any model in GGUF format you already have;
   - **A local server** — the address of Ollama (`http://localhost:11434`), LM Studio or llama-server, and the model name on the server.
3. **Run on**: the processor or the graphics card. **OK** keeps the choice; the model loads with the first question.
4. **Use the language model** (panel) switches it on and off; **Fluent answers** lets the model rephrase the answer (kept only when its numbers are exactly those of the database).

From the command line: `rockface llm setup` (the same choice, with the recommended one marked), `rockface llm use <file.gguf | http://localhost:11434>`, `rockface llm info`, and `rockface llm ask <project> "<question>"`.

Options

Field	What it changes
Use the language model	Off: the rules answer alone. On: the model reads the questions the rules leave words of
Fluent answers	The model rephrases the answer; a rephrasing with any number changed is discarded
Run on	Processor or graphics card (the card needs llama.cpp built with CUDA, Metal or Vulkan)
`ROCKFACE_LLM_PATH`, `ROCKFACE_LLM_URL`, `ROCKFACE_LLM_DEVICE`	The model without the interface (scripts, jobs)
`ROCKFACE_LLM_DIR`	Where downloaded models are kept

The models offered (all with permissive licences, Q4_K_M quantization):

Model	File	Licence	For
Qwen2.5 0.5B Instruct	0.40 GB	Apache-2.0	any computer
Qwen3 1.7B	1.28 GB	Apache-2.0	laptops (recommended with fewer than 8 threads)
Qwen3 4B · Phi-4 mini	2.5 GB	Apache-2.0 · MIT	8 threads or more, 8 GB of memory
Qwen2.5 7B Instruct · Qwen3 8B	4.7–5.0 GB	Apache-2.0	a graphics card

How it works

- **The database answers, never the model.** The question is read into criteria of the Explorer (the same criteria you build by hand) and the descriptions are found by the Explorer in the records. The model is only asked to turn the words the rules did not understand into phrases of RockFace's vocabulary, with numbers; its phrases are checked against the vocabulary (a phrase that is not in it is dropped or replaced by the closest one) and read again by the rules, so every condition is one the user sees. The answer constrained to a JSON schema (llama.cpp grammar) keeps the model from writing anything else.
- **The rules.** The vocabulary is the project's knowledge base (constituents, groups, habits, locations, modifiers in three languages), the values of the identification (wells, formations, facies…), the analyses (porosity, permeability, depth, totals) and the spot analyses; words are compared without accents and plurals, with one letter of tolerance for long words; numbers attach to the nearest number word, habits to the nearest constituent.
- **The tools.** `search`, `count`, `stats`, `values`, `describe`, `missing`, `suggest`, `guide` — all read only. Mio's **skills** chain them (*Mio's skills* page).
- **Privacy.** The model runs on the computer; a server address on another computer is refused unless you allow it explicitly, because the questions and the records shown to the model would leave the computer.
- **Accepted on a test set.** An ILLUSTRATIVE project of 48 invented descriptions of known content and 41 questions in English, Portuguese and Spanish: 41 right by the rules. Fifteen harder paraphrases: 8 right by the rules alone, 13 with Qwen2.5 0.5B on a processor without a graphics card (2 threads; 3–14 s, median 5 s, for the questions that call the model) — `docs/LLM.md`.

From a script

```
a = prj.ask("amostras com barita e porosidade acima de 20 %")      # rules only
print(a.count, a.names, a.explanation, a.unknown)
q = a.query()                                                  # the Explorer query, when it is one
mio = prj.mio(model=True, lang="pt")                               # with the language model
↪  configured
r = mio.ask("@mio quantas amostras do poço ILLUSTRATIVE-B têm caulinita?")
print(r.tool, r.text, r.data)
from rockface.api import llm
m = llm.LlamaModel("Qwen2.5-0.5B-Instruct-Q4_K_M.gguf", device="cpu")   # or
↪  llm.ServerModel("http://localhost:11434", "qwen2.5:0.5b")
print(prj.ask("plugs containing baryte", model=m).names)
```

Limits and common problems

- "No language model": choose one with **Language model...** or `rockface llm setup`; Mio still answers with her rules.
- A word ignored: it is listed under **Words not used**; use the term of the nomenclature or switch the model on.
- The first question with a model takes a few seconds more (the model is loaded once per session).
- On a server project the panel answers in the window's thread (the window waits while the model reads).
- Mio does not change the project: tools and skills only read; suggestions are written only when you accept them.

Coverage

L1 (natural-language search of the database), the assistant that calls the program's tools and the choice of the processor or the graphics card of the ROADMAP stage 0.27.

Mio’s skills

What it is for

A **skill** is a small recipe that Mio runs whenever a question asks for it: several of her tools one after the other, with the values taken from the question. “Well summary ILLUSTRATIVE-B” counts the samples of the well, lists its facies, gives porosity and permeability and says which descriptions lack porosity — in one answer. RockFace ships eight skills, active from the start, to use and to copy; you write your own and keep them for yourself or in the project, for the whole team.



In the program: Mio’s skills

Before you start

- A project open. Writing a skill needs no programming: it is a short JSON text edited in the **Skills** dialog.

Step by step

Using a skill

1. In the Mio panel type one of its phrases (“resumo do poço ILLUSTRATIVE-B”, “data gaps”, “melhores reservatórios”, “how do I ...”), or its name after a slash: `/well-summary ILLUSTRATIVE-B`.
2. If the skill needs a value the question did not give (the well, a sample), Mio asks for it.

The skills shipped with RockFace:

Skill	Called by (examples)	What it does
`well-summary`	well summary · resumo do poço · resumen del pozo	count, facies, porosity, permeability, missing porosity of a well
`formation-summary`	formation summary · resumo da formação	count, wells, facies, porosity, cement of a formation
`reservoir-quality`	reservoir quality · melhores reservatórios	samples with porosity ≥ 15 % and permeability ≥ 100 mD, and their wells
`cement-overview`	cement overview · cimentação	total and carbonate cement; samples with calcite, dolomite, quartz overgrowths
`dated-samples`	dated samples · amostras datadas	samples with dating spots and their wells
`data-gaps`	data gaps · lacunas · control de calidad	descriptions without depth, porosity or permeability
`sample-report`	sample report · ficha da amostra	the main facts of a description and the suggestions of its maps
`how-to`	how do I · como faço · cómo hago	the pages of this guide that explain a task

Writing a skill

1. Mio panel → **Skills...**. The list shows every skill with where it comes from: *RockFace*, *mine* (your folder), *project*.
2. Select one to see it, or click **New** for a template.
3. Edit the JSON (fields below) and click **Save for me** (your folder) or **Save in the project** (everyone who opens the project). The skill is checked before it is saved; problems are listed under the editor.
4. **Active** switches a skill off without deleting it. A skill saved with the name of another replaces it (a project skill named `how-to` replaces RockFace’s); **Delete** brings the other back.

Options

```
{
 "name": "kaolinite-by-well",
 "title": {"en": "Kaolinite by well", "pt": "Caulinita por poço", "es": "Caolinita por pozo"},
 "description": {"en": "How many samples of a well have kaolinite, and their porosity.", "pt":
 ↪  "…", "es": "…"},
 "triggers": ["kaolinite by well", "caulinita por poço", "caolinita por pozo"],
```

```
 "inputs": {"well": {"field": "well"}},
 "steps": [
  {"tool": "count", "question": "samples of well {well} with kaolinite"},
  {"tool": "stats", "number": "petrophysics.porosity", "question": "samples of well {well} with
   ↪  kaolinite"}
 ],
 "active": true
}
```

Field	What it is
`name`	lower case, digits and hyphens; also the `/name` that calls the skill
`title`, `description`	shown in the list, in English, Portuguese and Spanish
`triggers`	phrases that call the skill (accents and capitals do not matter)
`inputs`	values read from the question: `{"field": "well"}` (any identification field: well, formation, facies, field, basin, zone...) or `{"field": "sample"}` (a description's name); `{question}` is always available
`steps`	the tools, in order: `search`, `count` (`question`), `stats` (`number`, `question`), `values` (`field`, `question`), `missing` (`number`, `question`), `describe` (`sample`), `suggest` (`sample`), `guide` (`question`)

`number` is `concept.attribute` as in the Explorer: `petrophysics.porosity`, `petrophysics.permeability`, `petrophysics.grain_density`, `identification.top_depth`, `totals.cement`, `totals.carbonate_cement`...

How it works

Skills are data, never code: a skill can only call Mio's tools, and every tool only reads the project. Skills come from three places, read in this order (a later one with the same name replaces an earlier one): RockFace's folder `rockface/mio_skills`, your folder (`~/.rockface/mio/skills`, or the setting `mio.skills`, or `ROCKFACE_MIO_SKILLS`) and the project (kept in the project with an audit line, so a server project shares its skills with the team).

From a script

```
for s in prj.mio_skills():
    print(s.name, s.source, s.triggers)
prj.save_mio_skill({"name": "barite-count", "triggers": ["barite please"],
                    "steps": [{"tool": "count", "question": "samples with barite"}]})
print(prj.mio(lang="en").ask("/barite-count").text)
prj.delete_mio_skill("barite-count")
```

Limits and common problems

- "The skill cannot be used: ...": the check lists what to fix (a tool that does not exist, an input used but not declared, a `stats` without a number).
- A trigger that is also a normal question ("samples with calcite") makes every such question run the skill — choose phrases that say what the skill is.
- Skills that write to the project (apply a suggestion, run a script) are not allowed in this version: Mio proposes, the user decides.

Coverage

The assistant that calls the program's tools of the ROADMAP stage 0.27; the owner's request of 2026-11-13 (skills created by the user, examples active from the start).